



Chess Game & Artificial Intelligence

AQA GCE Computer Science NEA

Program to allow its users to play a game of chess, either with others or with a chess-playing AI I have created.

Alfie Jay Kunz 8158
Beckfoot School 37101
2021-2023

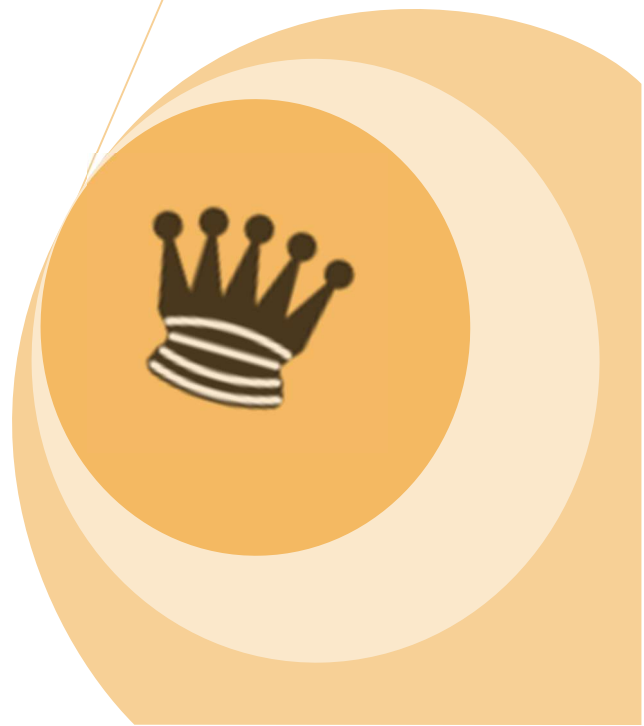
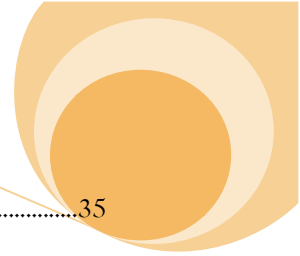


TABLE OF CONTENTS

Chess Terminology & Glossary	5
Section I: Analysis.....	7
Project Introduction.....	7
Project Outline / Description	7
Identifying my Supervisor.....	7
Interview with Supervisor.....	7
Initial Project Objectives	8
Research – Primary AI Algorithm.....	9
The Main Issue with Genetic Algorithms.....	10
MiniMax with Alpha-Beta Pruning: Noughts & Crosses Prototype	10
An Explanation of MiniMax	10
Alpha-Beta Pruning.....	15
Alpha-Beta Pruning Explanation	16
Secondary AI Research.....	18
Quiescence – Creating a Stable AI.....	18
Solved vs Non-Solved Games, and Measuring Success.....	19
Revised Project Objectives	19
Critical Path	20
Section 2: Design.....	22
Program Layout & General User Features	22
Data Structures.....	23
Representing the Chessboard and the Details of a Chess Position.....	24
An Aside: The Importance of The TrueFalseTable	27
Graphical User Interface	30
Displaying the Pieces on the Board.....	30
Moving a Piece – Drag & Drop Mechanics	32
Importing / Exporting Positions	34



FENConverter Algorithm	35
Eliminating Invalid FEN Strings	36
ConvertToFen Algorithm	37
Interacting With Text Files.....	37
Primary Algorithms.....	38
The Core of my Program – The PieceLegalMove Algorithms.....	38
The En Passant Pin.....	40
Resolving Checks Efficiently.....	42
Instantiating Our AI.....	43
Calculating the Optimal Depth for the Position.....	43
Multithreading	44
Source Code.....	45
The MiniMax Process.....	48
Finding and Ordering Moves.....	48
Making a Temporary Move	49
Evaluating a Chess position.....	51
Using Hashing to Calculate Piece Value	51
Adding Endgame Heuristics	52
Checking for End States.....	54
Section 3: Technical Solution.....	56
MainMenu Form	56
Customisation Forms.....	63
OnePlayerCustomisation Form	63
TwoPlayerCustomisation Form	65
Chess Form	66
Constructures & Board Creators.....	69
GUI Object Controls.....	79
Drag & Drop Mechanics.....	88
AI Hndlers & Thread Controls	95

AI Class	106
Base Controls.....	107
AI Core – Move Generators.....	112
AI Core - MiniMax	119
CoreMethods Class	123
FEN Converters	124
Pseudo-Legal Move Generators.....	126
FixTFTable Subroutine & Other Controls	145
SubObjects	148
InCheck Class.....	148
CanCastle Class	149
Move Structure	149
Section 4: Testing.....	150
Testing The GUI & its Features	150
Testing AI Features	155
Tracking AI Progress.....	160
Opening Test.....	160
Middlegame Test.....	161
Endgame Test.....	162
Section 5: Evaluation.....	163
Supervisor Assessment.....	163
Notes on Supervisor’s Comments	164
Fixing Quiescence.....	164
Objective Assessment.....	166
Calculating My AI’s Strength.....	170
Personal Evaluation.....	171

Front King Picture Credits: ‘4and4’ (<https://www.deviantart.com/4and4/art/King-Chess-Piece-Polygon-522963093>)

CHESS TERMINOLOGY & GLOSSARY

Welcome to my project report! Please note that this report does require some basic knowledge of chess, such as how the pieces move, castling, what a 'check' or 'checkmate' is, etc. However, I do believe that some more complex terminology does require some introduction for those who are not familiar with the game. With that in mind, here is my Project Glossary:

FEN = a string that represents the standard notation for a chess position, containing where all the pieces are on the board along with details such as who's turn it is, who can castle, En Passant rules, etc. For example, the starting position in chess is represented by the FEN string "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1". The steps required to generate / read from a FEN string is explained in [the design section](#) of this report.

Depth = how many 'moves deep' a computer searches from the starting position. If a chess AI, for example, was searching to a depth of 1, it would search all the possible moves a player can make on their turn. At a depth of 2, the AI would search all the possible moves a player can make, along with all the moves their opponent can make for each of the first player's possible moves. At a depth of 3, the computer would search all the possible moves a player can make, then perform a depth of 2 on each of these moves ($1+2=3$), and so forth. We usually use recursive algorithms (functions that call themselves) to perform this.

En Passant = A move in chess performed with pawns. Pawns can move two squares on their first move, and if one does this and ends up directly next to an opponent pawn, that pawn can be captured by the opponent pawn by taking it diagonally, as if it had only moved one square. For more details on this, please check out this helpful video: https://www.youtube.com/watch?v=c_KRIH0wnhE.

Trade = A set of moves in chess where a player's piece is exchanged for an opponent's piece of equal value. For example, if Player A were to take Player B's queen with their queen, and then player B immediately takes back A's queen, a trade of queens has occurred.

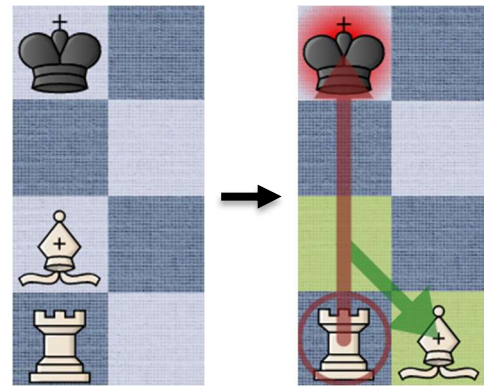
Pinned Piece = A piece that is attacked by an enemy piece, and is therefore unable to move out of the way as it would allow a devastating capture. There are 2 types of pins in chess, absolute pins and relative pins^[1]. An absolute pin is one where a piece is pinned to their own king; as a result, it is **illegal** to break this pin (or in other words, to move your pinned piece out of the pin). A relative pin is where a piece is pinned to another of their pieces of a higher value to itself (for example, a bishop may pin a knight to its queen, and if this knight is to move then the bishop could take its queen). Though it is legal to break a relative pin, it is strongly advised against for new players, and should only be played tactically.



The picture on the left shows an example of an absolute pin (red arrows) and a relative pin (amber arrows) in a typical game of chess, where a bishop is pinning an enemy knight to its king / queen. Though the white knight is legally able to move, the black knight is not, as that would result in the capture of a king.

Note that (unlike this example), there are some pins that allow restricted mobility of a piece. For example, a rook that is pinned by another rook can still move in one direction without breaking the pin.

Discovered Attack / Discovered Check = A technique in chess where, by moving one of your pieces, a different piece reveals an attack on your opponent. This can either be a discovered check, in which the king is checked by a piece that hasn't moved, or a discovered attack, where any other enemy piece falls victim to this technique. In the example on the right, white moves their bishop, unleashing a discovered attack by their rook.



Pseudo-legal = A common beginner coding practice when creating chess. In pseudo-legal chess, checks, checkmates & pins are not analysed when determining if a move is legal or not. Therefore, the main objective in pseudo-legal chess is to capture the opponent's king. In this project we will also discuss pseudo-legal moves. These are moves which conform with the rules in pseudo-legal chess, but not necessarily normal chess (we need to do extra checks to determine whether a pseudo-legal move is an actual legal move in chess). All legal moves are pseudo-legal moves, but not all pseudo-legal moves are legal moves.

REFERENCES

[1] = <https://www.regencychess.co.uk/blog/2012/07/chess-basic-tactics-series-I-pins-and-skewers/#:~:text=A%20pin%20in%20chess%20is,absolute%20pins%20and%20relative%20pins> ("A pin in chess is when you attack a piece that can not move out the way because it would allow capture of a piece of greater value. There are two types of pins, absolute pins and relative pins.")



SECTION I: ANALYSIS

PROJECT INTRODUCTION

For many, chess is a game that needs no introduction. Dating back to the 6th century in India, chess is a two-player turn-based strategy game that simulates a battle between two empires. Each player has an arsenal of pieces at their disposal, from pawns to queens, and are tasked with capturing, or 'checkmating', their opponent's king.

Chess has famously been called a pinnacle of Artificial Intelligence research due to the intense logic, strategy and intuition that sets players apart. If you ask a great chess player why they played a certain move, sometimes they will reply 'it just felt natural', or 'it was the only obvious move to me', and naturally this makes creating a strong chess Artificial Intelligence very strenuous.

I have been playing chess for almost two years, and since learning I have been fascinated by how computers can play such a complex game to a high-level. Therefore, choosing this task as my project was a no-brainer.

PROJECT OUTLINE / DESCRIPTION

The main purpose of my project is to devise a program that can allow a user to both play chess with their friends, or let them play against a chess AI of varying difficulties, which I have created. This will be achieved via Windows Forms to provide a graphical interface through which the user can access the full extent of my program. The program should also allow the user to import & export custom (but legal) chess positions and play these positions with others / my AI.

IDENTIFYING MY SUPERVISOR

I have chosen my supervisor to be my friend and fellow chess player, Hammad Ul-haq. He has been playing chess for multiple years and has sufficient knowledge of the game that he can pass onto me. My supervisor will also become my primary play-tester and will give me tips on how to improve my project along the way.

INTERVIEW WITH SUPERVISOR

Before creating my initial project objectives, I first conducted an interview with my supervisor, Hammad, in which we discussed what makes a good chess interface. He provided details on what he would like to see in a chess program, and what features would benefit him.

Transcript:

What purpose would my chess-playing program provide for you?

I want to use this program to play chess with my mates, and be able to discuss and analyse positions with them. I would also like to translate any position into the program and be able to play from it.

What purpose would my chess AI provide for you?

To analyse positions / openings with it, anytime I want. Also so that I can try to find the best move in any particular game, and see if the computer agrees or not.

How would you like to interact with my program? (Eg: how would you want the pieces to be displayed?)

Simple to use, pieces being represented graphically, simply and accurately. Should be able to drag and drop pieces to legal squares, and have those moves be put on the screen. Only the essential options from the playing screen.

In your opinion, what makes a chess program interface 'good'?

Only allowing legal moves, instead of pseudo-legal moves. Full functionality of chess, including en passant, checks, promoting, etc. A good chess program should also let me know when an end-position has been reached audibly (checkmate, stalemate etc) and stop the game.

What would be the most useful feature or features I could implement into my program's interface?

Definitely being able to type in my own chess positions from other chess programs, and have them being put onto the board. Similarly, I want to be able to export my current chess position so that I can use it on other programs. Also, being able to undo my or my opponent's previous move would be very useful for me.

What other special features would you like to see in my chess program or its interface?

Being able to view a piece's legal moves when you click on them, visually seeing the last move I or my opponent have made, and being able to flip the board for black's perspective any time I like.

What goal would you want my chess AI to have?

I would like it to give me a good, challenging game and make its moves in as short of a time as possible (preferably under 10s).

Would you want 'variants' of chess such as Chess960, Atomic Chess etc to be an essential feature of my program?

No, I don't find those features as important as the others I have mentioned.

On a scale of 1 to 10, with 1 being purely functional and 10 being purely user-friendly, where would you want my program (and its interface) to be?

Probably around a 6 or 7.

Do you have any other requirements for this project I should know about?

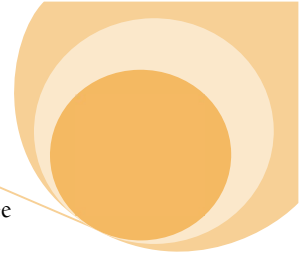
I would like it if I was able to change the board colours, and have my preferences be saved for the next time I open the program. Being able to play against multiple engine difficulties would also be cool.

INITIAL PROJECT OBJECTIVES

From my meeting with my supervisor, I have developed this list of measurable project objectives. These objectives may be subject to change as a result of my future analysis & research (please see the [research subsection](#) for more information), however with that being said, here are my initial objectives:

Essential objectives:

- 1) The user should be presented with a graphical and accurately scaled interface of a chessboard and its pieces.



- 2) The user should be able to drag and drop the pieces on the board, and if that move is legal (see Objective (3) for more details), have it be represented on the board.
- 3) The program should use the standard rules of chess to determine if a move the user has entered in is legal or not (and refuse it if it's not). This may require context about the board, such as restricting certain moves if a piece is blocked by another piece, if a king is in check, or if a piece is pinned by an opponent piece.
- 4) The user should be able to Castle both king-side and queen-side whilst conforming to the rules of chess.
- 5) The user should be able to enter in a chess FEN, and **if it is legal** (which will be determined by the program), it will be presented onto the chessboard.
- 6) The user can promote a pawn to a queen if it reaches the end of the board.
- 7) The program should identify when a check has occurred, and will notify the user of this both visually and audibly. This should also be used to restrict the moves an opponent can make, as checks need to be resolved immediately.
- 8) When a user is put into check, the program should accurately determine whether this is a Checkmate or a regular check. If it is a Checkmate then the game should terminate.
- 9) The user, at any time, should be able to let my chess AI make a move from the current position (for either player). My AI should take time-constraints into consideration when making its moves, and at the conclusion its move should be represented on the chessboard.

Desired objectives:

- 1) When allowed to, the user should be able to take En Passant.
- 2) The game should end if only kings are left on the board, or in other special circumstances (please see www.chessstrategyonline.com/content/tutorials/how-to-play-chess-draws for more detail).
- 3) The user will be given the option at all times to undo their / their opponent's last move.
- 4) The user should be able to select multiple / variable difficulties of the AI.
- 5) The user should have their last move be illustrated on the chessboard graphically.
- 6) The user should be able to see a piece's possible moves by clicking on it.
- 7) The user should be able to flip the board for black's perspective.
- 8) The user should have their last move be illustrated on the chessboard graphically.

RESEARCH – PRIMARY AI ALGORITHM

The bulk of my research comes from designing the algorithm from which my chess AI will play from. The main area of discussion here is the AI's searching algorithm, as this will provide its foundation. There are two main options I can implement here: a tree-searching algorithm such as MiniMax, or a genetic algorithm such as NEAT; whilst genetic algorithms have the capacity to play board games such as chess more efficiently and effectively than a tree-search, my research has detected 2 main problems with them...

- 1) Genetic algorithms are incredibly difficult to implement, and I believe that genetic algorithms in chess are beyond the scope of this A-level project. For a genetic algorithm such as NEAT to beat a tree-searching algorithm at chess, one needs a large group of very experienced programmers who fully understand both the game of chess and Artificial Intelligence. I, on the other hand, have never coded an AI before.

- 2) Genetic Algorithm AIs get better through experience, so if most of their games focus on the same skills in a game then they might not have a full understanding of some less common areas. To fully understand this concept, let's look at an example...

THE MAIN ISSUE WITH GENETIC ALGORITHMS

Chess can be broken down into 3 main categories: the opening phase, the middlegame and the endgame. The endgame of a chess game is distinctly different depending on which pieces are on the board, how many of them you have, and how many your opponent has. For example, a chess endgame with a rook requires very different skills to one where you have a knight and a few pawns. Now let's link this to creating a chess AI with genetic evolution. As the AI gets more and more experience, it may develop into a player who (for example) likes to trade knights very early on in the game (and therefore get rid of all the knights on the board well before the endgame phase) – this may happen almost every game, as the AI would be playing against similar versions of itself.

Once the AI has finished training, we'll put it against a skilled human chess player. What we would find is that, in the core, common aspects of the game, the computer would excel and show a real understanding of the game (maybe even to a superhuman level). However, if the game between the human and the computer resulted in an endgame of knights, the AI would begin to struggle. Because it has not had the chance to build up its knowledge of knight endgames, it would have next to no idea how to beat a competent human player, and would most likely lose as a result^[3].

As we cannot easily detect when the AI would enter that area of little understanding, it becomes almost impossible to ensure that an Artificial Intelligence using Genetic Algorithms is balanced in all aspects of play, so as a result with all this (and also due to the fact that this will be my first time coding Artificial Intelligence), I decided to create my AI using a tree-searching algorithm.

MINIMAX WITH ALPHA-BETA PRUNING: NOUGHTS & CROSSES PROTOTYPE

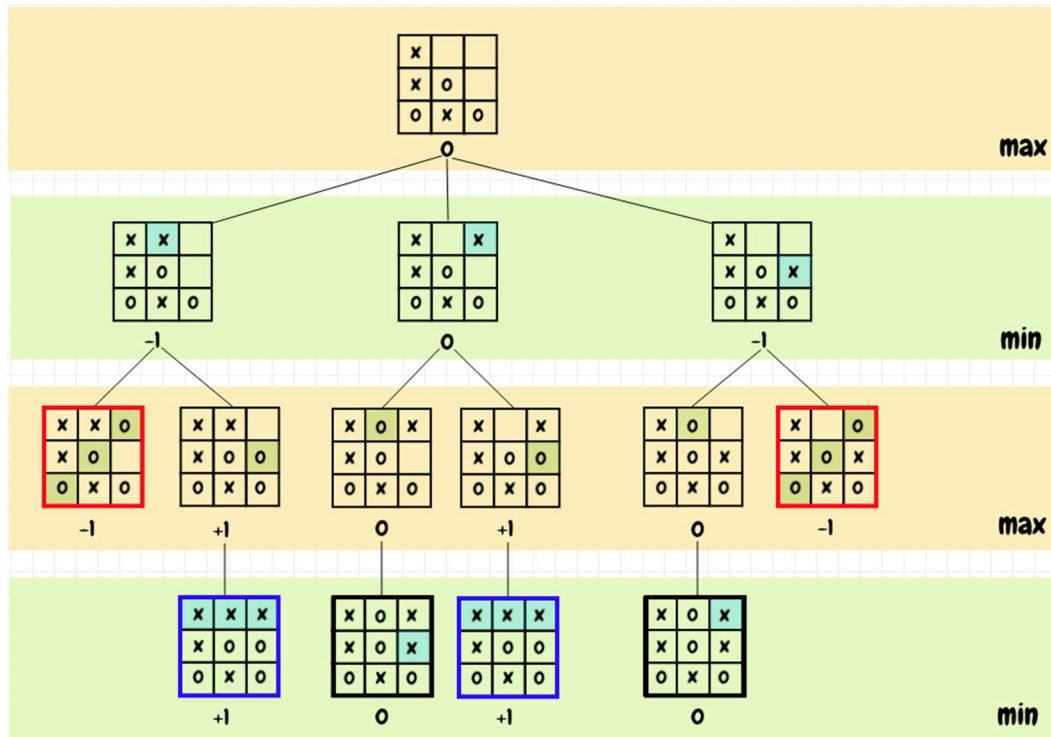
When researching for a tree-searching algorithm to play chess, nothing came close to the effectiveness and the adjustability of the MiniMax algorithm. The MiniMax algorithm is the gold-standard for creating chess Artificial Intelligences - it is an example of a recursive tree algorithm, recursive meaning that the algorithm calls itself for each position found, and tree meaning that its algorithm can be drawn as a tree, with the root node representing the position first given to MiniMax (the base position), and the leaf nodes representing all the end positions the algorithm will evaluate.

I chose to implement the MiniMax algorithm into my chess AI as it guarantees the same results every time, and is an example of a play-safe strategy – guaranteeing the 'best' results even if its opponent plays 'perfectly'. However, to first gain some experience with this algorithm, I chose to test this algorithm on the simpler game of Noughts & Crosses.

AN EXPLANATION OF MINIMAX

As stated previously, MiniMax is an example of recursive programming – each depth in a search is represented by a different 'layer' in the recursive algorithm, and is broken down into 2 sections: a maximising function (player X for my Noughts & Crosses Game), and a minimising function (player O). Player X will always choose the greatest outcome from a set of positions, whilst player O will always choose the lowest outcome from a set of positions. (As all game values are in terms of the maximising function X, a score of -1 would be losing for X

but **winning for O**, so O chooses the lowest value.) The best analogy for how MiniMax works is a tree of possible game-states:



(Picture Source: Nested Software – www.nestedsoftware.com/2019/06/15/tic-tac-toe-with-the-minimax-algorithm-5988.123625.html)

The board position in the top layer represents the current board position, and for this example **O** is the maximising function and **X** is the minimising function. The position is sent to MiniMax, and for each move possible it creates a new virtual board (as shown in the second layer). No end position has been reached, so each of these positions are called through MiniMax recursively to the 3rd layer. This continues until all end positions are reached, which are then evaluated as either winning for X (+1), losing for X (-1) or drawn (0).

Once all these positions have been evaluated, they move back up the tree. For the 4th layer positions going up to the 3rd layer this is quite easy, as no conflicts have to be resolved, however for positions going from the 3rd to the 2nd layer, a decision must be made between 2 board positions. Now, in the 2nd layer, the minimising function (X) is playing. Therefore, out of all the board positions that result from it, X will choose the position with the **lowest value** and pass that position up a layer.

Now we move from the 2nd layer to the 1st layer; again, there are multiple conflicts between board positions. In the 1st layer, the maximising function (O) is player. Therefore, out of all the conflicting positions, O will choose the board with the **highest value** and pass that position up a layer. Now we have the shallowest layer, and only **I board position remains**. This becomes the optimal strategy for player O, guaranteeing the best, safest result even if X plays perfectly.

My first attempts at MiniMax came from experimenting with a much simpler game: Noughts & Crosses. There are three main reasons why this task is simpler than programming this algorithm for chess (and hence why I chose N&C as my prototype game):

- 1) There are much fewer positions achievable in Noughts & Crosses (255,000)^[1] as opposed to chess (approximately 8.7×10^{45})^[2]. This makes searching to a higher depth much more achievable.

- 2) As the AI will be able to search through **all** these positions in a search, it will only need to evaluate **end** positions as either being won, drawn, or lost (discrete).
- 3) Noughts & Crosses is what is known as a 'space' game. As a result, it will be much easier to make / undo moves with MiniMax (as we don't need to worry about taking / removing pieces, or have to reject illegal moves).

As this was my first time investigating MiniMax, I took some support from an online YouTube tutorial from 'The Coding Train', in which the basics of MiniMax and its implementation in code is explained to beginners such as myself (note: 'The Coding Train' uses JS as their main coding language, which I am not at all familiar with, so their source code acted more like pseudocode to me). You can find that video here:

www.youtube.com/watch?v=trKjYdBASyQ.

After experimenting more, I became confident enough to code my first version of MiniMax with Noughts & Crosses. The source code below is broken into 3 distinct sections: the first containing the evaluation code, the second initiating MiniMax and selecting & outputting the AI's best move, and the last containing the MiniMax algorithm.

MINIMAX SOURCE CODE:

```
Function Evaluation(ByRef Depth As Integer) As Decimal
    Dim Result As Decimal
    If CheckWin(board) = "X" Then
        Result = 1 - CDec((Depth / 10)) 'Chooses the shortest pathway to winning.
        '(faster wins are given a higher score than slower wins).
    ElseIf CheckWin(board) = "O" Then
        Result = CDec((Depth / 10)) - 1 'Chooses the longest pathway to losing.
    ElseIf CheckWin(board) = "Draw" Then
        Result = 0
    End If
    Return Result
End Function
```

20 references

```
Public Function CheckWin(ByRef tempboard(,) As String) As String
    winner = ""
    tempcheck = "O"
    For n = 0 To 1 'Checks wins for O first, then for X.
        For x = 0 To 2
            'Checks horizontal + vertical wins.
            If tempboard(x, 0) = tempcheck And tempboard(x, 1) = tempcheck And tempboard(x, 2) = tempcheck Then winner = tempcheck
            If tempboard(0, x) = tempcheck And tempboard(1, x) = tempcheck And tempboard(2, x) = tempcheck Then winner = tempcheck
        Next
        'Checks diagonal wins.
        If tempboard(0, 0) = tempcheck And tempboard(1, 1) = tempcheck And tempboard(2, 2) = tempcheck Then winner = tempcheck
        If tempboard(0, 2) = tempcheck And tempboard(1, 1) = tempcheck And tempboard(2, 0) = tempcheck Then winner = tempcheck
        tempcheck = "X"
    Next
    If winner = "" Then
        draw = True
        'If all the elements in the board are full and a winner hasn't been decided, then the game must be a draw.
        'Otherwise, the end position has not been reached and the game continues.
        For a = 0 To 2
            For b = 0 To 2
                If tempboard(a, b) = "" Then
                    draw = False
                    Exit For
                End If
            Next
        Next
        If draw Then winner = "Draw"
    End If
    Return winner
End Function
```



```
Private Sub Initialise_MiniMax() Handles Button2.Click
    'Computer Plays as X.
    ProgressBar1.Value = 0
    ProgressBar1.Visible = True
    Dim score As Decimal = 0
    Dim bestscore As Decimal = Integer.MinValue
    'Represents the worst move for the computer.
    Dim bestmove As String = ""
    Dim depth As Integer = 1
    Dim RndInt As Integer

    Array.Copy(table, board, 9) 'creates virtual board.
    Randomize()
    For x = 0 To 2
        For y = 0 To 2
            If board(x, y) = "" Then
                board(x, y) = "X"
                'For each move possible, it plays it on the temp board.
                score = MiniMax(board, depth, False)
                'Position ran through MiniMax to generate score.
                If score > bestscore Then 'replaces BestScore.
                    bestscore = score
                    bestmove = x & y
                ElseIf score = bestscore Then '50% of the time, BestScore is relaxed.
                    RndInt = CInt(Rnd() * 1)
                    If RndInt = 1 Then
                        bestscore = score
                        bestmove = x & y
                    End If
                End If
                Array.Copy(table, board, 9) 'undos the last move back to OG position.
            End If
        Next
    Next

    'Plays the AI's chosen move on the board and displays it.
    table(CInt(bestmove.Substring(0, 1)), CInt(bestmove.Substring(1, 1))) = "X"
    'Checks for end positions.
    DisplayWin(CheckWin(table))
End Sub
```

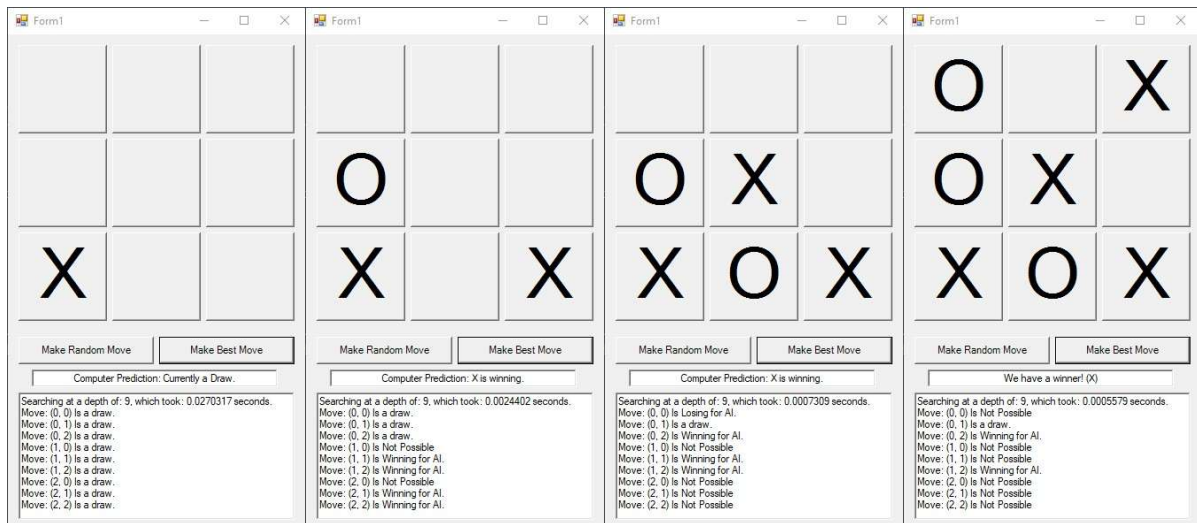


```

Public Function MiniMax(Board(,) As String, Depth As Integer, isMax As Boolean) As Decimal
    Dim Score As Decimal
    If isMax Then 'Computer, = X
        Dim BestScore As Decimal = Integer.MinValue
        If CheckWin(Board) <> "" Then
            'if end position has been reached, position is evaluated and returned up a layer.
            Return Evaluation(Depth)
        Else
            For a = 0 To 2
                For b = 0 To 2
                    If Board(a, b) = "" Then
                        Board(a, b) = "X"
                        'For each move possible, it plays it on the temporary board.
                        'Position ran through MiniMax to generate score.
                        Score = MiniMax(Board, Depth + 1, False)
                        'MiniMax called recursively for the opposite player.
                        If Score > BestScore Then
                            BestScore = Score
                        End If
                        Board(a, b) = "" 'Undos move.
                    End If
                Next
            Next
            Return BestScore
            'At the conclusion of the search, MiniMax returns its chosen move's score up a layer.
        End If
    Else 'Human, O.
        Dim BestScore As Decimal = Integer.MaxValue
        If CheckWin(Board) <> "" Then
            'if end position has been reached, position is evaluated and returned up a layer.
            Return Evaluation(Depth)
        Else
            For a = 0 To 2
                For b = 0 To 2
                    If Board(a, b) = "" Then
                        Board(a, b) = "O"
                        'For each move possible, it plays it on the temporary board.
                        'Position ran through MiniMax to generate score.
                        Score = MiniMax(Board, Depth + 1, True)
                        'MiniMax called recursively for the opposite player.
                        If Score < BestScore Then
                            BestScore = Score
                        End If
                        Board(a, b) = "" 'Undos move.
                    End If
                Next
            Next
            Return BestScore
            'At the conclusion of the search, MiniMax returns its chosen move's score up a layer.
        End If
    End If
End Function

```

Showcase of the MiniMax Algorithm – Computer playing as X:



ALPHA-BETA PRUNING

Alpha-Beta Pruning is an add-on to the MiniMax Algorithm, producing the same moves as MiniMax but much, much faster. How much faster? Here is a comparison of my Noughts & Crosses AI with and without Alpha-Beta Pruning:

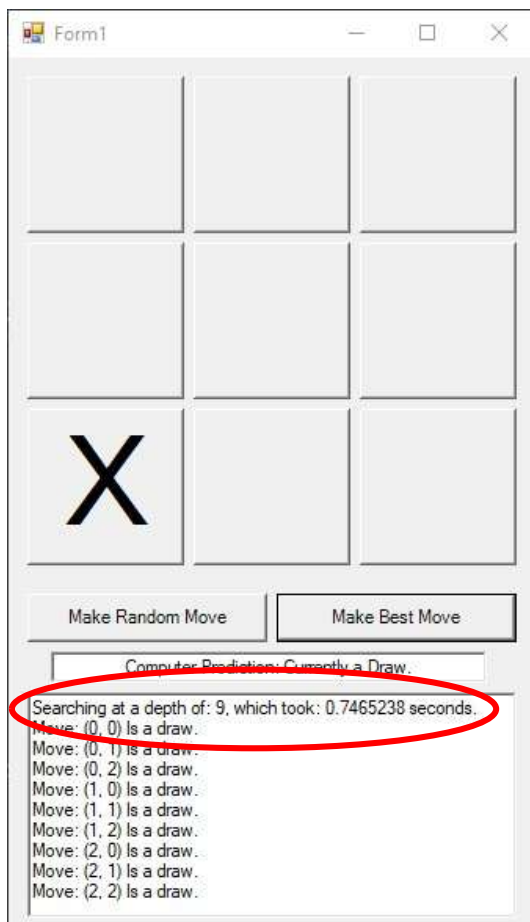


Figure 1: MiniMax without ABP – 0.747s

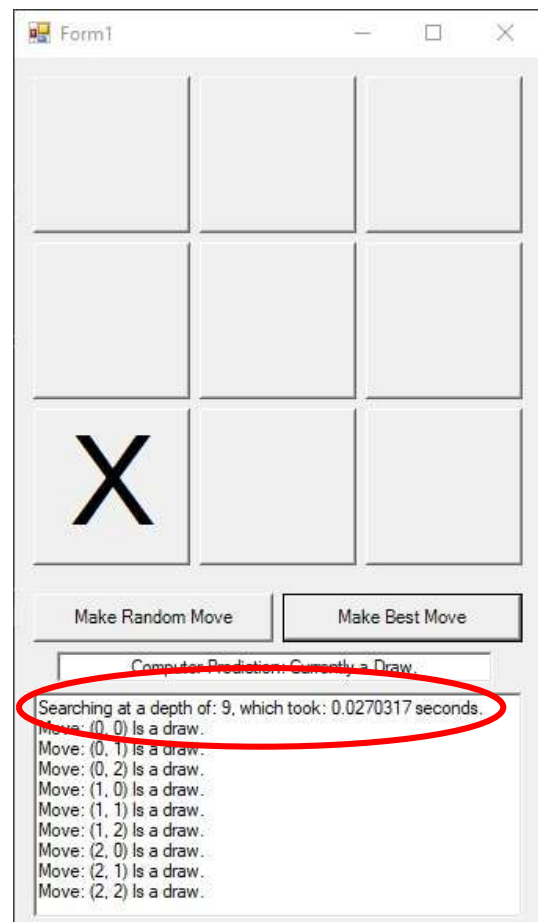


Figure 2: MiniMax with ABP – 0.027s

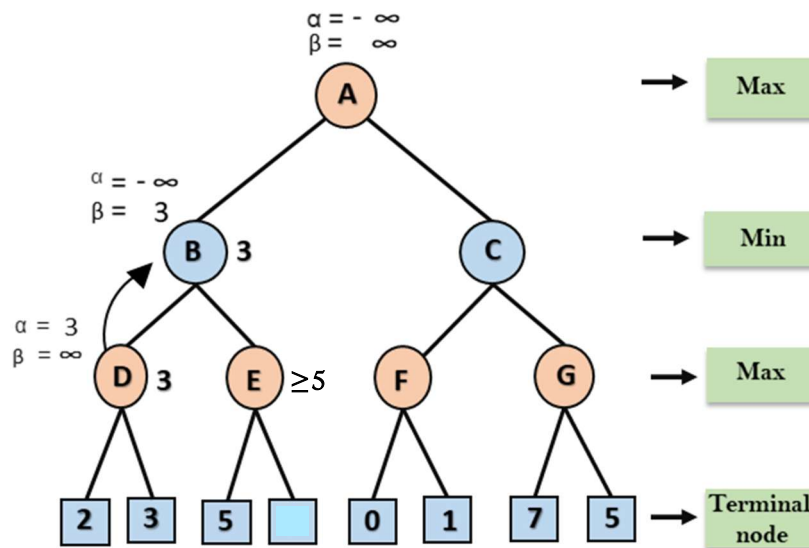
	Run 1 (s)	Run 2 (s)	Run 3 (s)	Run 4 (s)	Run 5 (s)	Mean Time (s)
Without ABP	0.7358487	0.7341821	0.7438502	0.7465238	0.7340544	0.730
With ABP	0.027199	0.0270369	0.0282634	0.0272613	0.0271759	0.027

Average speed increase with Alpha-Beta Pruning: **2698%** ($\approx 27x$ faster).

As you can see from my results, Alpha-Beta Pruning is arguably the most effective improvement to the MiniMax algorithm, so how does it work? The answer may be simpler than you would think:

ALPHA-BETA PRUNING EXPLANATION

So how does this amazing algorithm work? To find out, let us return to our tree analogy:

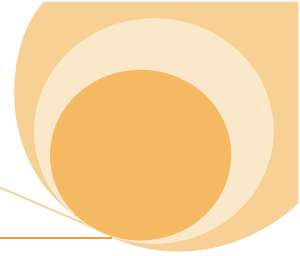


(Picture Source: Java T Point – www.javatpoint.com/ai-alpha-beta-pruning)

In this position, the maximising function at D selects position with value 3 ($3 > 2$), and passes that up a layer to B. Now B locates the sub-position E and sends that to MiniMax. At E, a sub-position is located, and is evaluated with a value of 5. Let's stop the search here for a second:

We know that E, being the maximising function, will choose either the position with value 5, or a position with value > 5 . Therefore, we can assign the value ≥ 5 to E, and pass that up to B. Now B, as a minimising function, will choose the lowest position value possible, and has the option of choosing the position with value 3 at D, or the position with value ≥ 5 at E. Of course, it will choose value 3, and pass that position up to A.

So, as a result of Alpha-Beta Pruning, we have found the exact same result of 3, however we were able to skip searching (or prune) a branch, giving us a $\approx 16\%$ increase. On a depth of 3, Alpha-Beta Pruning won't have much of an effect on speed, however as the depth gets greater and greater (such as in Noughts & Crosses, where we search to a depth of 9), the benefits from Alpha-Beta Pruning get larger and larger.



ALPHA-BETA PRUNING SOURCE CODE

Here is my source code implementing Alpha-Beta Pruning into my Noughts & Crosses MiniMax Algorithm:

```
score = MiniMax(board, depth, False, Alpha, Integer.MaxValue)  
Alpha = Math.Max(Alpha, score)
```

Figure 1 - changes when initializing MiniMax.

```
Score = MiniMax(Board, Depth + 1, False, Alpha, Beta) 'M  
Alpha = Math.Max(Alpha, Score)  
If Beta <= Alpha Then 'Move was too good - prune branch.  
    Return BestScore  
    Exit Function  
End If
```

Figure 2 - changes for X in MiniMax (maximising function).

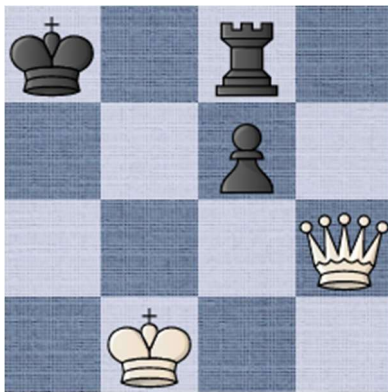
```
Score = MiniMax(Board, Depth + 1, True, Alpha, Beta) 'Mi  
Beta = Math.Min(Beta, Score)  
If Beta <= Alpha Then 'Move was too good - prune branch.  
    Return BestScore  
    Exit Function  
End If
```

Figure 3 - changes for O in MiniMax (minimising function).

SECONDARY AI RESEARCH

QUIESCENCE – CREATING A STABLE AI

When creating my MiniMax prototype for Noughts & Crosses and theorising how I would implement the algorithm into chess, I hypothesized an interesting question: what if the AI finished searching on an unstable position? In my mind, this problem seemed like a catastrophic failure of the MiniMax algorithm until I began researching solutions and came across the Quiescence Search. Thankfully, this completely fixed the instability problem (at a slight cost of speed). Explaining Quiescence (and the problem it solves) is very difficult to do with words, so instead let's imagine it visually...



Let's take this very simple chess position (white to move) where white is completely winning, and we set the AI to search to a depth of 1. The computer will search through every move in the position, and for each of these moves will count up the difference in material between the two sides – picking the position with the greatest (positive) material difference. This will cause the AI to get greedy - taking the pawn, as it thinks that will result in white getting an extra pawn. Now of course this is a horrible move, as on black's next turn they can just take white's queen and be completely winning, however our AI didn't see this move as the depth was too low. Thus, we can say that this position is 'unstable' (as MiniMax has evaluated this position incorrectly).



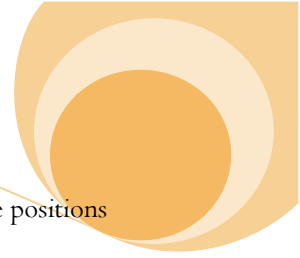
So therefore, we reach what is called the 'horizon limitation'^[4] – what happens if the AI finishes its search at one of these positions? (Or rather, what happens if a leaf position of a MiniMax search is one of these 'faulty' or 'instable' positions?)

Now you may propose "just increase the depth" - from 1 to 3 for example - however that only delays the problem, as these instable positions can still occur (eg: Qc2+ Ka3, Qxc3+ Rxc3). In fact, as you increase the depth more and more, and therefore create an exponentially increasing number of instable positions, more positions will be incorrectly evaluated and therefore the MiniMax search will become increasingly distorted. So then, what is the solution to this problem?

The surprisingly simple answer to this is to, at each of the leaf positions from MiniMax, continue searching deeper but **only include capture moves** in the move generation algorithm, until a position with no capture moves has been reached (at which we stop searching)^[5]. This new, end position is 'quiet' as there is no instability of pieces suddenly taking others, and therefore it is much less likely that the position will be incorrectly evaluated by MiniMax. Therefore, by using Quiescence, we have solved the instability problem.

This becomes slightly trickier when a player is put into check, however. As checks are inherently unstable, our Quiescence search mustn't stop searching in these positions, however to prevent incorrect evaluations we need to examine **all** the moves that could resolve the check – not just the capture moves.

Now in practice this explanation is a slight oversimplification, however Quiescence provides arguably the most simple and effective solution to the instability problem in chess, and therefore I will be implementing this



algorithm into my project. Although the depth my AI would search to would be limited (as many more positions have to be evaluated in a given search), this should make my AI a much stronger chess entity.

SOLVED VS NON-SOLVED GAMES, AND MEASURING SUCCESS

A solved game is one in which we can definitively say what the best move in a given position is. The only way we can make a game solved is by being able to search and evaluate every single position that branches from the game's starting position. Once we have done this, we can say with absolute confidence whether a given move in any position will lead to a win, a draw, or a loss. **Nothing else.**

Chess is an example of a non-solved game, meaning that, given the starting position, the game's outcome cannot be accurately predicted (or more accurately, a computer doesn't have the resources or time to feasibly locate and evaluate all the positions and game states that branch off from a given position). Unlike simpler games such as Connect 4, which have recently proved to be solved, it is predicted that chess will **never** become a solved game – primarily due to the astronomically high number of possible chess positions (approximately 8.7×10^{45} ^[2] – roughly equal to the number of atoms on planet Earth). With such an astronomically large number of possible positions that branch from the starting position, modern-day computers just don't have the resources nor the capability to locate them all.

As a result of this, given any position there is no way to definitively say what the best move is. We can make predictions as to what these may be, likewise we may be able to separate 'good' from 'bad' moves, however it is computationally impossible to perfectly predict which player is winning in a given position, and hence it is **impossible to create a 'perfect' chess Artificial Intelligence**. We therefore compare chess AIs based on how quickly they are able to search to a given depth, along with their estimated rating using the ELO system (a method of calculating the relative strength of a chess player).

REVISED PROJECT OBJECTIVES

After conducting my research and getting a better understanding of what my project will entail, I have revised my Initial Project Objectives to better reflect this project and my coding ability:

Essential objectives:

- 1) The project must be written in VB.net with the Windows Forms Application to create a graphical interface.
- 2) The user should be presented with a graphical and accurately scaled interface of a chessboard and its pieces.
- 3) The user should be able to drag and drop the pieces on the board, and if that move is legal (see Objective (4) for more details), have it be represented on the board.
- 4) The program should use the standard rules of chess to determine if a move the user has entered in is legal or not (and refuse it if it's not). This may require context about the board, such as restricting certain moves if a piece is blocked by another piece, or if a piece is pinned by an opponent's piece.
- 5) The user should be able to Castle both king-side and queen-side whilst conforming to the rules of chess.
- 6) The user should be able to enter in a chess FEN, and **if it is legal** (which will be determined by the program), it will be presented onto the chessboard.

- 7) The user should be able to export the current chess position on the board to a FEN string that can be used on other chess websites / engines.
- 8) The user can promote a pawn to a queen if it reaches the end of the board.
- 9) The program should identify when a check has occurred, and will notify the user of this both visually and audibly. This should also be used to restrict the moves an opponent can make, as checks need to be resolved immediately.
- 10) When a user is put into check, the program should accurately determine whether this is a Checkmate or a regular check. If it is a Checkmate then the game should terminate.
- 11) The user, at any time, should be able to let my chess AI make a move from the current position (for either player). This move should be represented on the chessboard.
- 12) The AI should be given either a certain amount of time to search for, and at the conclusion of this time its move should be chosen and displayed.
- 13) My AI should be **modular** and separate from my main chess program via classes and object-oriented programming.

Desired objectives:

- 1) When allowed to, the user should be able to take En Passant.
- 2) The game should end if only kings are left on the board, or in other special circumstances (please see <https://www.chessstrategyonline.com/content/tutorials/how-to-play-chess-draws> for more detail).
- 3) The user will be given the option at all times to undo their / their opponent's last move.
- 4) If the AI detects a forced checkmate at a shallow depth, it should terminate its searching and instantly play the move that results in that checkmate.
- 5) When the AI is creating its move, the user should be updated in real-time on its progress and current evaluation.
- 6) The user should be able to select multiple / variable difficulties of the AI.
- 7) By clicking on a piece, the user should be told visually the squares it can move to.

Preferred objectives:

- 1) The user should have their last move be illustrated on the chessboard graphically.
- 2) The user should be able to move the program window around at all times, including when waiting for the AI to make its move.
- 3) The user can pick from a variety of board colour schemes that can be modified in real-time. This setting should be stored in a file, so that when the user opens the application, their preferred colour scheme is retrieved and applied seamlessly.

CRITICAL PATH

The success of my project heavily relies on my MiniMax with Alpha-Beta Pruning algorithm that forms the basis of my chess AI, and therefore a lot of time and care must be spent in making that function efficient and bug-free. This compelled me to design a prototype of these algorithms, as if I am unable to implement them into a simpler game than chess then it would imply that I should not proceed with using it in chess. (This prototype can be seen in the [upper subsection](#).) However, my MiniMax algorithm will rely on many other algorithms, such as a move generation function that generates all the possible but **legal** moves a player can make in a given position, functions that deal with checks and checkmates, algorithms that can determine and store which pieces on a board are pinned, etc.

Another very critical section of my project will be the Graphical User Interface, as it will allow the user to use my program. It must be intuitive to use, smooth, and aesthetically pleasing, as a lack of these may cause annoyance or frustration from the user. Furthermore, my chess AI should be able to interact with my GUI, and vice versa – this should make my program easier to use, as they will only need to open one program.

REFERENCES

- [1] = www.se16.info/hgb/tictactoe.htm (“255168 possible games in total.”)
- [2] = https://en.wikipedia.org/wiki/Shannon_number (“Recent results^[3] improve that estimate, by proving an upper bound of 8.7×10^{45} .”)
- [3] = <https://www.youtube.com/watch?v=WXuK6gekUIY&t=3s> (23:40 – “I find [a] big weakness ... tricky lumps of knowledge that it understands very poorly, and it’s really hard for us to characterize when it is going to enter one of those lumps. But if it does, it can be completely delusional.” 19:21 – “We had our evaluation match. ... we lost a game in a way that would have made us look extremely foolish if that happened publicly”)
- [4] = https://www.chessprogramming.org/Horizon_Effect (“The Horizon Effect is caused by the depth limitation of the search algorithm. ... Because only a partial game tree has been analysed, it will appear to the system that the event can be avoided when in fact this is not the case.”)
- [5] = https://www.chessprogramming.org/Quiescence_Search (“The purpose of [the Quiescence] search is to only evaluate "quiet" positions, or positions where there are no winning tactical moves to be made.”)

SECTION 2: DESIGN

This section will be broken down into four distinct categories: the first discussing the main data structures I will use in my program, the second detailing my GUI, the third explaining the primary algorithms (bar MiniMax and Alpha-Beta Pruning) that will be used in my program, and the last containing information about my implementation of MiniMax. Before these however, we should first talk about the presentation of my program, and how the user will be able to access its full capacity of features...

PROGRAM LAYOUT & GENERAL USER FEATURES

When my users first runs my program, they will be presented with a main menu screen and will be able to pick one of four options (however, the user can return to the main menu from one of these options at any time).

These four options are:

- 1) One-Player Mode – allows the user to play a game of chess against my AI. They will also be able to customise their playing experience, such as choosing which side they will play with, what difficulty the AI will have, the starting FEN position, etc. When their game has ended, they will be notified of this, and will then be redirected back to the main menu.
- 2) Two-Player Mode – allows the user to play a game of chess against their friends, or themselves. Customisation is limited here, but the user will be allowed to flip the chessboard (both manually and automatically) and change the starting FEN position. When their game has ended, they will be notified of this, then redirected back to the main menu.
- 3) Analysis – gives the user access to all the functionality of my program, where they can customise, experiment and analyse to their heart's content. When their game has ended, they will be notified of this, but **will not** be redirected back to the main menu (incase the user wants to analyse the position further).
- 4) Exit – For obvious reasons, I won't be elaborating on the details of this feature.

If the user chooses either option 1 or option 2, they will be presented with a popup, where they can configure their settings. Moreover, the functionality of my program in the One-Player Mode will be slightly different to that in the Analysis Mode in order to improve user experience - as well as removing some features of Analysis Mode that are unnecessary when playing against the AI (such as letting the AI make a move on the player's turn, being able to flip the board, etc), some features will be tweaked. These changes include:

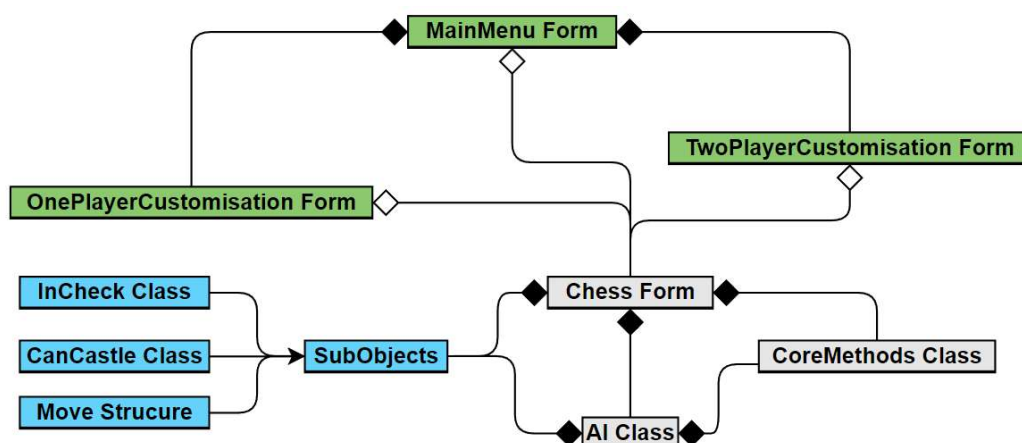
- i) In Analysis Mode, the AI's default difficulty will be set to 'Expert'. However, the user will be able to modify how long the AI will search for before it makes its move.
- ii) In Analysis Mode, the user must press a specific button that allows the AI to make its move. However, in One-Player Mode, the AI will automatically make its response once the user has finished their move. Furthermore, in Analysis Mode, the user can pre-emptively terminate an AI's search, and display its move on the board.
- iii) In Analysis Mode, the Undo Action button will revert to the last move made on the board (regardless of which player made it). However, in One-Player Mode, this button will revert to the last move that the **user made**. This is because the AI will always make the same decisions each time it is run (assuming that the position is the same).
- iv) In both the One-Player and Two-Player Modes, a custom FEN can only be imported at the start of the game, whilst in Analysis Mode a custom FEN can be imported at any time. To make up for this, in these two modes, the 'FENTextBox' will automatically export the current FEN position for the user (at all times).

DATA STRUCTURES

The module level of my program will consist of 6 main forms / classes, along with smaller subclasses & objects which link to the main ones. The subclasses & objects will be described in the [next subsection](#), however here are the main objects I will be using:

- **MainMenu Form** – This form contains the first bit of code that interacts with the user. Although the simplest computationally, it will be of huge importance to my project, as it will provide the user with a simple and aesthetically pleasing way to access the full extent of my project. It will also be the starting point for instantiating the **Chess Form**.
- **OnePlayerCustomisation** & **TwoPlayerCustomisation Forms** – I have linked these forms together here as they have the same general purpose – to allow the user to customise their game when selecting the One Player or Two Player Modes from the main menu. These customisation options are then passed onto the **Chess Form**.
- **Chess Form** – This form will form the main User Interface and the basic game of chess, along with its rules. It will control moving pieces, importing / exporting positions, manipulating the board & its visual design etc, along with converting a user's actions onto a computer-friendly interface (for the AI to use). This can be thought of as the main form, as it will call the **AI** and the **CoreMethods** classes when it requires them – furthermore, all game modes that the user can select from the MainMenu will be 'mutations' of the Chess Form, with the same underlying code but different visual representations & allowed features.
- **AI Class** – This class will form the AI part of my project, which will be instantiated from and interact briefly with the main **Chess Form**. As seen in my [Project Objectives](#), one aim of my project is to make this AI modular and independent from the rest of my program, and therefore there will be as little interaction between this class and my **Chess Class** as possible. In an ideal world, I would like this class to be instantiated with only the current chess FEN and the required depth, and for it to return only a set of coordinates, which represent the move it has chosen. This class will use algorithms such as 'CreateMoves', 'MiniMax', 'Quiescence', etc.
- **CoreMethods Class** – This class contains most of the primary algorithms I will be using in my project, and will link to both my **Chess** class and my **AI** class (either via instantiation or by inheritance). It will contain the algorithms that will be used by both my **Chess** & **AI** classes, such as the ['PieceLegalMoves' generator](#), the ['TFTable' Generator](#), and others. Many of these algorithms will be described in greater detail in the [Primary Algorithms subsection](#).

These classes, along with the subclasses, interact as is shown in the below UML class diagram:



REPRESENTING THE CHESSBOARD AND THE DETAILS OF A CHESS POSITION

There are a lot of hidden details in a chess board position, and to retrieve these we must first extract them from the position's equivalent FEN string. An example FEN, along with what information we can retrieve from one, is shown below:



Once we extract the details from the FEN, we can discover more details about the position, which will be vital to discover before the user is allowed to make actions. These secondary details about a chess position that my program uses include...

- The legal moves for the king, so that the piece will not be illegally captured on the next move.
- The location and type of pinned pieces.
- Check data, including the checking piece's location, and whether or not the king is in a 'double check'.
- The location of kings (used for calculating checks and castling).
- The location of the last move (used for illustration purposed – please read the SquareHistory row in the below table for more information).

And finally, there is extra data that my AI requires to determine an optimal move. These include:

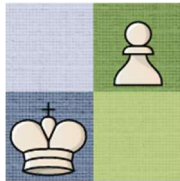
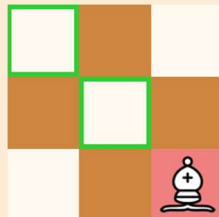
- The total material count of the board (used for determining the depth that the AI will search to).
- The value (or weight) of each piece.
- The data about a chess move.

To store all these details, I will be using a wide variety of data types across multiple classes. In the table below, you can see all the information regarding these.

Name	Object Type	Description / Purpose	Alternate Names in Program
FEN	String	Represents a chess position, along with its basic details. My program will, at all times, store the current position FEN, along with the FEN of the previous position (used for undoing moves).	StartingFEN, CurrentFEN, PreviousFEN, InvalidFEN, TempFEN.
Board	2D Array of Characters	8x8 array that represents where all the pieces on a chessboard are, using basic chess notation (P=Pawn, N=Knight, B=Bishop, Q=Queen, R=Rook, K=King). Uppercase letters represent white pieces, whilst lowercase letters represent black pieces.	Masterboard, Tempboard, PrimaryBoard.

Name	Object Type	Description / Purpose	Alternate Names in Program
TFTable	2D Array of Characters	8x8 array that has a one-to-one correlation with the Board array (element [1,2] on the TFTable gives information about element [1,2] on the Board) - contains information about pinned pieces and legal moves for the king.	WhiteTFTable, BlackTFTable, MasterWhiteTFTable, MasterBlackTFTable, PrimaryTFTable, TrueTable, MasterTrueTable, TFTable.
PlayerTurn	Boolean	Simple Boolean Value that determines who's turn it is to play. Influences validating FEN strings, preventing players / my AI from moving twice in a row, finding end positions etc.	isWhite, UserPlayer.
InCheck	Class	Class that holds the data for checks. Contains 3 variables: A Boolean IsInCheck value, a string Piece (which contains the location of the piece that is checking the king), and finally a Boolean DoubleCheck value (which indicates if two pieces are checking the king simultaneously). It also contains a NotInCheck Subroutine, which resets check data.	WInCheck, BInCheck, MasterWInCheck, MasterBInCheck, PrimaryMeInCheck, TempWInCheck, TempBInCheck, NotInCheck.
CanCastle	Class	Class that holds the castling data for a given player. Contains two boolean variables: KS and QS. There are two ways to castle in chess: castling king side (KS) and castling queen side (QS), so I have two variables which say if that player can castle either way. It also contains a CannotCastle Subroutine, which resets castling data.	WCanCastle, BCanCastle, MasterWCanCastle, MasterBCanCastle, PrimaryMeCanCastle, PrimaryEnemyCanCastle, TempWCanCastle, TempBCanCastle, CannotCastle
KPos	String	A two character string that represents the coordinates of a player's king. Used for efficiency purposes – making it easier to determine if a check has occurred, what pieces to scan when generating the TFTable, etc.	WKPos, BKPos, MasterWKPos, MasterBKPos, PrimaryMeKPos, PrimaryEnemyKPos, TempWKPos, TempBKPos
PieceInfo	Structure	Small structure containing information about a piece which is being moved by the user on the GUI. Contains the Boolean value IsMoving, along with 3 Point structures StartPoint, MidPoint & EndPoint – referring to where the user is moving / has moved their piece to. Used to determine if a user's move is legal, and to translate this move onto the board.	PieceMoving

Name	Object Type	Description / Purpose	Alternate Names in Program
PieceValue	Array of (Hashed) Decimals	This array of hashed values contains information about the 'weight' (aka: the value) of each piece type on the board – Pawns are worth 1 point, Bishops / Knights are worth 3, Rooks are 5 and a Queen is 9. For more information on my hashing technique, please see here .	N/A
EnPassant	String	A two character string which represents the coordinates of the En Passant square (ie: where the opponent can move their pawn to take En Passant).	MasterEnPassant, PrimaryEnPassant, TempEnPassant
Move	Structure	Object which contains the information about a computer's move. Consists of a decimal evaluation of the move it has chosen, the set of coordinates which represents the location of where the piece has been moved from, and the set of coordinates which represent where that piece is being moved to.	BestMove, TempMove
MaterialCount	Array of Integers	Represents the total weight / value of the board. Contains two indexes – one for white's total material count, and the other for black's total material count. Used to calculate both the depth the AI will search to (the more pieces on the board, the lower the depth) or the difference in material between both sides (when evaluating a position). When a piece is captured, its weight is taken away from its player's material count, instead of re-calculating the total each time.	PrimaryMaterialCount, TempMaterialCount
GameRunning	Boolean	Boolean Value that determines if a game's end position has been reached. If this value is set to False, neither player will be able to make a move until they start a new game (eg: if one player checkmates the other). This variable is similar to the ' <u>ComputerSearching</u> ' Boolean value (which is set to True if the AI is searching on the board) and prevents the user from making actions which may influence the board / UI.	N/A

Name	Object Type	Description / Purpose	Alternate Names in Program
PieceInfo	Structure	Contains information about a piece that the user is moving on the GUI, such as the Boolean <code>IsMovingPiece</code> value, an array containing the legal moves the piece can make (calculated when the user clicks on the piece), and finally three sets of coordinates (called <code>StartPoint</code> , <code>MidPoint</code> , and <code>EndPoint</code>). In order, these represent the piece's start coordinates, its current coordinates (when being dragged by the user), and its final coordinates (when the user completes their move).	PieceMoving
OrientForWhite	Boolean	Boolean value that indicates whether the board is flipped for white's perspective or not. If this value is set to <code>False</code> (meaning that the board is rotated in black's favour) then all moves / inputs made from the GUI must be rotated before they are represented on the board.	N/A
SquareHistory	2D Array of Strings	A set of four coordinates used to represent the squares last used by the user (2 for the current position & 2 for the previous position). This contains the start and end square of the piece that has just been moved, so that they can be highlighted on the board after each move. Please see the image on the right for a visual representation.	N/A 
LegalMoveSquares	2D Array of Boolean	An 8x8 grid representing the legal moves that a piece (picked up by the user) can move to on the board. This is so that the GUI checkerboard squares can be coloured in a different way (<code>True</code> = square on board is a legal move, <code>False</code> = square is not a legal move).	N/A 

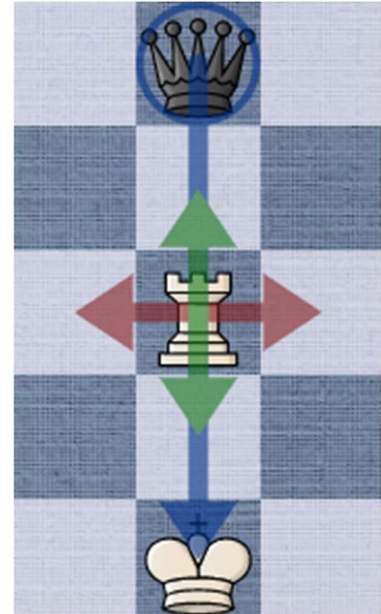
AN ASIDE: THE IMPORTANCE OF THE TRUEFALSETABLE

Before I fully explain the algorithms that use the above structures, it is necessary to provide some more information about the `TrueFalseTable` - abbreviated to `TFTTable` (seen in the 3rd row of the above table). This 2D array forms a large part of the backbone that makes my program function, more specifically containing details about the legal moves for each king, and the location & information about pinned pieces.

This 2D array has a one-to-one correlation with the `Board` data structure, meaning that each element in the `Board` array has its own corresponding element in the `TFTables`, with the **same** indexes. Each side will have its own `TFTTable`, which will be updated after each move. Let's explore how this structure is utilised, beginning first with the king's legal moves...

After each move played, the TFTable Generator “FixTFTables” is run for the side that is about to move (please see the next page for the flowchart of this algorithm). During this algorithm, all the squares around the king are labelled with either T or F, F meaning that that square is controlled by another piece (ie: if the king moved to that square, it would be taken by another piece), and T meaning it is a free square. Therefore, as soon as we click on the king to move it, we can instantly determine its legal moves.

Next, let's discuss pinned pieces. If a piece is pinned **absolutely**, then the location of the piece on the board is labelled on its corresponding position on the TFTable, using a number from 0-4. This number corresponds to the direction it is pinned from: 0 = pinned vertically, 1 = pinned diagonally (btm left to top right), 2 = pinned horizontally, 3 = pinned diagonally (top left to btm right), and 4 = *En Passant pinned*. The direction the piece is pinned from is very important, as some pinned pieces should still be able to move in certain directions. Referring to the picture on the right, a black queen is pinning white's rook to its king, and therefore we need to restrict the movement of the rook. We know it is pinned vertically, so we assign a value of 0 to the white rook in the TFTable. As it is pinned vertically, and the piece pinned is a rook, it should be able to move vertically as much as it pleases, however it should not be able to move horizontally, as this would result in the queen taking the white king. Thus, if we scan for and label pinned pieces before the player makes their move, it becomes much more efficient to determine a pinned piece's legal moves.



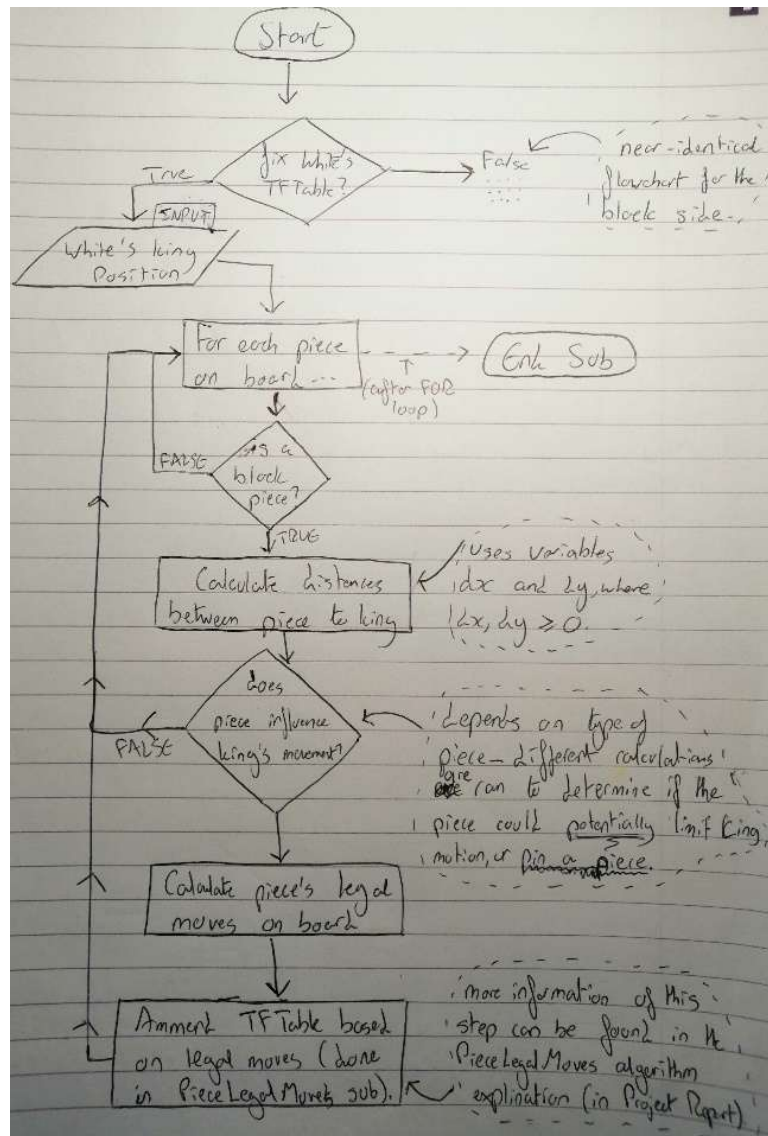
If a pawn is En Passant pinned (value of 4 on TFTable), then it is prevented from taking an enemy pawn En Passant for that turn only. For more information on why this is necessary, please see my explanation in the [PieceLegalMove Algorithm](#).

We can now look at how this algorithm functions in my program. We first take the board and the location of the enemy king, then scan for all the pieces on the board. For each piece, we find the horizontal distance (dx) and the vertical distance (dy) between it and the enemy king and use this to determine if that piece could potentially *influence* the enemy king's movement or pin a piece. As mentioned previously, all the squares around the king are labelled with either “T” or “F”, so by ‘influence’ I am referring to whether a piece could turn one or more of these “T” squares into an “F” square – hence preventing the king from moving to that square. This will be unique for each piece, depending on whether it is a sliding piece (in which case, we use dx and/or dy individually), or if it is a non-sliding piece (in which case, we calculate a given radius around the king and determine whether the piece is in that radius).

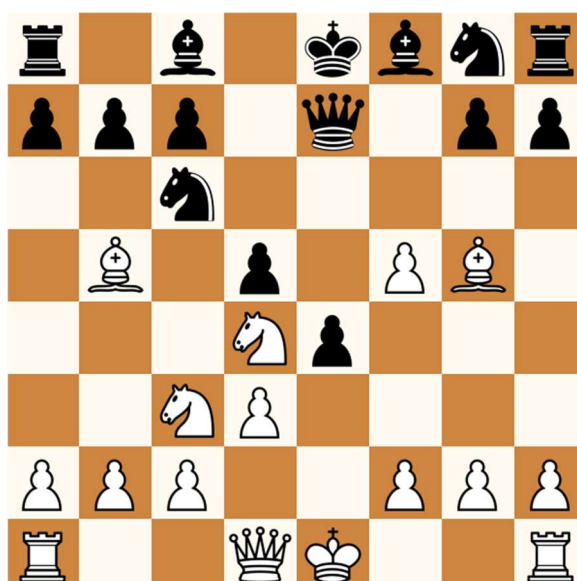
Once we have discarded the unneeded pieces, we run the pieces through the PieceLegalMoves function which, during its task of finding the piece's moves, will change the squares on the required TFTable to “F”, and will also pin pieces for us. If we do this for all the required pieces, we eventually build up the entire TFTable.

A great surprise to me was that this algorithm also calculates if a player has been put in check; as we scan all pieces that could influence the enemy king, this also includes those that are attacking the king. We can then detect when this occurs and flag it appropriately.

You can see the flowchart for this procedure below:



Example of the TFTables for each player in a real-game situation: (Note that the black knight is pinned diagonally by the white bishop.)



```

WhiteTFTable:
TTTTFFTT
TTFFTTFT
TTTTFFTT
TTFTFTFT
TFTTFTTT
FTTTTTTT
TTTTTTTT
TTTTTTTT
TTTTTTTT
BlackTFTable:
TTTFTTTT
TTTTFTTT
FT1TTFTF
TTTTTTTT
FTFTTFFF
TTTTFFTT
TTFFFTTT
FFFTFTTT
    
```

GRAPHICAL USER INTERFACE

The main method of presenting my code will be a Windows Form application containing buttons, TextBoxes and lots of PictureBoxes - 49 of them, to be precise. 32 of these PictureBoxes represent each piece on the board, an extra 16 are reserved for potential queens via pawn promotion, and the final one is used for the main chessboard. I chose PictureBoxes for this purpose as they are easy to manipulate in direction or design, they can be easily distinguished from one another, and they make my drag-and-drop mechanic much simpler. Furthermore, there is a neat feature of PictureBoxes that allows them to be inherited by others of the same type, and therefore we can have each piece PictureBox to be a child of the 'Chessboard' picture box:

For each PiecePictureBox: Checkerboard.Controls.Add(PictureBox)

The advantage of this is that each piece PictureBox now has its coordinates relative to their parent PictureBox, meaning that if, for example, we move the White King to location [0,0], it will be positioned at the top-left corner of the 'Chessboard' PictureBox, rather than the top-left corner of the Form – therefore making it easier to resize / reposition elements in my GUI.

The name of each PictureBox can be defined as such:

- First Letter of name = W or B, depending on which colour it is.
- Second Letter of name = the symbol of the piece (uppercase) – 'K', 'Q', 'R', 'N', 'B', 'P'.
- Third Letter of name = its numerical identifier (depending on how many of the piece can be legally placed onto the board).

For example, the name BN2 represents the 2nd Black Knight PictureBox.

Before we move on to the finer details of the GUI, I feel it important to first mention the chessboard itself - more specifically, its size: I have engineered my chessboard to be exactly 600 x 600 pixels – therefore each square will be 75 x 75 pixels. This is to make converting between the Board array and the graphical board much simpler – all we need to do to get a piece's graphical coordinates is to multiply its Board coordinates by 75 (and vice versa). For example, a piece at coordinates (2,7) on the Board array will be positioned at coordinates (150, 225) on the graphical board.

DISPLAYING THE PIECES ON THE BOARD

To display the pieces on the board, we first need to create the checkerboard pattern that they are positioned on. This will be achieved via the CreateBoard subroutine, which re-paints the chessboard every time the user interacts with it. The primary & secondary colours will be decided by the user depending on their board colour scheme preferences (which are saved to a text file for the next time the program is opened), however the colours for the 'previously used squares', along with the 'LegalMoveSquares' remain constant. The pseudocode for this algorithm can be found below:

```

1 Procedure CreateBoard(PrimaryColour as Colour, SecondaryColour as Colour, SquareHistory as
2   Array(of string), LegalMoveSquares as Array(of Boolean)) Handles Checkerboard.Paint
3   Def isLight As Boolean = True
4   For x = 0 To 7
5     For y = 0 To 7
6       Def Square As New Square(StartingCoords: [75x, 75y], Size: 75)
7       # If the coordinates are set to True in the LegalMoveSquare, then that square
8       # should be coloured differently, so that the user can see legal moves. Square
9       # given either a green or blue outline (depending on the colour scheme).
10      If LegalMoveSquares(x, y) Then
11        If PrimaryColour = Color.DarkSeaGreen Then # is green colour scheme - colour blue.
12          FillRectangle(Square) Using Colour(DarkTurquoise)
13        Else # is another colour scheme - colour green.
14          FillRectangle(Square) Using Colour(LimeGreen)
15        End If
16      End If
17      If isLight Then
18        # If Coords are True in the LegalMoveSquare, then thenNormal colour is filled
19        # at the centre of the legal move square to produce a green / blue highlight.
20        If LegalMoveSquares(x, y) Then
21          Def Square2 As New Square(StartingCoords: [75x + 5, 75y + 5], Size: 65)
22          FillRectangle(Square2) Using Colour(SecondaryColour)
23          # If the given square matches a SquareHistory coordinate,
24          # then it is coloured in a different way.
25          ElseIf Coords[x,y] is in SquareHistory Then
26            FillRectangle(Square) Using Colour(Color.YellowGreen) # SquareHistory colour.
27          Else # Is a normal square - colour with user's secondary colour.
28            FillRectangle(Square) Using Colour(SecondaryColour)
29          End If
30        Else # Identical code but for the dark squares (uses different colours).
31          If LegalMoveSquares(x, y) Then
32            Def Square2 As New Square(StartingCoords: [75x + 5, 75y + 5], Size: 65)
33            FillRectangle(Square2) Using Colour(PrimaryColour)
34            ElseIf Coords[x,y] is in SquareHistory Then
35              FillRectangle(Square) Using Colour(OliveDrab)
36            Else
37              FillRectangle(Square) Using Colour(PrimaryColour)
38            End If
39          End If
40          isLight = Not isLight # Produces alternating pattern.
41        Next
42        isLight = Not isLight
43      Next
44      # Gives the starting square for a piece a distinctive colour, when the user clicks on one.
45      If PieceMoving Then # the piece is being moved by the user.
46        Def Square As New Square(StartingCoords: [PieceMoving.StartingCoords], Size: 75)
47        FillRectangle(Square) Using Colour(LightCoral)
48      End If
49    End Procedure

```

Next, we need an algorithm that places the pieces onto the board, which will be called `DisplayPieces`. When the procedure is first run, we place all the pieces off-screen and make them invisible, then place each required piece one by one to correspond with the main Board array. However, complications arise when we try to do this: how do we know which PictureBox to move, and how can we make sure we are not moving those that have been already placed? To solve this, we construct the name of the PictureBox using the piece name in the Board array (eg: "P" → WP1), then check if that piece is visible (and has therefore been placed on the board). If it is, we move to the next piece of that type (WP2, WP3, etc); if it isn't, we move that PictureBox to the correct coordinates and make it visible. Here is the pseudocode for this subroutine:

```

1 Procedure DisplayPieces()
2   Def CurrentPiece As String
3   Def TempPiece As PictureBox
4   Def n As Byte
5   # Hides all pieces.
6   For x = 0 To 47
7     PieceArray(x).Location = Coords(-100, -100)
8     PieceArray(x).Visible = False
9   Next
10  For y = 0 To 7
11    For x = 0 To 7
12      If Square(at Coords [x,y] on Board).isPiece Then
13        n = 1
14        While True
15          # In the Board array, a lowercase letter represents a black piece,
16          # and an uppercase letter represents a white piece.
17          If Piece(at Coords [x,y] on Board).isBlack Then
18            CurrentPiece = "b"
19          Else
20            CurrentPiece = "w"
21          End If
22          # CurrentPiece is constructed, which has the same name as a corresponding
23          # piece on the board. Here, n represents a counter that starts at 1.
24          CurrentPiece &= UCase(Piece(at Coords [x,y] on Board)) & CStr(n)
25          TempPiece = PictureBox(with Name = CurrentPiece).Single()
26          # Single() is used so that only 1 PictureBox is returned.
27          If TempPiece.Visible = False Then
28            TempPiece.Location = New Point(x * 75, y * 75)
29            TempPiece.Visible = True
30            Exit While
31          End If
32          # If the PictureBox has already been placed on the board, the search
33          # continues with an increasing last digit (eg: the program searches for
34          # BP1, then BP2, then BP3 ext until it finds its match.)
35          n += 1
36        End While
37      End If
38    Next
39  Next
40 End Procedure

```

MOVING A PIECE – DRAG & DROP MECHANICS

The drag & drop mechanic for the chess pieces is incredibly fundamental to my project. There are three main stages to this mechanic – clicking on the piece to initiate the drag & drop, moving the piece and have it follow the mouse smoothly, and letting go of the piece, with its position being locked in place (and the user's move validated). Let's cover each of these parts separately:

(Please note that this mechanic interacts heavily with the 'PieceInfo' structure and its variables (which you can find information about [here](#).)

1) Initialisation:

This stage is by far the simplest. When we click on the piece, we first check its legal moves (so we can quickly check if a user's move is legal when they complete the move), then allow the user to move it (by setting IsMovingPiece to True). Meanwhile, we bring the PictureBox to the front of the screen and make a note of its starting coordinates (in StartPos).

2) Moving the Piece:

This section proved much more difficult than the others. I tried many different methods for moving PictureBoxes on the screen to follow the user's mouse, but after a while I stumbled upon a method that stood out amongst the rest for its simplicity and smoothness. Here is the annotated code for this part:


```

Sub Piece_MouseMove(sender As Object, e As System.Windows.Forms.MouseEventArgs) Handles AllPictureBoxes.MouseMove
# In this subroutine, 'Sender' represents the PictureBox the user is moving, and 'e' represents the data
# in the mouse (such as its coordinates).
If PieceMoving.IsMovingPiece Then
    # When the user first moves the object, MidPoint = StartPoint
    PieceMoving.EndPoint = sender.PointToScreen(New Point(e.X, e.Y)) # New point = coors of the mouse.
    sender.Left += (PieceMoving.EndPoint.X - PieceMoving.MidPoint.X)
    sender.Top += (PieceMoving.EndPoint.Y - PieceMoving.MidPoint.Y)
    # The 2 above lines move the PictureBox from MidPoint to EndPoint
    # sender.Left and sender.Top are build-in functions which move objects relative to their position.
    PieceMoving.MidPoint = PieceMoving.EndPoint
End If
End Sub

```

3) Completing the Move:

When the user completes their move, we first need to 'snap' the piece to the nearest square on the board, then check if that move is legal. If it is, then we represent that move on the Board array, perform the generic chess logic (checking for checks, game-end states etc), and allow the opponent to make their move. If the move is not legal, however, we need to send the PictureBox back to its original position and force the user to make another move. Starting from the beginning, to snap the piece into place we use these lines of code:

```

'Disables drag & drop mechanic and sets the piece's position on the board.
PieceMoving.IsMovingPiece = False
Dim TempXPoint As Decimal = sender.location.X + 37.5
Dim TempYPoint As Decimal = sender.location.Y + 37.5
sender.location = New Point(TempXPoint - (TempXPoint Mod 75), TempYPoint - (TempYPoint Mod 75))

```

Next, to check the validity of the move, we compare it to the coordinates given in from the **Initialisation** part of this subsection.

The process of 'applying a move' (as stated above) requires these actions (in order):

- a) Reset checking rules.
- b) If the king has moved, update the king's position and castling privileges.
- c) If the king has castled, move the rook to the other side of the king and remove castling privileges*.
- d) If a piece has been captured, remove the captured piece from the board *.
- e) If a pawn has promoted, replace its PictureBox with the PictureBox of a queen.
- f) If an En Passant capture has been performed, remove the captured piece *.
- g) If a rook has moved, update castling privileges accordingly.
- h) Update the Board array to compensate for the user's move.
- i) Make the correct sound for the move.
- j) Update the TFTTable for the other player (the player that will next make their move).
- k) Update the previously used squares, along with the previous board FEN position (as the previous one has now expired).
- l) Create an En Passant square, if the rules to make one have been satisfied.
- m) Check for checks, then for game-end states.
- n) Finally, end the player's turn and begin their opponent's turn.

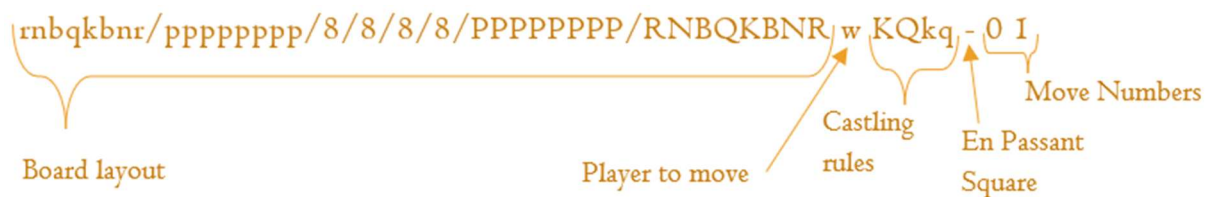
For the actions labelled with a * at the end, it is required for us to locate and modify other PictureBoxes on the board – ones that are not in direct contact with the user. I found overcoming this problem quite tricky, as we cannot use VB.net's built-in 'sender' function.

Instead, we use a modified version of the 'DisplayPieces' algorithm, but instead of using it to move pieces to their required places, we use it to return the PictureBox we are wanting to move, given the coordinates on the Board array where the piece is located. The function that will perform these actions is called `CoorToPieceConverter`; as ~90% of the code for this function is repeated from the `DisplayPieces` subroutine, I will not be including the pseudocode – just imagine this function to be a smaller, compact yet similar version of the `DisplayPieces` subroutine that returns the name of the PictureBox as soon as it is located, assuming that the PictureBox is located at the correct square on the GUI board.

IMPORTING / EXPORTING POSITIONS

It cannot be understated how vital the FEN Importer & Exporter algorithms are to my project. When the user first runs my program, the FEN for the starting position is sent to the FEN Importer and is converted to the starting board position, along with all the other data my program requires to play chess (the details of which can be found at the [start of this section](#)). When the user wishes to export their position, all the data for the given position is sent to the FEN exporter, which crunches it all down into a single FEN string. These two algorithms provide the primary link between my program and the wider world of chess, and the other programs that imitate it, and without them, testing & debugging my program would be much more cumbersome.

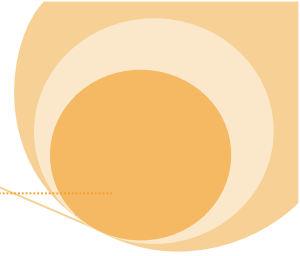
Below is a repeat of the FEN details example found in the [introduction for this section](#):



As you can see, the 'board layout' section is comprised of 8 'sets' of characters divided by slashes – these correspond with the 8 rows of a chessboard. Letters in these sets represent their corresponding piece names, whilst numbers represent a number of empty squares in the row. The 'Castling rules' are comprised of up to 4 letters. These represent the different ways each player can castle, where white's castling rules are denoted by capital letters (and black's with lowercase letters). King side castling is denoted via "K" or "k", and queen side castling is denoted via "Q" or "q". If a player can't castle in one of these ways then the appropriate symbol is not included, however if neither player can castle at all then we use the symbol '-' in place of this. Move Numbers provide information regarding three-fold repetition and the 50-move rule, both chess practices that I will not be considering in this project.



Figure 4 - A row of a chessboard, denoted in FEN notation with "npPK1b2".



FENCONVERTER ALGORITHM

To convert a FEN to a board position, we begin with resetting all castling rules and the En Passant square (with “-“). Then, we perform the following pseudocode:

```

1  Function FENConverter(FEN As String) As Array(of chars)
2      Def x As int = 0 # = Column index
3      Def y As int = 0 # = Row index
4      Def tempArray(7, 7) As Char
5      Def SpaceLocation As int
6      For n = 0 To FEN.Length - 1
7          Select Case FEN(n)
8              Case "/" # = end of board row. Reset column index and increment row index.
9                  y = y + 1
10                 x = 0
11
12                 Case "A" To "Z", "a" To "z" # Add name of piece to board index.
13                     tempArray(x, y) = FEN(n) # = name of piece.
14                     If FEN(n) = "K" Then # Update the position of the king.
15                         MasterWKPos = x & y
16                     ElseIf FEN(n) = "k" Then
17                         MasterBKPos = x & y
18                     End If
19                     x = x + 1
20
21                 Case "0" To "8" # = set of empty spaces.
22                     For m = 1 To FEN(n)
23                         tempArray(x, y) = " " # = empty square.
24                         x = x + 1
25                     Next
26
27                 Case " "
28                     # Checks for pawns incorrectly placed on the 1st or 8th rank. If we detect any, then we provoke an
29                     # intentional crash, which our encapsulating Try-Catch detects and promptly reverses.
30                     For x = 0 To 7
31                         If Piece(at coords (x,0) on tempArray) = "P" or Piece(at coords (x,7) on tempArray) = "P" Then
32                             FEN = "" # empty string.
33                         End If
34                     Next
35                     # Once the SPACE has been reached in the FEN, we know that we have read
36                     # the entire board. We can then move onto info about castling, who's
37                     # turn it is, En Passant and more.
38                     SpaceLocation = FEN.LocationOfCharacter(" ")
39                     If FEN(SpaceLocation + 1) = "w" Then # Is white's turn to move.
40                         PlayerTurn = True
41                     Else
42                         PlayerTurn = False
43                     End If
44
45                     FEN = FEN.Substring(FEN after the " ")
46                     For m = 0 To Len(FEN) - 1
47                         Select Case FEN(m)
48                             # Creates castling privileges.
49                             Case "K"
50                                 MasterWCanCastle.KS = True
51                             Case "Q"
52                                 MasterWCanCastle.QS = True
53                             Case "k"
54                                 MasterBCanCastle.KS = True
55                             Case "q"
56                                 MasterBCanCastle.QS = True
57                             Case "a" To "h"
58                                 # converts the a-h coordinate to the more computer-friendly
59                                 # index from 0-7 (eg: h3 --> 75)
60                                 MasterEnPassant = (FEN(m).ConvertFromASCII - 97) & (8 - FEN(m + 1))
61                         End Select
62                     Next
63
64                     # As the SPACE marks the end of the board position, we finish the loop here.
65                     Exit For
66                 End Select
67             Next
68             # At the end of the function, we return the newly constructed board.
69             Return tempArray
70 End Function

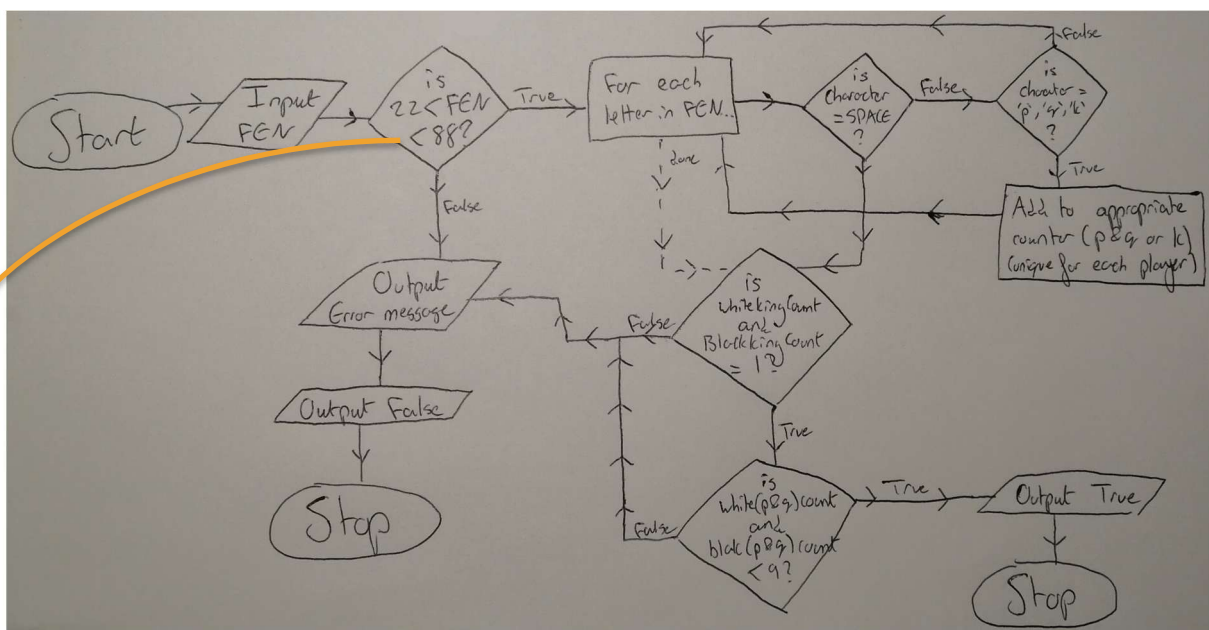
```

At the end of the algorithm, we double check that each player can *actually* castle (using the position of the player's king & rooks) and remove certain privileges accordingly. (Eg: if the king is not on its starting square then we remove that player's castling privileges.)

Note that this algorithm doesn't contain the detection of invalid FENs – when we give FENConverter a FEN, we assume it is legal. Therefore, we do most of the error detection when the user first enters their FEN into the program.

ELIMINATING INVALID FEN STRINGS

There are two sections of my code where an invalid FEN can be caught: the first is the FENErrorDetection subroutine, and the second is the set of algorithms that convert a FEN to a board position and display it on the board. Below is a flowchart containing the steps involved in the first section:



(Note: after doing some research, I found that the greatest possible length of a FEN comes to 87 characters^[1].)

If a FEN makes it through all these steps, we pass it on to the second section. Here, in what seems like a nonsensical turn of events, we assume that the FEN is valid, and run the FEN converting algorithms as normal. This includes:

- Converting the FEN to its board position.
- Bestowing castling & En Passant privileges.
- Displaying the pieces on the board.

If the FEN is invalid, then attempting to run one or more of these steps would cause the program to crash, and thus we can encapsulate these algorithms inside a 'Try-Catch' statement. Therefore, if we catch a crash then we reverse the above algorithms back to their original states, mark the FEN as invalid, and inform the user with an appropriate error message.

CONVERTTOFEN ALGORITHM

This algorithm is somewhat similar to the ConvertToFEN algorithm, but can be thought of as its 'inverse'; it has the opposite role - converting all the details of a board position to its equivalent FEN string.

```

1 Sub ConvertToFEN()
2   Def Counter As int # Here, "Counter" is the number of blank spaces in a row on the board.
3   CurrentFEN = ""
4   For y = 0 To 7
5     Counter = 0
6     For x = 0 To 7
7       If MasterBoard(x, y) = " " Then
8         Counter = Counter + 1
9       Else # If necessary, add Counter to the FEN & add the piece name to the FEN.
10        If Counter > 0 Then
11          CurrentFEN = CurrentFen & Counter
12          Counter = 0
13        End If
14        CurrentFEN = CurrentFen & MasterBoard(x, y)
15      End If
16    Next
17    If Counter > 0 Then CurrentFEN = CurrentFen & Counter
18    CurrentFEN = CurrentFen & "/" # Creates new row break, and begins on the next one.
19  Next
20  CurrentFEN = CurrentFEN.RemoveLastCharacter # Removes the last character from the FEN ("/").
21  If PlayerTurn Then
22    CurrentFEN = CurrentFen & " w "
23  Else
24    CurrentFEN = CurrentFen & " b "
25  End If
26  # Adds castling rules, then En Passant rules to FEN.
27  If MasterWCanCastle.KS Then CurrentFEN = CurrentFen & "K"
28  If MasterWCanCastle.QS Then CurrentFEN = CurrentFen & "Q"
29  If MasterBCanCastle.KS Then CurrentFEN = CurrentFen & "k"
30  If MasterBCanCastle.QS Then CurrentFEN = CurrentFen & "q"
31  If CurrentFEN.LastCharacter = " " Then CurrentFEN = CurrentFen & "-" # No Castling privileges
32  If MasterEnPassant <> "-" Then
33    # converts the computer-friendly index from 0-7 into the more human-friendly a-h coordinate (eg: 75 -> h3)
34    CurrentFEN = CurrentFen & " " & (MasterEnPassant(0) + 97).ConvertToASCII & 8 - MasterEnPassant(1) & " 0 1"
35  Else
36    CurrentFEN = CurrentFen & " - 0 1" # "0 1" represents the move numbers for a standard position.
37  End If
38 End Sub

```

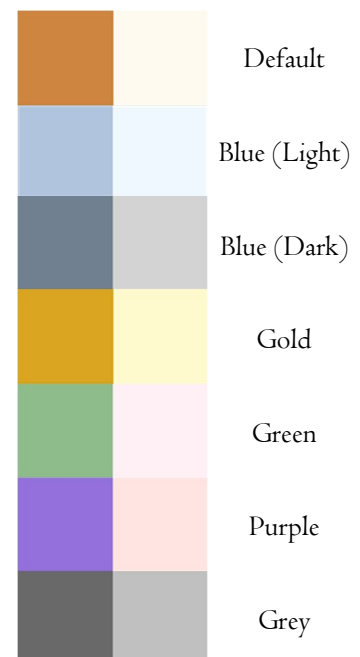
INTERACTING WITH TEXT FILES

One of my preferred project objectives is to allow the user to alter the board colour scheme to their liking, and this is achieved via text file handling. My program will contain a total of seven colour scheme options, which can be changed and updated in real time. Furthermore, it is essential that this colour scheme remains set - even if the user exits to the main menu, selects a new game mode, or closes & reopens the program, their colour profile should stick.

When the user first loads the program, the 'ColourPreferences' text file is read from, which will contain a three-letter code that corresponds to the user's chosen colour scheme ('def' for default, 'gld' for green, 'ppl' for purple, etc). This code is checked against the list of available colour schemes, and if there is a match, the variables 'PrimaryColour' and 'SecondaryColour' are modified appropriately. These variables are then used by the CreateBoard subroutine to create the checkerboard pattern using the chosen colours. If there are any errors in reading from the file, then the default colour scheme is automatically applied.

When the user modifies their colour scheme, PrimaryColour and SecondaryColour are modified, and the three-letter code is written to the 'ColourPreferences.txt' file. If this file does not exist, then the program automatically creates and writes to it.

All these actions are controlled via the built-in FileOpen function in VB.net.



PRIMARY ALGORITHMS

THE CORE OF MY PROGRAM – THE PIECELEGALMOVE ALGORITHMS

Arguably the most important part of my chess project will be determining if a move on the board is 'legal' (as in whether the move conforms with all the rules of chess) – this provides the foundation of both my chess program and my AI. This function should take the details about a chess position, along with the location of a piece, and with that information (along with the rules of chess) return all the legal moves that piece can make.

The main limitation of this algorithm is that it only generates the pseudo-legal moves a piece can make (with the addition of restricting the movement of pinned pieces), and hence this function does not take into consideration if a move resolves a player from check. To solve this, I will be making a secondary algorithm that takes a pseudo-legal move and calculates whether that move can be used to escape a check – thus confirming if the move is legal. You can find details about that function [here](#). With that said, here is some of the pseudocode for this algorithm:

(Note: as each function is nearly 500 lines of code long, and contains roughly the same general logic for each piece, the below pseudocode will not include the bishop, knight, and queen pieces. To view the full code for these algorithms, please see their counterparts in the [Technical Solution](#).)

```

3 ▾ # I have 2 functions which control the legal moves for the pieces on the board, called
4 # 'WhitePieceLegalMoves' and 'BlackPieceLegalMoves'. For this example, I will only provide the
5 # pseudocode for the White side, however you can imagine the Black side to be near identical, bar
6 # switched piece names. This subroutine receive a single piece's coordinates on the board and determines
7 # all of its possible moves - assigning them to the array LegalMoveArray. Alongside this, it also adds to
8 # one of the two TrueFalse Tables, using the squares which a piece attacks as markers (code commented as
9 # appropriate).
10
11 ▾ # As this function will likely be hundreds of lines of code long, I will only show the logic for the
12 # King piece, the pawn piece and the Rook piece. The other chess pieces' logic can be thought of in a
13 # similar way (remember, this pseudocode only serves to be a glorified proof-of-concept).
14
15
16 ▾ Function WhitePieceLegalMoves(Board, CoorX, CoorY, WhiteTFTable, BlackTFTable, BKPos, BInCheck, WInCheck, WCanCastle,
17 # EnPassant) as array(of strings))
18     define LegalMoveArray as array(of strings)
19     define n as int = 1 #Represents the number of moves the piece can make (+1)
20     define StartX as integer
21     define StartY as integer
22     define EndX as integer
23     define EndY as integer
24 ▾ define CheckingPiece as string #Coordinates of the piece that is checking the enemy king.
25     #Usually just represents the piece put into the function.
26
27
28     if Piece at coords (CoorX, CoorY) on Board = "K" Then
29 ▾ # Start & End Coords, in this case, represents the 8 squares around the king.
30         StartX = CoorX - 1
31         EndX = CoorX + 1
32         StartY = CoorY - 1
33         EndY = CoorY + 1
34 ▾ for y = StartY to EndY
35         for x = StartX to EndX
36 ▾ # Sets the appropriate square on the enemy TFTable to 'F'. This prevents the enemy king from moving to that square.
37             BlackTFTable(x, y) = "F"
38             # Important note: the piece's legal moves does NOT equate to the squares on the enemy TFTable to set to 'F'.
39             # This is because, even though the king might not be legally allowed to move to a certain square, it has the
40             # 'potential' to, meaning that it must have an effect on the enemy king's TFTable.
41             If (Square(at CoorX,CoorY) is Empty or Piece(at CoorX,CoorY).isBlack) And WhiteTFTable(x, y) = "T" Then
42 ▾ # Therefore is a legal move...
43                 LegalMoveArray(n) = x & y

```




```

44         n += 1
45     End If
46 Next
47 Next
48 # Code for castling.
49 If (White can castle king-side) And (White isn't in check) Then
50     If (there is space between king & rook) And (rook is on h1) And (White won't castle through check) Then
51         LegalMoveArray(n) = 67 #square king would go to to castle king-side.
52         n += 1
53     End If
54 End If
55 If (White can castle queen-side) And (White isn't in check) Then
56     If (there is space between king & rook) And (rook is on a1) And (White won't castle through check) Then
57         LegalMoveArray(n) = 27 #square king would go to to castle queen-side.
58         n += 1
59     End If
60 End If
61
62
63 elseif Piece = "P" Then
64     If Piece.IsPinned.Horizontally = False Then
65         If (Square above Pawn is Empty) and Piece.IsPinned.Diagonally = False Then
66             LegalMoveArray(n) = Square above Pawn
67             n += 1
68         If 2nd Square above Pawn is Empty Then
69             LegalMoveArray(n) = 2nd Square above Pawn
70             n += 1
71         End If
72     End If
73     If Piece.IsPinned.Vertically = False Then
74         If BlackTFTable(CoorX - 1, CoorY - 1) = "T" Then
75             # Sets the appropriate square on the enemy TTable to 'F'. This prevents the enemy king from moving to that square.
76             BlackTFTable(CoorX - 1, CoorY - 1) = "F"
77             # We do the above IF statement to prevent pinned pieces from being overwritten.
78             end if
79             If (Piece(at coords diagonal to pawn on board).isBlack) Or (CoorX - 1 & CoorY - 1 = EnPassant
80                 And Piece.IsPinned.EnPassantPinned = False) Then
81                 # Please see the TTable explanation in my Project Report for what 'EnPassantPinned' means.
82                 LegalMoveArray(n) = CoorX - 1 & CoorY - 1
83             If Piece(at coords CoorX - 1, CoorY - 1).isEnemyKing Then #The piece is checking the enemy king.
84                 BInCheck.IsInCheck = True
85                 CheckingPiece = CoorX & CoorY
86             End If
87             n += 1
88         End If
89     End If
90 End If
91
92
93 ElseIf Piece = "R" Or Piece = "Q" Then
94     # Please note that, to avoid repetition, I have only included the forwards motion for the rook in this pseudocode.
95     # In my actual program, the rook will be able to move in all 4 directions (using very similar code).
96     If Piece.IsPinned = False or Piece.IsPinned.Horizontally Then
97         For y = (CoorY + 1) To 7
98             If BlackTFTable(CoorX, y) = "T" Then
99                 # Sets the appropriate square on the enemy TTable to 'F'. This prevents the enemy king from moving to that square.
100                 BlackTFTable(CoorX, y) = "F"
101             end if
102             If Piece(at CoorX, y).isWhite Then
103                 # Stops the search if the rook's forwards movement runs into another white piece.
104                 Exit For
105             Else
106                 LegalMoveArray(n) = CoorX & y
107                 If Piece(at CoorX, y).isEnemyKing Then #The piece is checking the king.
108                     BInCheck.IsInCheck = True
109                     CheckingPiece = CoorX & CoorY
110                     If BlackTFTable(CoorX, y + 1) = "T" Then
111                         # As the search halts as soon as an enemy piece has been found, we run into a bug where the enemy king, when put
112                         # into check by a rook, can move 1 square forwards to illegally avoid it (as that square hasn't had a chance to be
113                         # flagged as illegal). Therefore, we forcefully set the next square (in the TTable) in the rook's For loop to 'F'.
114                         BlackTFTable(CoorX, y + 1) = "F"
115                     end if
116                     Exit For
117                 End If
118                 n += 1
119             End If
120         Next y
121     # If the piece is a sliding piece (one who's movement is not restricted), the function will allow checks for
122     # pinned pieces using the Enemy King's Position - extending a piece's line of sight if the King is on the same
123     # axis as it. If it finds a King, then the Pinned Piece is labelled from 0-3, which represent which direction it is
124     # pinned from (0 = vertical, 1 = btm left to top right diagonal, 2 = horizontal, 3 = top left to btm right diagonal).
125     # I have also implemented the logic for pieces moving depending on if / where they are pinned from.
126     If (Piece(at CoorX, y).isBlack) and (BKPos.XDirection = CoorX) And (BKPos.YDirection > y) Then
127         # Algorithm only runs if the function knows that the king is in the piece's line of sight.
128         For m = y + 1 To 7
129             If Piece(at CoorX, m).isEnemyKing Then

```



```

130 # therefore the enemy piece must be labelled as being pinned. For more information on my notation for pinned pieces,
131 # please see the TFTable explanation found in my Project Report.
132     Piece(at CoorX, y).isPinned = True
133     Piece(at CoorX, y).isPinned.Vertically = True
134     Exit For
135 End If
136 If Square(at CoorX, m).isPiece then
137 # Friendly piece is found - therefore terminate search.
138     Exit For
139 end if
140 Next
141 End If
142 If Square(at CoorX, y).isPiece then
143 # Friendly piece is found - therefore terminate search.
144     Exit For
145 end if
146 End If
147 Next
148 End If
149 end if
150
151
152
153 # At the end of the function, the program checks to see if any piece is attacking the enemy king.
154 # If it is, then the other player is signalled to be in check, and the attacking piece is recorded.
155 If CheckingPiece isNot Nothing Then
156     If BInCheck.Piece isNothing Or BInCheck.Piece = CheckingPiece Then
157         BInCheck.Piece = CheckingPiece
158     Else
159         BInCheck.DoubleCheck = True
160     End If
161 End If
162
163
164 # The first index of LegalMoveArray contains a counter of how many moves each piece can make. Useful for
165 # when the AI is picking moves (no need for array.length command as the array can be of numerous lengths).
166 If n > 1 Then
167     LegalMoveArray(0) = n
168 end if
169
170 return LegalMoveArray
171
172 end Function

```

As you can see from lines 120 to 141, sliding pieces (ie: rooks, bishops & queens) are able to pin enemy pieces, and label them with a given direction on the TFTable. This is used to build up a profile of the enemy king's legal moves and pinned pieces on the board. To optimise this section of the algorithm, we can stop the pin search if the piece detects a friendly piece (one of its own colour), as a rook obviously cannot pin one of its own pieces...

Or can it? Well, in a weird kind of betrayal, a rook or queen *can* pin one of its own to the enemy king – sort of. Introducing the En Passant Pin:

THE EN PASSANT PIN

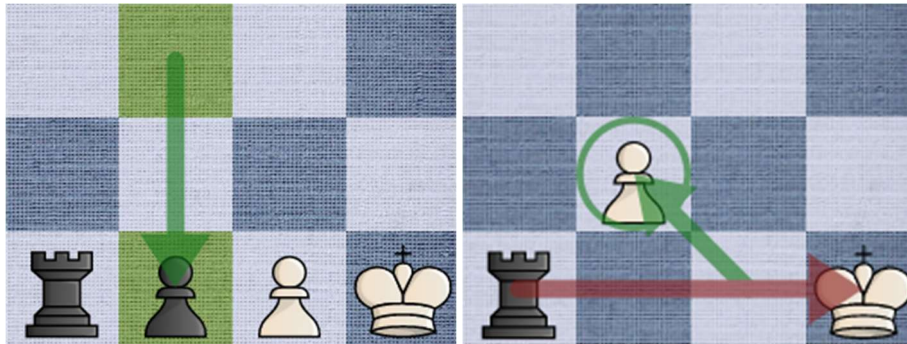
I briefly across this particular issue of En Passant during my initial research of chess AIs^[2] (to determine the feasibility of this project), however I didn't think about it too much at the time, as I was unsure as to what coding approach I would take during this project. This was until, ironically, I ran into the same problem myself whilst coding the pseudocode for my PieceLegalMoves function. Here's the issue:

En Passant is a tricky concept for beginners, not only for chess players but for programmers likewise. A brilliant example of this is an unwritten yet exclusive feature of En Passant, that states that, if a pawn is taken En Passant, **two** pieces leave that row (on the board) instead of the regular one – the pawn that is taking En Passant, and the enemy pawn that is being taken. This causes a huge problem for my pinning algorithm, as for these few cases we need to disable the above get-out clause whilst the rook is attempting to pin a pawn, and if it is successful, we should label this pawn with the En Passant pin (labelled as '4' in my TFTable).

Note that, unlike most other types of pins, the En Passant pin only restricts a pawn from taking En Passant – its general movement should not be restricted whatsoever. So, how can we differentiate En Passant pins from

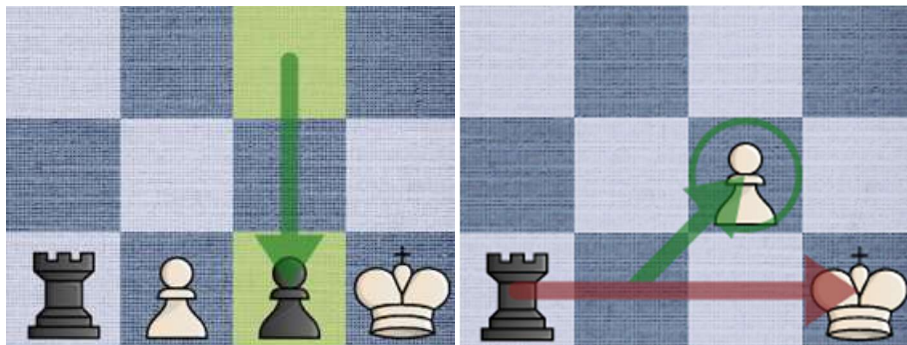
normal pins, and what do we need to change with our original pseudocode? Thankfully, there are only 4 situations where this can occur (for each rook) – two for the left direction and two for the right direction. For simplicity, I will only be including the explanations and pseudocode for the two rightward directions.

In both examples below, the black rook is En Passant pinning the white pawn to the white king. I have appropriately named these situations to make understanding my following pseudocode more trivial...



Example I - Friendly-First
En Passant pin:

Here, in relation to the black rook, the **friendly pawn** comes before the enemy piece, which is taking En Passant. White cannot take En Passant.



Example II - Enemy-First
En Passant pin:

Here, in relation to the black rook, the **enemy pawn** which is taking En Passant comes before the friendly pawn. White cannot take En Passant.

Whilst both these examples seem identical in terms of methodology, their pseudocode representations are distinctly different, as they change the order in which the black rook pins the white pawn (and therefore changes the algorithm's get-out clauses). To make this algorithm more efficient, we first check if the En Passant square is directly above the friendly (black) pawn. If it is, then it is implied that there is an enemy pawn next to it, and thus we can begin searching for En Passant pins.

Below are my pseudocode additions to the PieceLegalMoves function, which includes En Passant pins. Please note that, in my original pseudocode in the previous pages, only the forwards direction for the rook is explained, however an En Passant pin only applies to rooks moving horizontally – therefore please imagine that the below code (which explains the rightward En Passant pin) is made for the rightward rook movement logic (the code is near identical for each rook direction).

Under Line 102 in PieceLegalMoves Pseudocode:

```
# Pseudocode for 'friendly-first' En Passant Pin Creator.
if (Piece(at coords x, CoorY) = "P") And (EnPassant = x & (CoorY + 1)) and (y = BKPos.YDirection) and (x > BKPos.XDirection) then
# Then there will be a chance of that pawn being En Passant pinned (EnPassant Square is above friendly pawn & king is in the way).
  For m = x + 2 To 7 #Note: m is an incremental variable, similar to x and y. The reason we start at x+2 is that, because the
    #search starts at the friendly pawn, the king could only be at least 2 squares away (rightwardly).
    If Piece(at coords m, CoorY).isEnemyKing Then
#The enemy pawn must be subject to an En Passant pin.
      Piece(at coords x+1, CoorY).isPinned = true
      Piece(at coords x+1, CoorY).isPinned.EnPassantPinned = true
      Exit For
    End If
    If Piece(at coords m, CoorY).isPiece Then #If another friendly piece is found then the search stops.
      Exit For
    end if
  Next
end if
```

Under Line 136 in PieceLegalMoves Pseudocode:

```
# Pseudocode for 'enemy-first' En Passant Pin Creator.
if (Piece(at coords m, CoorY) = "P") And (EnPassant = m & (CoorY + 1)) and (y = BKPos.YDirection) and (x > BKPos.XDirection) then
# Then there will be a chance of that pawn being En Passant pinned (EnPassant Square is above friendly pawn & king is in the way).
  For a = m + 1 To 7 #Note: a & m are incremental variables, similar to x and y. The reason we start at m+1 is that, because the
    #search starts at the enemy pawn, the king must be on the next square to the right or more.
    If Piece(at coords a, CoorY).isEnemyKing Then
#The enemy pawn must be subject to an En Passant pin.
      Piece(at coords x, CoorY).isPinned = true
      Piece(at coords x, CoorY).isPinned.EnPassantPinned = true
      Exit For
    End If
    If Piece(at coords a, CoorY).isPiece Then #If another friendly piece is found then the search stops.
      Exit For
    end if
  Next
end if
```

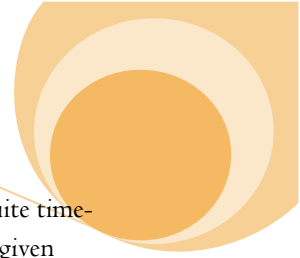
RESOLVING CHECKS EFFICIENTLY

There are three ways of escaping a check in chess. These methods are to...

- 1) Move your king out of the checking piece's way, to a legal square.
- 2) Capture the checking piece (either with the king or with any other piece).
- 3) Block the checking piece's line of attack via another piece.

There is also another fundamental concept to understand here, and that is a 'double-check'. A double check is when two pieces are checking the enemy king simultaneously (by the aid of discovered attacks). So, this is our problem – if a given player is in check, and on their turn they make move 'A', how can we determine whether this puts them out of check?

A common first-approach to solving this is to essentially brute-force the problem – we make the move on a temporary board, then use our AI to find all the opponent's responses to this move. If any of their moves is to capture our king, then we know that the original move is invalid. But whilst this may be practical for a two-player game of chess, this approach is awfully time-consuming for our AI to use, so instead I came up with a better method: instead of searching through all the opponent's responses to a move, we instead keep track of the piece that put us into check. When we want to see if a move resolves check, we make the move on a temporary board, then search through only the checking piece's response moves. If it still detects our king, then we exit the function and reject the move.



However, there are a lot of optimisations we can still make. As the check resolving algorithm will be quite time-consuming, we want to call it as scarcely as possible – therefore we need a way of quickly checking if a given piece’s pseudo-legal move would never resolve a check. The most obvious of these include: a check via a knight cannot be blocked by another piece, and double-checks must be resolved by moving the king out of the way (as you cannot take / block both checking pieces). Moreover, if we are moving the king out of the way to escape check, we actually don’t need to call the check resolving algorithm; as TFTables already contains the legal moves for the king, and because we update the player’s TFTable before they move, we can tell if a king’s move will escape check just by looking at that square on the TFTable (if it is “T” then the move is legal). Finally, if our move to resolve a check is to capture the checking piece, and the capturing move is legal, we can assume the move is legal (and therefore, we don’t need to call the algorithm).

INSTANTIATING OUR AI

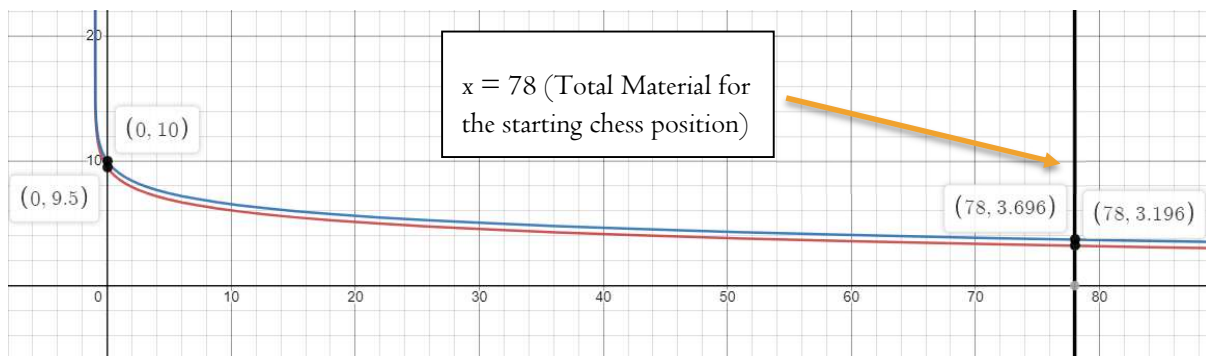
CALCULATING THE OPTIMAL DEPTH FOR THE POSITION

It is very important to note that the time an AI takes to search to a depth of N will be very different depending on the position and how many pieces are still on the board – therefore, I believe calculating the AI’s ‘optimal depth’ for a given position (before the AI performs on it) to be a necessary feature of my program. To calculate this depth, we will consider the total material count on the board, and put this value into an exponential formula. (I have chosen an exponential formula as the time to complete a search increases exponentially with depth.)

Unfortunately, it is impossible to say with absolute certainty what the optimal depth of the AI should be ahead of time, so instead I have created this formula to make an accurate assumption of what this depth should be:

$$y = -2 \log_4(x + 1) + 10 - (0.5 * Q)$$

... where y = the estimated optimal depth (rounded down), x = the total material on the board, and Q = ‘Is Quiescence Enabled?’ (1 for True, 0 for False)



And there we have it! – all we need to is input the total material count of the board, and we receive an estimated optimal depth, which we can then use on our AI’s search. This algorithm also includes some smart capabilities, such as limiting the depth if a checkmating sequence has been detected, and preventing the depth from being less than 2 (as this may result in errors).

The main disadvantage of this algorithm is that it is only an estimate to the problem, so naturally we may still get some weird values for the optimal depth - resulting in the user waiting an infuriating length of time for the AI to make its move. This is because, even though the relative weight of a piece generally corresponds with how many moves it can make (and therefore the time taken to search through its moves), this isn’t always the case.

For example, a queen stuck in the corner of the board, unable to move, might have less moves than an active bishop – making our depth estimation inaccurate.

To combat this, we will instead run the AI on five multiple depths, simultaneously – using our optimal depth as the medium. For example, if our optimal depth is calculated to be ‘6’, then we create five separate instantiations of AI – one that runs at a depth of 3, one to a depth of 4, 5, 6, and 7. We give the AI an allocated amount of time to search for, and at the conclusion of this time we retrieve the best move found at the highest depth successfully completed within that timeframe. Thankfully, the smart features of the optimal depth calculator still apply here; if the depth is calculated to be 3, then we search at a depth of 2, 3, 4, 5, and 6 (we always use five AIs). In the very rare case that our optimal depth algorithm returns an incredibly incorrect value, we can use these two extra rules:

- 1) If all the time has expired before any AI has finished searching then we give the AIs a further 50% of the allocated time (or an extra 10 seconds, whichever value is lowest) for searching. If, at the end of this time, we still don’t have a completed search, then we create a new search at a depth of 2 without Quiescence (which will be instantaneous), and use the move from that AI instead.
- 2) If all the AIs have finished searching before the time has expired, then we make the two AIs of lowest depth search to an increased depth (+5 of their previous). If these new searches still produce a move before the time has expired, then we stop the timer early and instantly make the move of highest depth.

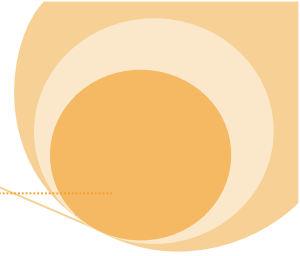
This method offers much more flexibility, both in AI difficulty and user experience. However, to not sacrifice on speed by creating multiple AIs, we need to implement multithreading into my code...

MULTITHREADING

I will be using multithreading in my program predominantly for creating multiple AIs (with different depths) that search on a position simultaneously. As well as providing the advantages stated in the previous subsection, this implementation will also allow for us to provide the user with visual updates on the AI’s progress (as we can see its evaluation of a position become more and more precise) - furthermore, if a mating sequence is detected in a lower depth AI, we can terminate the AIs of higher depths pre-emptively to save time!

As we have five AIs running simultaneously, we must create a list of five new ‘tasks’ that all run on separate system threads – meanwhile, our main thread will control the GUI, so that the user can still interact with my program whilst the AIs are searching.

When I was first designing my program, I originally planned to use multithreading in a completely different way – in these earlier models, multiple AIs would work on the same position simultaneously, each having access to only one row of the board and thus only searching through the possible moves that the pieces on that row could make. At the conclusion of the search, they would combine their moves & evaluations to generate a single, ‘best’ move. However, when I combined this technique with my move ordering algorithm (if we search the best moves in the position first, then Alpha-Beta Pruning can prune more branches – thus making the AI faster), I found that the speed increase that multithreading gave became almost negligible in most positions. This is because, as each AI is only given access to a sample of moves on the board, most AI instantiations will not detect the best move in the position, and hence less branch prunes are able to be made (as the AIs, running on separate threads, cannot interact with each other). Thus, I scrapped this method, and replaced it with the one previously explained.



SOURCE CODE

Please see below for the stripped-down code used for instantiating our AIs (including multithreading). One thing to note, however, is the mechanism for 'ABORT'-ing our AIs – my AI class contains the public variable ABORT, which is set to False at the start of a search. If this variable is set, the AI will complete its search immediately and return an incomplete move (with the EndState code "a" to show that its search has been abruptly terminated). We stop the search by adding this statement at the start of the MiniMax algorithm:

```
If ABORT = True Then Return 0
```

This serves the purpose of quickly & safely terminating an AI's search.

INstantiateAIs Subroutine

```
'Algorithm that creates all the threads & AIs that will search on a position.
Handles time management + overtime,
'interactions with the GUI chessboard, live updates via GUI objects, selecting
best moves, and more.
Private Sub InstantiateAIs() Handles AIMoveBtn.Click
    If GameRunning And Not ComputerIsSearching Then
        MainAI.Reconfigure(CurrentFEN) 'Recalibrates AI.
        Dim BestMove as Move
    = MainAI.CheckForEndState() 'Ensures that the position is valid, and that the
        'AIs won't crash whilst searching on the position.
        If BestMove.EndState = "f" Or BestMove.EndState = "o" Then
            If BestMove.EndState = "f" Then 'no forced move found.
                'Instantiations of threads (in a List of tasks format).
                Dim Tasks As New List(Of Task)
                Tasks.Add(New Task(AddressOf InitialiseThread1))
                Tasks.Add(New Task(AddressOf InitialiseThread2))
                Tasks.Add(New Task(AddressOf InitialiseThread3))
                Tasks.Add(New Task(AddressOf InitialiseThread4))
                Tasks.Add(New Task(AddressOf InitialiseThread5))

                CalculateAbsoluteDepth()
                ComputerIsSearching = True
                'Starts all tasks simultaneously.
                Tasks.All(Function(t As Task)
                    t.Start()
                    Return True
                End Function)
                'While any of the 5 threads are running...
                While Tasks.Any(Function(t As Task) Not (t.Status = TaskStatus.
                    Canceled OrElse t.Status = TaskStatus.Faulted OrElse t.Status = TaskStatus.
                    RanToCompletion))
                    Application.DoEvents() 'Allows the user to interact with
                    the GUI (limited).
                If Stopwatch.ElapsedMilliseconds / 1000 > TimeForSearch Then
                    'Time's up!
                    If Not MainAI.ABORT Then
                        'Aborts all AIs (apart from AI of lowest depth if
                        it has not finished).
                        If AIMovedToHigherDepth(0) Then AI2.ABORT = True
                        AI3.ABORT = True
                        MainAI.ABORT = True
                        AI4.ABORT = True
                        AI5.ABORT = True
                        ProgressBar.ForeColor = Color.Red
```

```

        End If
        If Stopwatch.ElapsedMilliseconds / 1000 > Math.Min(Ti
meForSearch * 1.5, TimeForSearch + 10000) Then
            '1.5 * time has expired - terminate all AIs immed
ately!
            AI2.ABORT = True
        End If
    End If
End While

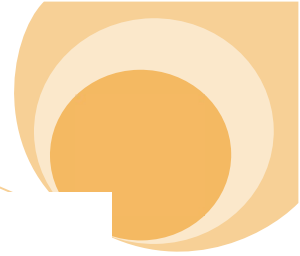
'Select the best move of the highest depth possible (if that
search has not been aborted).
If AIMovedToHigherDepth(1) And AIBestMoves(6).EndState <> "a"
Then
    BestMove = AIBestMoves(6)
ElseIf AIMovedToHigherDepth(0) And AIBestMoves(5).EndState <>
"a" Then
    BestMove = AIBestMoves(5)
ElseIf AIBestMoves(4).EndState <> "a" Then
    BestMove = AIBestMoves(4)
ElseIf AIBestMoves(3).EndState <> "a" Then
    BestMove = AIBestMoves(3)
ElseIf AIBestMoves(2).EndState <> "a" Then
    BestMove = AIBestMoves(2)
ElseIf AIBestMoves(1).EndState <> "a" Then
    BestMove = AIBestMoves(1)
ElseIf AIBestMoves(0).EndState <> "a" Then
    BestMove = AIBestMoves(0)
Else 'No AI have completed their search - start a new search
at a depth of 2 (without Quiescence) .
    'Set the result of this new to be the best move.
    BestMove = MainAI.Search(2, False)
End If

Else 'Move was forced - make move instantly without searching.
End If

GameRunning = True
'Makes the best move on the board, and updates the correct TFTabl
e.
If PlayerTurn Then
    MasterWInCheck.NotInCheck()
    MakeMove(MasterBoard, BestMove.OldMoveX, BestMove.OldMoveY, B
estMove.NewMoveX, BestMove.NewMoveY, MasterWCanCastle, MasterWKPos, MasterMat
erialCount, MasterEnPassant, True)
    CoreMethod.FixTFTables(MasterBoard, False, MasterBlackTFTTable
, MasterBKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
Else
    MasterBInCheck.NotInCheck()
    MakeMove(MasterBoard, BestMove.OldMoveX, BestMove.OldMoveY, B
estMove.NewMoveX, BestMove.NewMoveY, MasterBCanCastle, MasterBKPos, MasterMat
erialCount, MasterEnPassant, True)
    CoreMethod.FixTFTables(MasterBoard, True, MasterWhiteTFTTable,
MasterWKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
End If
DisplayPieces()
CheckChecker()
ComputerIsSearching = False
EnforceEndStates() 'Checks for end states found from the AI's mov

```

e.



```

        OutputDebugInfo()
    End If
End If
End Sub

```

CONTROLS FOR THREADS

The code for each thread controller is quite similar, so I will only provide the source code for one thread (in this case, the controller for the AI of least depth).

```

'Subroutines controlling each thread / AI of particular depth.
Private Sub InitialiseThread1()
    'Creates and starts the AI.
    AIBestMoves(0) = AI2.Search(AbsoluteDepth - 2, UseQuiescence)
    If Not AI2.ABORT Then 'The Move has completed successfully.
        If Math.Abs(AIBestMoves(0).Score) > 900 Then
            'Checkmating sequence found - terminate all other AIs.
            AI3.ABORT = True
            MainAI.ABORT = True
            AI4.ABORT = True
            AI5.ABORT = True
        ElseIf Stopwatch.ElapsedMilliseconds / 500 < TimeForSearch Then
            'Starts a new search at a higher depth (if there is >50% left on
the timer).
            AIMovedToHigherDepth = True
            AIBestMoves(5) = AI2.Search(AbsoluteDepth + 3, UseQuiescence)
            If Not AI2.ABORT Then 'The Move has completed successfully.
                If Math.Abs(AIBestMoves(5).Score) > 900 Then
                    'Checkmating sequence found - terminate all other AIs.
                    AI3.ABORT = True
                    MainAI.ABORT = True
                    AI4.ABORT = True
                    AI5.ABORT = True
                End If
            End If
        End If
    End If
End Sub

```

THE MINIMAX PROCESS

Once our AIs have been instantiated, we need to let them search on a position to produce what it considers to be the ‘best move’. For this, I will be using the MiniMax algorithm with Alpha-Beta Pruning. I have made a prototype for this algorithm that plays the game of Noughts & Crosses (you can read about that [here](#)), however as chess is a much more complex game, there are some key parts to describe in this subsection.

Here is the general algorithmic structure I will use for MiniMax in my program.

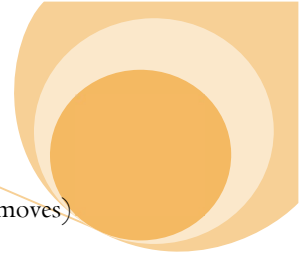
- If the depth limit has been reached, and the user is not using Quiescence, **evaluate the position and return this value**. If not...
- Set the score of the best move to -infinity or +infinity (depending on if it is white to move or black to move).
- Fix the TFTable for the player to move.
- **If the depth limit has been reached, and the user is using Quiescence, find the pseudo-legal capture moves that can be made in the position. Otherwise, find all the pseudo-legal moves that can be made.**
- **Order these moves from best to worst (estimation).**
- If the player is in check, filter out the moves that do not resolve the check.
- For each of these legal moves:
 - Copy all the position data structures to temporary variables.
 - **Make the move on a temporary board.**
 - Put this new position through MiniMax recursively.
 - Update the best move if necessary.
 - Use Alpha-Beta Pruning to potentially prune the branch.
 - Undo the temporary move (for my implementation, we skip this step, as we are making a set of temporary variables for each new board position).
- Return the score of the best move found this iteration. If no legal moves have been detected, however, we do the following:
 - If the depth limit has been reached, and the user is using Quiescence, **evaluate the position and return this value**. Otherwise...
 - If white is in check, return -1000 (checkmate).
 - If black is in check, return +1000 (checkmate).
 - Otherwise, return 0 (stalemate).

Now, most of these steps have either been explained in previous parts of this section, or are quite intuitive and don't provoke further explanations. However, the highlighted steps do require some more detail...

FINDING AND ORDERING MOVES

Due to the existence of our PieceLegalMoves algorithms, we can quickly find all the pseudo-legal moves a piece can make on the board – therefore, by running this algorithm for all the pieces on the board (of a certain colour), we can generate a list of all the possible pseudo-legal moves a player can make on their turn. However, this is only one part to the story – we then need to order these moves from best to worst.

In a nutshell, by searching the best moves in a position first, Alpha-Beta Pruning is able to cut off more branches in the MiniMax algorithm, therefore searching through fewer positions in a given depth^[3]. Obviously we cannot say for certain which moves are better than others without searching through them (as that is the entire purpose of my AI), however we can make some loose predictions based on certain criteria. To do this in my program, I



will break moves up into different categories (ranging from the 'best' set of moves to the 'worst' set of moves) which are then stored in multiple arrays. These moves are categorised by the following conditions:

- Section 1 = Capture Moves:
 - 1) Capture moves where the captured piece is worth more than the one that is capturing it (eg: a pawn captures a bishop).
 - 2) Capture moves where the captured piece is worth the same as the one that is capturing it (eg: a bishop captures a knight).
 - 3) Capture moves where the captured piece is worth less than the one that is capturing it (eg: a rook captures a knight).
- Section 2 = Non-Capture Moves:
 - 1) Moves that involve the promotion of a pawn.
 - 2) Moves where a sliding piece (ie: rooks, bishops, or queens) becomes in-line with the enemy king (which *may* indicate a check).
 - 3) Moves which do not satisfy any of the above or below categories in this section (ordinary moves).
 - 4) Moves that place the piece on a square marked with "F" on the enemy's TFTable (meaning the square is controlled by an enemy piece).
 - 5) Moves where the piece moves to a square controlled by an enemy pawn.

At the conclusion of the algorithm, we will combine these arrays into one large array (ordered from category 1 in section 1 to category 5 in section 2) and return this array as our complete list of pseudo-legal moves.

The last point of discussion is the Quiescence algorithm, and how it affects this algorithm. When Quiescence mode is activated, and the depth limit has been reached, the computer only searches through the capture moves in a position – only stopping when it reaches a position with no capture moves left (which we call 'quiet').

Therefore, for these circumstances we need to adjust the algorithm to only return capture moves; luckily, we've already done most of the work by ordering capture moves (in section 1) above regular moves (in section 2).

Therefore, if we need to only search through capture moves, we just eliminate section 2 from the categorisation, and only return section 1 at the conclusion of the algorithm.

MAKING A TEMPORARY MOVE

Unfortunately, making a move on a chessboard isn't as simple as changing the position of a piece on the board – we also need to account for other information such as castling rules, pawn promotion, and En Passant. This will all be handled via the MakeMove subroutine, which takes all the required information of a chess position and makes a given move – updating its parameters along the way.

There will be two versions of this algorithm in my program, one in the Chess class and the other in the AI Class. The major difference between the two is that, for the algorithm in the Chess class, appropriate sounds play when a move is made (depending on what type of move is being made) – this feature has been removed for the algorithm for in AI Class (so that the AI can complete the algorithm faster). You can see the pseudocode for the AI MakeMove algorithm below:


```

1 # Subroutine that makes a move on the board, given coordinates. Includes castling (& rights) and pawn promotion.
2 ▾ Subroutine MakeMove(Board As Array(of Chars), OldCoorX As String, OldCoorY As String, NewCoorX As String,
3   NewCoorY As String, CanCastle As CanCastle, KPos As String, MaterialCount As Integer, EnPassant As String)
4
5 # TempPiece represents the piece being moved by the user.
6 def TempPiece As String = Piece(at coords [OldCoorX, OldCoorY] on Board)
7 def HasEnPassanted As Boolean = False
8 ▾ If TempPiece.isWhitePiece Then
9   If TempPiece = "P" Then
10    # Code for Promoting Pawns and En Passant. Also increments the material count.
11    If NewCoorY = 0 Then
12      TempPiece = "Q" # Pawn is replaced by a queen.
13      # MaterialCount = + 9 for a new queen, - 1 for losing the pawn in the process.
14      MaterialCount = MaterialCount + PieceValueOf("Q") - PieceValueOf("P")
15    ElseIf NewCoorX & NewCoorY = EnPassant Then
16      Board(NewCoorX, NewCoorY + 1) = " " # Removes the enemy pawn that is being taken En Passant.
17      MaterialCount = MaterialCount + PieceValueOf("P")
18    ElseIf OldCoorY = 6 And NewCoorY = 4 then # Pawn moved 2 squares on turn.
19      If Piece(at coords [NewCoorX - 1 or NewCoorX + 1, 4] on Board) = "p" then
20        # Creation of EnPassant.
21        EnPassant = NewCoorX & 5
22        HasEnPassanted = True
23      End if
24    End If
25    # If piece is a Rook, part of Castling is disabled (depending on which Rook has moved).
26  ElseIf TempPiece = "R" Then
27    If OldCoorX = 0 And OldCoorY = 7 Then
28      # Queenside castling privileges removed.
29      CanCastle.Queenside = False
30    ElseIf OldCoorX = 7 And OldCoorY = 7 Then
31      # Kingside castling privileges removed.
32      CanCastle.Kingside = False
33    End If
34  ElseIf TempPiece = "K" Then
35    # Updates king position.
36    KPos = NewCoorX & NewCoorY
37    # Code for Castling.
38    If CanCastle.Kingside = True And (NewCoorX = 6 And NewCoorY = 7) Then
39      # Moves rook appropriately.
40      Board(5, 7) = "R"
41      Board(7, 7) = " "
42    ElseIf CanCastle.Queenside = True And (NewCoorX = 2 And NewCoorY = 7) Then
43      # Moves rook appropriately.
44      Board(0, 7) = " "
45      Board(3, 7) = "R"
46    End If
47    CanCastle.Kingside = False
48    CanCastle.Queenside = False
49  End If
50 ▾ Else
51   # Identical code as above but for the black pieces.
52 End If
53
54 # Removal of EnPassant (if required).
55 ▾ If Not (EnPassant = "-" or HasEnPassanted = True) Then
56   EnPassant = "-" # = Blank.
57 End If
58
59 # If the new position contains a piece, then the material count is updated (capture move).
60 ▾ If Square(at coords [NewCoorX, NewCoorY] on Board).isPiece Then
61   If Piece(at coords [NewCoorX, NewCoorY] on Board).isWhitePiece Then
62     MaterialCount = MaterialCount - PieceValueOf(Piece(at coords [NewCoorX, NewCoorY] on Board))
63   Else
64     MaterialCount = MaterialCount + PieceValueOf(Piece(at coords [NewCoorX, NewCoorY] on Board))
65   End If
66 End If
67 # At the end of the subroutine, the Piece is placed at the new coordinates,
68 # and the piece at the old position is cleared.
69 Square(at coords [NewCoorX, NewCoorY] on Board) = TempPiece
70 Piece(at coords [OldCoorX, OldCoorY] on Board) = " "
71 End Sub

```

As you can see, the above subroutine also handles the material count of the board, along with the king position of a given player. Whilst it isn't technically necessary to deal with that information here, it makes a lot more sense to append these variables (if necessary) when we make a move, rather than to recalculate them for every board position - this saves a lot of time during the MiniMax process.

EVALUATING A CHESS POSITION

My evaluation function performs on the leaf nodes found in a MiniMax search, and takes into account the difference in material between the two sides, along with my [endgame heuristic](#). The material count for each player is stored in the MaterialCount array (containing the total material count for white and black in separate indexes), and as we have [already mentioned](#), we amend the material count when a capture move is performed (or if a pawn promotes) instead of calculating this material count from scratch for every position. Meanwhile, the endgame heuristic is an add-on to the evaluation function, and is calculated every time the algorithm runs.

USING HASHING TO CALCULATE PIECE VALUE

It's easy to confuse the words 'material count' with 'piece value', or 'piece weight'; in a nutshell, the word 'material' doesn't link to the number of pieces, but instead the total *weight* for all the pieces. A piece's weight is its relative value in a game of chess, and it has been widely agreed what weight each piece should contain:

- Pawn = 1 point.
- Knight / Bishop = 3 points.
- Rook = 5 points.
- Queen = 9 points.

So, find the total material for each side, and calculate White's Material Count – Black's Material Count, and you have an estimate for a position's evaluation!

My first attempt at this was very primitive: a FOR statement looped through the entire board, and for each piece, ran it through a series of IF statements to determine which piece it was, and therefore what weight it should be assigned with. This linked to an Enumerated Structure called 'PieceValue', which contained the relative weights of all the pieces on the board. This meant that I could easily change how much or little specific pieces are worth to the computer (for debugging purposes). You can see the pseudocode for this attempt to the right:

```

1 Structure (Enumerated) PieceValue
2     Q = 9
3     R = 5
4     B = 3
5     N = 3
6     P = 1
7 End Structure
8
9
10 def TempPiece As Char
11 def WhiteMaterial As Integer = 0
12 For y = 0 To 7
13     For x = 0 To 7
14         If Square(at Coords (x, y) on Board).isPiece Then
15             TempPiece = Piece(at Coords (x, y) on Board)
16             If TempPiece.isWhite Then
17                 If TempPiece = "Q" Then
18                     WhiteMaterial = WhiteMaterial + PieceValue.Q
19                 ElseIf TempPiece = "R" Then
20                     WhiteMaterial = WhiteMaterial + PieceValue.R
21                 ElseIf TempPiece = "B" Then
22                     WhiteMaterial = WhiteMaterial + PieceValue.B
23                 ElseIf TempPiece = "N" Then
24                     WhiteMaterial = WhiteMaterial + PieceValue.N
25                 ElseIf TempPiece = "P" Then
26                     WhiteMaterial = WhiteMaterial + PieceValue.P
27                 end if
28             Else
29                 # ... = Identical code as above for the black pieces.
30             End If
31         End If
32     Next
33 Next
  
```

However, this method is of course incredibly slow, as we could check the piece's name up to 5 times before we assign its weight. A much more efficient method would be to collapse piece's name into a single number, which represents the weight associated to it. This provides the basis for my new implementation of this algorithm in my code, which involves hashing.

Hashing is the process of condensing an identifier name into a value, which refers to the index of an array. In this array index, the information about the identifier is stored. In my program, we hash the piece's name (as bound in the Board array) into a number from 0-9, then retrieve the constant found at that index in the array PieceValue. This variable will be the weight of the piece. My hashing process involves converting the piece name to its uppercase counterpart, then converting that letter to ASCII. We then take the modulus 11 operation of that number (by dividing it by 11, then returning the remainder). This number represents the PieceValue array index, which contains the appropriate value of each type of piece. Below is the pseudocode for this implementation.

```

Def PieceValue(9) as array (of float)
PieceValue(0) = 3 #Bishop's Weight
PieceValue(1) = 3 #Knight's Weight
PieceValue(3) = 1 #Pawn's Weight
PieceValue(4) = 9 #Queen's Weight
PieceValue(5) = 5 #Rook's Weight
PieceValue(9) = 0 #King's Weight
                #(not necessary)

Def Index as int
Def WeightofPiece as float
Index = Piece(at coords [x,y] on board).Uppercase
Index = (Index.ConvertToASCII) mod 11
WeightofPiece = PieceValue(Index)

```

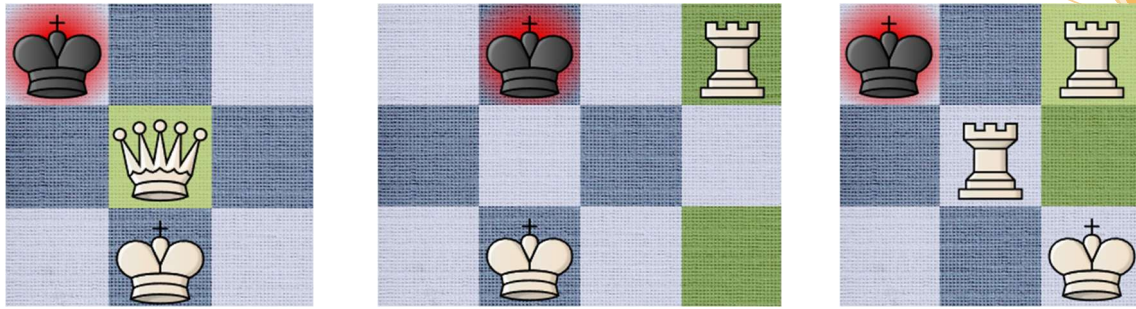
For example, if we want to calculate the weight of a black rook, we do the following:

$$r \rightarrow R \rightarrow 82 \rightarrow 5 \rightarrow \text{PieceValue}(5) = \text{Weight of 5}$$

Once we calculate this for all the pieces on the board (filtering them into either white's total or black's total appropriately), we subtract white's material count from black's, and return this as our simple evaluation of the position.

ADDING ENDGAME HEURISTICS

The above method for evaluating positions works well for middlegames in chess, but alas, when it comes to endgames this method isn't enough by itself. As a result, our AI appears to make no real progress in endgame positions, even in basic ones where beginners can find checkmate in a handful of moves – the AI just shuffles its pieces back & forth, with no aim or purpose behind its moves. This is because, if the AI's depth isn't high enough to find a checkmating pattern, and it cannot find any material to win, all the AI needs to do to 'secure its advantage' is to make pointless shuffling moves. Hence, as long as the AI doesn't lose any material, it is happy with its advantage. To fix this, we need to implement an endgame heuristic – one that will put the AI in a better position to find a checkmating pattern. Here's what I've decided to implement:



Taking a look at the above positions (where white has checkmated black) it becomes apparent that, in simple endgames such as these, it is much easier to checkmate a player if their king has been forced to the corner / edges of the board – furthermore, it appears that bringing one’s king closer to their opponent’s can also result in checkmates, as the player’s king can both cut off their enemy king’s moves, and protect its pieces when they deliver a checkmate (as is show in the left and right pictures above).

So, when we perform the evaluation function, at its conclusion we also check how much material the opponent player has (the player that is not evaluating the position). If it is less than 10, we flag the position as being in the endgame phase of a chess game, and run the following algorithm:

- 1) Calculate the opponent king’s distance from the centre of the board.
- 2) Calculate the distance between the two kings.
- 3) Evaluate positions where the opponent’s king is further from the centre, and where the distance between the kings is small, as being more advantageous for the player (as this may result in the discovery of a checkmating sequence).

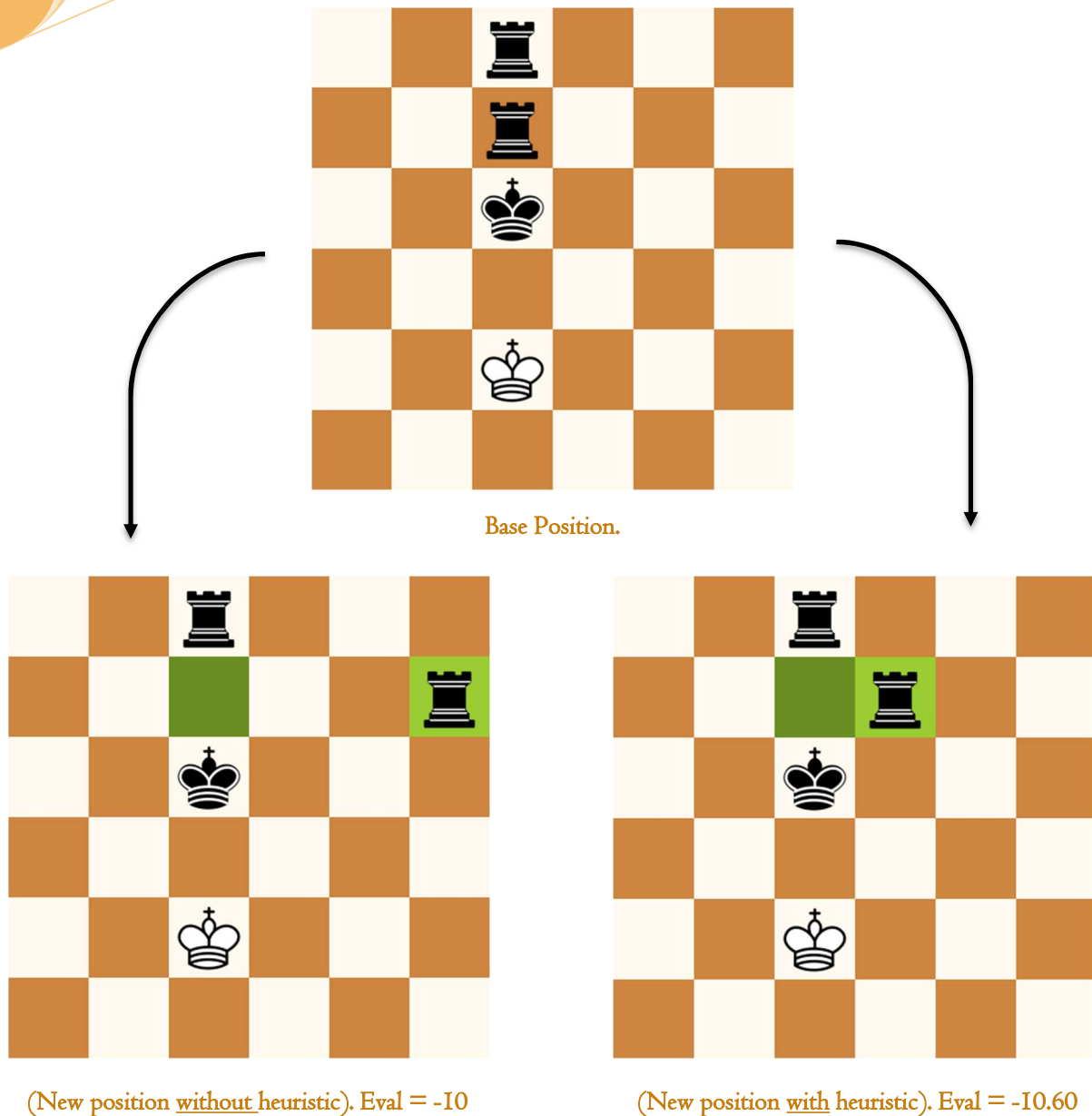
Finding the balance of how prevalent this heuristic should be in my evaluation process is quite tricky; in my code, I will have step (1) above to be 1.5 times more important than step (2), however I have made sure that this heuristic isn’t strong enough to overpower the effect of the material count algorithm. Finally, we want this heuristic to have more of a role on a position’s evaluation when the opponent has fewer and fewer pieces, as fewer pieces typically means a higher chance of checkmating an opponent. Here’s what all this looks like in pseudocode:

```
If OpponentMaterialCount < 10 Then
    # Finds distance from opponent's king to the centre of the board.
    Def HorizontalKingDistanceFromCentre as int = Max[(OpponentKPos.Column - 4), (3 - OpponentKPos.Column)]
    Def VerticalKingDistanceFromCentre as int = Max[(OpponentKPos.Rank - 4), (3 - OpponentKPos.Rank)]
    Def KingDistanceFromCentre as int = HorizontalKingDistanceFromCentre + VerticalKingDistanceFromCentre

    # Finds the distance between the two kings.
    Def KingDistance As int = Max[(WKPos.Rank - BKPos.Rank), (WKPos.Column - BKPos.Column)]

    Def HeuristicScore as float = (KingDistanceFromCentre * 1.5) + (7 - KingDistance)
    # Heuristic becomes more prevalent as the opponent has fewer and fewer pieces (exponential).
    EvaluationScore = EvaluationScore + (HeuristicScore * 1.3 ^ (9 - OpponentMaterialCount)) / 100
End If
```

To say how few lines of code make up this heuristic, its effect on the AI’s endgame performance is incredible. Having a look at a sample endgame position (black to move), let’s observe what the AI does with and without its new feature...

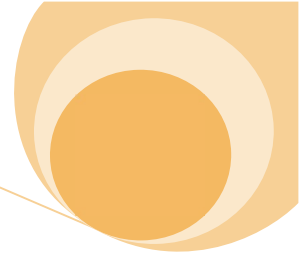


Even from the AI's first move, you can clearly see the improvement – on the left, black has made a seemingly 'random' move, whilst on the right, black has moved its rook to block off my king from the right side of the board. Playing this position as white without the heuristic, I was able to escape checkmate until I got bored 20 moves later. In contrast, when playing with the heuristic enabled, I got effortlessly checkmated by the AI after only three turns.

CHECKING FOR END STATES

After a move has been made on the chessboard (whether it be by the user or an AI), we need to detect whether the new position is an 'end state', meaning a position where the game should finish. The end states I will be including in my program are:

- Checkmates
- Stalemates (where a player has no legal moves, but they are not in check)



- Draws by insufficient material (please see <https://www.chessstrategyonline.com/content/tutorials/how-to-play-chess-draws> for more information).

Checking for draws by insufficient material is relatively trivial – we just count the total material on the board. If it is 0, we end the game. If it is 3, we end the game if we detect either a knight or bishop on the board. And if it is 6, we end the game if we detect two bishops of the same colour complex.

However, checkmates & stalemates are a bit trickier to detect, and for this we need some help from our AI.

Simplifying a little, to calculate checkmates / stalemates we run the AI on a depth of 1 (without Quiescence), and flag how many legal moves we detect. We do this because, before the AI conducts its search, it runs the TFTable Fixer subroutine, which will then tell us if a player is in check. Once the search has been completed, we do the following – if there are two or more moves detected, we return “f” (False = no end state) – if the AI doesn’t find any moves, we return either “c” (checkmate) if we are in check, or “s” (stalemate) if not. However, if there is only one move detected, we store this move and return “o” (one move).

Why do we return “o”, you may ask? Well, part of my FEN validation process is to check for end states, and we always run this algorithm before instantiating our AIs. This means that, if we only detect one (forced) legal move in the position, we don’t need to instantiate our AIs, as there are no decisions to make! – we just submit this only move as our ‘Best Move’, and make the move on the board instantly.

If any one of these end states are reached in a One-Player or Two-Player game of chess, then we notify the user with an appropriate message, and give them the option of either playing again, or returning back to the main menu.

REFERENCES

[1] = <https://chess.stackexchange.com/questions/30004/longest-possible-fen> (“So the total maximum number should be ... 87 characters.”)

[2] = www.youtube.com/watch?v=U4ogKOMIzqk (11:17 – “Here the [white] pawn is pinned to the king by the [black] rook... [but if] black then pushes this pawn two up, that breaks the pin... However, because there’s also that sneaky En Passant rule, where a pawn that moves two squares can be captured as if it had only moved one square, ... but in this case that reveals the attack of the rook again, and so the move is actually illegal.”)

[3] = <https://www.dailychess.com/rival/programming/moveorder.php> (“To improve the number of cut-offs made by the alpha-beta algorithm, the program attempts to order moves so that better moves are considered first.”)

SECTION 3: TECHNICAL SOLUTION

MAINMENU FORM



Imports System.Threading

'Class that allows the user to access the full extent of my program.
'Will be the first section of code that the user sees, and provides the start
ing
'point for instantiating all the other classes & forms in my program.

Public Class MainMenu

Dim PrimaryColour As Color 'Colour representing the dark squares on the Chessboard.

Dim SecondaryColour As Color 'Colour representing the light squares on the Chessboard.

Dim ColourScheme As String 'Colour code from the text file.

'Startup sound effect - sounds taken from Chess.com.

ReadOnly Sound_Startup As New Media.SoundPlayer With {
.SoundLocation = Application.StartupPath & "\\Sounds\\Chess_Startup.wav

"

}

Private Sub MainMenu_Load() **Handles** Me.Load

'Algorithm which retrieves the colour scheme from a file (represented by mnemonics).

Try

FileOpen(1, Application.StartupPath & "\\ColourPreferences.txt", OpenMode.Input)

ColourScheme = LineInput(1)

FileClose(1)

Catch ex As Exception

ColourScheme = "def"

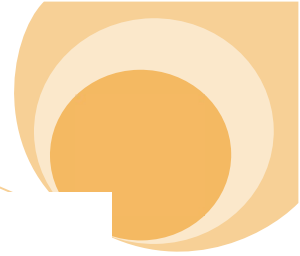
End Try

CreateColourProfile(ColourScheme)

'Sets up the text, buttons and objects.

SetupOptions()

SetupPieces()



End Sub

Private Sub CreateColourProfile(ByVal Code As String)

```
If LCase(Code) = "blu" Then
    PrimaryColour = Color.LightSteelBlue
    SecondaryColour = Color.AliceBlue
ElseIf LCase(Code) = "bl2" Then
    PrimaryColour = Color.SlateGray
    SecondaryColour = Color.Silver
ElseIf LCase(Code) = "gld" Then
    PrimaryColour = Color.Goldenrod
    SecondaryColour = Color.LemonChiffon
ElseIf LCase(Code) = "grn" Then
    PrimaryColour = Color.DarkSeaGreen
    SecondaryColour = Color.LavenderBlush
ElseIf LCase(Code) = "ppl" Then
    PrimaryColour = Color.MediumPurple
    SecondaryColour = Color.MistyRose
ElseIf LCase(Code) = "mon" Then
    PrimaryColour = Color.DimGray
    SecondaryColour = Color.Silver
Else 'def
    PrimaryColour = Color.Peru
    SecondaryColour = Color.FloralWhite
```

End If

End Sub

'A simpler version of the CreateBoard subroutine in the Chess.vb Form, which creates a checkerboard pattern on the screen.

Private Sub CreateBoard(ByVal sender As Object, ByVal e As PaintEventArgs) Handles MyBase.Paint

Dim g As Graphics = e.Graphics

Dim isLight As Boolean = True

'Creates checkerboard pattern. (Alternates between light and dark colours.)

For x = 0 To 7

For y = 0 To 7

Dim square As New Rectangle(0 + (75 * x), ((75 * y)), 75, 75)

If isLight Then

Using Brush As New SolidBrush(SecondaryColour)

g.FillRectangle(Brush, square)

End Using

Else

Using Brush As New SolidBrush(PrimaryColour)

g.FillRectangle(Brush, square)

End Using

End If

isLight = Not isLight

Next

isLight = Not isLight

Next

End Sub

Private Sub SetupPieces()

Dim Pieces(31) As PictureBox

Pieces(0) = WK1

Pieces(1) = WQ1

Pieces(2) = WB1

Pieces(3) = WB2

Pieces(4) = WN1

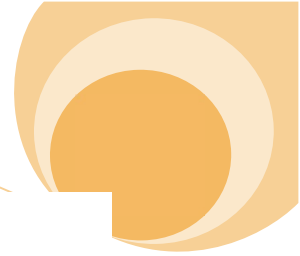
```

Pieces(5) = WN2
Pieces(6) = WR1
Pieces(7) = WR2
Pieces(8) = WP1
Pieces(9) = WP2
Pieces(10) = WP3
Pieces(11) = WP4
Pieces(12) = WP5
Pieces(13) = WP6
Pieces(14) = WP7
Pieces(15) = WP8
Pieces(16) = BK1
Pieces(17) = BQ1
Pieces(18) = BB1
Pieces(19) = BB2
Pieces(20) = BN1
Pieces(21) = BN2
Pieces(22) = BR1
Pieces(23) = BR2
Pieces(24) = BP1
Pieces(25) = BP2
Pieces(26) = BP3
Pieces(27) = BP4
Pieces(28) = BP5
Pieces(29) = BP6
Pieces(30) = BP7
Pieces(31) = BP8
For x = 0 To 31
    Pieces(x).Visible = False
Next

Dim Colour As String = "W"
For x = 0 To 16 Step 16
    Pieces(x).Image = Image.FromFile(Application.StartupPath & "\\Images\\Default\\" & Colour & "King.png")
    Pieces(x + 1).Image = Image.FromFile(Application.StartupPath & "\\Images\\Default\\" & Colour & "Queen.png")
    For n = x + 2 To x + 3
        Pieces(n).Image = Image.FromFile(Application.StartupPath & "\\Images\\Default\\" & Colour & "Bishop.png")
    Next
    For n = x + 4 To x + 5
        Pieces(n).Image = Image.FromFile(Application.StartupPath & "\\Images\\Default\\" & Colour & "Knight.png")
    Next
    For n = x + 6 To x + 7
        Pieces(n).Image = Image.FromFile(Application.StartupPath & "\\Images\\Default\\" & Colour & "Rook.png")
    Next
    For n = x + 8 To x + 15
        Pieces(n).Image = Image.FromFile(Application.StartupPath & "\\Images\\Default\\" & Colour & "Pawn.png")
    Next
    Colour = "B"
Next

'Sets the location of all the pieces on the board (in the starting position).
WK1.Location = New Point(300, 525)

```



```
WQ1.Location = New Point(225, 525)
BK1.Location = New Point(300, 0)
BQ1.Location = New Point(225, 0)
WB1.Location = New Point(150, 525)
WB2.Location = New Point(375, 525)
BB1.Location = New Point(150, 0)
BB2.Location = New Point(375, 0)
WN1.Location = New Point(75, 525)
WN2.Location = New Point(450, 525)
BN1.Location = New Point(75, 0)
BN2.Location = New Point(450, 0)
WR1.Location = New Point(0, 525)
WR2.Location = New Point(525, 525)
BR1.Location = New Point(0, 0)
BR2.Location = New Point(525, 0)
```

```
WP1.Location = New Point(0, 450)
WP2.Location = New Point(525, 450)
BP1.Location = New Point(0, 75)
BP2.Location = New Point(525, 75)
WP3.Location = New Point(75, 450)
WP4.Location = New Point(450, 450)
BP3.Location = New Point(75, 75)
BP4.Location = New Point(450, 75)
WP5.Location = New Point(150, 450)
WP6.Location = New Point(375, 450)
BP5.Location = New Point(150, 75)
BP6.Location = New Point(375, 75)
WP7.Location = New Point(225, 450)
WP8.Location = New Point(300, 450)
BP7.Location = New Point(225, 75)
BP8.Location = New Point(300, 75)
```

End Sub

Private Sub SetupOptions()

'Sets the location of all the labels and buttons, and makes them invisible.

```
Title.Visible = False
Title.Location = New Point(104, 160)
Title.BringToFront()
```

```
OnePlayer.Visible = False
OnePlayer.Location = New Point(127.5, 232.5)
OnePlayer.FlatAppearance.BorderSize = 0
OnePlayer.BringToFront()
```

```
TwoPlayer.Visible = False
TwoPlayer.Location = New Point(352.5, 232.5)
TwoPlayer.FlatAppearance.BorderSize = 0
TwoPlayer.BringToFront()
```

```
Analysis.Visible = False
Analysis.Location = New Point(127.5, 307.5)
Analysis.FlatAppearance.BorderSize = 0
Analysis.BringToFront()
```

```
ExitBtn.Visible = False
ExitBtn.Location = New Point(352.5, 307.5)
ExitBtn.FlatAppearance.BorderSize = 0
ExitBtn.BringToFront()
```



```

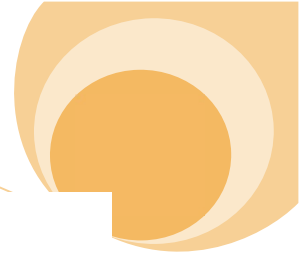
Credits.Visible = False
Credits.Location = New Point(172, 404)
Credits.BringToFront()
End Sub

Private Sub StartupAnimation() Handles Me.Activated
    MyBase.Refresh()
    Dim TempColour As String
    Try
        FileOpen(1, Application.StartupPath & "\ColourPreferences.txt", 0
penMode.Input)
        TempColour = LineInput(1)
        FileClose(1)
        If TempColour <> ColourScheme Then
            ColourScheme = TempColour
            CreateColourProfile(ColourScheme)
        End If
    Catch ex As Exception
    End Try
    MyBase.Refresh()

    Dim AlreadyBooted As Boolean
    If WP8.Visible = True Then AlreadyBooted = True
    'Shows the checkerboard pattern over time. This is achieved by having
    8 Rectangles which slowly move right off the screen.
    'These 8 Rectangles are made to be out-of-
    sync, so each Rectangle moves in the cycle after the one above it.
    'This gives the illusion of the checkerboard pattern being slowly con
    structed.
    For n = 1 To 15
        RowHider1.Left += 75
        If n >= 2 Then RowHider2.Left += 75
        If n >= 3 Then RowHider3.Left += 75
        If n >= 4 Then RowHider4.Left += 75
        If n >= 5 Then RowHider5.Left += 75
        If n >= 6 Then RowHider6.Left += 75
        If n >= 7 Then RowHider7.Left += 75
        If n >= 8 Then RowHider8.Left += 75
        Application.DoEvents()
        Thread.Sleep(100)
    Next
    Thread.Sleep(50)

    'Slowly makes the pieces on the board visible (in a spiral pattern).
    WK1.Visible = True
    WQ1.Visible = True
    BK1.Visible = True
    BQ1.Visible = True
    Application.DoEvents()
    Thread.Sleep(150)
    WB1.Visible = True
    WB2.Visible = True
    BB1.Visible = True
    BB2.Visible = True
    Application.DoEvents()
    Thread.Sleep(150)
    WN1.Visible = True
    WN2.Visible = True
    BN1.Visible = True

```



```

BN2.Visible = True
Application.DoEvents()
Thread.Sleep(150)
WR1.Visible = True
WR2.Visible = True
BR1.Visible = True
BR2.Visible = True
Application.DoEvents()
Thread.Sleep(150)
WP1.Visible = True
WP2.Visible = True
BP1.Visible = True
BP2.Visible = True
Application.DoEvents()
Thread.Sleep(150)
WP3.Visible = True
WP4.Visible = True
BP3.Visible = True
BP4.Visible = True
Application.DoEvents()
Thread.Sleep(150)
WP5.Visible = True
WP6.Visible = True
BP5.Visible = True
BP6.Visible = True
Application.DoEvents()
Thread.Sleep(150)
WP7.Visible = True
WP8.Visible = True
BP7.Visible = True
BP8.Visible = True
Application.DoEvents()
Thread.Sleep(150)

```

'Makes the labels & buttons visible, then audibly notifies the user that they can choose a menu option.

```

Title.Visible = True
OnePlayer.Visible = True
TwoPlayer.Visible = True
Analysis.Visible = True
ExitBtn.Visible = True
Credits.Visible = True
If Not AlreadyBooted Then Sound_Startup.Play()
End Sub

```

'Controls for the buttons.

```

Private Sub OnePlayer_MouseEnter() Handles OnePlayer.MouseEnter
    'Grows the Button.
    OnePlayer.Size = New Size(150, 75)
    OnePlayer.Left -= 15
    OnePlayer.Top -= 7.5
    OnePlayer.Font = New Font("Microsoft Sans Serif", 24, FontStyle.Bold)
End Sub
Private Sub OnePlayer_MouseLeave() Handles OnePlayer.MouseLeave
    'Shrinks the Button.
    OnePlayer.Size = New Size(120, 60)
    OnePlayer.Left += 15
    OnePlayer.Top += 7.5
    OnePlayer.Font = New Font("Microsoft Sans Serif", 18, FontStyle.Bold)

```

```

End Sub
Private Sub OnePlayer_Click() Handles OnePlayer.Click
    Dim Customisation As New OnePlayerCustomisation
    Customisation.Show()
    Me.Hide()
End Sub

Private Sub TwoPlayer_MouseEnter() Handles TwoPlayer.MouseEnter
    'Grows the Button.
    TwoPlayer.Size = New Size(150, 75)
    TwoPlayer.Left -= 15
    TwoPlayer.Top -= 7.5
    TwoPlayer.Font = New Font("Microsoft Sans Serif", 24, FontStyle.Bold)
End Sub
Private Sub TwoPlayer_MouseLeave() Handles TwoPlayer.MouseLeave
    'Shrinks the Button.
    TwoPlayer.Size = New Size(120, 60)
    TwoPlayer.Left += 15
    TwoPlayer.Top += 7.5
    TwoPlayer.Font = New Font("Microsoft Sans Serif", 18, FontStyle.Bold)
End Sub
Private Sub TwoPlayer_Click() Handles TwoPlayer.Click
    Dim Customisation As New TwoPlayerCustomisation
    Customisation.Show()
    Me.Hide()
End Sub

Private Sub Analysis_MouseEnter() Handles Analysis.MouseEnter
    'Grows the Button.
    Analysis.Size = New Size(150, 75)
    Analysis.Left -= 15
    Analysis.Top -= 7.5
    Analysis.Font = New Font("Microsoft Sans Serif", 24, FontStyle.Bold)
End Sub
Private Sub Analysis_MouseLeave() Handles Analysis.MouseLeave
    'Shrinks the Button.
    Analysis.Size = New Size(120, 60)
    Analysis.Left += 15
    Analysis.Top += 7.5
    Analysis.Font = New Font("Microsoft Sans Serif", 18, FontStyle.Bold)
End Sub
Private Sub Analysis_Click() Handles Analysis.Click
    Dim ChessGame As New Chess()
    ChessGame.Show()
    Me.Hide()
End Sub

Private Sub ExitBtn_MouseEnter() Handles ExitBtn.MouseEnter
    'Grows the Button.
    ExitBtn.Size = New Size(150, 75)
    ExitBtn.Left -= 15
    ExitBtn.Top -= 7.5
    ExitBtn.Font = New Font("Microsoft Sans Serif", 24, FontStyle.Bold)
End Sub
Private Sub ExitBtn_MouseLeave() Handles ExitBtn.MouseLeave
    'Shrinks the Button.
    ExitBtn.Size = New Size(120, 60)
    ExitBtn.Left += 15
    ExitBtn.Top += 7.5
    ExitBtn.Font = New Font("Microsoft Sans Serif", 18, FontStyle.Bold)

```

```

End Sub
Private Sub ExitBtn_Click() Handles ExitBtn.Click
    Me.Close()
End Sub

```

'Button that displays the credits information onto the screen (in the form of a pop-up).

```

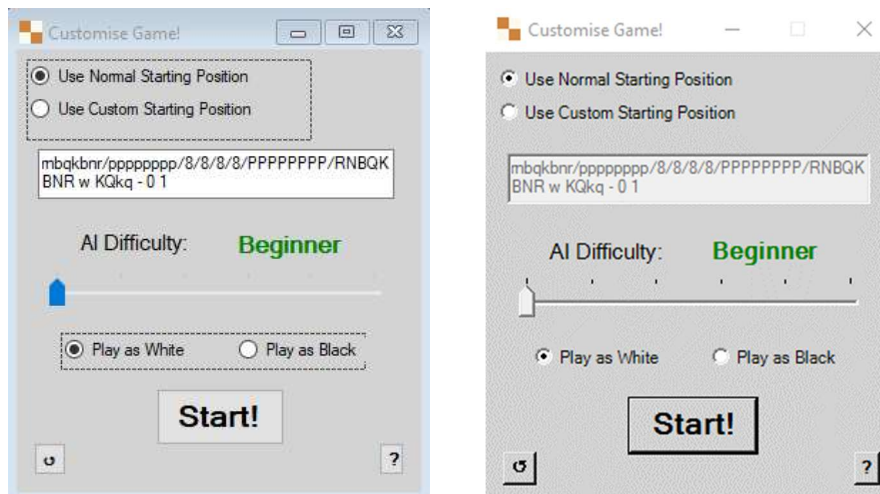
Private Sub Credits_Click() Handles Credits.Click
    MsgBox(Strings.StrDup(14, " ") & "Chess Game & Artificial Intelligence" & vbCrLf & Strings.StrDup(21, " ") & "Created by Alfie Kunz (8158)" & vbCrLf & Strings.StrDup(22, " ") & "of Beckfoot School (37101)" & vbCrLf & "Project used for the AQA GCE Computer Science NEA" & vbCrLf & Strings.StrDup(35, " ") & "(2021 - 2023)", vbInformation + vbApplicationModal, "Credits")
End Sub

```

```
End Class
```

CUSTOMISATION FORMS

ONEPLAYERCUSTOMISATION FORM



'Class that allows the user to customise their one-player game of chess.
'Called from the main menu, and is involved in instantiating the Chess form.

```

Public Class OnePlayerCustomisation
    Private Sub StartBtn_MouseEnter() Handles StartBtn.MouseEnter
        'Grows the Button.
        StartBtn.Size = New Size(112.5, 50)
        StartBtn.Left -= 11.25
        StartBtn.Top -= 5
        StartBtn.Font = New Font("Microsoft Sans Serif", 21, FontStyle.Bold)
    End Sub
    Private Sub StartBtn_MouseLeave() Handles StartBtn.MouseLeave
        'Shrinks the Button.
        StartBtn.Size = New Size(90, 40)
        StartBtn.Left += 11.25
        StartBtn.Top += 5
        StartBtn.Font = New Font("Microsoft Sans Serif", 16, FontStyle.Bold)
    End Sub
    Private Sub StartBtn_Click() Handles StartBtn.Click

```

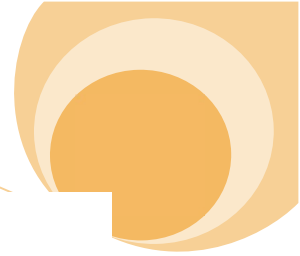
```

'Takes the user to their Chess game, which varies based on their preferences.
Dim UserStartingFEN As String
Dim PlayAsWhite As Boolean
If PosBtn2.Checked AndAlso FENTextBox.Text <> "" Then
    UserStartingFEN = FENTextBox.Text
Else
    UserStartingFEN = "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w
KQkq - 0 1"
End If
If White.Checked Then 'Will they play as white?
    PlayAsWhite = True
Else
    PlayAsWhite = False
End If
'Instantiates a new game of Chess.
Dim ChessGame As New Chess(1, UserStartingFEN, DifficultySlider.Value
, PlayAsWhite)
ChessGame.Show()
End Sub

'Code for Radio buttons.
Private Sub PosBtn1_Checked() Handles PosBtn1.CheckedChanged
    FENTextBox.Enabled = False
End Sub
Private Sub PosBtn2_Checked() Handles PosBtn2.CheckedChanged
    FENTextBox.Enabled = True
End Sub

'Button that displays information regarding the various AI difficulties.
Private Sub Button1_Click() Handles InfoBtn.Click
    MsgBox("Beginner: 1s per search, Quiescence off." & vbCrLf & "Easy: 3
s per search, Quiescence off." & vbCrLf & "Medium: 5s per search, Quiescence
off." & vbCrLf & "Hard: 5s per search, Quiescence on." & vbCrLf & "Expert: 10
s per search, Quiescence on." & vbCrLf & "Pain: 20s per search, Quiescence on
.", vbInformation + vbApplicationModal, "AI Difficulty Information")
End Sub
'Button that takes the user back to the main menu.
Private Sub BackBtn_Click() Handles BackBtn.Click
    'Locates MainMenu Form.
    Dim Menu As Form = CType(Application.OpenForms("MainMenu"), MainMenu)
    Me.Close()
    Menu.Show()
End Sub

'Controls difficulty slider (as well as changing the cursor).
Private Sub DifficultySlider_MouseDown() Handles DifficultySlider.MouseDown
    Me.Cursor = New Cursor(New System.IO.MemoryStream(My.Resources.Cursor
16))
End Sub
Private Sub DifficultySlider_MouseUp() Handles DifficultySlider.MouseUp
    Me.Cursor = Cursors.Default
End Sub
Private Sub DifficultySlider_ValueChanged() Handles DifficultySlider.Value
eChanged
    If DifficultySlider.Value = 1 Then
        AIDiffLabel2.Text = "Beginner"
        AIDiffLabel2.ForeColor = Color.Green
    
```

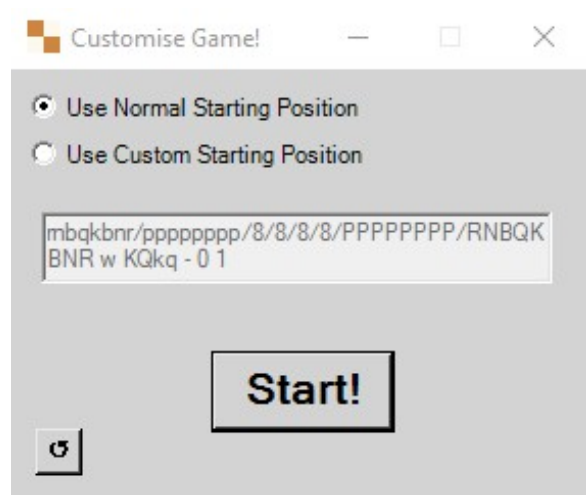
```

ElseIf DifficultySlider.Value = 2 Then
    AIDiffLabel2.Text = "Easy"
    AIDiffLabel2.ForeColor = Color.Goldenrod
ElseIf DifficultySlider.Value = 3 Then
    AIDiffLabel2.Text = "Medium"
    AIDiffLabel2.ForeColor = Color.DarkOrange
ElseIf DifficultySlider.Value = 4 Then
    AIDiffLabel2.Text = "Hard"
    AIDiffLabel2.ForeColor = Color.Red
ElseIf DifficultySlider.Value = 5 Then
    AIDiffLabel2.Text = "Expert"
    AIDiffLabel2.ForeColor = Color.DarkRed
Else
    AIDiffLabel2.Text = "Pain"
    AIDiffLabel2.ForeColor = Color.Black
End If
End Sub

End Class

```

TWOPLAYERCUSTOMISATION FORM



'Class that allows the user to customise their two-player game of chess.
'Called from the main menu, and is involved in instantiating the Chess form.

```

Public Class TwoPlayerCustomisation
    Private Sub StartBtn_MouseEnter() Handles StartBtn.MouseEnter
        'Grows the Button.
        StartBtn.Size = New Size(112.5, 50)
        StartBtn.Left -= 11.25
        StartBtn.Top -= 5
        StartBtn.Font = New Font("Microsoft Sans Serif", 21, FontStyle.Bold)
    End Sub
    Private Sub StartBtn_MouseLeave() Handles StartBtn.MouseLeave
        'Shrinks the Button.
        StartBtn.Size = New Size(90, 40)
        StartBtn.Left += 11.25
        StartBtn.Top += 5
        StartBtn.Font = New Font("Microsoft Sans Serif", 16, FontStyle.Bold)
    End Sub
    Private Sub StartBtn_Click() Handles StartBtn.Click

```

'Takes the user to their Chess game, which varies based on their preferences.

```
Dim UserStartingFEN As String
If PosBtn2.Checked AndAlso FENTextBox.Text <> "" Then
    UserStartingFEN = FENTextBox.Text
Else
    UserStartingFEN = "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w
KQkq - 0 1"
End If
'Instantiates a new game of Chess.
Dim ChessGame As New Chess(2, UserStartingFEN, 0, True)
ChessGame.Show()
End Sub
```

'Code for Radio buttons.

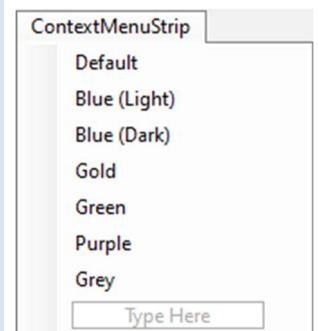
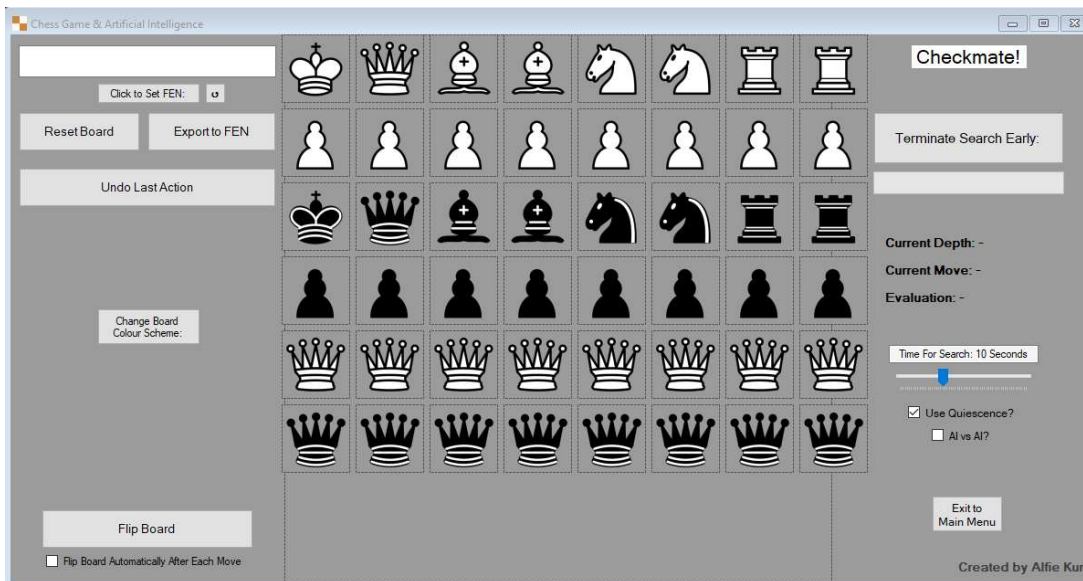
```
Private Sub PosBtn1_Checked() Handles PosBtn1.CheckedChanged
    FENTextBox.Enabled = False
End Sub
Private Sub PosBtn2_Checked() Handles PosBtn2.CheckedChanged
    FENTextBox.Enabled = True
End Sub
```

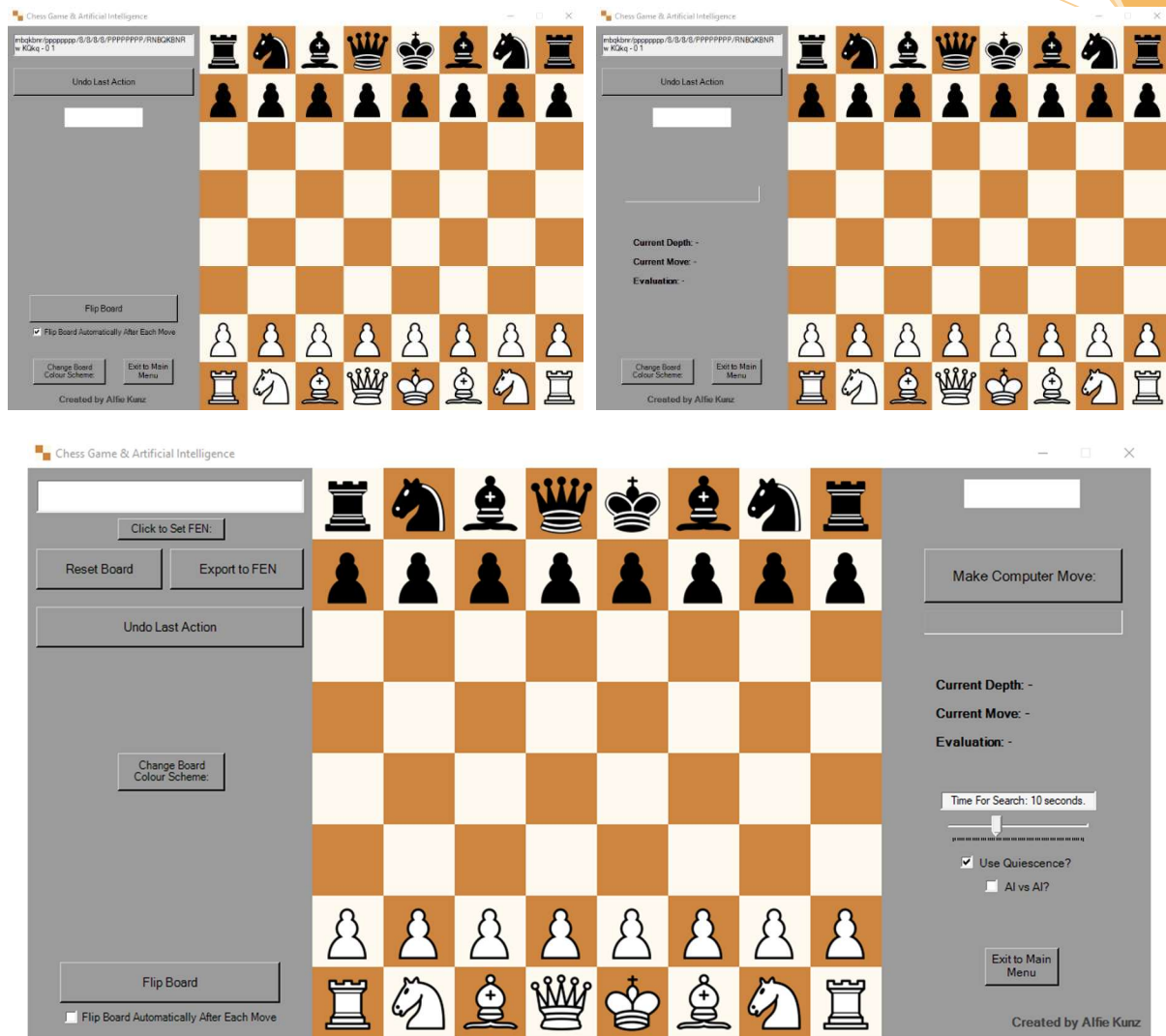
'Button that takes the user back to the main menu.

```
Private Sub BackBtn_Click() Handles BackBtn.Click
    'Locates MainMenu Form.
    Dim Menu As Form = CType(Application.OpenForms("MainMenu"), MainMenu)
    Me.Close()
    Menu.Show()
End Sub
```

End Class

CHESS FORM





'Working Version of Chess, along with AI using MiniMax and Alpha Beta Pruning . Created by Alfie Kunz - 8158.

Imports System.Threading

'This class forms the main User Interface and the basic game of chess, along with its rules. It will control moving pieces, 'importing / exporting positions, manipulating the board + its visual design ext, along with converting a user's actions onto 'a computer-friendly interface (for the AI to use). This also instantiates the AI class and interacts with it.

Public Class Chess

Structure PieceInfo 'Contains information about a piece that the user is moving on the GUI

Dim IsMovingPiece As Boolean 'is the piece being moved by the user?

Dim LegalMoves() As String 'Array containing the legal Moves of the piece being dragged.

Dim StartPoint As Point

Dim MidPoint As Point

Dim EndPoint As Point

End Structure

Private Shared Algorithms As New CoreMethods 'Calls all objects from the CoreMethods class.

```

'Here are the computer's primary "eyes" - MasterBoard (which contains each piece's location on the board) and,
'derived from this, Master TrueFalse Tables for each player. These Tables display information such as Legal
'Moves for the players' kings, along with pinned pieces and their location, and make handling Move Generation,
'Checks and Evaluations much easier and more efficient.
Private MasterBoard(7, 7) As Char
Private MasterWhiteTFTable(7, 7), MasterBlackTFTable(7, 7) As Char
Private GameRunning As Boolean = True
Private ReadOnly GameMode As Byte = 3 '1 = 1-Player, 2 = 2-Player, 3 = Analysis

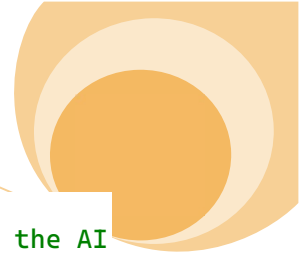
'FEN = Standard Chess notation that displays where all the pieces on the board are supposed to go.
Private StartingFEN As String = "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1"
Private CurrentFEN, PreviousFEN, InvalidFEN As String
Private PlayerTurn As Boolean = True 'True = Is White's Turn, False = Is Black's Turn
Private UserPlayer As Boolean = True 'Represents what colour the user is playing with.
Private MasterWKPos, MasterBKPos As String 'Location of Kings.
Private MasterEnPassant As String 'Location of EnPassant Square

'Data for castling & checks for each player.
Private MasterWCanCastle As New CanCastle
Private MasterBCanCastle As New CanCastle
Private MasterWInCheck As New InCheck
Private MasterBInCheck As New InCheck

Private ReadOnly PieceArray(47) As PictureBox 'Contains all the PictureBoxes on the board.
Private PieceMoving As PieceInfo
Private PrimaryColour As Color 'Colour representing the dark squares on the Chessboard.
Private SecondaryColour As Color 'Colour representing the light squares on the Chessboard.
Private SquareHistory(3, 1) As SByte 'Array containing the previously used squares on the board (for prev moves).
Private LegalMoveSquares(7, 7) As Boolean 'Array containing the coordinates of where a piece can move on the board,
'... generated when the user clicks on a piece. Used when redrawing the checkerboard pattern.
Private OrientForWhite As Boolean = True 'Represents which way the board is flipped.

'Data structures that bridge the gap between user and AI.
Private AbsoluteDepth As SByte 'Estimate of the optimal depth used for searching.
Private TimeForSearch As Decimal = 10 'Time the AIs are allowed to search for.
Private CurrentDepth As Byte 'Highest depth successfully searched to.
Private CurrentMove As String 'Move of the highest depth successfully searched to.
Private CurrentEvaluation As String 'Eval of the highest depth successfully searched to.
Private TerminateSearch As Boolean 'Set to True if the user preemptively aborts the AI search.

```



```

Private ComputerIsSearching As Boolean = False
Private UseQuiescence As Boolean = True 'Determines whether or not the AI
will use Quiescence in its searching.
'Please see my project report for information about Quiescence.

'Below are the sets of AIs (that will all run simultaneously), along with
their corresponding Move Outputs.
'Also contains data such as which AIs have moved to a higher depth, and w
hich have finished their search
'(so that GUI controls can be updated).
Private MainAI As New AI()
Private AI2 As New AI()
Private AI3 As New AI()
Private AI4 As New AI()
Private AI5 As New AI()
Private AIBestMoves(6) As Move
Private AIMovedToHigherDepth(1) As Boolean
Private AIFinishedSearch As Boolean
Private AISTopwatch As New Stopwatch 'For timing AI searches.

'Sets up sound effects. All sounds used by Chess.com
Private ReadOnly Sound_Move As New Media.SoundPlayer With {
    .SoundLocation = Application.StartupPath & "\Sounds\Chess_Move.wav"
}
Private ReadOnly Sound_Capture As New Media.SoundPlayer With {
    .SoundLocation = Application.StartupPath & "\Sounds\Chess_Capture.wav"
}
Private ReadOnly Sound_Check As New Media.SoundPlayer With {
    .SoundLocation = Application.StartupPath & "\Sounds\Chess_Check.wav"
}
Private ReadOnly Sound_Castle As New Media.SoundPlayer With {
    .SoundLocation = Application.StartupPath & "\Sounds\Chess_Castle.wav"
}
Private ReadOnly Sound_Checkmate As New Media.SoundPlayer With {
    .SoundLocation = Application.StartupPath & "\Sounds\Chess_Checkmate.w
av"
}
Private ReadOnly Sound_Stalemate As New Media.SoundPlayer With {
    .SoundLocation = Application.StartupPath & "\Sounds\Chess_Stalemate.w
av"
}

```

CONSTRUCTURES & BOARD CREATORS

```

Public Sub New() 'Call used for a standard game of chess (usually in Analysis
Mode).
    InitializeComponent() ' This call is required by the designer.
    StartingFEN = "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0
1"
End Sub

'The below constructor method rearranges the location of objects on the fo
rm (to save space on the screen).
'This will be called for 1-Player Games and 2-Player Games.
Public Sub New(ByVal Mode As Byte, ByVal UserStartingFEN As String, ByVal
Difficulty As Byte, ByVal PlayAsWhite As Boolean)
    InitializeComponent() ' This call is required by the designer.
    GameMode = Mode

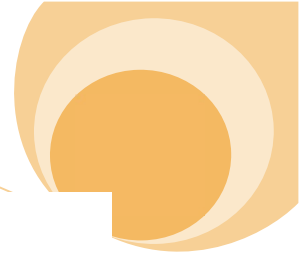
```



```

'Edits the size of the forms, and modifies the object now off-screen.
Me.Size = New Size(916, 639)
AIMoveBtn.Visible = False
FENTextBox.ReadOnly = True
Reset_Btn.Visible = False
FENButton.Visible = False
FENExport.Visible = False
UndoMove.Top -= 80
CheckLabel.Location = New Point(89, 128)
UserTimeBar.Visible = False
UserTimeBox.Visible = False
QuiescenceBox.Visible = False
AIEndlessMode.Visible = False
ColourChangerButton.Location = New Point(40, 520)
ExitBtn.Location = New Point(181, 520)
Credits.Location = New Point(78.5, 575)
StartingFEN = UserStartingFEN
If Mode = 1 Then 'Mode-Specific Movement.
    'Hides / sets appropriate objects.
    FlipperButton.Visible = False
    AutoFlipper.Visible = False
    ProgressBar.Location = New Point(46, 250)
    CurrentAIDepth.Location = New Point(ProgressBar.Location.X + 10, P
rogressBar.Location.Y + 80)
    CurrentAIMove.Location = New Point(CurrentAIDepth.Location.X, Curr
entAIDepth.Location.Y + 30)
    CurrentAIEval.Location = New Point(CurrentAIDepth.Location.X, Curr
entAIDepth.Location.Y + 60)
    UserPlayer = PlayAsWhite
    'Difficulty generator.
    If Difficulty = 1 Then
        TimeForSearch = 1
        UseQuiescence = False
    ElseIf Difficulty = 2 Then
        TimeForSearch = 3
        UseQuiescence = False
    ElseIf Difficulty = 3 Then
        TimeForSearch = 5
        UseQuiescence = False
    ElseIf Difficulty = 4 Then
        TimeForSearch = 5
        UseQuiescence = True
    ElseIf Difficulty = 5 Then
        TimeForSearch = 10
        UseQuiescence = True
    Else
        TimeForSearch = 20
        UseQuiescence = True
    End If
Else
    FlipperButton.Top -= 100
    AutoFlipper.Top -= 100
    'Hides / sets appropriate objects.
    ProgressBar.Visible = False
    CurrentAIEval.Visible = False
    AutoFlipper.Checked = True
    CurrentAIDepth.Visible = False
    CurrentAIMove.Visible = False
    CurrentAIEval.Visible = False
End If

```



End Sub

```

Private Sub Form_Load() Handles Me.Load
    Application.EnableVisualStyles()
    Dim StartupStopwatch As New Stopwatch
    StartupStopwatch.Start()
    'Sets the colour scheme for the board (from txt file). If all else fails, use the default settings.
    Dim ColourScheme As String
    Try
        FileOpen(1, Application.StartupPath & "\ColourPreferences.txt", OpenMode.Input)
        ColourScheme = LineInput(1)
    Catch ex As Exception
        ColourScheme = "def"
    End Try
    FileClose(1)
    If LCase(ColourScheme) = "blu" Then
        PrimaryColour = Color.LightSteelBlue
        SecondaryColour = Color.AliceBlue
    ElseIf LCase(ColourScheme) = "bl2" Then
        PrimaryColour = Color.SlateGray
        SecondaryColour = Color.Silver
    ElseIf LCase(ColourScheme) = "gld" Then
        PrimaryColour = Color.Goldenrod
        SecondaryColour = Color.LemonChiffon
    ElseIf LCase(ColourScheme) = "grn" Then
        PrimaryColour = Color.DarkSeaGreen
        SecondaryColour = Color.LavenderBlush
    ElseIf LCase(ColourScheme) = "ppl" Then
        PrimaryColour = Color.MediumPurple
        SecondaryColour = Color.MistyRose
    ElseIf LCase(ColourScheme) = "mon" Then
        PrimaryColour = Color.DimGray
        SecondaryColour = Color.Silver
    Else 'def
        PrimaryColour = Color.Peru
        SecondaryColour = Color.FloralWhite
    End If

    'Assigns each piece to PieceArray and sets the 'board' picturebox as their parent.
    PieceArray(0) = WK1
    PieceArray(1) = WQ1
    PieceArray(2) = WB1
    PieceArray(3) = WB2
    PieceArray(4) = WN1
    PieceArray(5) = WN2
    PieceArray(6) = WR1
    PieceArray(7) = WR2
    PieceArray(8) = WP1
    PieceArray(9) = WP2
    PieceArray(10) = WP3
    PieceArray(11) = WP4
    PieceArray(12) = WP5
    PieceArray(13) = WP6
    PieceArray(14) = WP7
    PieceArray(15) = WP8
    PieceArray(16) = BK1

```

```

PieceArray(17) = BQ1
PieceArray(18) = BB1
PieceArray(19) = BB2
PieceArray(20) = BN1
PieceArray(21) = BN2
PieceArray(22) = BR1
PieceArray(23) = BR2
PieceArray(24) = BP1
PieceArray(25) = BP2
PieceArray(26) = BP3
PieceArray(27) = BP4
PieceArray(28) = BP5
PieceArray(29) = BP6
PieceArray(30) = BP7
PieceArray(31) = BP8
PieceArray(32) = WQ2
PieceArray(33) = WQ3
PieceArray(34) = WQ4
PieceArray(35) = WQ5
PieceArray(36) = WQ6
PieceArray(37) = WQ7
PieceArray(38) = WQ8
PieceArray(39) = WQ9
PieceArray(40) = BQ2
PieceArray(41) = BQ3
PieceArray(42) = BQ4
PieceArray(43) = BQ5
PieceArray(44) = BQ6
PieceArray(45) = BQ7
PieceArray(46) = BQ8
PieceArray(47) = BQ9
For x = 0 To 47
    Checkerboard.Controls.Add(PieceArray(x))
Next

```

'Assigns all the PictureBoxes with their associated images.
 'All images are taken from the 'cburnett' piece theme of Lichess.org – an open source, charity chess website.
 'Please see here for information about using these assets: <https://lichess.org/contact#help-authorize>.

```

Dim Colour As String = "W"
For x = 0 To 16 Step 16
    PieceArray(x).Image = Image.FromFile(Application.StartupPath & "\Images\Default\" & Colour & "King.png")
    PieceArray(x + 1).Image = Image.FromFile(Application.StartupPath & "\Images\Default\" & Colour & "Queen.png")
    For n = x + 2 To x + 3
        PieceArray(n).Image = Image.FromFile(Application.StartupPath & "\Images\Default\" & Colour & "Bishop.png")
    Next
    For n = x + 4 To x + 5
        PieceArray(n).Image = Image.FromFile(Application.StartupPath & "\Images\Default\" & Colour & "Knight.png")
    Next
    For n = x + 6 To x + 7
        PieceArray(n).Image = Image.FromFile(Application.StartupPath & "\Images\Default\" & Colour & "Rook.png")
    Next
    For n = x + 8 To x + 15

```



```

        PieceArray(n).Image = Image.FromFile(Application.StartupPath &
"\Images\Default\" & Colour & "Pawn.png")
    Next
    Colour = "B"
Next
For x = 32 To 39
    PieceArray(x).Image = Image.FromFile(Application.StartupPath & "\I
mages\Default\WQueen.png")
Next
For x = 40 To 47
    PieceArray(x).Image = Image.FromFile(Application.StartupPath & "\I
mages\Default\BQueen.png")
Next

'Sets up more objects...
UndoFENChange.Visible = False
CheckLabel.Text = "
For n = 0 To 3
    SquareHistory(n, 0) = -1
    SquareHistory(n, 1) = -1
Next

'If the FEN is valid, we send the start postion to be converted to the
Board array, then display the pieces on the board.
'If it is not valid, we replace the FEN with the Default StartingFEN.
Dim FENErrorMessage As String = ""
Dim FENIsValid As Boolean = False
Dim CustomisationForm As Form = Nothing 'We first locate the cusomisa
tion form (if the user used it).
If GameMode = 1 Then
    CustomisationForm = CType(Application.OpenForms("OnePlayerCustomis
ation"), OnePlayerCustomisation)
ElseIf GameMode = 2 Then
    CustomisationForm = CType(Application.OpenForms("TwoPlayerCustomis
ation"), TwoPlayerCustomisation)
End If
If FENErrorDetection(StartingFEN, False, FENErrorMessage) Then 'First
step of error-detection.
    Try
        'Forms the chessboard, along with the required variables.
        MasterBoard = SharedAlgorithms.FENConverter(StartingFEN, Maste
rWCanCastle, MasterBCanCastle, MasterWKPos, MasterBKPos, MasterEnPassant, Pla
yerTurn)
        DisplayPieces()
        'Forms the TFTTable for the players.
        SharedAlgorithms.FixTFTables(MasterBoard, True, MasterWhiteTFT
able, MasterWKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
        SharedAlgorithms.FixTFTables(MasterBoard, False, MasterBlackTF
Table, MasterBKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
        If GameMode < 3 Then CustomisationForm.Close() 'Closes the cus
tomisation form.
        CheckChecker()
        FENIsValid = True
    Catch ex As Exception 'FEN is not valid
        If MsgBox("Starting Position Rejected - Invalid FEN Code. Plea
se Input a Genuine FEN and try again." & vbCrLf & "Please Click 'Retry' to en
ter another FEN, or Click 'Cancel' to Start the Game with the Standard Starti
ng Position.", vbCritical + vbRetryCancel + vbApplicationModal) = 4 Then
            'The user wants to enter another FEN.

```

```

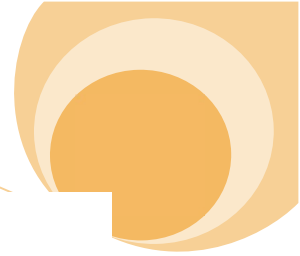
        Me.Close()
    End If
End Try
Else
    'Displays the appropriate error message, as stated by the FENError
Detection Sub.
    If MsgBox("Starting " & FENErrorMessage & vbCrLf & "Please Click 'Re
try' to Enter Another FEN, or Click 'Cancel' to Start the Game with the Stand
ard Starting Position.", vbCritical + vbRetryCancel + vbApplicationModal) = 4
    Then
        'The user wants to enter another FEN.
        Me.Close()
    End If
End If

If Not FENIsValid AndAlso Application.OpenForms().OfType(Of Chess).Any
Then 'If customisation form hasn't been closed.
    'Sets the board to the standard FEN postion, and forms variables &
objects.
    CustomisationForm.Close()
    StartingFEN = "rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq
- 0 1"
    MasterBoard = SharedAlgorithms.FENConverter(StartingFEN, MasterWCa
nCastle, MasterBCanCastle, MasterWKPos, MasterBKPos, MasterEnPassant, PlayerT
urn)
    DisplayPieces()
    SharedAlgorithms.FixTFTables(MasterBoard, True, MasterWhiteTFTTable
, MasterWKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
    Array.Copy(SharedAlgorithms.MasterTrueTable, MasterBlackTFTTable, 6
4)
End If

If Me.IsHandleCreated Then 'If this form is open...
    If Not UserPlayer Then FlipBoard() 'Flips board in user's favour.
    If MasterWInCheck.IsInCheck AndAlso Not PlayerTurn Then
        'If White is in check, and it is black to move, then black can
take white's king. This is illegal.
        GameRunning = False
        Console.WriteLine("The Game has Ended. Cause = Invalid Check."
)
    ElseIf MasterBInCheck.IsInCheck AndAlso PlayerTurn Then
        'If Black is in check, and it is white to move, then white can
take black's king. This is illegal.
        Console.WriteLine("The Game has Ended. Cause = Invalid Check."
)
        GameRunning = False
    ElseIf Math.Abs(CStr(MasterWKPos(0)) - CStr(MasterBKPos(0))) <= 1
AndAlso Math.Abs(CStr(MasterWKPos(1)) - CStr(MasterBKPos(1))) <= 1 Then
        'Kings are too close together.
        GameRunning = False
        Console.WriteLine("The Game has Ended. Cause = Invalid King Po
sitions.")
        CheckLabel.Text = " Stalemate! "
        Sound_Stalemate.Play()
    Else
        Console.WriteLine("The Game has Begun.")
    End If

    'Creates our AI (for legal move generation).
    CurrentFEN = StartingFEN

```

```

MainAI.Reconfigure(CurrentFEN)

StartupStopwatch.Stop()
Console.WriteLine("Startup Time: " & StartupStopwatch.Elapsed.Total
lMilliseconds & " Milliseconds." & vbCrLf)
StartupStopwatch.Reset()

If GameMode < 3 Then
    If GameMode = 1 AndAlso Not (UserPlayer = PlayerTurn) Then
        'The AI needs to make the first move.
        InstantiateAIs()
        PreviousFEN = CurrentFEN
    Else 'Prevents the user from undoing the starting move.
        PreviousFEN = StartingFEN
    End If
    FENTextBox.Text = StartingFEN
    FENTextBox.Select(FENTextBox.Text.Length, 0) 'Stops the TextBo
x from being highlighted.
Else
    PreviousFEN = StartingFEN 'if CurrentFEN = PreviousFEN then th
e user cannot undo a move.
End If
EnforceEndStates() 'Checks for end positions in the starting FEN.
End If
End Sub

'Subroutine that creates checkerboard pattern. (Alternates between light a
nd dark colours,
'and includes controls for Previously Used Squares, Legal Move Squares, an
d more.)
Private Sub CreateBoard(ByVal sender As System.Object, ByVal e As System.W
indows.Forms.PaintEventArgs) Handles Checkerboard.Paint
    Dim g As Graphics = e.Graphics
    Dim isLight As Boolean = True
    For x = 0 To 7
        For y = 0 To 7
            Dim Square As New Rectangle(75 * x, 75 * y, 75, 75)
            'If the coordinates are set to True in the LegalMoveSquare, th
en that square should be coloured differently, so
            'that the user can see legal moves. Square given either a gree
n or blue outline (depending on the colour scheme).
            If LegalMoveSquares(x, y) Then
                If PrimaryColour = Color.DarkSeaGreen Then 'is green colou
r scheme - colour blue.
                    Using Brush As New SolidBrush(Color.DarkTurquoise)
                        g.FillRectangle(Brush, Square)
                    End Using
                Else 'is another colour scheme - colour green.
                    Using Brush As New SolidBrush(Color.LimeGreen)
                        g.FillRectangle(Brush, Square)
                    End Using
                End If
            End If
            If isLight Then
                If LegalMoveSquares(x, y) Then 'Normal colour is filled at
the centre of the legal move square to produce a green / blue highlight.
                    Dim Square2 As New Rectangle((75 * x) + 5, (75 * y) +
5, 65, 65)
                    Using Brush As New SolidBrush(SecondaryColour)

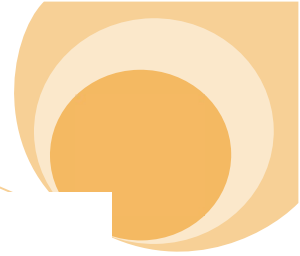
```

```

        g.FillRectangle(Brush, Square2)
    End Using
    'If the given square matches a SquareHistory coordinate, then it is coloured in a different way.
    ElseIf (x = SquareHistory(0, 0) AndAlso y = SquareHistory(0, 1)) OrElse (x = SquareHistory(1, 0) AndAlso y = SquareHistory(1, 1)) Then
        Using Brush As New SolidBrush(Color.YellowGreen) 'SquareHistory secondary colour.
            g.FillRectangle(Brush, Square)
        End Using
    Else 'Is a normal square - colour with user's secondary colour.
        Using Brush As New SolidBrush(SecondaryColour)
            g.FillRectangle(Brush, Square)
        End Using
    End If
Else 'Identical code but for the dark squares (uses different colours).
    If LegalMoveSquares(x, y) Then
        Dim Square2 As New Rectangle((75 * x) + 5, (75 * y) + 5, 65, 65)
        Using Brush As New SolidBrush(PrimaryColour)
            g.FillRectangle(Brush, Square2)
        End Using
        ElseIf (x = SquareHistory(0, 0) AndAlso y = SquareHistory(0, 1)) OrElse (x = SquareHistory(1, 0) AndAlso y = SquareHistory(1, 1)) Then
            Using Brush As New SolidBrush(Color.OliveDrab) 'SquareHistory primary colour.
                g.FillRectangle(Brush, Square)
            End Using
        Else
            Using Brush As New SolidBrush(PrimaryColour) 'User's primary colour.
                g.FillRectangle(Brush, Square)
            End Using
        End If
    End If
    isLight = Not isLight 'Produces alternating pattern.
Next
isLight = Not isLight
Next
'Gives the starting square for a piece a distinctive colour, when the user clicks on one.
If PieceMoving.IsMovingPiece Then 'the piece is being moved by the user.
    Dim Square As New Rectangle(PieceMoving.StartPoint.X, PieceMoving.StartPoint.Y, 75, 75)
    Using Brush As New SolidBrush(Color.LightCoral)
        g.FillRectangle(Brush, Square)
    End Using
End If
End Sub

'Subroutine that adjusts the values on the LegalMoveSquares array when a given move is being made
'(for illustration purposes when the AI is deciding between moves).
Private Sub AmendLegalMoveSquares(ByVal TempMove As Move)
    'Resets LMS array.
    For y = 0 To 7
        For x = 0 To 7

```



```

        LegalMoveSquares(x, y) = False
    Next
Next
'Sets specific squares on the array (start & end square of the AI's move).
If OrientForWhite Then
    LegalMoveSquares(TempMove.OldMoveX, TempMove.OldMoveY) = True
    LegalMoveSquares(TempMove.NewMoveX, TempMove.NewMoveY) = True
Else
    LegalMoveSquares(7 - TempMove.OldMoveX, 7 - TempMove.OldMoveY) = True
    LegalMoveSquares(7 - TempMove.NewMoveX, 7 - TempMove.NewMoveY) = True
End If
End Sub

'Algorithm that displays all the pieces on the board, given a board array.
Private Sub DisplayPieces()
    Dim CurrentPiece As String
    Dim TempPiece As PictureBox
    Dim Counter As Byte
    'Hides all pieces.
    For x = 0 To 47
        PieceArray(x).Location = New Point(-100, -100)
        PieceArray(x).Visible = False
    Next
    For y = 7 To 0 Step -1
        For x = 0 To 7
            'For each piece on the board...
            If MasterBoard(x, y) <> " " Then
                Counter = 1
                While True
                    'In MasterBoard(), a lowercase letter represents a black piece, and an uppercase letter represents a white piece.
                    If Char.IsLower(MasterBoard(x, y)) Then
                        CurrentPiece = "B"
                    Else
                        CurrentPiece = "W"
                    End If
                    'CurrentPiece is formed, which has the same name as a corresponding PictureBox on the board.
                    CurrentPiece &= UCase(MasterBoard(x, y)) & CStr(Counter)

                    'Searches for piece that has not been placed on the board (that matches CurrentPiece / TempPiece's name), and moves its match into place on the board.
                    TempPiece = Me.Controls.Find(CurrentPiece, True).Single()

                    If TempPiece.Visible = False Then
                        If OrientForWhite Then
                            TempPiece.Location = New Point(((x) * 75), ((7 - y) * 75))
                        Else 'Location flipped 180 degrees.
                            TempPiece.Location = New Point(((7 - x) * 75), ((75 * (7 - y))))
                        End If
                        TempPiece.Visible = True
                    End If
                End While
            End If
        Next x
    Next y
End Sub

```

```

        Exit While
    End If
    'If not found, the search continues with an increasing
    last digit (eg: the program seaches for BP1, then BP2, then BP3 ext until it
    finds its match.)

```

```

        Counter += 1
    End While
End If
Next
Next
End Sub

```

'Outputs the board position, along with the TFTables & King positions (to the console).

```

'Used for debugging purposes.
Private Sub OutputDebugInfo()
    Console.WriteLine(vbCrLf & "Board Position:")
    For y = 0 To 7
        For x = 0 To 7
            Console.Write(MasterBoard(x, y))
        Next
        Console.WriteLine()
    Next
    Console.WriteLine()
    Console.WriteLine("WhiteTFTable:")
    For y = 0 To 7
        For x = 0 To 7
            Console.Write(MasterWhiteTFTTable(x, y))
        Next
        Console.WriteLine()
    Next
    Console.WriteLine("BlackTFTable:")
    For y = 0 To 7
        For x = 0 To 7
            Console.Write(MasterBlackTFTTable(x, y))
        Next
        Console.WriteLine()
    Next
    Console.WriteLine("WKPos: " & MasterWKPos & ". BKPos: " & MasterBKPos)
End Sub

```

'Alternate version of DisplayPieces which is more efficient and only converts a single piece in

'MasterBoard to its PictureBox counterpart on the board.

```

Private Function CoorToPieceConverter(ByVal CoorX As Int16, ByVal CoorY As Int16) As PictureBox
    Dim CurrentPiece As String
    Dim TempPiece As PictureBox = Nothing
    Dim Counter As Byte
    If MasterBoard(CoorX, CoorY) <> " " Then
        Counter = 1
        While True
            'In MasterBoard(), a lowercase letter represents a black piece
            , and an uppercase letter represents a white piece.
            If Char.IsLower(MasterBoard(CoorX, CoorY)) Then
                CurrentPiece = "B"
            Else
                CurrentPiece = "W"
            End If

```



```

'CurrentPiece is formed, which has the same name as a correspo
nding PictureBox on the board.
CurrentPiece &= UCase(MasterBoard(CoorX, CoorY)) & CStr(Counte
r)

'Searches for piece that has not been placed on the board (tha
t matches CurrentPiece / TempPiece's name + location), and moves its match in
to place on the board.
TempPiece = Me.Controls.Find(CurrentPiece, True).Single()
If OrientForWhite AndAlso (CInt(TempPiece.Location.X) = CoorX
* 75 AndAlso CInt(TempPiece.Location.Y) = CoorY * 75) Then
    Exit While
ElseIf Not OrientForWhite AndAlso (CInt(TempPiece.Location.X)
= (7 - CoorX) * 75 AndAlso CInt(TempPiece.Location.Y) = (7 - CoorY) * 75) The
n
    Exit While
End If
'If not found, the search continues with an increasing last di
git (eg: the program seaches for BP1, then BP2, then BP3 ext until it finds i
ts match.)
Counter += 1
End While
End If
Return TempPiece
End Function

```

GUI OBJECT CONTROLS

'Subroutine that detects some errors in a user-
entered FEN string, given the below rules:

```

Private Function FENErrorDetection(ByVal FEN As String, ByVal OutputToBox
As Boolean, ByRef FENErrorMessage As String) As Boolean
    'If OutputToBox = True then the error message is stored in FENTextBox
    . If it is false then the message is stored in FENErrorMessage.
    Dim MaxWQueens As Byte = 0
    Dim MaxBQueens As Byte = 0
    Dim WKingCount As Byte = 0
    Dim BKingCount As Byte = 0
    If FEN.Length > 22 AndAlso FEN.Length < 88 Then 'Length validation
        'Checks if each player has a king, and if they don't have too man
y queens & pawns (that could promote to a queen) - creating too
many queens on the board and therefore potentially overloading t
he PictureBox system.
        For n = 0 To Len(FEN) - 1
            If FEN(n) = " " Then Exit For 'Early exit clause.
            If FEN(n) = "P" OrElse FEN(n) = "Q" Then
                MaxWQueens += 1
            ElseIf FEN(n) = "p" OrElse FEN(n) = "q" Then
                MaxBQueens += 1
            ElseIf FEN(n) = "K" Then
                WKingCount += 1
            ElseIf FEN(n) = "k" Then
                BKingCount += 1
            End If
        Next
        'Displays appropriate error messages.
        If Not (WKingCount = 1 AndAlso BKingCount = 1) Then
            If OutputToBox Then
                InvalidFEN = FEN
            End If
        End If
    End Function

```



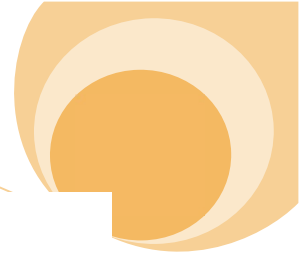
```

        FENTextBox.Text = "Position Rejected - Invalid number of
Kings. Please Input a Genuine FEN and try again."
        UndoFENChange.Visible = True
    Else 'Returns message as string.
        FENErrorMessage = "Position Rejected - No King."
    End If
    Return False
ElseIf MaxWQueens > 9 OrElse MaxBQueens > 9 Then
    If OutputToBox Then
        InvalidFEN = FEN
        FENTextBox.Text = "Position Rejected - Too many Pawns / Q
ueens. Please Input a Genuine FEN and try again."
        UndoFENChange.Visible = True
    Else
        FENErrorMessage = "Position Rejected - Too many Pawns / Q
ueens."
    End If
    Return False
End If
'Has passed all tests :).
Return True
Else
    If OutputToBox Then
        InvalidFEN = FEN
        FENTextBox.Text = "Position Rejected - Invalid FEN Length. Pl
ease Input a Genuine FEN and try again."
        UndoFENChange.Visible = True
    Else
        FENErrorMessage = "Position Rejected - Invalid FEN Length."
    End If
    Return False
End If
End Function

'Sends the FEN a user types in to be converted & displayed.
Private Sub FENButton_Click() Handles FENButton.Click
    If FENTextBox.Text <> "" AndAlso FENTextBox.Text <> CurrentFEN AndAls
o Not ComputerIsSearching Then
        'Checks for first form of validation (if the 2nd letter is "o", t
hen that usually implies that
        'the TextBox contains an error message.
        If FENTextBox.Text(1) <> "o" AndAlso FENErrorDetection(FENTextBox
.Text, True, "") Then
            If UndoFENChange.Visible = True Then UndoFENChange.Visible =
False

            'Resets Check and Castling Properties.
            MasterWInCheck.NotInCheck()
            MasterBInCheck.NotInCheck()
            Dim TempFEN As String = PreviousFEN
            'Try displaying the FEN on the board graphically. If this fai
ls, then the FEN is invalid.
            'In this case, the board is reset to the previous position.
            Try
                PreviousFEN = CurrentFEN
                CurrentFEN = FENTextBox.Text
                MasterBoard = SharedAlgorithms.FENConverter(FENTextBox.Te
xt, MasterWCanCastle, MasterBCanCastle, MasterWKPos, MasterBKPos, MasterEnPas
sant, PlayerTurn)
                DisplayPieces()
            Catch ex As Exception

```



```

InvalidFEN = FENTextBox.Text
CurrentFEN = PreviousFEN
PreviousFEN = TempFEN
MasterBoard = SharedAlgorithms.FENConverter(CurrentFEN, M
asterWCanCastle, MasterBCanCastle, MasterWKPos, MasterBKPos, MasterEnPassant,
PlayerTurn)

DisplayPieces()
UndoFENChange.Visible = True
FENTextBox.Text = "Position Rejected - Invalid FEN Code.
Please Input a Genuine FEN and try again."
End Try

If UndoFENChange.Visible = False Then 'Therefore FEN is valid
.
    Console.Clear()
    'Edits location of Previously Used Squares.
    SquareHistory(2, 0) = SquareHistory(0, 0)
    SquareHistory(2, 1) = SquareHistory(0, 1)
    SquareHistory(3, 0) = SquareHistory(1, 0)
    SquareHistory(3, 1) = SquareHistory(1, 1)
    SquareHistory(0, 0) = -1
    SquareHistory(0, 1) = -1
    SquareHistory(1, 0) = -1
    SquareHistory(1, 1) = -1
    If PlayerTurn Xor OrientForWhite Then
        FlipBoard()
    Else
        Checkerboard.Refresh()
    End If
    GameRunning = True
    'Resets TrueFalse Tables, then checks for Checks.
    SharedAlgorithms.FixTFTables(MasterBoard, True, MasterWhi
teTFTTable, MasterWKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
    SharedAlgorithms.FixTFTables(MasterBoard, False, MasterBl
ackTFTTable, MasterBKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
    Sound_Move.Play()
    CheckChecker()

    If MasterWInCheck.IsInCheck AndAlso Not PlayerTurn Then
        'If White is in check, and it is black to move, then
black can take white's king. This is illegal.
        GameRunning = False
        Console.WriteLine("The Game has Ended. Cause = Invali
d Check.")
    ElseIf MasterBInCheck.IsInCheck AndAlso PlayerTurn Then
        'If Black is in check, and it is white to move, then
white can take black's king. This is illegal.
        Console.WriteLine("The Game has Ended. Cause = Invali
d Check.")
        GameRunning = False
    ElseIf Math.Abs(CStr(MasterWKPos(0)) - CStr(MasterBKPos(0
))) <= 1 AndAlso Math.Abs(CStr(MasterWKPos(1)) - CStr(MasterBKPos(1))) <= 1 T
hen
        'Kings are too close together.
        GameRunning = False
        Console.WriteLine("The Game has Ended. Cause = Invali
d King Positions.")
        CheckLabel.Text = " Stalemate! "
        Sound_Stalemate.Play()
    End If

```

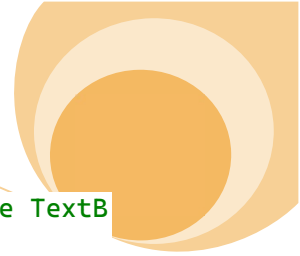
```

'Recalibrates AI and resets AI move info.
MainAI.Reconfigure(CurrentFEN)
CurrentDepth = 2
CurrentMove = "-"
CurrentEvaluation = "-"
CurrentAIDepth.Text = "Current Depth: -"
CurrentAIMove.Text = "Current Move: -"
CurrentAIEval.Text = "Evaluation: -"
EnforceEndStates()
End If
End If
End If
End Sub

'Button which resets the FEN in the FENTextBox, in case it is invalid.
Private Sub UndoFENChange_Click(sender As Object, e As EventArgs) Handles
UndoFENChange.Click
    FENTextBox.Text = InvalidFEN
    sender.Visible = False
End Sub

'Resets the board to the starting position and sets white to move.
Private Sub Reset_Btn_Click() Handles Reset_Btn.Click
    If Not ComputerIsSearching Then
        PreviousFEN = CurrentFEN
        CurrentFEN = StartingFEN
        'Resets Check Properties.
        MasterWInCheck.NotInCheck()
        MasterBInCheck.NotInCheck()
        'Can assume that the StartingFEN is valid, so we display it graph
ically.
        MasterBoard = SharedAlgorithms.FENConverter(CurrentFEN, MasterWCa
nCastle, MasterBCanCastle, MasterWKPos, MasterBKPos, MasterEnPassant, PlayerT
urn)
        DisplayPieces()
        'Edits location of Previously Used Squares.
        SquareHistory(2, 0) = SquareHistory(0, 0)
        SquareHistory(2, 1) = SquareHistory(0, 1)
        SquareHistory(3, 0) = SquareHistory(1, 0)
        SquareHistory(3, 1) = SquareHistory(1, 1)
        SquareHistory(0, 0) = -1
        SquareHistory(0, 1) = -1
        SquareHistory(1, 0) = -1
        SquareHistory(1, 1) = -1
        Checkerboard.Refresh()
        'Resets TrueFalse Table, then check for Checks.
        If PlayerTurn Then
            SharedAlgorithms.FixTFTables(MasterBoard, True, MasterWhiteTF
Table, MasterWKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
        Else
            SharedAlgorithms.FixTFTables(MasterBoard, False, MasterBlackT
FTable, MasterBKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
        End If
        CheckChecker()
        If PlayerTurn Xor OrientForWhite Then FlipBoard()
        GameRunning = True
        If GameMode < 3 Then
            FENTextBox.Text = StartingFEN

```



```

        FENTextBox.Select(FENTextBox.Text.Length, 0) 'Stops the TextB
ox from being highlighted.
    End If

    'Recalibrates AI and resets AI move info.
    MainAI.Reconfigure(CurrentFEN)
    CurrentDepth = 2
    CurrentMove = "-"
    CurrentEvaluation = "-"
    CurrentAIDepth.Text = "Current Depth: -"
    CurrentAIMove.Text = "Current Move: -"
    CurrentAIEval.Text = "Evaluation: -"
    EnforceEndStates()
    Console.Clear()
    Sound_Move.Play()
End If
End Sub

'Outputs the current FEN into the FEN textbox.
Private Sub FENExport_Click() Handles FENExport.Click
    FENTextBox.Text = CurrentFEN
End Sub

'Converts the board to the previous FEN Position.
Private Sub UndoMove_Click() Handles UndoMove.Click
    'If CurrentFEN = PreviousFEN then it is implied that it is the starti
ng position.
    'The user should not be able to undo in this circumstance.
    If Not ComputerIsSearching AndAlso Not (CurrentFEN = PreviousFEN) The
n
        Dim TempSH(1, 1) As SByte 'Temp Square History array.
        Dim TempFEN As String = CurrentFEN
        CurrentFEN = PreviousFEN
        PreviousFEN = TempFEN
        MasterWInCheck.NotInCheck()
        MasterBInCheck.NotInCheck()
        'Converts the FEN to a board position, and displays it.
        MasterBoard = SharedAlgorithms.FENConverter(CurrentFEN, MasterWCa
nCastle, MasterBCanCastle, MasterWKPos, MasterBKPos, MasterEnPassant, PlayerT
urn)
        DisplayPieces()
        'Swaps location of Previously Used Squares.
        Array.Copy(SquareHistory, TempSH, 4)
        SquareHistory(0, 0) = SquareHistory(2, 0)
        SquareHistory(0, 1) = SquareHistory(2, 1)
        SquareHistory(1, 0) = SquareHistory(3, 0)
        SquareHistory(1, 1) = SquareHistory(3, 1)
        SquareHistory(2, 0) = TempSH(0, 0)
        SquareHistory(2, 1) = TempSH(0, 1)
        SquareHistory(3, 0) = TempSH(1, 0)
        SquareHistory(3, 1) = TempSH(1, 1)
        If AutoFlipper.Checked Then
            FlipBoard()
        Else
            Checkerboard.Refresh() 'This makes the newly-
changed Previously used Squares visible to the user.
        End If

        GameRunning = True
        'Resets TrueFalse Tables, then checks for Checks.

```

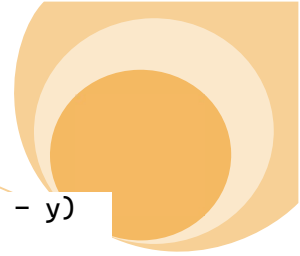
```

    If PlayerTurn Then
        SharedAlgorithms.FixTFTables(MasterBoard, True, MasterWhiteTF
Table, MasterWKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
    Else
        SharedAlgorithms.FixTFTables(MasterBoard, False, MasterBlackTF
FTable, MasterBKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
    End If
    Sound_Move.Play()
    CheckChecker()

    If MasterWInCheck.IsInCheck AndAlso Not PlayerTurn Then
        'If White is in check, and it is black to move, then black ca
n take white's king. This is illegal.
        GameRunning = False
        Console.WriteLine("The Game has Ended. Cause = Invalid Check.
")
    ElseIf MasterBInCheck.IsInCheck AndAlso PlayerTurn Then
        'If Black is in check, and it is white to move, then white ca
n take black's king. This is illegal.
        Console.WriteLine("The Game has Ended. Cause = Invalid Check.
")
        GameRunning = False
    ElseIf Math.Abs(CStr(MasterWKPos(0)) - CStr(MasterBKPos(0))) <= 1
AndAlso Math.Abs(CStr(MasterWKPos(1)) - CStr(MasterBKPos(1))) <= 1 Then
        'Kings are too close together.
        GameRunning = False
        Console.WriteLine("The Game has Ended. Cause = Invalid King P
ositions.")
        CheckLabel.Text = " Stalemate! "
        Sound_Stalemate.Play()
    Else
        Sound_Move.Play()
    End If

    If GameMode < 3 Then FENTextBox.Text = CurrentFEN
    'Recalibrates AI, then checks for end positions.
    MainAI.Reconfigure(CurrentFEN)
    EnforceEndStates()
End If
End Sub

'Subroutine which rotates the board 180 degrees in favour of the other p
layer.
Private Sub FlipBoard() Handles FlipperButton.Click
    OrientForWhite = Not OrientForWhite
    'Reorientates Previously used squares.
    SquareHistory(0, 0) = 7 - SquareHistory(0, 0)
    SquareHistory(0, 1) = 7 - SquareHistory(0, 1)
    SquareHistory(1, 0) = 7 - SquareHistory(1, 0)
    SquareHistory(1, 1) = 7 - SquareHistory(1, 1)
    SquareHistory(2, 0) = 7 - SquareHistory(2, 0)
    SquareHistory(2, 1) = 7 - SquareHistory(2, 1)
    SquareHistory(3, 0) = 7 - SquareHistory(3, 0)
    SquareHistory(3, 1) = 7 - SquareHistory(3, 1)
    'If the AI is searching, rotate the location of the LegalMoveSquares.
    If ComputerIsSearching Then
        Dim TempVar As Boolean
        For y = 0 To 3
            For x = 0 To 7
                TempVar = LegalMoveSquares(x, y)
            
```

```

LegalMoveSquares(x, y) = LegalMoveSquares(7 - x, 7 - y)
LegalMoveSquares(7 - x, 7 - y) = TempVar
    Next
Next
End If
'Displays the new board on the screen and redraws the checkerboard.
DisplayPieces()
Checkerboard.Refresh()
End Sub

'Subroutines Controlling each Colour Scheme Option change. When each butt
on is clicked,
'the Primary & Secondary colours are appended, and these preferences are
written to a file.
Private Sub ColourChangerButton_Click() Handles ColourChangerButton.Click
    'Reveals drop-down menu.
    ColourChanger.Show(ColourChangerButton, 0, ColourChangerButton.Height
)
End Sub
Private Sub DefColourChanger() Handles Def.Click
    'Sets colours and redraws the checkerboard.
    PrimaryColour = Color.Peru
    SecondaryColour = Color.FloralWhite
    Checkerboard.Refresh()
    'Writes the colour preferences to a file (creates the file if one doe
s not exist).
    FileOpen(1, Application.StartupPath & "\ColourPreferences.txt", OpenM
ode.Output)
    PrintLine(1, "def")
    FileClose(1)
End Sub
Private Sub LBluColourChanger() Handles LBlu.Click
    PrimaryColour = Color.LightSteelBlue
    SecondaryColour = Color.AliceBlue
    Checkerboard.Refresh()
    FileOpen(1, Application.StartupPath & "\ColourPreferences.txt", OpenM
ode.Output)
    PrintLine(1, "blu")
    FileClose(1)
End Sub
Private Sub DBluColourChanger() Handles DBlu.Click
    PrimaryColour = Color.SlateGray
    SecondaryColour = Color.LightGray
    Checkerboard.Refresh()
    FileOpen(1, Application.StartupPath & "\ColourPreferences.txt", OpenM
ode.Output)
    PrintLine(1, "bl2")
    FileClose(1)
End Sub
Private Sub GldColourChanger() Handles Gld.Click
    PrimaryColour = Color.Goldenrod
    SecondaryColour = Color.LemonChiffon
    Checkerboard.Refresh()
    FileOpen(1, Application.StartupPath & "\ColourPreferences.txt", OpenM
ode.Output)
    PrintLine(1, "gld")
    FileClose(1)
End Sub
Private Sub GrnColourChanger() Handles Grn.Click
    PrimaryColour = Color.DarkSeaGreen

```

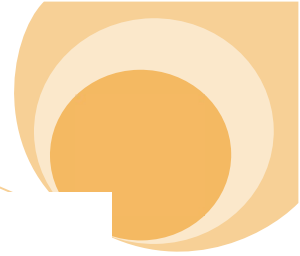
```

SecondaryColour = Color.LavenderBlush
Checkerboard.Refresh()
FileOpen(1, Application.StartupPath & "\ColourPreferences.txt", OpenM
ode.Output)
PrintLine(1, "grn")
FileClose(1)
End Sub
Private Sub PplColourChanger() Handles Ppl.Click
PrimaryColour = Color.MediumPurple
SecondaryColour = Color.MistyRose
Checkerboard.Refresh()
FileOpen(1, Application.StartupPath & "\ColourPreferences.txt", OpenM
ode.Output)
PrintLine(1, "ppl")
FileClose(1)
End Sub
Private Sub MonColourChanger() Handles Mon.Click
PrimaryColour = Color.DimGray
SecondaryColour = Color.Silver
Checkerboard.Refresh()
FileOpen(1, Application.StartupPath & "\ColourPreferences.txt", OpenM
ode.Output)
PrintLine(1, "mon")
FileClose(1)
End Sub

'Subroutines that alter the design of the cursor when the user interacts
with the TimeBar slider.
Private Sub UserTimeBar_MouseDown() Handles UserTimeBar.MouseDown
'Custom cursor - "Open Hand" Pointer used on MacOS.
'https://support.apple.com/en-gb/guide/mac-help/mh35695/mac
Me.Cursor = New Cursor(New System.IO.MemoryStream(My.Resources.Cursor
16))
End Sub
Private Sub UserTimeBar_MouseUp() Handles UserTimeBar.MouseUp
Me.Cursor = Cursors.Default
End Sub

'Subroutine that turns the value of the time slider (in Analysis Mode) in
to the time for an AI to search.
'Also updates TimeBox.
Private Sub UserTimeBar_ValueChanged() Handles UserTimeBar.ValueChanged
'Updates UserTimeBox.
UserTimeBox.Text = "Time For Search: " & Math.Max(UserTimeBar.Value /
2, 0.1) & " seconds."
'Gives the UserTimeBox a red highlight if the time allocated might no
t be enough for a search to complete (estimate).
If (UseQuiescence AndAlso UserTimeBar.Value < 4) OrElse Not UseQuiesc
ence AndAlso UserTimeBar.Value < 2 Then
UserTimeBox.BackColor = Color.LightCoral
Else
UserTimeBox.BackColor = Color.WhiteSmoke
End If
TimeForSearch = Math.Max(UserTimeBar.Value / 2, 0.1)
End Sub
'Subroutine that allows the user to enter in the time for the AI to search
h - the user's input
'is reflected on the TimeBar and the TimeBox. Includes error detection.
Private Sub UserTimeBox_Click() Handles UserTimeBox.Click

```



```

If Not ComputerIsSearching Then
    Dim Temp As String
    Dim UserTime As Decimal
    While True
        Try
            'Displays text box popup to user, where they can enter th
            'eir time.
            Temp = InputBox("Please input how long you want the AI to
                search for (in seconds)." & vbCrLf & vbCrLf & "(Min Time = 0.1s, Max Time =
                600s)", "Time Inputter")
            If Temp = "" OrElse Temp = " " Then 'Cancel / Exit button
                was pressed - abort.
                UserTime = TimeForSearch
                Exit While
            End If
            UserTime = Math.Round(CDec(Temp), 1)
            If UserTime >= 0.1 AndAlso UserTime <= 600 Then
                Exit While 'Inside range - input has passed all the c
                hecks :).
            Else 'Input was outside the given range - displays approp
                riate message.
                If MsgBox("Invalid Number - Please make sure your inp
                    ut is in the correct range.", vbCritical + vbRetryCancel + vbApplicationModal
                    ) = 2 Then
                    'User has cancelled - revert back to original tim
                    e.
                    UserTime = TimeForSearch
                    Exit While
                End If
            End If
            Catch ex As Exception 'Input was not a decimal / integer numb
                er - displays appropriate message.
                If MsgBox("Invalid Entry - Please make sure that your inp
                    ut is in the correct format.", vbCritical + vbRetryCancel + vbApplicationModa
                    l) = 2 Then
                    'User has cancelled - revert back to original time.
                    UserTime = TimeForSearch
                    Exit While
                End If
            End Try
        End While

        'Edits GUI objects, and updates TimeForSearch.
        UserTimeBox.Select(UserTimeBox.Text.Length, 0)
        UserTimeBar.Value = 2 * Math.Round(Math.Min(UserTime, 30))
        UserTimeBox.Text = "Time For Search: " & UserTime & " seconds."
        TimeForSearch = UserTime
    End If
End Sub
Private Sub QuiescenceBox_CheckedChanged() Handles QuiescenceBox.CheckedC
hanged
    'Toggles Quiescence depending on the state of the box.
    If QuiescenceBox.Checked Then UseQuiescence = True : Else UseQuiescen
ce = False
    If (UseQuiescence AndAlso UserTimeBar.Value < 4) OrElse Not UseQuiesc
ence AndAlso UserTimeBar.Value < 2 Then
        UserTimeBox.BackColor = Color.LightCoral
    Else
        UserTimeBox.BackColor = Color.WhiteSmoke
    End If
End If

```

End Sub

'Subroutine that terminates the AI search, and displays on the board the AI's current move.

```
Private Sub AITerminator_Click() Handles AITerminator.Click
    TerminateSearch = True
End Sub
```

'Button that returns the user back to the Main Menu.

```
Private Sub ExitBtn_Click() Handles ExitBtn.Click
    'Locates MainMenu Form and shows it.
    Dim Menu As Form = CType(Application.OpenForms("MainMenu"), MainMenu)
    Me.Close() 'Closes the current form.
    Menu.Show()
End Sub
```

'Button that displays the credits information onto the screen (in the form of a pop-up).

```
Private Sub Credits_Click() Handles Credits.Click
    MsgBox(Strings.StrDup(14, " ") & "Chess Game & Artificial Intelligence" & vbCrLf & Strings.StrDup(21, " ") & "Created by Alfie Kunz (8158)" & vbCrLf & Strings.StrDup(22, " ") & "of Beckfoot School (37101)" & vbCrLf & "Project used for the AQA GCE Computer Science NEA" & vbCrLf & Strings.StrDup(35, " ") & "(2021 - 2023)", vbInformation + vbApplicationModal, "Credits")
End Sub
```

DRAG & DROP MECHANICS

'The below three subroutines control the drag & drop mechanics for the pieces, and translates a user's move onto the board.

```
Private Sub Piece_MouseDown(ByVal sender As Object, ByVal e As System.Windows.Forms.MouseEventArgs) Handles WK1.MouseDown, WQ1.MouseDown, WB1.MouseDown, WB2.MouseDown, WN1.MouseDown, WN2.MouseDown, WR1.MouseDown, WR2.MouseDown, WP1.MouseDown, WP2.MouseDown, WP3.MouseDown, WP4.MouseDown, WP5.MouseDown, WP6.MouseDown, WP7.MouseDown, WP8.MouseDown, BK1.MouseDown, BQ1.MouseDown, BB1.MouseDown, BB2.MouseDown, BN1.MouseDown, BN2.MouseDown, BR1.MouseDown, BR2.MouseDown, BP1.MouseDown, BP2.MouseDown, BP3.MouseDown, BP4.MouseDown, BP5.MouseDown, BP6.MouseDown, BP7.MouseDown, BP8.MouseDown, WQ8.MouseDown, WQ7.MouseDown, WQ6.MouseDown, WQ5.MouseDown, WQ4.MouseDown, WQ3.MouseDown, BQ6.MouseDown, BQ8.MouseDown, BQ5.MouseDown, BQ7.MouseDown, BQ2.MouseDown, BQ4.MouseDown, BQ3.MouseDown, WQ2.MouseDown, WQ9.MouseDown, BQ9.MouseDown
```

'Subroutine that activates the drag + drop mechanic if the user clicks on a piece.

```
If e.Button = Windows.Forms.MouseButtons.Left AndAlso GameRunning AndAlso Not ComputerIsSearching Then
```

'If the piece corresponds to the player allowed to move...

```
If (CStr(sender.name)(0) = "W" AndAlso PlayerTurn) OrElse (CStr(sender.name)(0) = "B" AndAlso (Not PlayerTurn)) Then
```

```
Me.Cursor = New Cursor(New System.IO.MemoryStream(My.Resources.Cursor16)) 'Can be Cursor32 or Cursor192
```

'Generates the Legal Moves for the piece the user has picked up (using the AI).

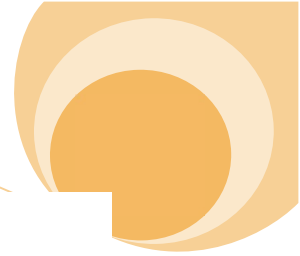
```
If OrientForWhite Then
```

```
PieceMoving.LegalMoves = MainAI.ReturnPiecesLegalMoves(sender.location.x / 75, sender.location.y / 75)
```

'Updates LegalMoveSquares array.

```
For n = 1 To CInt(PieceMoving.LegalMoves(0))
```

```
LegalMoveSquares(CStr(PieceMoving.LegalMoves(n)(0)), CStr(PieceMoving.LegalMoves(n)(1))) = True
```



```

        Next
    Else
        PieceMoving.LegalMoves = MainAI.ReturnPiecesLegalMoves(7
- (sender.location.x / 75), 7 - (sender.location.y / 75))
        'Updates LegalMoveSquares array.
        For n = 1 To CInt(PieceMoving.LegalMoves(0))
            LegalMoveSquares(7 - CStr(PieceMoving.LegalMoves(n)(0
)), 7 - CStr(PieceMoving.LegalMoves(n)(1))) = True
        Next
    End If
    'Creates the PieceMoving values, then redraws the checkerboard (creates LegalMoveSquares).
    sender.bringtofront()
    PieceMoving.IsMovingPiece = True
    'Notes location of the piece for drag & drop mechanics.
    PieceMoving.StartPoint = sender.location
    PieceMoving.MidPoint = sender.PointToScreen(New Point(e.X, e.Y))

    Checkerboard.Refresh()
Else
    'Stops the piece from being moved abruptly.
    PieceMoving.StartPoint = sender.location
End If
End If
End Sub

```

```

Private Sub Piece_MouseMove(ByVal sender As Object, ByVal e As System.Windows.Forms.MouseEventArgs) Handles WK1.MouseMove, WQ1.MouseMove, WB1.MouseMove, WB2.MouseMove, WN1.MouseMove, WN2.MouseMove, WR1.MouseMove, WR2.MouseMove, WP1.MouseMove, WP2.MouseMove, WP3.MouseMove, WP4.MouseMove, WP5.MouseMove, WP6.MouseMove, WP7.MouseMove, WP8.MouseMove, BK1.MouseMove, BQ1.MouseMove, BB1.MouseMove, BB2.MouseMove, BN1.MouseMove, BN2.MouseMove, BR1.MouseMove, BR2.MouseMove, BP1.MouseMove, BP2.MouseMove, BP3.MouseMove, BP4.MouseMove, BP5.MouseMove, BP6.MouseMove, BP7.MouseMove, BP8.MouseMove, WQ2.MouseMove, WQ3.MouseMove, WQ4.MouseMove, WQ5.MouseMove, WQ6.MouseMove, WQ7.MouseMove, WQ8.MouseMove, BQ2.MouseMove, BQ3.MouseMove, BQ4.MouseMove, BQ5.MouseMove, BQ6.MouseMove, BQ7.MouseMove, BQ8.MouseMove, WQ9.MouseMove, BQ9.MouseMove
    If PieceMoving.IsMovingPiece Then
        'The piece is bound to the position of the user's mouse.
        PieceMoving.EndPoint = sender.PointToScreen(New Point(e.X, e.Y))
        sender.Left += (PieceMoving.EndPoint.X - PieceMoving.MidPoint.X)
        sender.Top += (PieceMoving.EndPoint.Y - PieceMoving.MidPoint.Y)
        PieceMoving.MidPoint = PieceMoving.EndPoint
    End If
End Sub

```

```

Private Sub Piece_MouseUp(ByVal sender As Object, ByVal e As System.Windows.Forms.MouseEventArgs) Handles WK1.MouseUp, WQ1.MouseUp, WB1.MouseUp, WB2.MouseUp, WN1.MouseUp, WN2.MouseUp, WR1.MouseUp, WR2.MouseUp, WP1.MouseUp, WP2.MouseUp, WP3.MouseUp, WP4.MouseUp, WP5.MouseUp, WP6.MouseUp, WP7.MouseUp, WP8.MouseUp, BK1.MouseUp, BQ1.MouseUp, BB1.MouseUp, BB2.MouseUp, BN1.MouseUp, BN2.MouseUp, BR1.MouseUp, BR2.MouseUp, BP1.MouseUp, BP2.MouseUp, BP3.MouseUp, BP4.MouseUp, BP5.MouseUp, BP6.MouseUp, BP7.MouseUp, BP8.MouseUp, WQ2.MouseUp, WQ3.MouseUp, WQ4.MouseUp, WQ5.MouseUp, WQ6.MouseUp, WQ7.MouseUp, WQ8.MouseUp, BQ2.MouseUp, BQ3.MouseUp, BQ4.MouseUp, BQ5.MouseUp, BQ6.MouseUp, BQ7.MouseUp, BQ8.MouseUp, WQ9.MouseUp, BQ9.MouseUp
    Dim JustCastled, IsLegalMove As Boolean
    Dim ReplacedPiece As PictureBox = Nothing
    Dim TempPoint As Point

```

```

If e.Button = Windows.Forms.MouseButtons.Left AndAlso PieceMoving.IsMovingPiece Then
    Me.Cursor = Cursors.Default 'Resets cursor.
    'Disables drag & drop mechanic and sets the piece's position on the board (into the centre of the square).
    PieceMoving.IsMovingPiece = False
    sender.location = New Point(sender.location.X + 37.5 - (sender.location.X + 37.5) Mod 75, sender.location.y + 37.5 - (sender.location.y + 37.5) Mod 75)
    'Resets LegalMoveSquares array & redraws checkerboard.
    For y = 0 To 7
        For x = 0 To 7
            LegalMoveSquares(x, y) = False
        Next
    Next
    Checkerboard.Refresh()
    'If the user has actually moved the piece, and the move is legal, the program continues.
    If sender.location <> PieceMoving.StartPoint Then

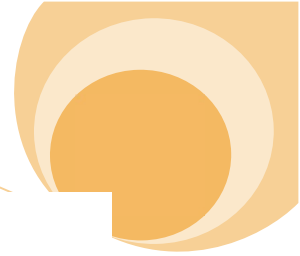
        TempPoint.X = sender.location.x / 75
        TempPoint.Y = sender.location.y / 75
        If Not OrientForWhite Then 'Flips coordinates 180 degrees.
            TempPoint.X = 7 - TempPoint.X
            TempPoint.Y = 7 - TempPoint.Y
            PieceMoving.StartPoint.X = 525 - PieceMoving.StartPoint.X
            PieceMoving.StartPoint.Y = 525 - PieceMoving.StartPoint.Y
        End If
        'Checks if the user's move is in the list of that piece's legal moves.

        IsLegalMove = False
        For n = 1 To CInt(PieceMoving.LegalMoves(0))
            If CStr(TempPoint.X & TempPoint.Y) = PieceMoving.LegalMoves(n) Then

                IsLegalMove = True
                Exit For
            End If
        Next

        If IsLegalMove Then
            'Resets checking rules.
            If PlayerTurn Then
                MasterWInCheck.NotInCheck()
            Else
                MasterBInCheck.NotInCheck()
            End If
            If UCase(sender.name(1)) = "K" Then
                'Disables Castling if the King has moved.
                If sender.name(0) = "W" Then
                    MasterWKPos = TempPoint.X & TempPoint.Y
                    MasterWCanCastle.CannotCastle()
                Else
                    MasterBKPos = TempPoint.X & TempPoint.Y
                    MasterBCanCastle.CannotCastle()
                End If
                'Code for Castling (Queen Side & King Side).
                If (TempPoint.X & TempPoint.Y) = ((PieceMoving.StartPoint.X / 75) + 2 & PieceMoving.StartPoint.Y / 75) Then
                    JustCastled = True
                    Sound_Castle.Play()
                End If
            End If
        End If
    End If
End If

```

```

'Adjusts rook position.
If OrientForWhite Then
    CoordToPieceConverter(7, TempPoint.Y).Left -
= 150

Else
    CoordToPieceConverter(7, TempPoint.Y).Left +=
150

End If
MasterBoard(5, TempPoint.Y) = CStr(MasterBoard(7,
TempPoint.Y))
MasterBoard(7, TempPoint.Y) = " "
ElseIf (TempPoint.X & TempPoint.Y) = ((PieceMoving.St
artPoint.X / 75) - 2 & PieceMoving.StartPoint.Y / 75) Then
    'Queenside castling.
    JustCastled = True
    Sound_Castle.Play()
    'Adjusts rook position.
    If OrientForWhite Then
        CoordToPieceConverter(0, TempPoint.Y).Left +=
225

    Else
        CoordToPieceConverter(0, TempPoint.Y).Left -
= 225

    End If
    MasterBoard(3, TempPoint.Y) = CStr(MasterBoard(0,
TempPoint.Y))
    MasterBoard(0, TempPoint.Y) = " "
End If
End If

'If the move is a capture, the required sound is played a
nd the captured piece is removed from the board.
If MasterBoard(TempPoint.X, TempPoint.Y) <> " " Then
    Sound_Capture.Play()
    ReplacedPiece = CoordToPieceConverter(TempPoint.X, Tem
pPoint.Y) 'Finds captured PictureBox.
    ReplacedPiece.Visible = False
    ReplacedPiece.Location = New Point(-100, -100)
ElseIf JustCastled = False Then
    Sound_Move.Play()
End If
'MasterBoard is updated by clearing the piece's previous
position and reallocating its current position.
MasterBoard(PieceMoving.StartPoint.X / 75, PieceMoving.St
artPoint.Y / 75) = " "

'Updates SquareHistory array,
SquareHistory(2, 0) = SquareHistory(0, 0)
SquareHistory(2, 1) = SquareHistory(0, 1)
SquareHistory(3, 0) = SquareHistory(1, 0)
SquareHistory(3, 1) = SquareHistory(1, 1)
If OrientForWhite Then
    SquareHistory(0, 0) = PieceMoving.StartPoint.X / 75
    SquareHistory(0, 1) = PieceMoving.StartPoint.Y / 75
    SquareHistory(1, 0) = TempPoint.X
    SquareHistory(1, 1) = TempPoint.Y
Else
    SquareHistory(0, 0) = 7 - (PieceMoving.StartPoint.X /
75)

```

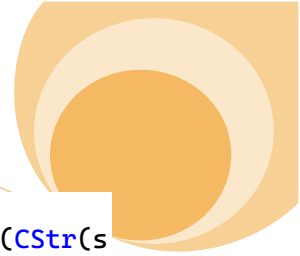
75)

```

SquareHistory(0, 1) = 7 - (PieceMoving.StartPoint.Y /
SquareHistory(1, 0) = 7 - TempPoint.X
SquareHistory(1, 1) = 7 - TempPoint.Y
End If

'Handles moving a piece to its new coordinates, along with
special pawn moves.
If CStr(sender.name)(0) = "W" Then
    MasterBoard(TempPoint.X, TempPoint.Y) = UCase((CStr(s
ender.name))(1))
    'If a pawn has made it to the end of the board, it is
    promoted to a Queen.
    If sender.name(1) = "P" Then
        If TempPoint.Y = 0 Then
            'Pawn promotion - finds a queen to replace the
            pawn.
            For n = 1 To 9
                ReplacedPiece = Me.Controls.Find("WQ" & n
.ToString, True).Single()
                If ReplacedPiece.Visible = False Then Exit
                For
                    Next
                    ReplacedPiece.Location = New Point(sender.location.X, sender.location.Y)
                    ReplacedPiece.Visible = True
                    ReplacedPiece.BringToFront()
                    'Replaces the Board index with a queen, then
                    removes the pawn PictureBox from the screen.
                    MasterBoard(TempPoint.X, TempPoint.Y) = "Q"
                    sender.Visible = False
                    sender.Location = New Point(-100, -100)
                ElseIf TempPoint.X & TempPoint.Y = MasterEnPassant
                    Then
                        'Rules for En Passant - removes the piece behind it.
                        Sound_Capture.Play()
                        ReplacedPiece = CoordToPieceConverter(TempPoint.X, TempPoint.Y + 1)
                        ReplacedPiece.Visible = False
                        ReplacedPiece.Location = New Point(-100, -100)
                        MasterBoard(TempPoint.X, (TempPoint.Y) + 1) =
                        " "
                    End If
                End If
            End If
            'If the rook has moved, then part of castling is disabled.
            If sender.name(1) = "R" Then
                If sender.name(2) = "1" Then
                    MasterWCanCastle.QS = False
                Else
                    MasterWCanCastle.KS = False
                End If
            End If
            'The TrueFalse Table is generated for the Black Pieces.
            SharedAlgorithms.FixTFTTables(MasterBoard, False, MasterBlackTFTTable, MasterBKPos, SharedAlgorithms.NotInCheck, MasterBInCheck, MasterEnPassant)

```



```

Else
    MasterBoard(TempPoint.X, TempPoint.Y) = LCase((CStr(s
ender.name)))(1))
    'Identical Code as above but for the black pieces.
    If sender.name(1) = "P" Then
        If TempPoint.Y = 7 Then
            For n = 1 To 9
                ReplacedPiece = Me.Controls.Find("BQ" & n
.ToString, True).Single()
                If ReplacedPiece.Visible = False Then Exi
t For
                    Next
                    ReplacedPiece.Location = New Point(sender.loc
ation.X, sender.location.Y)
                    ReplacedPiece.Visible = True
                    MasterBoard(TempPoint.X, TempPoint.Y) = "q"
                    sender.Visible = False
                    sender.Location = New Point(-100, -100)
                ElseIf TempPoint.X & TempPoint.Y = MasterEnPassan
t Then
                    Sound_Capture.Play()
                    ReplacedPiece = CoorToPieceConverter(TempPoi
nt.X, TempPoint.Y - 1)
                    ReplacedPiece.Visible = False
                    ReplacedPiece.Location = New Point(-100, -
100)
                    MasterBoard(TempPoint.X, (TempPoint.Y) - 1) =
" "
                End If
            End If
            If sender.name(1) = "R" Then
                If sender.name(2) = "1" Then
                    MasterBCanCastle.QS = False
                Else
                    MasterBCanCastle.KS = False
                End If
            End If
            'The TrueFalse Table is generated for the White Piece
s.
            SharedAlgorithms.FixTFTTables(MasterBoard, True, Maste
rWhiteTFTTable, MasterWKPos, MasterWInCheck, SharedAlgorithms.NotInCheck, Maste
rEnPassant)
        End If

        'En Passant Generation.
        If Not OrientForWhite Then
            PieceMoving.StartPoint.X = 525 - PieceMoving.StartPoi
nt.X
            PieceMoving.StartPoint.Y = 525 - PieceMoving.StartPoi
nt.Y
        End If
        If sender.name(1) = "P" AndAlso Math.Abs(sender.location.
Y - PieceMoving.StartPoint.Y) = 150 Then
            If sender.name(0) = "W" AndAlso (MasterBoard(Math.Max
((TempPoint.X) - 1, 0), TempPoint.Y) = "p" OrElse MasterBoard(Math.Min((TempP
oint.X) + 1, 7), TempPoint.Y) = "p") Then
                MasterEnPassant = TempPoint.X & 5
            ElseIf sender.name(0) = "B" AndAlso (MasterBoard(Math
.Max((TempPoint.X) - 1, 0), TempPoint.Y) = "P" OrElse MasterBoard(Math.Min((T
empPoint.X) + 1, 7), TempPoint.Y) = "P") Then

```

```

        MasterEnPassant = TempPoint.X & 2
    Else 'No En Passant exists.
        MasterEnPassant = "-"
    End If
Else 'Remove En Passant.
    MasterEnPassant = "-"
End If

If AutoFlipper.Checked Then
    FlipBoard()
Else 'Redraws checkerboard.
    Checkerboard.Refresh()
End If
'Checks if any player is in check, and if so notifies the
player with audio and visuals.
CheckChecker()

'Starts the next turn by updating the Current FEN and che
cking for end positions.
PlayerTurn = Not PlayerTurn
PreviousFEN = CurrentFEN
CurrentFEN = SharedAlgorithms.ConvertToFEN(MasterBoard, M
asterWCanCastle, MasterBCanCastle, MasterEnPassant, PlayerTurn)
MainAI.Reconfigure(CurrentFEN) 'Recalibrates AI before ch
ecking for end states.
EnforceEndStates()
If CurrentFEN = PreviousFEN Then Exit Sub 'Stops AI from
running if it is the start position.    OutputDebugInfo()
If GameMode < 3 Then
    FENTextBox.Text = CurrentFEN
    If GameMode = 1 Then InstantiateAIs()
End If

Else 'Resets piece to previous position
If Not OrientForWhite Then
    PieceMoving.StartPoint.X = 525 - PieceMoving.StartPoi
nt.X
    PieceMoving.StartPoint.Y = 525 - PieceMoving.StartPoi
nt.Y
End If
sender.location = PieceMoving.StartPoint
End If
End If
End Sub

'Subroutine which updates the Sprites / Gamestate Textbox depending on wh
ether a player is in check (or not).
Private Sub CheckChecker()
    If MasterWInCheck.IsInCheck OrElse MasterBInCheck.IsInCheck Then
        Sound_Check.Play()
        CheckLabel.Text = "    Check!    "
    Else
        CheckLabel.Text = "                "
    End If
    'Modifies king picture accordingly.
    If MasterWInCheck.IsInCheck Then
        WK1.Image = Image.FromFile(Application.StartupPath & "\Images\Def
ault\WKingCheck.png")
    End If
End Sub

```



```

Else
    WK1.Image = Image.FromFile(Application.StartupPath & "\\Images\\Default\\WKing.png")
End If
If MasterBInCheck.IsInCheck Then
    BK1.Image = Image.FromFile(Application.StartupPath & "\\Images\\Default\\BKingCheck.png")
Else
    BK1.Image = Image.FromFile(Application.StartupPath & "\\Images\\Default\\BKing.png")
End If
End Sub

```

AI HANDLERS & THREAD CONTROLS

```

'Algorithm that creates all the threads & AIs that will search on a position.
Handles time management + overtime,
'interactions with the GUI chessboard, live updates via GUI objects, selecting best moves, and more.
Private Sub InstantiateAIs() Handles AIMoveBtn.Click
    If GameRunning AndAlso Not ComputerIsSearching Then
        Dim IsLate As Boolean 'Represents whether the computer has used more time than it was originally allocated.
        Dim BestMove As Move = MainAI.CheckForEndState() 'Ensures that the position is valid, and that the
        'AIs won't crash whilst searching on the position.
        If BestMove.EndState = "f" OrElse BestMove.EndState = "o" Then
            If BestMove.EndState = "f" Then 'no forced move found.
                'Instantiations of threads (in a List of tasks format).
                Dim Tasks As New List(Of Task) With {
                    .Capacity = 6
                }
                Tasks.Add(New Task(AddressOf InitialiseThread1))
                Tasks.Add(New Task(AddressOf InitialiseThread2))
                Tasks.Add(New Task(AddressOf InitialiseThread3))
                Tasks.Add(New Task(AddressOf InitialiseThread4))
                Tasks.Add(New Task(AddressOf InitialiseThread5))

                CalculateAbsoluteDepth()
                AIMovedToHigherDepth(0) = False
                AIMovedToHigherDepth(1) = False
                'Resets GUI objects & changes cursor design.
                Me.Cursor = Cursors.AppStarting
                ProgressBar.ForeColor = Color.FromArgb(0, 192, 0)
                ProgressBar.Value = 0
                UserTimeBar.Enabled = False
                QuiescenceBox.Enabled = False
                'Erases the information from the last AI search.
                CurrentDepth = 2
                CurrentMove = "-"
                CurrentEvaluation = "-"
                TerminateSearch = False
                AITerminator.Visible = True
                AIFinishedSearch = True

                ComputerIsSearching = True
                Console.WriteLine(vbCrLf & "The AI is now searching..." & vbCrLf & "Time for Search = " & TimeForSearch & " Seconds. Quiescence Mode: " & UseQuiescence & vbCrLf)
            End If
        End If
    End If
End Sub

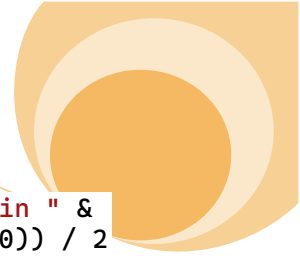
```

```

AIStopwatch.Reset()
AIStopwatch.Start()
'Starts all tasks simultaneously.
Tasks.All(Function(t As Task)
            t.Start()
            Return True
        End Function)
'While any of the 5 threads are running...
While Tasks.Any(Function(t As Task) Not (t.Status = TaskSt
atus.Canceled OrElse t.Status = TaskStatus.Faulted OrElse t.Status = TaskStat
us.RanToCompletion))
    Application.DoEvents() 'Allows the user to interact wi
th the GUI (limited).
    'Updates progress bar to reflect time.
    If IsLate Then
        '2x speed with red colour.
        ProgressBar.Value = Math.Min((Math.Max(AIStopwatch
.ElapsedMilliseconds - TimeForSearch * 1000, 0) / 5) / TimeForSearch, 100)
    Else 'Normal speed with green colour.
        ProgressBar.Value = Math.Min(((AIStopwatch.Elapsed
Milliseconds / 10) / TimeForSearch), 100)
    End If

    If AIStopwatch.ElapsedMilliseconds / 1000 > TimeForSea
rch OrElse TerminateSearch Then 'Time's up!
        If Not MainAI.ABORT Then
            'Aborts all AIs (apart from AI of lowest depth
            if it has not finished).
            If AIMovedToHigherDepth(0) Then AI2.ABORT = Tr
ue
            AI3.ABORT = True
            MainAI.ABORT = True
            AI4.ABORT = True
            AI5.ABORT = True
            IsLate = True
            ProgressBar.ForeColor = Color.Red 'Red progres
s bar - visual indication to user that time is up.
        End If
        If AIStopwatch.ElapsedMilliseconds / 1000 > Math.M
in(TimeForSearch * 1.5, TimeForSearch + 10000) OrElse TerminateSearch Then
            '1.5 * time (or extra 10s) has expired - termi
            nate all AIs immediately!
            If Not AI2.ABORT Then AI2.ABORT = True
            If Not AI3.ABORT Then AI3.ABORT = True
            If Not MainAI.ABORT Then MainAI.ABORT = True
            If Not AI4.ABORT Then AI4.ABORT = True
            If Not AI5.ABORT Then AI5.ABORT = True
        End If
        ElseIf AIFinishedSearch Then 'Updates GUI controls on
latest search & redraws checkerboard.
            Checkerboard.Refresh()
            CurrentAIDepth.Text = "Current Depth: " & CurrentD
epth
            CurrentAIMove.Text = "Current Move: " & CurrentMov
e
            If CurrentEvaluation <> "-
" AndAlso Math.Abs(CInt(CurrentEvaluation)) > 900 Then
                'Mating pattern has been found.

```

```

        CurrentAIEval.Text = "Evaluation: Mate in " &
Math.Truncate((CurrentDepth - (Math.Abs(CInt(CurrentEvaluation)) - 1000)) / 2
)
        Else
            CurrentAIEval.Text = "Evaluation: " & CurrentE
valuation
            End If
            AIFinishedSearch = False
        End If
        Thread.Sleep(5) 'Allows more processing time to be spe
nt on the AIs.
    End While
    AIStopwatch.Stop()
    If TerminateSearch Then Console.WriteLine("Search Terminat
ed Preemptively (by user).")

    'Select the best move of the highest depth possible (if th
at search has not been aborted).
    If AIMovedToHigherDepth(1) AndAlso AIBestMoves(6).EndState
    <> "a" Then
        BestMove = AIBestMoves(6)
    ElseIf AIMovedToHigherDepth(0) AndAlso AIBestMoves(5).EndS
tate <> "a" Then
        BestMove = AIBestMoves(5)
    ElseIf AIBestMoves(4).EndState <> "a" Then
        BestMove = AIBestMoves(4)
    ElseIf AIBestMoves(3).EndState <> "a" Then
        BestMove = AIBestMoves(3)
    ElseIf AIBestMoves(2).EndState <> "a" Then
        BestMove = AIBestMoves(2)
    ElseIf AIBestMoves(1).EndState <> "a" Then
        BestMove = AIBestMoves(1)
    ElseIf AIBestMoves(0).EndState <> "a" Then
        BestMove = AIBestMoves(0)
    Else 'No AI have completed their search - start a new sear
ch at a depth of 2 (without Quiescence).
        'Sets the result of this new to be the best move.
        BestMove = MainAI.Search(2, False)
        CurrentMove = SharedAlgorithms.MoveConverter(MasterBoa
rd, BestMove, MasterEnPassant)
    End If
    Console.WriteLine(vbCr & "Search Complete. Total Time: " &
AIStopwatch.Elapsed.TotalMilliseconds & " Milliseconds.")

    'Updates GUI labels (containing info on search).
    AITerminator.Visible = False
    TerminateSearch = False
    CurrentAIDepth.Text = "Current Depth: " & CurrentDepth
    CurrentEvaluation = BestMove.Score
    CurrentAIMove.Text = "Current Move: " & CurrentMove
    If CurrentEvaluation <> "-"
" AndAlso Math.Abs(CInt(CurrentEvaluation)) > 900 Then
        'Mating pattern has been found - update GUI accordingl
y
        CurrentAIEval.Text = "Evaluation: Mate in " & Math.Tru
ncate((CurrentDepth - (Math.Abs(CInt(CurrentEvaluation)) - 1000)) / 2)
    Else
        CurrentAIEval.Text = "Evaluation: " & CurrentEvaluatio
n
    End If

```

```

Else 'Move was forced - make move instantly without searching.
    Console.WriteLine(vbCrLf & "Search Aborted - Only 1 Move i
n Position.")

    CurrentAIDepth.Text = "Current Depth: 1"
    CurrentAIMove.Text = "Current Move: " & SharedAlgorithms.M
oveConverter(MasterBoard, BestMove, MasterEnPassant) & " (FORCED)."
End If

If Me.IsHandleCreated Then 'If form is still open...
    GameRunning = True
    'Makes the best move on the board, and updates the correct
TFTable.

    If PlayerTurn Then
        MasterWInCheck.NotInCheck() 'Player is no longer in ch
eck.

        MakeMove(MasterBoard, BestMove.OldMoveX, BestMove.OldM
oveY, BestMove.NewMoveX, BestMove.NewMoveY, MasterWCanCastle, MasterWKPos, Ma
sterEnPassant, True)

        SharedAlgorithms.FixTFTables(MasterBoard, False, Maste
rBlackTFTable, MasterBKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
    Else
        MasterBInCheck.NotInCheck() 'Player is no longer in ch
eck.

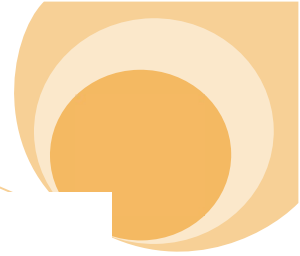
        MakeMove(MasterBoard, BestMove.OldMoveX, BestMove.OldM
oveY, BestMove.NewMoveX, BestMove.NewMoveY, MasterBCanCastle, MasterBKPos, Ma
sterEnPassant, True)

        SharedAlgorithms.FixTFTables(MasterBoard, True, Master
WhiteTFTable, MasterWKPos, MasterWInCheck, MasterBInCheck, MasterEnPassant)
    End If
    DisplayPieces()

    'Update Previously Used Squares, then flips the board if n
ecessary.

    If GameMode = 3 Then
        SquareHistory(2, 0) = SquareHistory(0, 0)
        SquareHistory(2, 1) = SquareHistory(0, 1)
        SquareHistory(3, 0) = SquareHistory(1, 0)
        SquareHistory(3, 1) = SquareHistory(1, 1)
    End If
    SquareHistory(0, 0) = BestMove.NewMoveX
    SquareHistory(0, 1) = BestMove.NewMoveY
    SquareHistory(1, 0) = BestMove.OldMoveX
    SquareHistory(1, 1) = BestMove.OldMoveY
    If Not OrientForWhite Then
        SquareHistory(0, 0) = 7 - SquareHistory(0, 0)
        SquareHistory(0, 1) = 7 - SquareHistory(0, 1)
        SquareHistory(1, 0) = 7 - SquareHistory(1, 0)
        SquareHistory(1, 1) = 7 - SquareHistory(1, 1)
    End If
    'Resets LegalMoveSquares.
    For y = 0 To 7
        For x = 0 To 7
            LegalMoveSquares(x, y) = False
        Next
    Next
    If AutoFlipper.Checked Then
        FlipBoard()
    Else
        Checkerboard.Refresh()

```



```

End If
CheckChecker()

'Ends the turn, then calculates the new position's FEN.
PlayerTurn = Not PlayerTurn
If GameMode = 3 Then
    PreviousFEN = CurrentFEN
Else
    FENTextBox.Text = SharedAlgorithms.ConvertToFEN(Master
Board, MasterWCanCastle, MasterBCanCastle, MasterEnPassant, PlayerTurn)
End If
    CurrentFEN = SharedAlgorithms.ConvertToFEN(MasterBoard, Ma
sterWCanCastle, MasterBCanCastle, MasterEnPassant, PlayerTurn)
    MainAI.Reconfigure(CurrentFEN) 'Recalibrates AI.

'Resets GUI objects & cursor design.
Me.Cursor = Cursors.Default
ProgressBar.ForeColor = Color.FromArgb(0, 192, 0)
ProgressBar.Value = 0
UserTimeBar.Enabled = True
QuiescenceBox.Enabled = True
ComputerIsSearching = False
EnforceEndStates() 'Checks for end states found from the A
I's move.

OutputDebugInfo()
'The below lines fix a bug, where the AI wouldn't make a m
ove after the user restarts their game (1P mode only).
If AIEndlessMode.Checked OrElse (GameMode = 1 AndAlso Curr
entFEN = PreviousFEN AndAlso Not UserPlayer = PlayerTurn) Then
    Thread.Sleep(5) 'Allows the system to recalibrate (to
prevent spam).

    InstantiateAIs()
End If

End If
End If
End If
End Sub

'Subroutines controlling each thread / AI of particular depth.
Private Sub InitialiseThread1()
    'Creates and starts the AI.
    AIBestMoves(0) = AI2.Search(AbsoluteDepth - 2, UseQuiescence)
    If Not AI2.ABORT Then
        'Outputs the move if the move has completed successfully.
        Console.WriteLine("Depth Of " & AbsoluteDepth - 2 & " Completed i
n: " & AIStopwatch.ElapsedMilliseconds & " Milliseconds." & vbCrLf)
        AmmendLegalMoveSuares(AIBestMoves(0))
        AIFinishedSearch = True
        CurrentDepth = AbsoluteDepth - 2
        CurrentMove = SharedAlgorithms.MoveConverter(MasterBoard, AIBestM
oves(0), MasterEnPassant)
        CurrentEvaluation = AIBestMoves(0).Score
        If Math.Abs(AIBestMoves(0).Score) > 900 Then
            'Checkmating sequence found - terminate all other AIs.
            AI3.ABORT = True
            MainAI.ABORT = True
            AI4.ABORT = True
            AI5.ABORT = True
        ElseIf AIStopwatch.ElapsedMilliseconds / 500 < TimeForSearch Then

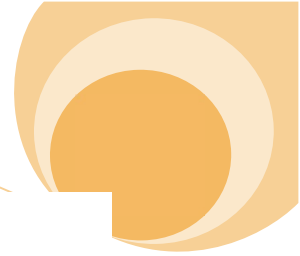
```

```

'Starts a new search at a higher depth (if there is >50% left
on the timer).
    AIMovedToHigherDepth(0) = True
    AIBestMoves(5) = AI2.Search(AbsoluteDepth + 3, UseQuiescence)
    If Not AI2.ABORT Then
        'Outputs the move if the move has completed successfully.
        Console.WriteLine("Depth Of " & AbsoluteDepth + 3 & " Completed in: " & AIStopwatch.ElapsedMilliseconds & " Milliseconds." & vbCrLf)
        AmendLegalMoveSquares(AIBestMoves(5))
        AIFinishedSearch = True
        CurrentDepth = AbsoluteDepth + 3
        CurrentMove = SharedAlgorithms.MoveConverter(MasterBoard,
AIBestMoves(5), MasterEnPassant)
        CurrentEvaluation = AIBestMoves(5).Score
        If Math.Abs(AIBestMoves(5).Score) > 900 Then
            'Checkmating sequence found - terminate all other AIs
            .
                AI3.ABORT = True
                MainAI.ABORT = True
                AI4.ABORT = True
                AI5.ABORT = True
            End If
        End If
    End If
End Sub

'The below 4 subroutines are very similar to the one above - just for different depths.
'(also note that, apart from the immediately below subroutine, the below algorithms do
'not contain the 'increase to higher depth once completed' mechanism).
Private Sub InitialiseThread2()
    AIBestMoves(1) = AI3.Search(AbsoluteDepth - 1, UseQuiescence)
    If Not AI3.ABORT Then
        Console.WriteLine("Depth Of " & AbsoluteDepth - 1 & " Completed in: " & AIStopwatch.ElapsedMilliseconds & " Milliseconds." & vbCrLf)
        AmendLegalMoveSquares(AIBestMoves(1))
        AIFinishedSearch = True
        CurrentDepth = AbsoluteDepth - 1
        CurrentMove = SharedAlgorithms.MoveConverter(MasterBoard, AIBestMoves(1), MasterEnPassant)
        CurrentEvaluation = AIBestMoves(1).Score
        If Math.Abs(AIBestMoves(1).Score) > 900 Then
            MainAI.ABORT = True
            AI4.ABORT = True
            AI5.ABORT = True
            AI2.ABORT = True
        ElseIf AIStopwatch.ElapsedMilliseconds / 500 < TimeForSearch Then
            AIMovedToHigherDepth(1) = True
            AIBestMoves(6) = AI3.Search(AbsoluteDepth + 4, UseQuiescence)
            If Not AI3.ABORT Then
                Console.WriteLine("Depth Of " & AbsoluteDepth + 4 & " Completed in: " & AIStopwatch.ElapsedMilliseconds & " Milliseconds." & vbCrLf)
                AmendLegalMoveSquares(AIBestMoves(6))
                AIFinishedSearch = True
                CurrentDepth = AbsoluteDepth + 4
                CurrentMove = SharedAlgorithms.MoveConverter(MasterBoard,
AIBestMoves(6), MasterEnPassant)
                CurrentEvaluation = AIBestMoves(6).Score
                If Math.Abs(AIBestMoves(6).Score) > 900 Then

```



```

        MainAI.ABORT = True
        AI4.ABORT = True
        AI5.ABORT = True
        AI2.ABORT = True
    End If
End If
End If
End Sub
Private Sub InitialiseThread3()
    AIBestMoves(2) = MainAI.Search(AbsoluteDepth, UseQuiescence)
    If Not MainAI.ABORT Then
        Console.WriteLine("Depth Of " & AbsoluteDepth & " Completed in: "
& AISTopwatch.ElapsedMilliseconds & " Milliseconds." & vbCrLf)
        AmendLegalMoveSuares(AIBestMoves(2))
        AIFinishedSearch = True
        CurrentDepth = AbsoluteDepth
        CurrentMove = SharedAlgorithms.MoveConverter(MasterBoard, AIBestM
oves(2), MasterEnPassant)
        CurrentEvaluation = AIBestMoves(2).Score
        If Math.Abs(AIBestMoves(2).Score) > 900 Then
            AI4.ABORT = True
            AI5.ABORT = True
            AI2.ABORT = True
            AI3.ABORT = True
        End If
    End If
End Sub
Private Sub InitialiseThread4()
    AIBestMoves(3) = AI4.Search(AbsoluteDepth + 1, UseQuiescence)
    If Not AI4.ABORT Then
        Console.WriteLine("Depth Of " & AbsoluteDepth + 1 & " Completed i
n: " & AISTopwatch.ElapsedMilliseconds & " Milliseconds." & vbCrLf)
        AmendLegalMoveSuares(AIBestMoves(3))
        AIFinishedSearch = True
        CurrentDepth = AbsoluteDepth + 1
        CurrentMove = SharedAlgorithms.MoveConverter(MasterBoard, AIBestM
oves(3), MasterEnPassant)
        CurrentEvaluation = AIBestMoves(3).Score
        If Math.Abs(AIBestMoves(3).Score) > 900 Then
            AI5.ABORT = True
            AI2.ABORT = True
            AI3.ABORT = True
            MainAI.ABORT = True
        End If
    End If
End Sub
Private Sub InitialiseThread5()
    AIBestMoves(4) = AI5.Search(AbsoluteDepth + 2, UseQuiescence)
    If Not AI5.ABORT Then
        Console.WriteLine("Depth Of " & AbsoluteDepth + 2 & " Completed i
n: " & AISTopwatch.ElapsedMilliseconds & " Milliseconds." & vbCrLf)
        AmendLegalMoveSuares(AIBestMoves(4))
        AIFinishedSearch = True
        CurrentDepth = AbsoluteDepth + 2
        CurrentMove = SharedAlgorithms.MoveConverter(MasterBoard, AIBestM
oves(4), MasterEnPassant)
        CurrentEvaluation = AIBestMoves(4).Score
        If Math.Abs(AIBestMoves(4).Score) > 900 Then
            AI2.ABORT = True

```

```

        AI3.ABORT = True
        MainAI.ABORT = True
        AI2.ABORT = True
    End If
End If
End Sub

```

'Subroutine that makes a move on the board, given coordinates. Includes castling (& rights) and pawn promotion.

'Note: This algorithm is very similar to the MakeMove subroutine in the CoreMethods Class, however here material

'count is not taken into account, and there is an option for sounds to be played when a move is made. This is removed

'in the CoreMethods version of this algorithm for efficiency purposes for MiniMax (less IF statements).

```

Private Sub MakeMove(ByVal Board(,) As Char, ByVal OldCoorX As String, ByVal OldCoorY As String, ByVal NewCoorX As String, ByVal NewCoorY As String, ByRef CanCastle As Boolean, ByRef KPos As String, ByRef EnPassant As String, ByVal MakeSounds As Boolean)

```

```

    Dim TempPiece As Char = Board(OldCoorX, OldCoorY)

```

```

    Dim HasEnPassanted As Boolean

```

```

    If MakeSounds Then Sound_Move.Play()

```

```

    If Char.IsUpper(TempPiece) Then

```

```

        If TempPiece = "P" Then

```

'Code for Promoting Pawns and En Passant. Also increments the material count.

```

        If NewCoorY = 0 Then

```

```

            TempPiece = "Q"

```

```

        ElseIf NewCoorX & NewCoorY = EnPassant Then

```

```

            Board(NewCoorX, NewCoorY + 1) = " "

```

```

            If MakeSounds Then Sound_Capture.Play()

```

```

            ElseIf OldCoorY = 6 AndAlso NewCoorY = 4 AndAlso (Board(Math.Max(NewCoorX - 1, 0), 4) = "p" OrElse Board(Math.Min(NewCoorX + 1, 7), 4) = "p") Then

```

```

                'EnPassant Creation.

```

```

                EnPassant = NewCoorX & 5

```

```

                HasEnPassanted = True

```

```

            End If

```

'If piece is a Rook, part of Castling is disabled (depending on which Rook has moved).

```

            ElseIf TempPiece = "R" AndAlso CanCastle.CanICastle Then

```

```

                If OldCoorX = 0 AndAlso OldCoorY = 7 Then

```

```

                    CanCastle.QS = False

```

```

                ElseIf OldCoorX = 7 AndAlso OldCoorY = 7 Then

```

```

                    CanCastle.KS = False

```

```

                End If

```

```

            ElseIf TempPiece = "K" Then

```

```

                KPos = NewCoorX & NewCoorY

```

```

                'Code for Castling.

```

```

                If CanCastle.KS AndAlso NewCoorX = "6" AndAlso NewCoorY = "7"

```

```

Then

```

```

                    Board(5, 7) = "R"

```

```

                    Board(7, 7) = " "

```

```

                    If MakeSounds Then Sound_Castle.Play()

```

```

                ElseIf CanCastle.QS AndAlso NewCoorX = "2" AndAlso NewCoorY =

```

```

"7" Then

```

```

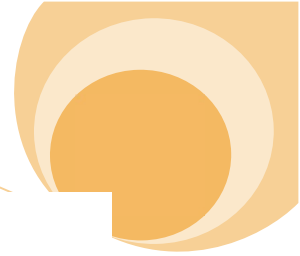
                    Board(0, 7) = " "

```

```

                    Board(3, 7) = "R"

```

```

        If MakeSounds Then Sound_Castle.Play()
    End If
    CanCastle.CannotCastle()
End If
Else
    'Identical Code for the Black Pieces.
    If TempPiece = "p" Then
        If NewCoorY = 7 Then
            TempPiece = "q"
        ElseIf NewCoorX & NewCoorY = EnPassant Then
            Board(NewCoorX, NewCoorY - 1) = " "
            If MakeSounds Then Sound_Capture.Play()
        ElseIf OldCoorY = 1 AndAlso NewCoorY = 3 AndAlso (Board(Math.
Max(NewCoorX - 1, 0), 3) = "P" OrElse Board(Math.Min(NewCoorX + 1, 7), 3) = "
P") Then
            'EnPassant Creation.
            EnPassant = NewCoorX & 2
            HasEnPassanted = True
        End If
    ElseIf TempPiece = "r" AndAlso CanCastle.CanICastle Then
        If OldCoorX = 0 AndAlso OldCoorY = 0 Then
            CanCastle.QS = False
        ElseIf OldCoorX = 7 AndAlso OldCoorY = 0 Then
            CanCastle.KS = False
        End If
    ElseIf TempPiece = "k" Then
        KPos = NewCoorX & NewCoorY
        If CanCastle.KS AndAlso NewCoorX = "6" AndAlso NewCoorY = "0"
Then
            Board(5, 0) = "r"
            Board(7, 0) = " "
            If MakeSounds Then Sound_Castle.Play()
        ElseIf CanCastle.QS AndAlso NewCoorX = "2" AndAlso NewCoorY =
"0" Then
            Board(0, 0) = " "
            Board(3, 0) = "r"
            If MakeSounds Then Sound_Castle.Play()
        End If
        CanCastle.CannotCastle()
    End If
End If

    If Not (EnPassant = "-" OrElse HasEnPassanted) Then EnPassant = "-"
    'Removal of EnPassant (if required).
    'At the end of the subroutine, the Piece is placed at the new coordin
ates, and the old position is cleared.
    'If the new position contains a piece, then the material count is upd
ated for only that piece.
    If Board(NewCoorX, NewCoorY) <> " " AndAlso MakeSounds Then Sound_Cap
ture.Play()
    Board(NewCoorX, NewCoorY) = TempPiece
    Board(OldCoorX, OldCoorY) = " "
End Sub

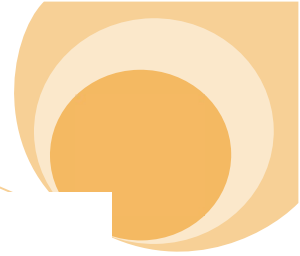
```

'Checks for Checkmate & Stalemate are calculated by running the opponent AI on a special mode,
 'where the AI runs to a depth of 1 but makes no moves on the board. This determines if the opponent

'actually has any legal moves (or if there is only 1 forced move), and if they don't, then the game will end. This algorithm also checks for draws by insufficient material. If the procedure does detect any of these end states, then it stops the game and/or notifies the user (depending on gamemode).

```
Private Sub EnforceEndStates()
    Dim TempMove = MainAI.CheckForEndState()
    Select Case TempMove.EndState
        Case "c" 'Position is checkmate.
            CheckLabel.Text = "Checkmate!"
            Sound_Checkmate.Play()
            GameRunning = False
            Console.WriteLine("The Game has Ended. Cause = Checkmate.")
            If GameMode < 3 Then NotifyGameEnd("c")
            Exit Sub
        Case "s" 'Position is stalemate.
            CheckLabel.Text = "Stalemate!"
            Sound_Stalemate.Play()
            GameRunning = False
            Console.WriteLine("The Game has Ended. Cause = Stalemate.")
            If GameMode < 3 Then NotifyGameEnd("d")
            Exit Sub
    End Select

    'Draw by insufficient material is then checked for. In principle, if
    it is physically impossible for one
    'player to checkmate the other (such as king vs king, or king vs king
    + a knight / bishop), then the
    'position is declared 'dead' and the game ends in a draw.
    Dim TempMaterialCount As SByte() = SharedAlgorithms.CountMaterial(Mas
terBoard)
    If TempMaterialCount(0) + TempMaterialCount(1) = 0 Then 'only kings r
emain.
        CheckLabel.Text = "Draw!"
        Sound_Stalemate.Play()
        GameRunning = False
        Console.WriteLine("The Game has Ended. Cause = Draw by Insufficie
nt Material (K v K).")
        If GameMode < 3 Then NotifyGameEnd("d")
        ElseIf TempMaterialCount(0) + TempMaterialCount(1) = 3 Then 'could be
king vs king + knight / bishop.
            For y = 0 To 7
                For x = 0 To 7
                    'scans for knights / bishops.
                    If UCase(MasterBoard(x, y)) = "B" OrElse UCase(MasterBoar
d(x, y)) = "N" Then
                        'Game ends in a draw.
                        CheckLabel.Text = "Draw!"
                        Sound_Stalemate.Play()
                        GameRunning = False
                        Console.WriteLine("The Game now Ended. Cause = Draw b
y Insufficient Material (K v K+B/N).")
                        If GameMode < 3 Then NotifyGameEnd("d")
                        Exit Sub
                    End If
                Next
            Next
        ElseIf TempMaterialCount(0) = 3 AndAlso TempMaterialCount(1) = 3 Then
            'only possibility is king + bishop vs king + bishop (of same type).
```



```

Dim NoOfBishopsFound As Byte
Dim BishopType As Boolean 'True = Light, False = Dark
For y = 0 To 7
    For x = 0 To 7
        If UCase(MasterBoard(x, y)) = "B" Then
            Select Case NoOfBishopsFound
                Case 0
                    'Calculates if the bishop is on a light square
                    'or a dark square.
                    If (x Mod 2 = 0 AndAlso y Mod 2 = 0) OrElse (
x Mod 2 = 1 AndAlso y Mod 2 = 1) Then
                        BishopType = True
                    Else
                        BishopType = False
                    End If
                Case 1
                    'If this bishop is on the same colour complex
                    (light / dark) as the previous
                    bishop,
                    'then we end the game.
                    If (x Mod 2 = 0 AndAlso y Mod 2 = 0 AndAlso B
ishopType) OrElse (x Mod 2 = 1 AndAlso y Mod 2 = 1 AndAlso BishopType) OrElse
(x Mod 2 = 1 AndAlso y Mod 2 = 0 AndAlso Not BishopType) OrElse (x Mod 2 = 0
AndAlso y Mod 2 = 1 AndAlso Not BishopType) Then
                        CheckLabel.Text = "      Draw!      "
                        Sound_Stalemate.Play()
                        GameRunning = False
                        Console.WriteLine("The Game has now Ended
. Cause = Draw by Insufficient Material (K v K+B+B).")
                        If GameMode < 3 Then NotifyGameEnd("d")
                    End If
                End Select
                NoOfBishopsFound += 1
            ElseIf Not (UCase(MasterBoard(x, y)) = "K" OrElse UCase(M
asterBoard(x, y)) = " ") Then
                'Another piece found - exit search.
                Exit Sub
            End If
        Next
    Next
End Sub

'Notifies the user if the game has concluded - giving them an appropriate
message and an option to play again.
'If they deny, then they are sent back to the main menu.
Private Sub NotifyGameEnd(ByVal EndState As Char)
    Dim Text As String
    If GameMode = 1 Then
        If EndState = "c" Then
            'Checks if the player or the AI is the winner of the game.
            If (MasterWinCheck.IsInCheck AndAlso Not UserPlayer) OrElse (
MasterBinCheck.IsInCheck AndAlso UserPlayer) Then
                Text = "Congratulations - You have Beaten the AI!"
            Else
                Text = "Unfortunately, the AI has Beaten you."
            End If
        Else
            Text = "The Game has Concluded in a Draw!"
        End If
    Else 'Gives unbiased comment.

```

```

Text = "The Game has Concluded"
If EndState = "c" Then
    If MasterBInCheck.IsInCheck Then
        Text &= ", with White Victorious!"
    Else
        Text &= ", with Black Victorious!"
    End If
Else
    Text &= " in a Draw!"
End If
End If

Application.DoEvents() 'Finishes making the move.
Thread.Sleep(1000)
'Displays popup message.
If MsgBox(Text & vbCr & "Would you Like to Play Again?", vbInformatio
n + vbYesNo + vbApplicationModal) = 6 Then
    'User wants to play again - reset the board.
    Reset_Btn_Click()
    PreviousFEN = CurrentFEN
Else 'Quit.
    ExitBtn_Click()
End If
End Sub

End Class

```

AI CLASS

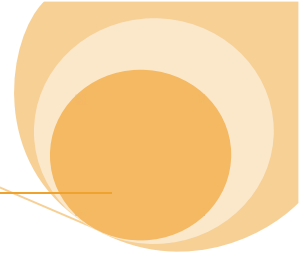
'This class contains my Chess AI, which involves the MiniMax algorithm with Alpha-Beta Pruning, Move ordering, Quiescence (optional), along with the ability to check for end states. This class is modular from my Chess class - being constructed only from the FEN position, and only returning a Move (see the structure below). Some other interacting is done, however, such as allowing the AI to be remotely aborted.

```

Public Class AI
    Inherits CoreMethods
    Private Shared HasBeenInstantiated As Boolean 'The AI will not perform methods if it has not been fully instantiated with a FEN.
    'Below are the details that the AI requires for a search. Please see their counterparts in the Chess form for their info.
    Private Shared PrimaryBoard(7, 7), PrimaryTFTable(7, 7) As Char
    Private Shared PrimaryMeCanCastle, PrimaryEnemyCanCastle As New CanCastle
    Private Shared PrimaryMeInCheck As New InCheck
    Private Shared PrimaryMeKPos, PrimaryEnemyKPos As String
    Private Shared PrimaryEnPasant As String
    Private Shared PrimaryMaterialCount(1) As SByte
    Private Shared PlayerTurn As Boolean

    Private Shared KingSymbol As Char '"K" for white, "k" for black. Used for helping to resolve checks.
    Private UseQuiescence As Boolean 'Set by the user - determines whether the AI will use the Quiescence algorithm.
    Private TotalPositionsSearched As UInt32
    Public ABORT As Boolean 'Controlled by the thread handlers - an AI is aborted if another AI has
    'found a mating pattern, or if the AI has ran out of time.

```



BASE CONTROLS

```

'Constructor methods.
Public Sub New()
    'AI variables instantiated but without a FEN. Used when the program i
s first booted up.
End Sub
Public Sub New(ByVal FEN As String)
    'Configures the AI using a given FEN.
    Reconfigure(FEN)
End Sub

'Subroutine which converts a user's input FEN into all the details needed
to conduct a MiniMax search.
Public Sub Reconfigure(ByVal FEN As String)
    'Converts the user's FEN into a board position, then resets checking
rules.
    PrimaryBoard = FENConverter(FEN, PrimaryMeCanCastle, PrimaryEnemyCanC
astle, PrimaryMeKPos, PrimaryEnemyKPos, PrimaryEnPassant, PlayerTurn)
    PrimaryMeInCheck.NotInCheck()
    If PlayerTurn Then
        'Creates the TFTable for the white pieces, then creates king symb
ol.
        FixTFTables(PrimaryBoard, True, PrimaryTFTable, PrimaryMeKPos, Pr
imaryMeInCheck, NotInCheck, PrimaryEnPassant)
        KingSymbol = "K"
    Else
        'Swaps the Primary & Enemy Castling privileges, along with the Pr
imary & Enemy King privileges.
        'This is because, for the AI, all variables are in context of whi
ch player the AI is controlling,
        'so 'me' and 'enemy' can refer to different colours depending on
the board position.
        Dim TempCanCastle As New CanCastle
        TempCanCastle.CopyFrom(PrimaryMeCanCastle)
        PrimaryMeCanCastle.CopyFrom(PrimaryEnemyCanCastle)
        PrimaryEnemyCanCastle.CopyFrom(TempCanCastle)
        Dim TempKPos As String = PrimaryMeKPos
        PrimaryMeKPos = PrimaryEnemyKPos
        PrimaryEnemyKPos = TempKPos
        'Creates the TFTable for the black pieces, then creates king symb
ol.
        FixTFTables(PrimaryBoard, False, PrimaryTFTable, PrimaryMeKPos, N
otInCheck, PrimaryMeInCheck, PrimaryEnPassant)
        KingSymbol = "k"
    End If
    'Finds the material count of the board.
    PrimaryMaterialCount = CountMaterial(PrimaryBoard)
    HasBeenInstantiated = True
End Sub

'Algorithm that finds the AI's 'Best Move' for a given position, using th
e MiniMax algorithm.
Public Function Search(ByVal Depth As SByte, ByVal UserQuiescence As Bool
ean) As Move
    Dim BestMove As New Move
    Dim CurrentScore As Decimal
    BestMove.Score = Integer.MinValue
    If HasBeenInstantiated AndAlso Depth > 0 Then

```

```

UseQuiescence = UserQuiescence
'Creates alpha (white's best move) and beta (black's best move).
Dim Alpha As Decimal = Integer.MinValue
Dim Beta As Decimal = Integer.MaxValue
ABORT = False
TotalPositionsSearched = 0
'Creates the pseudo-legal moves for the chosen player.
Dim PieceMoves(200, 2) As String
If PlayerTurn Then
    PieceMoves = CreateMoves(PrimaryBoard, True, PrimaryTFTable,
PrimaryEnemyKPos, PrimaryMeInCheck, NotInCheck, PrimaryMeCanCastle, PrimaryEn
Passant, False)
Else
    PieceMoves = CreateMoves(PrimaryBoard, False, PrimaryTFTable,
PrimaryEnemyKPos, NotInCheck, PrimaryMeInCheck, PrimaryMeCanCastle, PrimaryEn
nPassant, False)
End If
If PieceMoves(0, 0) > 0 Then 'If any move exists...
    'Creates temp variables.
    Dim TempBoard(7, 7) As Char
    Dim TempMaterialCount(1) As SByte
    Dim TempMeKPos As String
    Dim TempMeCanCastle As New CanCastle
    Dim TempMeInCheck As New InCheck
    Dim TempEnPassant As String
    For n = 1 To CInt(PieceMoves(0, 0)) 'for each move...
        If ABORT Then Exit For
        'If the user is in check, eliminate the moves that do not
        escape check. King moves are not considered
        'as the player's TFTable will ensure that all king moves
        are legal.
        If PrimaryMeInCheck.IsInCheck AndAlso PrimaryBoard(PieceM
oves(n, 0), PieceMoves(n, 1)) <> KingSymbol Then
            'Runs move through the DoesMoveResolveCheck algorithm
            .
            TempMeInCheck.CopyFrom(PrimaryMeInCheck)
            If PlayerTurn AndAlso DoesMoveResolveCheck(PrimaryBoa
rd, PieceMoves(n, 0), PieceMoves(n, 1), PieceMoves(n, 2)(0), PieceMoves(n, 2)
(1), TempMeInCheck, NotInCheck, PrimaryEnPassant) Then
                'Move has resolved check.
                TempMeInCheck.NotInCheck()
            ElseIf Not PlayerTurn AndAlso DoesMoveResolveCheck(Pr
imaryBoard, PieceMoves(n, 0), PieceMoves(n, 1), PieceMoves(n, 2)(0), PieceMov
es(n, 2)(1), NotInCheck, TempMeInCheck, PrimaryEnPassant) Then
                'Move has resolved check.
                TempMeInCheck.NotInCheck()
            End If
        Else
            TempMeInCheck.IsInCheck = False
        End If
        If Not TempMeInCheck.IsInCheck Then 'Therefore, is a lega
l move.
            'Copies board info to temp variables.
            Array.Copy(PrimaryBoard, TempBoard, 64)
            Array.Copy(PrimaryMaterialCount, TempMaterialCount, 2
)

            TempMeKPos = PrimaryMeKPos
            TempMeCanCastle.CopyFrom(PrimaryMeCanCastle)
            TempEnPassant = PrimaryEnPassant

```




```

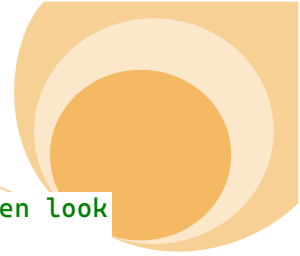
        'Makes move on temp board, then calls MiniMax for this
s new position.
        MakeMove(TempBoard, PieceMoves(n, 0), PieceMoves(n, 1
), PieceMoves(n, 2)(0), PieceMoves(n, 2)(1), TempMeCanCastle, TempMeKPos, TempMaterialCount, TempEnPassant)
        If Depth = 1 AndAlso Not UseQuiescence Then
            'We have reached a leaf position - return the evaluation for this position.
            TotalPositionsSearched += 1
            If PlayerTurn Then
                CurrentScore = Evaluate(TempMaterialCount, TempMeKPos, PrimaryEnemyKPos)
            Else
                CurrentScore = Evaluate(TempMaterialCount, PrimaryEnemyKPos, TempMeKPos)
            End If
            ElseIf PlayerTurn Then
                CurrentScore = MiniMax(TempBoard, Depth - 1, False, TempMeCanCastle, PrimaryEnemyCanCastle, TempMeKPos, PrimaryEnemyKPos, TempEnPassant, TempMaterialCount, Alpha, Beta)
            Else
                'Alpha & Beta are replaced with -Beta & -Alpha so that all moves' scores are given in context
                'of the player to move. This is so that the largest number will always be the best score.
                CurrentScore = -
MiniMax(TempBoard, Depth - 1, True, PrimaryEnemyCanCastle, TempMeCanCastle, PrimaryEnemyKPos, TempMeKPos, TempEnPassant, TempMaterialCount, -Beta, -Alpha)
            End If
            Alpha = Math.Max(Alpha, CurrentScore)
            If CurrentScore > BestMove.Score Then
                'Move has been beaten - replace it.
                BestMove.Score = CurrentScore
                BestMove.OldMoveX = PieceMoves(n, 0)
                BestMove.OldMoveY = PieceMoves(n, 1)
                BestMove.NewMoveX = PieceMoves(n, 2)(0)
                BestMove.NewMoveY = PieceMoves(n, 2)(1)
            End If
        End If
    Next
End If
If ABORT Then
    'Search has been terminated / no move found - make a note on the Move.
    BestMove.EndState = "a"
Else 'Search successfully completed.
    If Not PlayerTurn Then BestMove.Score = -BestMove.Score
    Console.WriteLine("Depth Of " & Depth & " Completed. Move = " & MoveConverter(PrimaryBoard, BestMove, PrimaryEnPassant) & ", with Evaluation = " & BestMove.Score & vbCrLf & "Positions searched = " & TotalPositionsSearched & vbCrLf)
End If
Else 'AI not correctly instantiated.
    Console.WriteLine("Error when Attempting Search - FEN Position not set / Depth set too low.")
    BestMove.EndState = "a"
End If
Return BestMove
End Function

```

```

'Function that returns all the legal moves of a given piece on the board.
'Used for when the user is attempting to move a piece on the GUI.
Public Function ReturnPiecesLegalMoves(ByVal CoorX As String, ByVal CoorY
As String) As String()
    Dim LegalMoves(27) As String
    If HasBeenInstantiated Then
        Dim TotalMoves As Byte = 0
        'Creates the pseudo-legal moves for the chosen player.
        Dim PieceMoves(27) As String
        If PlayerTurn AndAlso Char.IsUpper(PrimaryBoard(CoorX, CoorY)) Th
en
            PieceMoves = WhitePieceLegalMoves(PrimaryBoard, CoorX, CoorY,
PrimaryTFTable, TrueTable, PrimaryEnemyKPos, NotInCheck, PrimaryMeInCheck, P
rimaryMeCanCastle, PrimaryEnPassant)
        ElseIf Not (PlayerTurn OrElse Char.IsUpper(PrimaryBoard(CoorX, Co
orY))) Then
            PieceMoves = BlackPieceLegalMoves(PrimaryBoard, CoorX, CoorY,
PrimaryTFTable, TrueTable, PrimaryEnemyKPos, NotInCheck, PrimaryMeInCheck, P
rimaryMeCanCastle, PrimaryEnPassant)
        Else
            Console.WriteLine("Error - Illegal to Move Piece.")
            Return Nothing
        End If
        If PieceMoves(0) IsNot Nothing Then 'If any move exists...
            Dim TempMeInCheck As New InCheck 'Creates temp check class.
            For n = 1 To CInt(PieceMoves(0)) - 1 'for each move...
                'If the user is in check, eliminate the moves that do not
                escape check. King moves are not considered
                'as the player's TFTable will ensure that all king moves
                are legal.
                If PrimaryMeInCheck.IsInCheck AndAlso PrimaryBoard(CoorX,
CoorY) <> KingSymbol Then
                    'Runs move through the DoesMoveResolveCheck algorithm
                    .
                    TempMeInCheck.CopyFrom(PrimaryMeInCheck)
                    If PlayerTurn AndAlso DoesMoveResolveCheck(PrimaryBoa
rd, CoorX, CoorY, CStr(PieceMoves(n)(0)), CStr(PieceMoves(n)(1)), TempMeInChe
ck, NotInCheck, PrimaryEnPassant) Then
                        'Move has resolved check.
                        TempMeInCheck.NotInCheck()
                    ElseIf Not PlayerTurn AndAlso DoesMoveResolveCheck(Pr
imaryBoard, CoorX, CoorY, CStr(PieceMoves(n)(0)), CStr(PieceMoves(n)(1)), Not
InCheck, TempMeInCheck, PrimaryEnPassant) Then
                        'Move has resolved check.
                        TempMeInCheck.NotInCheck()
                    End If
                Else
                    TempMeInCheck.IsInCheck = False
                End If
                If Not TempMeInCheck.IsInCheck Then
                    'The move is legal - update the list of the piece's l
                    egal moves.
                    TotalMoves += 1
                    LegalMoves(TotalMoves) = PieceMoves(n)
                End If
            Next
        End If
        'Make the first index of the array to be the total amount of lega
        l moves.

```



```

        'This helps know how many indexes of the array to check when looking for legal moves.
        LegalMoves(0) = TotalMoves
    Else 'AI not correctly instantiated.
        Console.WriteLine("Error when Attempting Search - FEN Position not set.")
    End If
    Return LegalMoves
End Function

'Function that scans the position for End States - positions where the game must terminate.
Public Function CheckForEndState() As Move
    Dim CurrentMove As New Move
    If HasBeenInstantiated Then
        CurrentMove.EndState = "c" 'Result defaults to checkmate unless proven otherwise.
        Dim TotalMoves As Byte = 0
        'Creates the pseudo-legal moves for the chosen player.
        Dim PieceMoves(200, 2) As String
        If PlayerTurn Then
            PieceMoves = CreateMoves(PrimaryBoard, True, PrimaryTFTable, PrimaryEnemyKPos, PrimaryMeInCheck, NotInCheck, PrimaryMeCanCastle, PrimaryEnPassant, False)
        Else
            PieceMoves = CreateMoves(PrimaryBoard, False, PrimaryTFTable, PrimaryEnemyKPos, NotInCheck, PrimaryMeInCheck, PrimaryMeCanCastle, PrimaryEnPassant, False)
        End If
        If PieceMoves(0, 0) > 0 Then 'If any move exists...
            Dim TempMeInCheck As New InCheck 'Creates temp check class.
            For n = 1 To CInt(PieceMoves(0, 0)) 'for each move...
                'If the user is in check, eliminate the moves that do not escape check. King moves are not considered
                'as the player's TFTable will ensure that all king moves are legal.
                If PrimaryMeInCheck.IsInCheck AndAlso PrimaryBoard(PieceMoves(n, 0), PieceMoves(n, 1)) <> KingSymbol Then
                    'Runs move through the DoesMoveResolveCheck algorithm
                    TempMeInCheck.CopyFrom(PrimaryMeInCheck)
                    If PlayerTurn AndAlso DoesMoveResolveCheck(PrimaryBoard, PieceMoves(n, 0), PieceMoves(n, 1), PieceMoves(n, 2)(0), PieceMoves(n, 2)(1), TempMeInCheck, NotInCheck, PrimaryEnPassant) Then
                        'Move has resolved check.
                        TempMeInCheck.NotInCheck()
                    ElseIf Not PlayerTurn AndAlso DoesMoveResolveCheck(PrimaryBoard, PieceMoves(n, 0), PieceMoves(n, 1), PieceMoves(n, 2)(0), PieceMoves(n, 2)(1), NotInCheck, TempMeInCheck, PrimaryEnPassant) Then
                        'Move has resolved check.
                        TempMeInCheck.NotInCheck()
                    End If
                Else
                    TempMeInCheck.IsInCheck = False
                End If
                If Not TempMeInCheck.IsInCheck Then
                    'Therefore is a legal move. This means that there is no checkmate / stalemate in the position.
                    TotalMoves += 1
                    If TotalMoves > 1 Then

```

```

        'Multiple moves detected - set EndState code as s
        CurrentMove.EndState = "f"
        Return CurrentMove
    Else 'Only one (forced) move detected - note Move info.
        CurrentMove.OldMoveX = PieceMoves(n, 0)
        CurrentMove.OldMoveY = PieceMoves(n, 1)
        CurrentMove.NewMoveX = PieceMoves(n, 2)(0)
        CurrentMove.NewMoveY = PieceMoves(n, 2)(1)
        CurrentMove.EndState = "o"
    End If
End If
Next
ElseIf Not PrimaryMeInCheck.IsInCheck Then
    'Position is a stalemate.
    CurrentMove.EndState = "s"
End If
Else 'AI not correctly instantiated.
    Console.WriteLine("Error when Attempting Search - FEN Position not set.")
    CurrentMove.EndState = "a"
End If
Return CurrentMove
End Function

```

AI CORE – MOVE GENERATORS

'Function which creates and orders all the pseudo-legal moves a player can make, given certain criteria.

```

Private Function CreateMoves(ByVal Board(,) As Char, ByVal isWhite As Boolean, ByVal TrueFalseTable(,) As Char, ByVal KPos As String, ByVal WInCheck As Boolean, ByVal BInCheck As Boolean, ByVal CanCastle As Boolean, ByVal EnPassant As String, ByVal OnlyCaptures As Boolean) As String(,)

```

```

    Dim PieceMoves(27) As String
    'Creates category arrays, along with their lengths variables.
    Dim AmazingCaptureMoves(200, 2) As String
    Dim CaptureMoves(19, 2) As String
    Dim OtherCaptureMoves(19, 2) As String
    Dim PawnPromotionMoves(7, 2) As String
    Dim GoodMoves(49, 2) As String
    Dim OtherMoves(99, 2) As String
    Dim BadMoves(49, 2) As String
    Dim TerribleMoves(24, 2) As String

```

```

    Dim ArrLens(7) As Byte 'Represents the number of moves in each move category array.
    ArrLens(0) = 1

```

'dx and dy represent the horizontal & vertical distances between the piece and the enemy king.

```

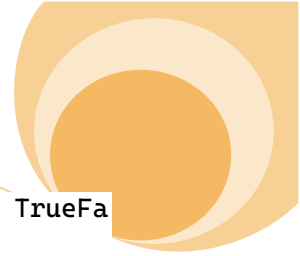
    Dim dx, dy As Byte
    Dim PieceValueDif As SByte 'Difference in weight between the capturing piece, and the piece being captured.

```

```

    If isWhite Then
        For y = 0 To 7
            For x = 0 To 7
                If Char.IsUpper(Board(x, y)) Then
                    'Generates the moves that the piece can make.

```



```

PieceMoves = WhitePieceLegalMoves(Board, x, y, TrueFa
lseTable, TrueTable, KPos, BInCheck, WInCheck, CanCastle, EnPassant)
    If PieceMoves(0) IsNot Nothing Then 'If there are any
        moves...
            For n = 1 To CInt(PieceMoves(0)) - 1 'for each mo
ve...
                If Board(CStr(PieceMoves(n)(0)), CStr(PieceMo
ves(n)(1))) <> " " Then '= capture move.
                    'Gets the difference in weight between th
e capturing piece, and the captured piece.
                    PieceValueDif = ReturnPieceValue(Board(CS
tr(PieceMoves(n)(0)), CStr(PieceMoves(n)(1)))) - ReturnPieceValue(Board(x, y)
)
                    If PieceValueDif >= 0 Then 'Capturing pie
ce weighs less than captured piece.
                        'Ammend move list.
                        AmazingCaptureMoves(ArrLens(0), 0) =
x
                        AmazingCaptureMoves(ArrLens(0), 1) =
y
                        AmazingCaptureMoves(ArrLens(0), 2) =
PieceMoves(n)
                        ArrLens(0) += 1
                    ElseIf PieceValueDif = 0 Then 'Capturing
piece weighs the same as captured piece.
                        'Ammend move list.
                        CaptureMoves(ArrLens(1), 0) = x
                        CaptureMoves(ArrLens(1), 1) = y
                        CaptureMoves(ArrLens(1), 2) = PieceMo
ves(n)
                        ArrLens(1) += 1
                    Else 'Capturing piece weighs more than ca
ptured piece.
                        'Ammend move list.
                        OtherCaptureMoves(ArrLens(2), 0) = x
                        OtherCaptureMoves(ArrLens(2), 1) = y
                        OtherCaptureMoves(ArrLens(2), 2) = Pi
eceMoves(n)
                        ArrLens(2) += 1
                    End If
                ElseIf Not OnlyCaptures Then 'Non-
Capture moves are not considered when Quiescence mode has been activated.
                    'Finds distances between piece & enemy pi
ece.
                    dx = Math.Abs(CStr(KPos(0)) - CStr(PieceM
oves(n)(0)))
                    dy = Math.Abs(CStr(KPos(1)) - CStr(PieceM
oves(n)(1)))
                    If PieceMoves(n)(1) = "0" AndAlso Board(x
, y) = "P" Then 'User is promoting a pawn.
                        'Ammend move list.
                        PawnPromotionMoves(ArrLens(3), 0) = x
                        PawnPromotionMoves(ArrLens(3), 1) = y
                        PawnPromotionMoves(ArrLens(3), 2) = P
ieceMoves(n)
                        ArrLens(3) += 1
                    ElseIf Board(Math.Max(CStr(PieceMoves(n)(
0)) - 1, 0), Math.Max(CStr(PieceMoves(n)(1)) - 1, 0)) = "p" OrElse Board(Math
.Min(CStr(PieceMoves(n)(0)) + 1, 7), Math.Max(CStr(PieceMoves(n)(1)) - 1, 0))
= "p" Then

```

pawn - ammend move list.

```

oves(n)
    'New square is controlled by an enemy
    TerribleMoves(ArrLens(7), 0) = x
    TerribleMoves(ArrLens(7), 1) = y
    TerribleMoves(ArrLens(7), 2) = PieceM

    ArrLens(7) += 1
    ElseIf TrueFalseTable(CStr(PieceMoves(n)(
0))), CStr(PieceMoves(n)(1))) = "F" Then
        'Piece is positioned on a "False" on
the TTable, meaning the square is controlled by an enemy piece.
        'Ammend move list.
        BadMoves(ArrLens(6), 0) = x
        BadMoves(ArrLens(6), 1) = y
        BadMoves(ArrLens(6), 2) = PieceMoves(
n)

        ArrLens(6) += 1
        ElseIf (Board(x, y) = "R" OrElse Board(x,
y) = "B" OrElse Board(x, y) = "Q") AndAlso (dx = 0 OrElse dy = 0 OrElse dx =
dy) Then
            'Piece may influece king's motion - l
eading to a possible check. Ammend move list.
            GoodMoves(ArrLens(4), 0) = x
            GoodMoves(ArrLens(4), 1) = y
            GoodMoves(ArrLens(4), 2) = PieceMoves
(n)

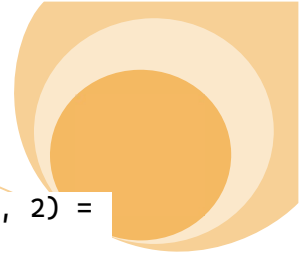
            ArrLens(4) += 1
            Else 'Is a regular move. Ammend move list

            OtherMoves(ArrLens(5), 0) = x
            OtherMoves(ArrLens(5), 1) = y
            OtherMoves(ArrLens(5), 2) = PieceMove

s(n)

            ArrLens(5) += 1
            End If
        End If
    Next
End If
Next
End If
Next
Next
Else 'Identical code for the black pieces.
    For y = 7 To 0 Step -1
        For x = 0 To 7
            If Char.IsLower(Board(x, y)) Then
                PieceMoves = BlackPieceLegalMoves(Board, x, y, TrueFa
lseTable, TrueTable, KPos, WInCheck, BInCheck, CanCastle, EnPassant)
                If PieceMoves(0) IsNot Nothing Then
                    For n = 1 To CInt(PieceMoves(0)) - 1
                        If Board(CStr(PieceMoves(n)(0)), CStr(PieceMo
ves(n)(1))) <> " " Then
                            PieceValueDif = ReturnPieceValue(Board(CS
tr(PieceMoves(n)(0)), CStr(PieceMoves(n)(1)))) - ReturnPieceValue(Board(x, y)
)
                            If PieceValueDif >= 0 Then
                                AmazingCaptureMoves(ArrLens(0), 0) =
x
                                AmazingCaptureMoves(ArrLens(0), 1) =
y

```

```

PieceMoves(n)
    AmazingCaptureMoves(ArrLens(0), 2) =
        ArrLens(0) += 1
        ElseIf PieceValueDif = 0 Then
            CaptureMoves(ArrLens(1), 0) = x
            CaptureMoves(ArrLens(1), 1) = y
            CaptureMoves(ArrLens(1), 2) = PieceMo

ves(n)
    ArrLens(1) += 1
    Else
        OtherCaptureMoves(ArrLens(2), 0) = x
        OtherCaptureMoves(ArrLens(2), 1) = y
        OtherCaptureMoves(ArrLens(2), 2) = Pi

ieceMoves(n)
    ArrLens(2) += 1
    End If
    ElseIf Not OnlyCaptures Then
        dx = Math.Abs(CStr(KPos(0)) - CStr(PieceM
oves(n)(0)))
        dy = Math.Abs(CStr(KPos(1)) - CStr(PieceM
oves(n)(1)))
        If PieceMoves(n)(1) = "7" AndAlso Board(x
, y) = "p" Then
            PawnPromotionMoves(ArrLens(3), 0) = x
            PawnPromotionMoves(ArrLens(3), 1) = y
            PawnPromotionMoves(ArrLens(3), 2) = P
ieceMoves(n)
            ArrLens(3) += 1
            ElseIf Board(Math.Max(CStr(PieceMoves(n)(
0)) - 1, 0), Math.Min(CStr(PieceMoves(n)(1)) + 1, 7)) = "p" OrElse Board(Math
.Min(CStr(PieceMoves(n)(0)) + 1, 7), Math.Min(CStr(PieceMoves(n)(1)) + 1, 7))
= "p" Then
                TerribleMoves(ArrLens(7), 0) = x
                TerribleMoves(ArrLens(7), 1) = y
                TerribleMoves(ArrLens(7), 2) = PieceM
oves(n)
                ArrLens(7) += 1
                ElseIf TrueFalseTable(CStr(PieceMoves(n)(
0)), CStr(PieceMoves(n)(1))) = "F" Then
                    BadMoves(ArrLens(6), 0) = x
                    BadMoves(ArrLens(6), 1) = y
                    BadMoves(ArrLens(6), 2) = PieceMoves(
n)
                    ArrLens(6) += 1
                    ElseIf (Board(x, y) = "r" OrElse Board(x,
y) = "b" OrElse Board(x, y) = "q") AndAlso (dx = 0 OrElse dy = 0 OrElse dx =
dy) Then
                        GoodMoves(ArrLens(4), 0) = x
                        GoodMoves(ArrLens(4), 1) = y
                        GoodMoves(ArrLens(4), 2) = PieceMoves
(n)
                        ArrLens(4) += 1
                        Else
                            OtherMoves(ArrLens(5), 0) = x
                            OtherMoves(ArrLens(5), 1) = y
                            OtherMoves(ArrLens(5), 2) = PieceMove
s(n)
                            ArrLens(5) += 1
                            End If
                        End If

```

```

        Next
      End If
    End If
  Next
Next
End If

```

'At the end of the function, we merge all the category arrays into one - producing a huge, tiered, 'list of a player's total pseudo-legal moves. If the user is in Quiescence mode, all non-capture moves 'are discounted, and are not merged.

```

    Array.Copy(CaptureMoves, 0, AmazingCaptureMoves, 3 * ArrLens(0), 3 *
ArrLens(1))
    Array.Copy(OtherCaptureMoves, 0, AmazingCaptureMoves, 3 * (ArrLens(0)
+ ArrLens(1)), 3 * ArrLens(2))
    ArrLens(0) += ArrLens(1) + ArrLens(2)
    If Not OnlyCaptures Then 'Copies all the non-
capture moves to the main array.
        If ArrLens(3) > 0 Then Array.Copy(PawnPromotionMoves, 0, AmazingC
aptureMoves, 3 * ArrLens(0), 3 * ArrLens(3))
        Array.Copy(GoodMoves, 0, AmazingCaptureMoves, 3 * (ArrLens(0) + A
rrLens(3)), 3 * ArrLens(4))
        Array.Copy(OtherMoves, 0, AmazingCaptureMoves, 3 * (ArrLens(0) +
ArrLens(3) + ArrLens(4)), 3 * ArrLens(5))
        ArrLens(0) += ArrLens(3) + ArrLens(4) + ArrLens(5)
        Array.Copy(BadMoves, 0, AmazingCaptureMoves, 3 * ArrLens(0), 3 *
ArrLens(6))
        Array.Copy(TerribleMoves, 0, AmazingCaptureMoves, 3 * (ArrLens(0)
+ ArrLens(6)), 3 * ArrLens(7))
        ArrLens(0) += ArrLens(6) + ArrLens(7)
    End If
    'Make the first index of the array to be the total amount of pseudo-
legal moves.
    'This helps know how many indexes of the array to check when looking
for moves.
    AmazingCaptureMoves(0, 0) = ArrLens(0) - 1
    Return AmazingCaptureMoves
End Function

```

'Function which receives a game position and a possible move. The function makes this move on the board (using 'shortcuts that can only be made on a virtual board), and then generates the possible moves of the attacking 'piece(s). If the king is no longer being threatened, then the check has been resolved.

```

Private Function DoesMoveResolveCheck(ByVal Board(,) As Char, ByVal OldPosX As String, ByVal OldPosY As String, ByVal NewPosX As String, ByVal NewPosY As String, ByRef WInCheck As InCheck, ByRef BInCheck As InCheck, ByVal EnPassant As String) As Boolean

```

'If the player is in a double check, then only king moves could be legal. Therefore, as this part is done in the

'TFTable generation, we skip this here.

```

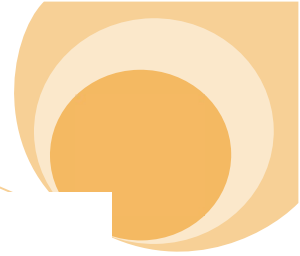
    If WInCheck.IsInCheck AndAlso Not WInCheck.DoubleCheck Then
        If NewPosX & NewPosY = WInCheck.Piece OrElse (NewPosX & NewPosY =
EnPassant AndAlso Board(OldPosX, OldPosY) = "P") Then
            'Move captures the attacking piece. Therefore we assume it is
legal.

```

```

        Return True
    End If
End Function

```



```

ElseIf Board(NewPosX, NewPosY) = " " Then
    WInCheck.IsInCheck = False
    Board(NewPosX, NewPosY) = "0" 'Move made on temporary board.
    'Calculate the legal moves of the attacking piece. If the king
    'is still in check, then WInCheck.IsInCheck
    'will flag from False to True - therefore it is illegal.
    BlackPieceLegalMoves(Board, CStr(WInCheck.Piece(0)), CStr(WIn
Check.Piece(1)), TrueTable, TrueTable, "00", WInCheck, BInCheck, CannotCastle
, "-")
    Board(NewPosX, NewPosY) = " " 'Move unmade on temporary board
.
    If WInCheck.IsInCheck = False Then Return True
    Else 'Move is a capture move, but not capturing the attacking pie
ce. Therefore, it has to be illegal.
        Return False
    End If
ElseIf BInCheck.IsInCheck AndAlso Not BInCheck.DoubleCheck Then
    'Identical code but for the black pieces.
    If NewPosX & NewPosY = BInCheck.Piece OrElse (NewPosX & NewPosY =
EnPassant AndAlso Board(OldPosX, OldPosY) = "p") Then
        Return True
    ElseIf Board(NewPosX, NewPosY) = " " Then
        BInCheck.IsInCheck = False
        Board(NewPosX, NewPosY) = "o"
        WhitePieceLegalMoves(Board, CStr(BInCheck.Piece(0)), CStr(BIn
Check.Piece(1)), TrueTable, TrueTable, "00", BInCheck, WInCheck, CannotCastle
, "-")
        Board(NewPosX, NewPosY) = " "
        If BInCheck.IsInCheck = False Then Return True
    Else
        Return False
    End If
End If
Return False
End Function

```

'Subroutine that makes a move on the board, given coordinates. Includes castling (& rights) and pawn promotion.

```

Private Sub MakeMove(ByVal Board(,) As Char, ByVal OldCoorX As String, By
Val OldCoorY As String, ByVal NewCoorX As String, ByVal NewCoorY As String, B
yRef CanCastle As CanCastle, ByRef KPos As String, ByRef MaterialCount() As S
Byte, ByRef EnPassant As String)
    Dim TempPiece As Char = Board(OldCoorX, OldCoorY)
    Dim HasEnPassanted As Boolean
    If Char.IsUpper(TempPiece) Then
        If TempPiece = "P" Then
            'Code for Promoting Pawns and En Passant. Also increments the
            material count.
            If NewCoorY = 0 Then
                TempPiece = "Q"
                MaterialCount(0) += ReturnPieceValue("Q") - ReturnPieceVa
lue("P") ' + 9 for a new queen, - 1 for losing the pawn in the process.
            ElseIf NewCoorX & NewCoorY = EnPassant Then
                Board(NewCoorX, NewCoorY + 1) = " "
                MaterialCount(1) -= ReturnPieceValue("P")
            ElseIf OldCoorY = 6 AndAlso NewCoorY = 4 AndAlso (Board(Math.
Max(NewCoorX - 1, 0), 4) = "p" OrElse Board(Math.Min(NewCoorX + 1, 7), 4) = "
p") Then
                'EnPassant creation.

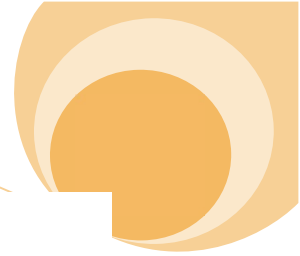
```

```

        EnPassant = NewCoorX & 5
        HasEnPassanted = True
    End If
    'If piece is a Rook, part of Castling is disabled (depending
on which Rook has moved).
    ElseIf TempPiece = "R" AndAlso CanCastle.CanICastle Then
        If OldCoorX = 0 AndAlso OldCoorY = 7 Then
            CanCastle.QS = False
        ElseIf OldCoorX = 7 AndAlso OldCoorY = 7 Then
            CanCastle.KS = False
        End If
    ElseIf TempPiece = "K" Then
        KPos = NewCoorX & NewCoorY
        'Code for Castling.
        If CanCastle.KS AndAlso NewCoorX = 6 AndAlso NewCoorY = 7 The
n
            Board(5, 7) = "R"
            Board(7, 7) = " "
        ElseIf CanCastle.QS AndAlso NewCoorX = 2 AndAlso NewCoorY = 7
Then
            Board(0, 7) = " "
            Board(3, 7) = "R"
        End If
        CanCastle.CannotCastle()
    End If
Else
    'Identical Code for the Black Pieces.
    If TempPiece = "p" Then
        If NewCoorY = 7 Then
            TempPiece = "q"
            MaterialCount(1) += ReturnPieceValue("Q") - ReturnPieceVa
lue("P")

            ElseIf NewCoorX & NewCoorY = EnPassant Then
                Board(NewCoorX, NewCoorY - 1) = " "
                MaterialCount(0) -= ReturnPieceValue("P")
            ElseIf OldCoorY = 1 AndAlso NewCoorY = 3 AndAlso (Board(Math.
Max(NewCoorX - 1, 0), 3) = "P" OrElse Board(Math.Min(NewCoorX + 1, 7), 3) = "
P") Then
                EnPassant = NewCoorX & 2
                HasEnPassanted = True
            End If
        ElseIf TempPiece = "r" AndAlso CanCastle.CanICastle Then
            If OldCoorX = 0 AndAlso OldCoorY = 0 Then
                CanCastle.QS = False
            ElseIf OldCoorX = 7 AndAlso OldCoorY = 0 Then
                CanCastle.KS = False
            End If
        ElseIf TempPiece = "k" Then
            KPos = NewCoorX & NewCoorY
            If CanCastle.KS AndAlso NewCoorX = 6 AndAlso NewCoorY = 0 The
n
                Board(5, 0) = "r"
                Board(7, 0) = " "
            ElseIf CanCastle.QS AndAlso NewCoorX = 2 AndAlso NewCoorY = 0
Then
                Board(0, 0) = " "
                Board(3, 0) = "r"
            End If
            CanCastle.CannotCastle()
        End If
    End If

```



```

End If

If Not (EnPassant = "-" OrElse HasEnPassanted) Then EnPassant = "-"
' Removal of EnPassant (if required).
' At the end of the subroutine, the Piece is placed at the new coordinates, and the old position is cleared.
' If the new position contains a piece, then the material count is updated for only that piece.
If Board(NewCoorX, NewCoorY) <> " " Then
    If Char.IsUpper(Board(NewCoorX, NewCoorY)) Then
        MaterialCount(0) -
    = ReturnPieceValue(Board(NewCoorX, NewCoorY))
    Else
        MaterialCount(1) -
    = ReturnPieceValue(Board(NewCoorX, NewCoorY))
    End If
End If
Board(NewCoorX, NewCoorY) = TempPiece
Board(OldCoorX, OldCoorY) = " "
End Sub

```

AI CORE - MINIMAX

```

' This function contains my MiniMax algorithm using Alpha-Beta Pruning and Quiescence (optional).
Private Function MiniMax(ByVal Board(,) As Char, ByVal depth As SByte, ByVal isWhite As Boolean, ByVal WCanCastle As CanCastle, ByVal BCanCastle As CanCastle, ByVal WKPos As String, ByVal BKPos As String, ByVal EnPassant As String, ByVal MaterialCount() As SByte, ByVal Alpha As Decimal, ByVal Beta As Decimal) As Decimal
    If ABORT Then Return 0
    TotalPositionsSearched += 1
    ' Creates moves & checking rules.
    Dim CurrentMove, BestMove As Decimal
    Dim WInCheck As New InCheck
    Dim BInCheck As New InCheck

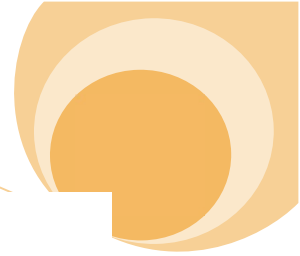
    If isWhite Then
        ' Creates and forms the TFTable for the player to move.
        Dim WhiteTFTable(7, 7) As Char
        FixTFTables(Board, True, WhiteTFTable, WKPos, WInCheck, NotInCheck, EnPassant)
        If Not (depth > 0 OrElse WInCheck.IsInCheck) Then 'Quiescence mode activated.
            ' Evaluation of board is the current move to beat.
            BestMove = Evaluate(MaterialCount, WKPos, BKPos)
            Alpha = Math.Max(Alpha, BestMove)
            If Beta <= Alpha Then 'ABP.
                Return BestMove
            End If
        Else
            BestMove = Integer.MinValue
        End If
        ' Creates the pseudo-legal moves for the chosen player.
        Dim PieceMoves(200, 2) As String
        PieceMoves = CreateMoves(Board, True, WhiteTFTable, BKPos, WInCheck, BInCheck, WCanCastle, EnPassant, Not (depth > 0 OrElse WInCheck.IsInCheck))
        ' If Quiescence mode is activated then use capture moves only.
        If PieceMoves(0, 0) > 0 Then 'If any move exists...

```

```

'Creates temp variables.
Dim TempBoard(7, 7) As Char
Dim TempMaterialCount(1) As SByte
Dim TempWKPos As String
Dim TempWCanCastle As New CanCastle
Dim TempWInCheck As New InCheck
Dim TempEnPassant As String
For n = 1 To CInt(PieceMoves(0, 0)) 'for each move...
    'If the user is in check, eliminate the moves that do not
    escape check. King moves are not considered
    'as the player's TTable will ensure that all king moves
    are legal.
    If WInCheck.IsInCheck AndAlso Board(PieceMoves(n, 0), Pie
ceMoves(n, 1)) <> "K" Then
        'Runs move through the DoesMoveResolveCheck algorithm
        TempWInCheck.CopyFrom(WInCheck)
        If DoesMoveResolveCheck(Board, PieceMoves(n, 0), Piec
eMoves(n, 1), PieceMoves(n, 2)(0), PieceMoves(n, 2)(1), TempWInCheck, BInChec
k, EnPassant) Then
            'Move has resolved check.
            TempWInCheck.NotInCheck()
        End If
    Else
        TempWInCheck.IsInCheck = False
    End If
    If Not TempWInCheck.IsInCheck Then 'Therefore, is a legal
    move.
        'Copies board info to temp variables.
        Array.Copy(Board, TempBoard, 64)
        TempMaterialCount(0) = MaterialCount(0)
        TempMaterialCount(1) = MaterialCount(1)
        TempWKPos = WKPos
        TempWCanCastle.CopyFrom(WCanCastle)
        TempEnPassant = EnPassant
        'Makes move on temp board, then calls MiniMax for thi
s new position.
        MakeMove(TempBoard, PieceMoves(n, 0), PieceMoves(n, 1
), PieceMoves(n, 2)(0), PieceMoves(n, 2)(1), TempWCanCastle, TempWKPos, TempM
aterialCount, TempEnPassant)
        If Not UseQuiescence AndAlso depth = 1 Then
            'We have reached a leaf position - return the eva
luation for this position.
            TotalPositionsSearched += 1
            CurrentMove = Evaluate(TempMaterialCount, TempWKP
os, BKPos) 'Evaluate position for opponent.
        Else 'No leaf node (or are using Quiescence) - put po
sition through MiniMax recursively.
            CurrentMove = MiniMax(TempBoard, depth - 1, False
, TempWCanCastle, BCanCastle, TempWKPos, BKPos, TempEnPassant, TempMaterialCo
unt, Alpha, Beta)
        End If
        If CurrentMove > BestMove Then
            BestMove = CurrentMove 'Best Move has been beaten
            - replace it.
            Alpha = Math.Max(Alpha, BestMove) 'Alpha = best m
ove found for white.
            'Move was too strong for player; opponent will no
t choose this branch.
        End If
        If Beta <= Alpha Then

```

```

Return BestMove 'Alpha-
Beta Pruning - return best move.
End If
End If
End If
Next
End If
Else 'Near-identical code for the black side.
Dim BlackTFTTable(7, 7) As Char
FixTFTTables(Board, False, BlackTFTTable, BKPos, NotInCheck, BInChe
ck, EnPassant)
If Not (depth > 0 OrElse BInCheck.IsInCheck) Then
BestMove = Evaluate(MaterialCount, WKPos, BKPos)
Beta = Math.Min(Beta, BestMove)
If Beta <= Alpha Then
Return BestMove
End If
Else
BestMove = Integer.MaxValue
End If
Dim PieceMoves(200, 2) As String
PieceMoves = CreateMoves(Board, False, BlackTFTTable, WKPos, WInCh
eck, BInCheck, BCanCastle, EnPassant, Not (depth > 0 OrElse BInCheck.IsInChec
k))
If PieceMoves(0, 0) > 0 Then
Dim TempBoard(7, 7) As Char
Dim TempMaterialCount(1) As SByte
Dim TempBKPos As String
Dim TempBCanCastle As New CanCastle
Dim TempBInCheck As New InCheck
Dim TempEnPassant As String
For n = 1 To CInt(PieceMoves(0, 0))
If BInCheck.IsInCheck AndAlso Board(PieceMoves(n, 0), Pie
ceMoves(n, 1)) <> "k" Then
TempBInCheck.CopyFrom(BInCheck)
If DoesMoveResolveCheck(Board, PieceMoves(n, 0), Piec
eMoves(n, 1), PieceMoves(n, 2)(0), PieceMoves(n, 2)(1), WInCheck, TempBInChec
k, EnPassant) Then
TempBInCheck.NotInCheck()
End If
Else
TempBInCheck.IsInCheck = False
End If
If Not TempBInCheck.IsInCheck Then
Array.Copy(Board, TempBoard, 64)
TempMaterialCount(0) = MaterialCount(0)
TempMaterialCount(1) = MaterialCount(1)
TempBKPos = BKPos
TempBCanCastle.CopyFrom(BCanCastle)
TempEnPassant = EnPassant
MakeMove(TempBoard, PieceMoves(n, 0), PieceMoves(n, 1
), PieceMoves(n, 2)(0), PieceMoves(n, 2)(1), TempBCanCastle, TempBKPos, TempM
aterialCount, TempEnPassant)
If Not UseQuiescence AndAlso depth = 1 Then
TotalPositionsSearched += 1
CurrentMove = Evaluate(TempMaterialCount, WKPos,
TempBKPos)
Else

```

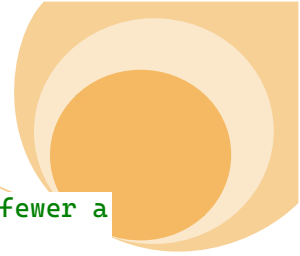
```

        CurrentMove = MiniMax(TempBoard, depth - 1, True,
WCanCastle, TempBCanCastle, WKPos, TempBKPos, TempEnPassant, TempMaterialCou
nt, Alpha, Beta)
    End If
    If CurrentMove < BestMove Then
        BestMove = CurrentMove
        Beta = Math.Min(Beta, BestMove) 'Beta = best move
found for white.
    End If
    If Beta <= Alpha Then
        Return BestMove
    End If
End If
Next
End If
End If

If BestMove = Integer.MinValue OrElse BestMove = Integer.MaxValue The
n
    'No legal move found for the player.
    If WInCheck.IsInCheck Then
        'Checkmate for white: return -
(1000 + depth) so that the quickest path to checkmate is chosen by the AI.
        Return -(1000 + depth)
    ElseIf BInCheck.IsInCheck Then
        'Checkmate for black: return (1000 + depth) so that the quick
est path to checkmate is chosen by the AI.
        Return (1000 + depth)
    Else 'Stalemate: return 0
        Return 0
    End If
End If
'Otherwise, return the best move's score found this iteration.
Return BestMove
End Function

'This algorithm is used to condense a board position into an evaluation s
core, used to determine best moves.
'We take into account the difference in material between the two sides, a
long with a heuristic to help
'the AI find checkmated in simple endgame positions.
Private Function Evaluate(ByVal MaterialCount() As SByte, ByVal WKPos As
String, ByVal BKPos As String) As Decimal
    'Finds difference in material between both sides.
    Dim Score As Decimal = MaterialCount(0) - MaterialCount(1)
    'If the opponent has little material left, we try to find positions w
here the opponent king is close to
    'the edge / corner of the board, and where the kings are closer toget
her. This can help find a checkmate.
    If MaterialCount(0) < 10 OrElse MaterialCount(1) < 10 Then
        'Finds distances between kings.
        Dim KingDistance As Byte = Math.Max(Math.Abs(CStr(WKPos(0)) - CSt
r(BKPos(0))), Math.Abs(CStr(WKPos(1)) - CStr(BKPos(1))))
        Dim KingCentreDistance As Byte
        If MaterialCount(1) < 10 Then
            'Finds distance from opponent's king to the centre of the boa
rd.
            KingCentreDistance = Math.Max(CStr(BKPos(0)) - 4, 3 - CStr(BK
Pos(0))) + Math.Max(CStr(BKPos(1)) - 4, 3 - CStr(BKPos(1)))

```



```

        'Heuristic becomes more prevelant as the opponent has fewer a
nd fewer pieces (exponential curve).
        Score += (KingCentreDistance * 1.5 + (7 - KingDistance)) * (1
.3 ^ (9 - MaterialCount(1))) / 100
    End If
    If MaterialCount(0) < 10 Then 'Similar code for the white pieces.
        KingCentreDistance = Math.Max(CStr(WKPos(0)) - 4, 3 - CStr(WK
Pos(0))) + Math.Max(CStr(WKPos(1)) - 4, 3 - CStr(WKPos(1)))
        Score -
= (KingCentreDistance * 1.5 + (7 - KingDistance)) * (1.3 ^ (9 - MaterialCount
(0))) / 100
    End If
End If
Return Math.Round(Score, 2) 'Prevents huge floating point numbers fro
m entering the system.
End Function

End Class

```

COREMETHODS CLASS

'This class contains most of the primary algorithms I will be using in my pro
ject, and will link to both my Chess class and
'my AI class (either via instavintiation or by inheritance). It will contain
the algorithms that will be used by both my Chess
'& AI classes, such as the 'PieceLegalMoves' generator, the 'TFTable' Generat
or, 'DoesMoveResolveCheck', and others.

Public Class CoreMethods

'TrueTables are just TrueFalse Tables containing only the letter T. Very
useful for resetting TrueFalse Tables

'and debugging.

Public MasterTrueTable(7, 7), TrueTable(7, 7) **As Char**

Public CannotCastle **As New** CanCastle

Public NotInCheck **As New** InCheck

Private ReadOnly PieceValue(9) **As Decimal** 'Array Containing the Value or
Weight of each Piece.

Public Sub New()

'Sets PieceValues variables using a Hash Function (Upper Case letter
--> ASCII, then MOD 11). This

'creates() a unique index / row in the PieceValue array for each piec
e and its corresponding weight,

'so its value can be searched up quickly. PieceValue(Asc(UCase(Board(
x, y))) Mod 11)

PieceValue(0) = 3 'Bishop Weight

PieceValue(1) = 3 'Knight Weight

PieceValue(3) = 1 'Pawn Weight

PieceValue(4) = 9 'Queen Weight

PieceValue(5) = 5 'Rook Weight

PieceValue(9) = 0 'King Weight

'Creates MasterTrueTable and TrueTable.

For x = 0 **To** 7

For y = 0 **To** 7

MasterTrueTable(x, y) = "T"

TrueTable(x, y) = "T"

Next

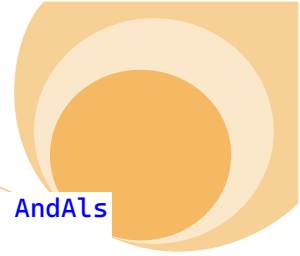
Next

End Sub

FEN CONVERTERS

'Function which puts the required pieces in a FEN board position into an 8x8 board array.

```
Public Function FENConverter(ByVal FEN As String, ByRef WCanCastle As Can
Castle, ByRef BCanCastle As CanCastle, ByRef WKPos As String, ByRef BKPos As
String, ByRef EnPassant As String, ByRef IsWhite As Boolean) As Char(,)
    'Resets castling & EnPassant rules
    WCanCastle.CannotCastle()
    BCanCastle.CannotCastle()
    EnPassant = "-"
    Dim x As Byte = 0
    Dim y As Byte = 0
    Dim tempArray(7, 7) As Char
    Dim SpaceLocation As Byte
    For n = 0 To Len(FEN) - 1
        Select Case FEN(n)
            Case "/" 'end of board row. Reset column index and incremen
t row index.
                y += 1
                x = 0
            Case "A" To "Z", "a" To "z" 'name of piece - add name of pi
ece to board index.
                tempArray(x, y) = FEN(n)
                If FEN(n) = "K" Then
                    WKPos = x & y
                ElseIf FEN(n) = "k" Then
                    BKPos = x & y
                End If
                x += 1
            Case "0" To "8" 'Set of empty squares.
                For m = 1 To CInt(CStr((FEN(n))))
                    tempArray(x, y) = " "
                    x += 1
                Next
            Case " "
                'Checks for pawns incorrectly placed on the 1st or 8th ra
nk. If we detect any, then we provoke an
                'intentional crash, which our encapsulating Try-
Catch detects and promptly reverses.
                For x = 0 To 7
                    If UCase(tempArray(x, 0)) = "P" OrElse UCase(tempArra
y(x, 7)) = "P" Then FEN = ""
                Next
                ' Once the SPACE has been reached In the FEN, we know tha
t we have read the entire board. We can then move onto
                ' info about castling, who's turn it is, En Passant And m
ore.
                SpaceLocation = FEN.IndexOf(" ") 'Following characters re
volve around location of SPACE.
                If FEN(SpaceLocation + 1) = "w" Then
                    IsWhite = True
                Else
                    IsWhite = False
                End If
                FEN = Right(FEN, Len(FEN) - SpaceLocation - 3)
                'Creates castling privileges (as long as the pieces are i
n the correct space).
                For m = 0 To Len(FEN) - 1
```



```

        If FEN(m) = "K" AndAlso (tempArray(4, 7) = "K" AndAlso
o tempArray(7, 7) = "R") Then
            WCanCastle.KS = True
        ElseIf FEN(m) = "Q" AndAlso (tempArray(4, 7) = "K" An
dAlso tempArray(0, 7) = "R") Then
            WCanCastle.QS = True
        ElseIf FEN(m) = "k" AndAlso (tempArray(4, 0) = "k" An
dAlso tempArray(7, 0) = "r") Then
            BCanCastle.KS = True
        ElseIf FEN(m) = "q" AndAlso (tempArray(4, 0) = "k" An
dAlso tempArray(0, 0) = "r") Then
            BCanCastle.QS = True
        ElseIf FEN(m) >= "a" AndAlso FEN(m) <= "h" Then
            'converts the a-
h coordinate to the more computer-friendly index from 0-7 (eg: h3 --> 75)
            EnPassant = Asc(FEN(m)) - 97 & 8 - CStr(FEN(m + 1
))
        End If
    Next
Exit For
End Select
Next
Return tempArray
End Function

```

'Function which converts the current board position into its FEN counterpart.

```

Public Function ConvertToFEN(ByVal Board(,) As Char, ByVal WCanCastle As
CanCastle, ByVal BCanCastle As CanCastle, ByVal EnPassant As String, ByVal is
White As Boolean) As String
    Dim Counter As Byte = 0 ' = the number of blank spaces in a row on the
board.
    ConvertToFEN = ""
    For y = 0 To 7
        For x = 0 To 7
            If Board(x, y) = " " Then
                Counter += 1
            Else 'If necessary, add Counter to the FEN & add the piece na
me to the FEN.
                If Counter > 0 Then
                    ConvertToFEN &= Counter
                    Counter = 0
                End If
                ConvertToFEN &= Board(x, y)
            End If
        Next
        If Counter > 0 Then ConvertToFEN &= Counter
        ConvertToFEN &= "/" 'Creates new row break, and begins on the nex
t one.
        Counter = 0
    Next
    ConvertToFEN = ConvertToFEN.TrimEnd("/") 'Removes the last character
from the FEN ("/").
    If isWhite Then
        ConvertToFEN &= " w "
    Else
        ConvertToFEN &= " b "
    End If
    If WCanCastle.KS Then ConvertToFEN &= "K"
    If WCanCastle.QS Then ConvertToFEN &= "Q"

```

```

If BCanCastle.KS Then ConvertToFEN &= "k"
If BCanCastle.QS Then ConvertToFEN &= "q"
If ConvertToFEN.EndsWith(" ") Then ConvertToFEN &= "-"
" '= therefore no castling privileges
If EnPassant <> "-" Then
    'converts the computer-friendly index from 0-
7 into the more human-friendly a-h coordinate (eg: 75 -> h3)
    ConvertToFEN &= " " & Chr(CStr(EnPassant(0)) + 97) & 8 - CStr(EnP
assant(1)) & " 0 1"
Else
    ConvertToFEN &= " - 0 1" 'represents the move numbers for a stand
ard position.
End If
Return ConvertToFEN
End Function

```

PSEUDO-LEGAL MOVE GENERATORS

'The next 2 functions (WhitePieceLegalMoves and BlackPieceLegalMoves) receive a single piece on the board, and determines all its possible moves - assigning them to the array LegalMoveArray. Alongside this, it also builds to one of the two TrueFalse Tables using the squares which a piece attacks as markers. In this table, the program checks for pinned pieces using the Enemy King's Position - extending a piece's line of sight if the King is on the same axis as it. If it finds a King, then the Pinned Piece is labelled from 0-3, which represent which direction it is pinned from (0 = vertical, 1 = btm left to top right diagonal, 2 = horizontal, 3 = top left to btm right diagonal). This is useful as a pinned pawn pinned vertically can still move upwardly, but cannot take an enemy piece diagonally (which would expose the king), so I have also implemented the logic for pieces moving depending on if / where they are pinned from.

'Note: to help reduce unnecessary commenting, and due to the fact that much of this code is somewhat similar

'for each type of piece, comments will only be included for the first appearance of a particular method / technique.

'For more in-depth commenting & info, please see the section on Pseudo-legal Move Generation in my Project Report (Design).

```

Public Function WhitePieceLegalMoves(ByVal Board(,) As Char, ByVal CoorX As SByte, ByVal CoorY As SByte, ByRef WhiteTFTable(,) As Char, ByRef BlackTFTable(,) As Char, ByVal BKPos As String, ByRef BInCheck As InCheck, ByVal WInCheck As InCheck, ByVal WCanCastle As CanCastle, ByVal EnPassant As String) As String()

```

```

    Dim LegalMoveArray(27) As String
    Dim n As Byte = 1
    Dim StartX, EndX, StartY, EndY As SByte 'Pointers for the coordinate bounds.
    Dim CheckingPiece As String = " "

```

```

    If Board(CoorX, CoorY) = "K" Then 'Legal Moves for the King (along with Castling).

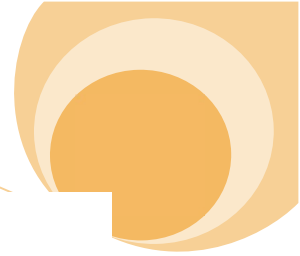
```

'Sets coordinate bounds. Start & End Coords, in this case, represents the 8 squares around the king.

```

    StartX = Math.Max(CoorX - 1, 0)

```

```

EndX = Math.Min(CoorX + 1, 7)
StartY = Math.Max(CoorY - 1, 0)
EndY = Math.Min(CoorY + 1, 7)
For y = StartY To EndY
    For x = StartX To EndX
        'Sets the appropriate square on the enemy TFTable to 'F'.
        This prevents the enemy king from moving to that square.
        BlackTFTable(x, y) = "F"
        'Important note: the piece's legal moves does NOT equate
        to the squares on the enemy TFTable to set to 'F'.
        'This Is because, even though the king might Not be legal
        ly allowed To move To a certain square, it has the
        'potential' to, meaning that it must have an effect on th
        e enemy king's TFTable.
        If Not Char.IsUpper(Board(x, y)) AndAlso WhiteTFTable(x,
y) = "T" Then
            'Therefore is a legal move...
            LegalMoveArray(n) = x & y
            n += 1
        End If
    Next
Next
If WCanCastle.KS AndAlso WInCheck.IsInCheck = False Then
    If Board(5, 7) = " " AndAlso Board(6, 7) = " " AndAlso Board(
7, 7) = "R" AndAlso WhiteTFTable(5, 7) = "T" AndAlso WhiteTFTable(6, 7) = "T"
Then
        LegalMoveArray(n) = 67 'Square king would go to to castle
king-side.
        n += 1
    End If
End If
If WCanCastle.QS AndAlso WInCheck.IsInCheck = False Then
    If Board(3, 7) = " " AndAlso Board(2, 7) = " " AndAlso Board(
1, 7) = " " AndAlso Board(0, 7) = "R" AndAlso WhiteTFTable(3, 7) = "T" AndAls
o WhiteTFTable(2, 7) = "T" Then
        LegalMoveArray(n) = 27 'Square king would go to to castle
queen-side.
        n += 1
    End If
End If

ElseIf Board(CoorX, CoorY) = "P" Then 'Legal Moves for the Pawn (alon
g with First Moves).
    If WhiteTFTable(CoorX, CoorY) <> "2" Then
        'If the pawn is not pinned diagonally...
        If WhiteTFTable(CoorX, CoorY) <> "1" AndAlso WhiteTFTable(Coo
rX, CoorY) <> "3" Then
            If Board(CoorX, CoorY - 1) = " " Then
                LegalMoveArray(n) = CoorX & CoorY - 1
                n += 1
                If CoorY = 6 AndAlso Board(CoorX, 4) = " " Then 'Pawn
s can move two squares on their first move.
                    LegalMoveArray(n) = CoorX & 4
                    n += 1
                End If
            End If
        End If
    End If
    If WhiteTFTable(CoorX, CoorY) <> "0" Then
        'Code for pawn left captures.

```

```

    If WhiteTFTable(CoorX, CoorY) <> "1" AndAlso CoorX > 0 Th
en
        If BlackTFTable(CoorX - 1, CoorY - 1) = "T" Then Blac
kTFTable(CoorX - 1, CoorY - 1) = "F"
        'We do the above IF statement to prevent pinned piece
s from being overwritten.
        If Char.IsLower(Board(CoorX - 1, CoorY - 1)) OrElse (
CoorX - 1 & CoorY - 1 = EnPassant AndAlso WhiteTFTable(CoorX, CoorY) <> "4")
Then
            LegalMoveArray(n) = CoorX - 1 & CoorY - 1
            If Board(CoorX - 1, CoorY - 1) = "k" Then
                BInCheck.IsInCheck = True
                CheckingPiece = CoorX & CoorY
            End If
            n += 1
        End If
    End If
    'Code for pawn right captures.
    If WhiteTFTable(CoorX, CoorY) <> "3" AndAlso CoorX < 7 Th
en
        If BlackTFTable(CoorX + 1, CoorY - 1) = "T" Then Blac
kTFTable(CoorX + 1, CoorY - 1) = "F"
        If Char.IsLower(Board(CoorX + 1, CoorY - 1)) OrElse (
CoorX + 1 & CoorY - 1 = EnPassant AndAlso WhiteTFTable(CoorX, CoorY) <> "4")
Then
            LegalMoveArray(n) = CoorX + 1 & CoorY - 1
            If Board(CoorX + 1, CoorY - 1) = "k" Then
                BInCheck.IsInCheck = True
                CheckingPiece = CoorX & CoorY
            End If
            n += 1
        End If
    End If
End If

ElseIf Board(CoorX, CoorY) = "N" Then 'Legal Moves for the Knight. (I
f pinned, then it cannot move at all.)
    If WhiteTFTable(CoorX, CoorY) >= "5" Then
        If CoorX >= 2 Then
            For x = -1 To 1 Step 2
                If (CoorY + x) >= 0 AndAlso (CoorY + x) <= 7 Then
                    If BlackTFTable(CoorX - 2, CoorY + x) = "T" Then
                        BlackTFTable(CoorX - 2, CoorY + x) = "F"
                        If Not Char.IsUpper(Board(CoorX - 2, CoorY + x))
Then
                            LegalMoveArray(n) = (CoorX - 2) & (CoorY + x)
                            If Board(CoorX - 2, CoorY + x) = "k" Then
                                BInCheck.IsInCheck = True
                                CheckingPiece = CoorX & CoorY
                            Exit For
                        End If
                        n += 1
                    End If
                End If
            Next
        End If
        If CoorX <= 5 Then
            For x = -1 To 1 Step 2

```



```

        If (CoorY + x) >= 0 AndAlso (CoorY + x) <= 7 Then
            If BlackTFTable(CoorX + 2, CoorY + x) = "T" Then
                BlackTFTable(CoorX + 2, CoorY + x) = "F"
                If Not Char.IsUpper(Board(CoorX + 2, CoorY + x))
                    Then
                        LegalMoveArray(n) = (CoorX + 2) & (CoorY + x)
                        If Board(CoorX + 2, CoorY + x) = "k" Then
                            BInCheck.IsInCheck = True
                            CheckingPiece = CoorX & CoorY
                            Exit For
                        End If
                        n += 1
                    End If
                End If
            End If
        Next
    End If
    If CoorY >= 2 Then
        For x = -1 To 1 Step 2
            If (CoorX + x) >= 0 AndAlso (CoorX + x) <= 7 Then
                If BlackTFTable(CoorX + x, CoorY - 2) = "T" Then
                    BlackTFTable(CoorX + x, CoorY - 2) = "F"
                    If Not Char.IsUpper(Board(CoorX + x, CoorY - 2))
                        Then
                            LegalMoveArray(n) = (CoorX + x) & (CoorY - 2)
                            If Board(CoorX + x, CoorY - 2) = "k" Then
                                BInCheck.IsInCheck = True
                                CheckingPiece = CoorX & CoorY
                                Exit For
                            End If
                            n += 1
                        End If
                    End If
                End If
            End If
        Next
    End If
    If CoorY <= 5 Then
        For x = -1 To 1 Step 2
            If (CoorX + x) >= 0 AndAlso (CoorX + x) <= 7 Then
                If BlackTFTable(CoorX + x, CoorY + 2) = "T" Then
                    BlackTFTable(CoorX + x, CoorY + 2) = "F"
                    If Not Char.IsUpper(Board(CoorX + x, CoorY + 2))
                        Then
                            LegalMoveArray(n) = (CoorX + x) & (CoorY + 2)
                            If Board(CoorX + x, CoorY + 2) = "k" Then
                                BInCheck.IsInCheck = True
                                CheckingPiece = CoorX & CoorY
                                Exit For
                            End If
                            n += 1
                        End If
                    End If
                End If
            End If
        Next
    End If
    End If

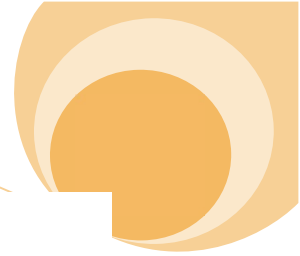
    ElseIf Board(CoorX, CoorY) = "R" OrElse Board(CoorX, CoorY) = "Q" The
n 'Legal Moves for the Rook / part of Queen.
        If WhiteTFTable(CoorX, CoorY) <> "1" AndAlso WhiteTFTable(CoorX,
CoorY) <> "3" Then

```

```

    If WhiteTFTable(CoorX, CoorY) <> "0" Then 'if rook is not pin
ned vertically...
        For x = (CoorX + 1) To 7
            If BlackTFTable(x, CoorY) = "T" Then BlackTFTable(x,
CoorY) = "F"
                If Char.IsUpper(Board(x, CoorY)) Then
                    If Board(x, CoorY) = "P" AndAlso x & CoorY + 1 =
EnPassant AndAlso CInt(CStr(BKPos(0))) > x AndAlso CInt(CStr(BKPos(1))) = CoorY Then
                        'Initiate pin checks (inc friendly-
first EnPassant pins).
                            For m = x + 2 To 7
                                If Board(m, CoorY) = "k" Then
                                    BlackTFTable(x + 1, CoorY) = "4"
                                    Exit For
                                End If
                                'There is another piece in the way - cancel the search.
                                If Board(m, CoorY) <> " " Then Exit For
                            Next
                        End If
                        Exit For
                    Else
                        LegalMoveArray(n) = (x) & (CoorY)
                        If Board(x, CoorY) = "k" Then
                            BInCheck.IsInCheck = True
                            CheckingPiece = CoorX & CoorY
                            'Marks the next square as illegal for the king to move to.
                                If BlackTFTable(Math.Min(x + 1, 7), CoorY) =
"T" Then BlackTFTable(Math.Min(x + 1, 7), CoorY) = "F"
                                    Exit For
                                End If
                                n += 1
                                If Char.IsLower(Board(x, CoorY)) AndAlso CInt(CStr(BKPos(0))) > x AndAlso CInt(CStr(BKPos(1))) = CoorY Then
                                    'Initiate pin checks.
                                    For m = x + 1 To 7
                                        If Board(m, CoorY) = "k" Then
                                            'Enemy king spotted - mark piece as pinned.
                                                BlackTFTable(x, CoorY) = "2"
                                                Exit For
                                        End If
                                        'Checks for enemy-first EnPassant pins.
                                        If Board(m, CoorY) <> " " Then
                                            If Board(m, CoorY) = "P" AndAlso m &
CoorY + 1 = EnPassant Then
                                                'Pawn should be stopped from taking En-Passant.
                                                    For a = m + 1 To 7
                                                        If Board(a, CoorY) = "k" Then
                                                            BlackTFTable(x, CoorY) =
"4"
                                                                Exit For
                                                        End If
                                                        If Board(a, CoorY) <> " " Then
                                                            Exit For
                                                        End If
                                                    Next
                                                End If
                                            End If
                                        Next
                                    End If
                                End If
                            End If
                        End If
                    End If
                End If
            End If
        Next
    End If

```



```

Exit For
End If
Next
End If

If Board(x, CoorY) <> " " Then Exit For
End If
Next

For x = (CoorX - 1) To 0 Step -1
  If BlackTFTable(x, CoorY) = "T" Then BlackTFTable(x,
CoorY) = "F"
    If Char.IsUpper(Board(x, CoorY)) Then
      If Board(x, CoorY) = "P" AndAlso x & CoorY + 1 =
EnPassant AndAlso CInt(CStr(BKPos(0))) < x AndAlso CInt(CStr(BKPos(1))) = CoorY Then
        For m = x - 2 To 0 Step -1
          If Board(m, CoorY) = "k" Then
            BlackTFTable(x - 1, CoorY) = "4"
            Exit For
          End If
          If Board(m, CoorY) <> " " Then Exit For
        Next
      End If
      Exit For
    Else
      LegalMoveArray(n) = (x) & (CoorY)
      If Board(x, CoorY) = "k" Then
        BInCheck.IsInCheck = True
        CheckingPiece = CoorX & CoorY
        If BlackTFTable(Math.Max(x - 1, 0), CoorY) =
"T" Then BlackTFTable(Math.Max(x - 1, 0), CoorY) = "F"
        Exit For
      End If
      n += 1
      If Char.IsLower(Board(x, CoorY)) AndAlso CInt(CStr(BKPos(0))) < x AndAlso CInt(CStr(BKPos(1))) = CoorY Then
        For m = x - 1 To 0 Step -1
          If Board(m, CoorY) = "k" Then
            BlackTFTable(x, CoorY) = "2"
            Exit For
          End If
          If Board(m, CoorY) <> " " Then
            If Board(m, CoorY) = "P" AndAlso m &
CoorY + 1 = EnPassant Then
              For a = m - 1 To 0 Step -1
                If Board(a, CoorY) = "k" Then
                  BlackTFTable(x, CoorY) =
"4"

                  Exit For
                End If
                If Board(a, CoorY) <> " " The
n Exit For
              Next
            End If
          Exit For
        End If
      End If
    Next
  End If
End If
Next
End If

```

```

        If Board(x, CoorY) <> " " Then Exit For
    End If
Next

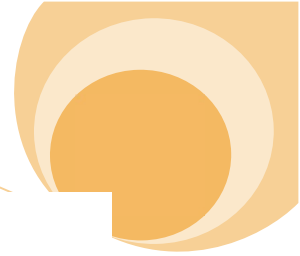
End If
If WhiteTFTable(CoorX, CoorY) <> "2" Then
    For y = (CoorY + 1) To 7
        If BlackTFTable(CoorX, y) = "T" Then BlackTFTable(Coo
rX, y) = "F"

        If Char.IsUpper(Board(CoorX, y)) Then
            Exit For
        Else
            LegalMoveArray(n) = (CoorX) & (y)
            If Board(CoorX, y) = "k" Then
                BInCheck.IsInCheck = True
                CheckingPiece = CoorX & CoorY
                If BlackTFTable(CoorX, Math.Min(y + 1, 7)) =
"T" Then BlackTFTable(CoorX, Math.Min(y + 1, 7)) = "F"
                Exit For
            End If
            n += 1
            If Char.IsLower(Board(CoorX, y)) AndAlso CInt(CSt
r(BKPos(0))) = CoorX AndAlso CInt(CStr(BKPos(1))) > y Then
                For m = y + 1 To 7
                    If Board(CoorX, m) = "k" Then
                        BlackTFTable(CoorX, y) = "0"
                        Exit For
                    End If
                    If Board(CoorX, m) <> " " Then Exit For
                Next
            End If
            If Board(CoorX, y) <> " " Then Exit For
        End If
    Next

    For y = (CoorY - 1) To 0 Step -1
        If BlackTFTable(CoorX, y) = "T" Then BlackTFTable(Coo
rX, y) = "F"

        If Char.IsUpper(Board(CoorX, y)) Then
            Exit For
        Else
            LegalMoveArray(n) = (CoorX) & (y)
            If Board(CoorX, y) = "k" Then
                BInCheck.IsInCheck = True
                CheckingPiece = CoorX & CoorY
                If BlackTFTable(CoorX, Math.Max(y - 1, 0)) =
"T" Then BlackTFTable(CoorX, Math.Max(y - 1, 0)) = "F"
                Exit For
            End If
            n += 1
            If Char.IsLower(Board(CoorX, y)) AndAlso CInt(CSt
r(BKPos(0))) = CoorX AndAlso CInt(CStr(BKPos(1))) < y Then
                For m = y - 1 To 0 Step -1
                    If Board(CoorX, m) = "k" Then
                        BlackTFTable(CoorX, y) = "0"
                        Exit For
                    End If
                    If Board(CoorX, m) <> " " Then Exit For
                Next
            End If
        End If
    Next
End If

```

```

        If Board(CoorX, y) <> " " Then Exit For
    End If
Next
End If
End If
End If

If Board(CoorX, CoorY) = "B" OrElse Board(CoorX, CoorY) = "Q" Then 'Legal Moves for the Bishop / Part of Queen.
    Dim o, p As SByte 'These represent temporary x and y coordinates, used for pin detection.
    If WhiteTFTable(CoorX, CoorY) <> "0" AndAlso WhiteTFTable(CoorX, CoorY) <> "2" Then
        If WhiteTFTable(CoorX, CoorY) <> "1" Then
            StartX = CoorX + 1
            StartY = CoorY + 1
            Do Until StartX > 7 OrElse StartY > 7
                If BlackTFTable(StartX, StartY) = "T" Then BlackTFTable(StartX, StartY) = "F"
                If Char.IsUpper(Board(StartX, StartY)) Then
                    Exit Do
                Else
                    LegalMoveArray(n) = StartX & StartY
                    If Board(StartX, StartY) = "k" Then
                        BInCheck.IsInCheck = True
                        CheckingPiece = CoorX & CoorY
                        If StartX < 7 AndAlso StartY < 7 Then
                            If BlackTFTable(StartX + 1, StartY + 1) = "T" Then BlackTFTable(StartX + 1, StartY + 1) = "F"
                        End If
                    End If
                    Exit Do
                End If
                n += 1
                If Char.IsLower(Board(StartX, StartY)) AndAlso (CInt(CStr(BKPos(0))) - CoorX = CInt(CStr(BKPos(1))) - CoorY) AndAlso CInt(CStr(BKPos(0))) > CoorX Then
                    o = StartX + 1
                    p = StartY + 1
                    Do Until o > 7 OrElse p > 7 'until the end of the board is reached.
                        If Board(o, p) = "k" Then
                            BlackTFTable(StartX, StartY) = "3"
                            Exit Do
                        End If
                        If Board(o, p) <> " " Then Exit Do
                        o += 1
                        p += 1
                    Loop
                End If
                If Board(StartX, StartY) <> " " Then Exit Do
            End If
            StartX += 1
            StartY += 1
        Loop
        StartX = CoorX - 1
        StartY = CoorY - 1
        Do Until StartX < 0 OrElse StartY < 0

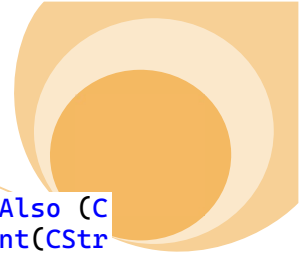
```

```

If BlackTFTable(StartX, StartY) = "T" Then BlackTFTab
le(StartX, StartY) = "F"
If Char.IsUpper(Board(StartX, StartY)) Then
Exit Do
Else
LegalMoveArray(n) = StartX & StartY
If Board(StartX, StartY) = "k" Then
BInCheck.IsInCheck = True
CheckingPiece = CoorX & CoorY
If StartX > 0 AndAlso StartY > 0 Then
If BlackTFTable(StartX - 1, StartY - 1) =
"T" Then BlackTFTable(StartX - 1, StartY - 1) = "F"
End If
Exit Do
End If
n += 1
If Char.IsLower(Board(StartX, StartY)) AndAlso (C
Int(CStr(BKPos(0))) - CoorX = CInt(CStr(BKPos(1))) - CoorY) AndAlso CInt(CStr
(BKPos(0))) < CoorX Then
o = StartX - 1
p = StartY - 1
Do Until o < 0 OrElse p < 0
If Board(o, p) = "k" Then
BlackTFTable(StartX, StartY) = "3"
Exit Do
End If
If Board(o, p) <> " " Then Exit Do
o -= 1
p -= 1
Loop
End If
If Board(StartX, StartY) <> " " Then Exit Do
End If
StartX -= 1
StartY -= 1
Loop

End If
If WhiteTFTable(CoorX, CoorY) <> "3" Then
StartX = CoorX + 1
StartY = CoorY - 1
Do Until StartX > 7 OrElse StartY < 0
If BlackTFTable(StartX, StartY) = "T" Then BlackTFTab
le(StartX, StartY) = "F"
If Char.IsUpper(Board(StartX, StartY)) Then
Exit Do
Else
LegalMoveArray(n) = StartX & StartY
If Board(StartX, StartY) = "k" Then
BInCheck.IsInCheck = True
CheckingPiece = CoorX & CoorY
If StartX < 7 AndAlso StartY > 0 Then
If BlackTFTable(StartX + 1, StartY - 1) =
"T" Then BlackTFTable(StartX + 1, StartY - 1) = "F"
End If
Exit Do
End If
n += 1

```



```

        If Char.IsLower(Board(StartX, StartY)) AndAlso (C
Int(CStr(BKPos(0))) - CoorX = CoorY - CInt(CStr(BKPos(1))) AndAlso CInt(CStr
(BKPos(0))) > CoorX Then
            o = StartX + 1
            p = StartY - 1
            Do Until o > 7 OrElse p < 0
                If Board(o, p) = "k" Then
                    BlackTFTable(StartX, StartY) = "1"
                    Exit Do
                End If
                If Board(o, p) <> " " Then Exit Do
                o += 1
                p -= 1
            Loop
        End If

        If Board(StartX, StartY) <> " " Then Exit Do
    End If
    StartX += 1
    StartY -= 1
Loop

StartX = CoorX - 1
StartY = CoorY + 1
Do Until StartX < 0 OrElse StartY > 7
    If BlackTFTable(StartX, StartY) = "T" Then BlackTFTab
le(StartX, StartY) = "F"
    If Char.IsUpper(Board(StartX, StartY)) Then
        Exit Do
    Else
        LegalMoveArray(n) = StartX & StartY
        If Board(StartX, StartY) = "k" Then
            BInCheck.IsInCheck = True
            CheckingPiece = CoorX & CoorY
            If StartX > 0 AndAlso StartY < 7 Then
                If BlackTFTable(StartX - 1, StartY + 1) =
"T" Then BlackTFTable(StartX - 1, StartY + 1) = "F"
            End If
            Exit Do
        End If
        n += 1
        If Char.IsLower(Board(StartX, StartY)) AndAlso (C
Int(CStr(BKPos(0))) - CoorX = CoorY - CInt(CStr(BKPos(1))) AndAlso CInt(CStr
(BKPos(0))) < CoorX Then
            o = StartX - 1
            p = StartY + 1
            Do Until o < 0 OrElse p > 7
                If Board(o, p) = "k" Then
                    BlackTFTable(StartX, StartY) = "1"
                    Exit Do
                End If
                If Board(o, p) <> " " Then Exit Do
                o -= 1
                p += 1
            Loop
        End If
        If Board(StartX, StartY) <> " " Then Exit Do
    End If
    StartX -= 1
    StartY += 1

```

```

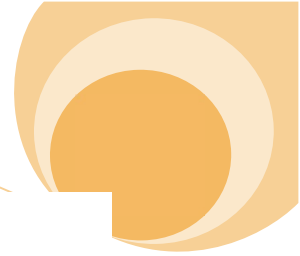
        Loop
        End If
    End If
End If

'At the end of the function, the program checks to see if any piece is
attacking the enemy king.
'If it is, then the other player is signalled to be in check, and the
attacking piece is recorded.
If CheckingPiece <> " " Then
    If BInCheck.Piece = " " OrElse BInCheck.Piece = CheckingPiece Then
n
        BInCheck.Piece = CheckingPiece
    Else
        BInCheck.DoubleCheck = True
    End If
End If
'The first index of LegalMoveArray contains a counter of how many moves
each piece can make. Useful for
'when the AI is picking moves.
If n <> 1 Then LegalMoveArray(0) = n
Return LegalMoveArray
End Function

'As the below subroutine is very similar to its equivalent 'WhitePieceLegalMoves'
function, commenting will be limited.
Public Function BlackPieceLegalMoves(ByVal Board(,) As Char, ByVal CoorX
As SByte, ByVal CoorY As SByte, ByRef BlackTFTable(,) As Char, ByRef WhiteTFT
able(,) As Char, ByVal WKPos As String, ByRef WInCheck As InCheck, ByVal BInC
heck As InCheck, ByVal BCanCastle As CanCastle, ByVal EnPassant As String) As
String()
    Dim LegalMoveArray(27) As String
    Dim n As Byte = 1
    Dim StartX, EndX, StartY, EndY As SByte 'Pointers for the coordinate
bounds.
    Dim CheckingPiece As String = " "

    If Board(CoorX, CoorY) = "k" Then 'Legal Moves for the King (Along with
Castling).
        StartX = Math.Max(CoorX - 1, 0)
        EndX = Math.Min(CoorX + 1, 7)
        StartY = Math.Max(CoorY - 1, 0)
        EndY = Math.Min(CoorY + 1, 7)
        For y = StartY To EndY
            For x = StartX To EndX
                If WhiteTFTable(x, y) = "T" Then WhiteTFTable(x, y) = "F"
                If Not Char.IsLower(Board(x, y)) AndAlso BlackTFTable(x,
y) = "T" Then
                    LegalMoveArray(n) = (x) & (y)
                    n += 1
                End If
            Next
        Next
        If BCanCastle.KS AndAlso BInCheck.IsInCheck = False Then
            If Board(5, 0) = " " AndAlso Board(6, 0) = " " AndAlso Board(
7, 0) = "r" AndAlso BlackTFTable(5, 0) = "T" AndAlso BlackTFTable(6, 0) = "T"
Then
                LegalMoveArray(n) = 60
                n += 1
            End If
        End If
    End If
End Function

```



```

End If
If BCanCastle.QS AndAlso BInCheck.IsInCheck = False Then
    If Board(3, 0) = " " AndAlso Board(2, 0) = " " AndAlso Board(
1, 0) = " " AndAlso Board(0, 0) = "r" AndAlso BlackTFTable(3, 0) = "T" AndAls
o BlackTFTable(2, 0) = "T" Then
        LegalMoveArray(n) = 20
        n += 1
    End If
End If

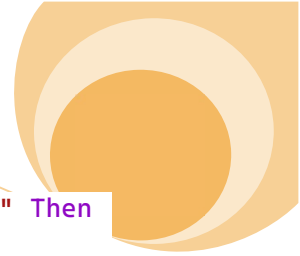
ElseIf Board(CoorX, CoorY) = "p" Then 'Legal Moves for the Pawn (along with First Moves.)
    If BlackTFTable(CoorX, CoorY) <> "2" Then
        If BlackTFTable(CoorX, CoorY) <> "1" AndAlso BlackTFTable(CoorX, CoorY) <> "3" Then
            If Board(CoorX, CoorY + 1) = " " Then
                LegalMoveArray(n) = CoorX & CoorY + 1
                n += 1
                If CoorY = 1 AndAlso Board(CoorX, 3) = " " Then
                    LegalMoveArray(n) = CoorX & 3
                    n += 1
                End If
            End If
        End If
    End If
    If BlackTFTable(CoorX, CoorY) <> "0" Then
        If BlackTFTable(CoorX, CoorY) <> "3" AndAlso CoorX > 0 Then
en
            If WhiteTFTable(CoorX - 1, CoorY + 1) = "T" Then WhiteTFTable(CoorX - 1, CoorY + 1) = "F"
            If Char.IsUpper(Board(CoorX - 1, CoorY + 1)) OrElse (CoorX - 1 & CoorY + 1 = EnPassant AndAlso BlackTFTable(CoorX, CoorY) <> "4") Then
                LegalMoveArray(n) = CoorX - 1 & CoorY + 1
                If Board(CoorX - 1, CoorY + 1) = "K" Then
                    WInCheck.IsInCheck = True
                    CheckingPiece = CoorX & CoorY
                End If
                n += 1
            End If
        End If
    End If
    If BlackTFTable(CoorX, CoorY) <> "1" AndAlso CoorX < 7 Then
en
            If WhiteTFTable(CoorX + 1, CoorY + 1) = "T" Then WhiteTFTable(CoorX + 1, CoorY + 1) = "F"
            If Char.IsUpper(Board(CoorX + 1, CoorY + 1)) OrElse (CoorX + 1 & CoorY + 1 = EnPassant AndAlso BlackTFTable(CoorX, CoorY) <> "4") Then
                LegalMoveArray(n) = CoorX + 1 & CoorY + 1
                If Board(CoorX + 1, CoorY + 1) = "K" Then
                    WInCheck.IsInCheck = True
                    CheckingPiece = CoorX & CoorY
                End If
                n += 1
            End If
        End If
    End If
End If
End If

```

```

ElseIf Board(CoorX, CoorY) = "n" Then 'Legal Moves for the Knight. (I
f pinned, then it cannot move at all.)
  If BlackTFTable(CoorX, CoorY) >= "5" Then
    If CoorX >= 2 Then
      For x = -1 To 1 Step 2
        If (CoorY + x) >= 0 AndAlso (CoorY + x) <= 7 Then
          If WhiteTFTable(CoorX - 2, CoorY + x) = "T" Then
            WhiteTFTable(CoorX - 2, CoorY + x) = "F"
            If Not Char.IsLower(Board(CoorX - 2, CoorY + x))
Then
              LegalMoveArray(n) = (CoorX - 2) & (CoorY + x)
              If Board(CoorX - 2, CoorY + x) = "K" Then
                WInCheck.IsInCheck = True
                CheckingPiece = CoorX & CoorY
                Exit For
              End If
              n += 1
            End If
          End If
        Next
      End If
    If CoorX <= 5 Then
      For x = -1 To 1 Step 2
        If (CoorY + x) >= 0 AndAlso (CoorY + x) <= 7 Then
          If WhiteTFTable(CoorX + 2, CoorY + x) = "T" Then
            WhiteTFTable(CoorX + 2, CoorY + x) = "F"
            If Not Char.IsLower(Board(CoorX + 2, CoorY + x))
Then
              LegalMoveArray(n) = (CoorX + 2) & (CoorY + x)
              If Board(CoorX + 2, CoorY + x) = "K" Then
                WInCheck.IsInCheck = True
                CheckingPiece = CoorX & CoorY
                Exit For
              End If
              n += 1
            End If
          End If
        Next
      End If
    If CoorY >= 2 Then
      For x = -1 To 1 Step 2
        If (CoorX + x) >= 0 AndAlso (CoorX + x) <= 7 Then
          If WhiteTFTable(CoorX + x, CoorY - 2) = "T" Then
            WhiteTFTable(CoorX + x, CoorY - 2) = "F"
            If Not Char.IsLower(Board(CoorX + x, CoorY - 2))
Then
              LegalMoveArray(n) = (CoorX + x) & (CoorY - 2)
              If Board(CoorX + x, CoorY - 2) = "K" Then
                WInCheck.IsInCheck = True
                CheckingPiece = CoorX & CoorY
                Exit For
              End If
              n += 1
            End If
          End If
        Next
      End If
    If CoorY <= 5 Then
      For x = -1 To 1 Step 2
        If (CoorX + x) >= 0 AndAlso (CoorX + x) <= 7 Then

```

```

        If WhiteTFTable(CoorX + x, CoorY + 2) = "T" Then
WhiteTFTable(CoorX + x, CoorY + 2) = "F"
        If Not Char.IsLower(Board(CoorX + x, CoorY + 2))
Then
            LegalMoveArray(n) = (CoorX + x) & (CoorY + 2)
            If Board(CoorX + x, CoorY + 2) = "K" Then
                WInCheck.IsInCheck = True
                CheckingPiece = CoorX & CoorY
                Exit For
            End If
            n += 1
        End If
        End If
    Next
End If
End If

    ElseIf Board(CoorX, CoorY) = "r" OrElse Board(CoorX, CoorY) = "q" The
n 'Legal Moves for the Rook / Part of Queen.
        If BlackTFTable(CoorX, CoorY) <> "1" AndAlso BlackTFTable(CoorX,
CoorY) <> "3" Then
            If BlackTFTable(CoorX, CoorY) <> "0" Then
                For x = (CoorX + 1) To 7
                    If WhiteTFTable(x, CoorY) = "T" Then WhiteTFTable(x,
CoorY) = "F"
                    If Char.IsLower(Board(x, CoorY)) Then
                        If Board(x, CoorY) = "p" AndAlso x & CoorY - 1 =
EnPassant Then
                            For m = x + 2 To 7
                                If Board(m, CoorY) = "K" Then
                                    BlackTFTable(x + 1, CoorY) = "4"
                                    Exit For
                                End If
                                If Board(m, CoorY) <> " " Then Exit For
                            Next
                        End If
                    End If
                Exit For
            Else
                LegalMoveArray(n) = (x) & (CoorY)
                If Board(x, CoorY) = "K" Then
                    WInCheck.IsInCheck = True
                    CheckingPiece = CoorX & CoorY
                    If WhiteTFTable(Math.Min(x + 1, 7), CoorY) =
"T" Then WhiteTFTable(Math.Min(x + 1, 7), CoorY) = "F"
                    Exit For
                End If
                n += 1
                If Char.IsUpper(Board(x, CoorY)) AndAlso CInt(CStr
r(WKPos(0))) > x AndAlso CInt(CStr(WKPos(1))) = CoorY Then
                    For m = x + 1 To 7
                        If Board(m, CoorY) = "K" Then
                            WhiteTFTable(x, CoorY) = "2"
                            Exit For
                        End If
                        If Board(m, CoorY) <> " " Then
                            If Board(m, CoorY) = "p" AndAlso m &
CoorY - 1 = EnPassant Then
                                For a = m + 1 To 7

```

"4"

n Exit For

CoorY) = "F"

EnPassant Then

"T" Then WhiteTFTable(Math.Max(x - 1, 0), CoorY) = "F"

CoorY - 1 = EnPassant Then

"4"

n Exit For

```
If Board(a, CoorY) = "K" Then
    BlackTFTable(x, CoorY) =
```

```
Exit For
```

```
End If
```

```
If Board(a, CoorY) <> " " The
```

```
Next
```

```
End If
```

```
Exit For
```

```
End If
```

```
Next
```

```
End If
```

```
If Board(x, CoorY) <> " " Then Exit For
```

```
End If
```

```
Next
```

```
For x = (CoorX - 1) To 0 Step -1
```

```
If WhiteTFTable(x, CoorY) = "T" Then WhiteTFTable(x,
```

```
If Char.IsLower(Board(x, CoorY)) Then
```

```
If Board(x, CoorY) = "p" AndAlso x & CoorY - 1 =
```

```
For m = x - 2 To 0 Step -1
```

```
If Board(m, CoorY) = "K" Then
```

```
BlackTFTable(x - 1, CoorY) = "4"
```

```
Exit For
```

```
End If
```

```
If Board(m, CoorY) <> " " Then Exit For
```

```
Next
```

```
End If
```

```
Exit For
```

```
Else
```

```
LegalMoveArray(n) = (x) & (CoorY)
```

```
If Board(x, CoorY) = "K" Then
```

```
WInCheck.IsInCheck = True
```

```
CheckingPiece = CoorX & CoorY
```

```
If WhiteTFTable(Math.Max(x - 1, 0), CoorY) =
```

```
"T" Then WhiteTFTable(Math.Max(x - 1, 0), CoorY) = "F"
```

```
Exit For
```

```
End If
```

```
n += 1
```

```
If Char.IsUpper(Board(x, CoorY)) AndAlso CInt(CStr
```

```
r(WKPos(0))) < x AndAlso CInt(CStr(WKPos(1))) = CoorY Then
```

```
For m = x - 1 To 0 Step -1
```

```
If Board(m, CoorY) = "K" Then
```

```
WhiteTFTable(x, CoorY) = "2"
```

```
Exit For
```

```
End If
```

```
If Board(m, CoorY) <> " " Then
```

```
If Board(m, CoorY) = "p" AndAlso m &
```

```
For a = m - 1 To 0 Step -1
```

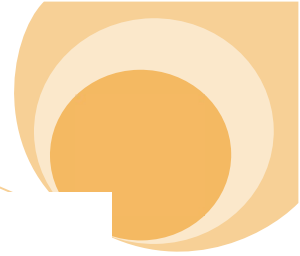
```
If Board(a, CoorY) = "K" Then
```

```
BlackTFTable(x, CoorY) =
```

```
Exit For
```

```
End If
```

```
If Board(a, CoorY) <> " " The
```



```

Next
End If
Exit For
End If
Next
End If
If Board(x, CoorY) <> " " Then Exit For
End If
Next
End If

If BlackTFTable(CoorX, CoorY) <> "2" Then
For y = (CoorY + 1) To 7
If WhiteTFTable(CoorX, y) = "T" Then WhiteTFTable(Coo
rX, y) = "F"

If Char.IsLower(Board(CoorX, y)) Then
Exit For
Else
LegalMoveArray(n) = (CoorX) & (y)
If Board(CoorX, y) = "K" Then
WInCheck.IsInCheck = True
CheckingPiece = CoorX & CoorY
If WhiteTFTable(CoorX, Math.Min(y + 1, 7)) =
"T" Then WhiteTFTable(CoorX, Math.Min(y + 1, 7)) = "F"
Exit For
End If
n += 1
If Char.IsUpper(Board(CoorX, y)) AndAlso CInt(CSt
r(WKPos(0))) = CoorX AndAlso CInt(CStr(WKPos(1))) > y Then
For m = y + 1 To 7
If Board(CoorX, m) = "K" Then
WhiteTFTable(CoorX, y) = "0"
Exit For
End If
If Board(CoorX, m) <> " " Then Exit For
Next
End If
If Board(CoorX, y) <> " " Then Exit For
End If
Next

For y = (CoorY - 1) To 0 Step -1
If WhiteTFTable(CoorX, y) = "T" Then WhiteTFTable(Coo
rX, y) = "F"

If Char.IsLower(Board(CoorX, y)) Then
Exit For
Else
LegalMoveArray(n) = (CoorX) & (y)
If Board(CoorX, y) = "K" Then
WInCheck.IsInCheck = True
CheckingPiece = CoorX & CoorY
If WhiteTFTable(CoorX, Math.Max(y - 1, 0)) =
"T" Then WhiteTFTable(CoorX, Math.Max(y - 1, 0)) = "F"
Exit For
End If
n += 1
If Char.IsUpper(Board(CoorX, y)) AndAlso CInt(CSt
r(WKPos(0))) = CoorX AndAlso CInt(CStr(WKPos(1))) < y Then
For m = y - 1 To 0 Step -1
If Board(CoorX, m) = "K" Then

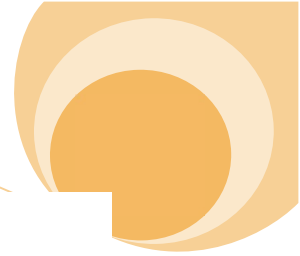
```

```

        WhiteTFTable(CoorX, y) = "0"
        Exit For
    End If
    If Board(CoorX, m) <> " " Then Exit For
Next
End If
If Board(CoorX, y) <> " " Then Exit For
End If
Next
End If
End If
End If

If Board(CoorX, CoorY) = "b" OrElse Board(CoorX, CoorY) = "q" Then 'Legal Moves for the Bishop / Part of Queen.
    Dim o, p As SByte
    If BlackTFTable(CoorX, CoorY) <> "0" AndAlso BlackTFTable(CoorX, CoorY) <> "2" Then
        If BlackTFTable(CoorX, CoorY) <> "1" Then
            StartX = CoorX + 1
            StartY = CoorY + 1
            Do Until StartX > 7 OrElse StartY > 7
                If WhiteTFTable(StartX, StartY) = "T" Then WhiteTFTable(StartX, StartY) = "F"
                If Char.IsLower(Board(StartX, StartY)) Then
                    Exit Do
                Else
                    LegalMoveArray(n) = StartX & StartY
                    If Board(StartX, StartY) = "K" Then
                        WInCheck.IsInCheck = True
                        CheckingPiece = CoorX & CoorY
                        If StartX < 7 AndAlso StartY < 7 Then
                            If WhiteTFTable(StartX + 1, StartY + 1) = "T" Then WhiteTFTable(StartX + 1, StartY + 1) = "F"
                        End If
                    End If
                    Exit Do
                End If
                n += 1
            Do
                If Char.IsUpper(Board(StartX, StartY)) AndAlso (CInt(CStr(WKPos(0))) - CoorX = CInt(CStr(WKPos(1))) - CoorY) AndAlso CInt(CStr(WKPos(0))) > CoorX Then
                    o = StartX + 1
                    p = StartY + 1
                    Do Until o > 7 OrElse p > 7
                        If Board(o, p) = "K" Then
                            WhiteTFTable(StartX, StartY) = "3"
                            Exit Do
                        End If
                        If Board(o, p) <> " " Then Exit Do
                        o += 1
                        p += 1
                    Loop
                End If
                If Board(StartX, StartY) <> " " Then Exit Do
            End If
            StartX += 1
            StartY += 1
        Loop
    End If
End If

```



```

    StartX = CoorX - 1
    StartY = CoorY - 1
    Do Until StartX < 0 OrElse StartY < 0
        If WhiteTFTable(StartX, StartY) = "T" Then WhiteTFTab
le(StartX, StartY) = "F"
        If Char.IsLower(Board(StartX, StartY)) Then
            Exit Do
        Else
            LegalMoveArray(n) = StartX & StartY
            If Board(StartX, StartY) = "K" Then
                WInCheck.IsInCheck = True
                CheckingPiece = CoorX & CoorY
                If StartX > 0 AndAlso StartY > 0 Then
                    If WhiteTFTable(StartX - 1, StartY - 1) =
"T" Then WhiteTFTable(StartX - 1, StartY - 1) = "F"
                    End If
                    Exit Do
                End If
                n += 1
                If Char.IsUpper(Board(StartX, StartY)) AndAlso (C
Int(CStr(WKPos(0))) - CoorX = CInt(CStr(WKPos(1))) - CoorY) AndAlso CInt(CStr
(WKPos(0))) < CoorX Then
                    o = StartX - 1
                    p = StartY - 1
                    Do Until o < 0 OrElse p < 0
                        If Board(o, p) = "K" Then
                            WhiteTFTable(StartX, StartY) = "3"
                            Exit Do
                        End If
                        If Board(o, p) <> " " Then Exit Do
                        o -= 1
                        p -= 1
                    Loop
                End If
                If Board(StartX, StartY) <> " " Then Exit Do
            End If
            StartX -= 1
            StartY -= 1
        Loop
    End If

    If BlackTFTable(CoorX, CoorY) <> "3" Then
        StartX = CoorX + 1
        StartY = CoorY - 1
        Do Until StartX > 7 OrElse StartY < 0
            If WhiteTFTable(StartX, StartY) = "T" Then WhiteTFTab
le(StartX, StartY) = "F"
            If Char.IsLower(Board(StartX, StartY)) Then
                Exit Do
            Else
                LegalMoveArray(n) = StartX & StartY
                If Board(StartX, StartY) = "K" Then
                    WInCheck.IsInCheck = True
                    CheckingPiece = CoorX & CoorY
                    If StartX < 7 AndAlso StartY > 0 Then
                        If WhiteTFTable(StartX + 1, StartY - 1) =
"T" Then WhiteTFTable(StartX + 1, StartY - 1) = "F"
                        End If
                        Exit Do
                    End If
                End If
            End If
        Loop
    End If

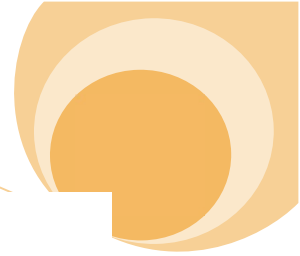
```

```

n += 1
If Char.IsUpper(Board(StartX, StartY)) AndAlso (C
Int(CStr(WKPos(0))) - CoorX = CoorY - CInt(CStr(WKPos(1))) AndAlso CInt(CStr
(WKPos(0))) > CoorX Then
    o = StartX + 1
    p = StartY - 1
    Do Until o > 7 OrElse p < 0
        If Board(o, p) = "K" Then
            WhiteTFTable(StartX, StartY) = "1"
            Exit Do
        End If
        If Board(o, p) <> " " Then Exit Do
        o += 1
        p -= 1
    Loop
End If
If Board(StartX, StartY) <> " " Then Exit Do
End If
StartX += 1
StartY -= 1
Loop

StartX = CoorX - 1
StartY = CoorY + 1
Do Until StartX < 0 OrElse StartY > 7
    If WhiteTFTable(StartX, StartY) = "T" Then WhiteTFTab
le(StartX, StartY) = "F"
    If Char.IsLower(Board(StartX, StartY)) Then
        Exit Do
    Else
        LegalMoveArray(n) = StartX & StartY
        If Board(StartX, StartY) = "K" Then
            WInCheck.IsInCheck = True
            CheckingPiece = CoorX & CoorY
            If StartX > 0 AndAlso StartY < 7 Then
                If WhiteTFTable(StartX - 1, StartY + 1) =
"T" Then WhiteTFTable(StartX - 1, StartY + 1) = "F"
            End If
            Exit Do
        End If
    End If
    n += 1
    If Char.IsUpper(Board(StartX, StartY)) AndAlso (C
Int(CStr(WKPos(0))) - CoorX = CoorY - CInt(CStr(WKPos(1))) AndAlso CInt(CStr
(WKPos(0))) < CoorX Then
        o = StartX - 1
        p = StartY + 1
        Do Until o < 0 OrElse p > 7
            If Board(o, p) = "K" Then
                WhiteTFTable(StartX, StartY) = "1"
                Exit Do
            End If
            If Board(o, p) <> " " Then Exit Do
            o -= 1
            p += 1
        Loop
    End If
    If Board(StartX, StartY) <> " " Then Exit Do
End If
StartX -= 1
StartY += 1

```

```

        Loop
        End If
    End If
End If

'At the end of the function, the program checks to see if any piece i
s attacking the enemy king.
'If it is, then the other player is signalled to be in check, and the
attacking piece is recorded.
If CheckingPiece <> " " Then
    If WInCheck.Piece = " " OrElse WInCheck.Piece = CheckingPiece The
n
        WInCheck.Piece = CheckingPiece
    Else
        WInCheck.DoubleCheck = True
    End If
End If
'The first index of LegalMoveArray contains a counter of how many mov
es each piece can make. Useful for
'when the AI is picking moves.
If n <> 1 Then LegalMoveArray(0) = n
Return LegalMoveArray
End Function

```

FIXTFTABLE SUBROUTINE & OTHER CONTROLS

```

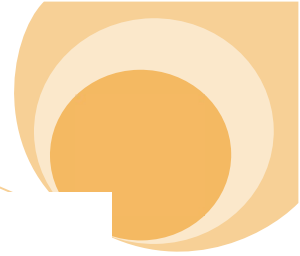
'Subroutine which creates the TrueFalse Table of the selected player (control
led by the Variable FixWhite).
'This is done by generating all the legal moves of the pieces that could
influence the enemy king's motion.
'This creates a 'field' around the king (stating where its legal moves ar
e), along with creating pinned pieces
'and checks.
Public Sub FixTFTables(ByVal Board(,) As Char, ByVal FixWhite As Boolean,
ByRef TrueFalseTable(,) As Char, ByRef KPos As String, ByRef WInCheck As InC
heck, ByRef BInCheck As InCheck, ByVal EnPasant As String)
    Dim dx, dy As SByte
    'Resets TFTables.
    Array.Copy(MasterTrueTable, TrueFalseTable, 64)
    Array.Copy(MasterTrueTable, TrueTable, 64)
    If FixWhite Then
        For y = 0 To 7
            For x = 0 To 7
                If Char.IsLower(Board(x, y)) Then
                    'Calculates distances between piece and the enemy kin
g.
                    dx = Math.Abs(CStr(KPos(0)) - x)
                    dy = Math.Abs(CStr(KPos(1)) - y)
                    If Board(x, y) = "p" Then
                        If Math.Max(dx, dy) <= 2 AndAlso CStr(KPos(1)) >=
y Then
                            'Pawn could influence king motion - calculate
legal moves.
                            BlackPieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, WInCheck, BInCheck, CannotCastle, EnPasant)
                        End If
                    ElseIf Board(x, y) = "b" Then
                        If Math.Abs(dx - dy) <= 2 Then

```

```

'Piece could influence king motion - calculate
e legal moves.
BlackPieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, WInCheck, BInCheck, CannotCastle, EnPassant)
End If
ElseIf Board(x, y) = "n" Then
If Math.Max(dx, dy) <= 3 AndAlso dx + dy <= 5 Then
n
'Piece could influence king motion - calculate
e legal moves.
BlackPieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, WInCheck, BInCheck, CannotCastle, EnPassant)
End If
ElseIf Board(x, y) = "r" Then
If Math.Min(dx, dy) <= 1 Then
'Piece could influence king motion - calculate
e legal moves.
BlackPieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, WInCheck, BInCheck, CannotCastle, EnPassant)
End If
ElseIf Board(x, y) = "q" Then
If Math.Min(dx, dy) <= 1 OrElse Math.Abs(dx - dy)
<= 2 Then
'Piece could influence king motion - calculate
e legal moves.
BlackPieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, WInCheck, BInCheck, CannotCastle, EnPassant)
End If
ElseIf Math.Max(dx, dy) <= 2 Then 'is a king.
'Piece could influence king motion - calculate le
gal moves.
BlackPieceLegalMoves(Board, x, y, TrueTable, True
FalseTable, KPos, WInCheck, BInCheck, CannotCastle, EnPassant)
End If
End If
Next
Next
Else 'Identical code but for the white pieces (fixing the Black TFTab
le).
For y = 0 To 7
For x = 0 To 7
If Char.IsUpper(Board(x, y)) Then
dx = Math.Abs(CStr(KPos(0)) - x)
dy = Math.Abs(CStr(KPos(1)) - y)
If Board(x, y) = "P" Then
If Math.Max(dx, dy) <= 2 AndAlso CStr(KPos(1)) <=
y Then
WhitePieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, BInCheck, WInCheck, CannotCastle, EnPassant)
End If
ElseIf Board(x, y) = "B" Then
If Math.Abs(dx - dy) <= 2 Then
WhitePieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, BInCheck, WInCheck, CannotCastle, EnPassant)
End If
ElseIf Board(x, y) = "N" Then
If Math.Max(dx, dy) <= 3 AndAlso dx + dy <= 5 Then
n
WhitePieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, BInCheck, WInCheck, CannotCastle, EnPassant)

```



```

        End If
    ElseIf Board(x, y) = "R" Then
        If Math.Min(dx, dy) <= 1 Then
            WhitePieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, BInCheck, WInCheck, CannotCastle, EnPassant)
        End If
    ElseIf Board(x, y) = "Q" Then
        If Math.Min(dx, dy) <= 1 OrElse Math.Abs(dx - dy)
<= 2 Then
            WhitePieceLegalMoves(Board, x, y, TrueTable,
TrueFalseTable, KPos, BInCheck, WInCheck, CannotCastle, EnPassant)
        End If
    ElseIf Math.Max(dx, dy) <= 2 Then 'is a king.
        WhitePieceLegalMoves(Board, x, y, TrueTable, True
FalseTable, KPos, BInCheck, WInCheck, CannotCastle, EnPassant)
    End If
End If
Next
Next
End If
End Sub

```

'Algorithm that returns the weight / value of a given piece. Links to the array of hashed values PieceValue.

```

Public Function ReturnPieceValue(ByVal Piece As Char) As Decimal
    Return PieceValue(Asc(UCase(Piece)) Mod 11)
End Function

```

'Function which counts up all the material (and their values) on the board. Stores this information in two

'variables - one for white's total material count, and the other for black's total material count.

```

Public Function CountMaterial(ByVal Board(,) As Char) As SByte()
    Dim MaterialCount(1) As SByte
    For y = 0 To 7
        For x = 0 To 7
            If Board(x, y) <> " " Then
                'Add the value of the piece to either White or Black's total.
                If Char.IsUpper(Board(x, y)) Then
                    MaterialCount(0) += ReturnPieceValue(Board(x, y))
                Else
                    MaterialCount(1) += ReturnPieceValue(Board(x, y))
                End If
            End If
        Next
    Next
    Return MaterialCount
End Function

```

'Subroutine that converts a Move into standard chess notation (eg: e4, Nf4, Ka2).

```

Public Function MoveConverter(ByVal Board(,) As Char, ByVal TempMove As Move, ByVal EnPassant As String) As String
    Dim MovedPiece As Char = UCase(Board(TempMove.OldMoveX, TempMove.OldMoveY))

```

```

'Pawns operate differently with standard chess notation - when moving
a pawn, we give its column (a-h),
'then add its file that it is moving to. For all other pieces, we sta
te the name of the piece, then its
'end coordinates.
If MovedPiece <> " " Then
    If MovedPiece = "P" Then
        MoveConverter = Chr(CStr(TempMove.OldMoveX) + 97)
        'Code for detecting castling.
    ElseIf MovedPiece = "K" AndAlso TempMove.OldMoveX = 4 AndAlso (Te
mpMove.NewMoveY = 0 OrElse TempMove.NewMoveY = 7) Then
        If TempMove.NewMoveX = 6 Then 'Is king-side castling.
            Return "O-O" 'Notation for KS castling.
        ElseIf TempMove.NewMoveX = 2 Then 'Is queen-side castling.
            Return "O-O-O" 'Notation for QS castling.
        Else
            MoveConverter = MovedPiece
        End If
    Else
        MoveConverter = MovedPiece
    End If
    If Board(TempMove.NewMoveX, TempMove.NewMoveY) <> " " OrElse (UCa
se(MovedPiece) = "P" AndAlso EnPassant = TempMove.NewMoveX & TempMove.NewMove
Y) Then
        'Is a capture move - add an "x" followed by the coordinates o
f the captured piece.
        MoveConverter &= "x" & Chr(CStr(TempMove.NewMoveX) + 97) & 8
- TempMove.NewMoveY
    Else
        If MovedPiece <> "P" Then MoveConverter &= Chr(CStr(TempMove.
NewMoveX) + 97)
        MoveConverter &= 8 - TempMove.NewMoveY
    End If
    'Code for pawn promotions.
    If MovedPiece = "P" AndAlso (TempMove.NewMoveY = 0 OrElse TempMov
e.NewMoveY = 7) Then MoveConverter &= "=Q"
    Else
        Return "ERROR"
    End If
End If
End Function

End Class

```

SUBOBJECTS

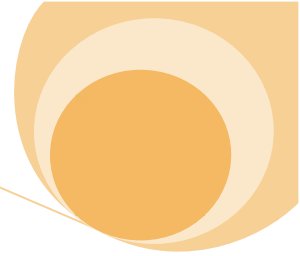
'Below are the subobjects containing information about castling & checking ru
les,
'along with some getter (used mainly to reset privileges) & setter functions.

INCHECK CLASS

```

Public Class InCheck
    Public IsInCheck As Boolean
    Public Piece As String
    Public DoubleCheck As Boolean
    Public Sub New()
        Piece = " "
    End Sub

```



```

Public Sub NotInCheck() 'Resets all rules.
    IsInCheck = False
    Piece = " "
    DoubleCheck = False
End Sub
Public Sub CopyFrom(ByVal Copier As InCheck) 'Copies all the data from
the parameter class.
    IsInCheck = Copier.IsInCheck
    Piece = Copier.Piece
    DoubleCheck = Copier.DoubleCheck
End Sub

```

CANCASTLE CLASS

```

Public Class CanCastle
    Public KS As Boolean
    Public QS As Boolean
    Public Sub CannotCastle() 'Resets Castling Privileges.
        KS = False
        QS = False
    End Sub
    Public Sub CopyFrom(ByVal Copier As CanCastle) 'Copies all the data from the
parameter class.
        KS = Copier.KS
        QS = Copier.QS
    End Sub
    Public Function CanICastle() As Boolean 'Returns True if any castling
privileges exist.
        Return (KS OrElse QS)
    End Function
End Class

```

MOVE STRUCTURE

```

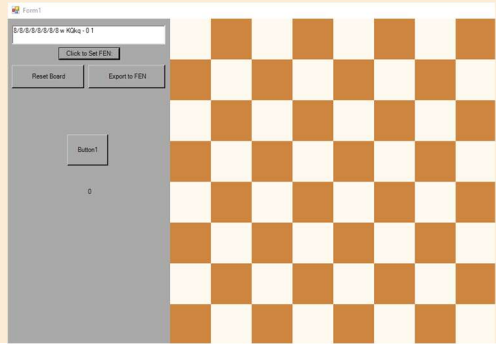
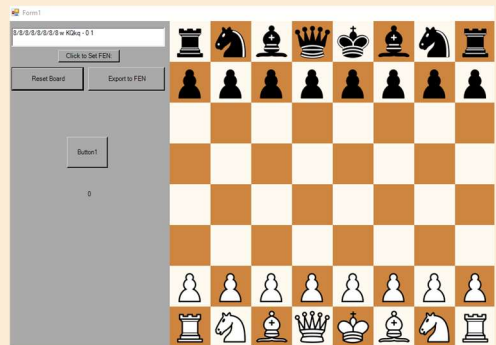
Public Structure Move 'Information about an AI's move.
    Public Score As Decimal 'Move Evaluation.
    Public OldMoveX As String
    Public OldMoveY As String
    Public NewMoveX As String
    Public NewMoveY As String
    Public EndState As Char 'Contains information about an AI's search:
    'f = no end-
state detected, a = search manually aborted, c = checkmate found, s = stalema
te, o = one move found (forced)
End Structure

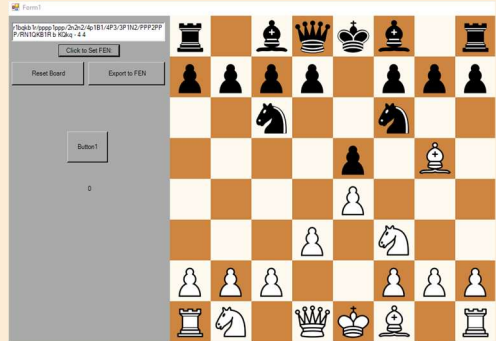
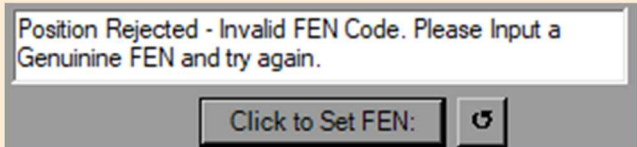
```

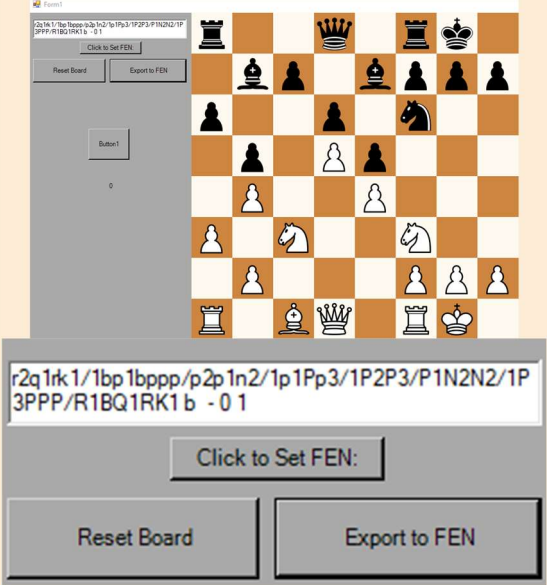
SECTION 4: TESTING

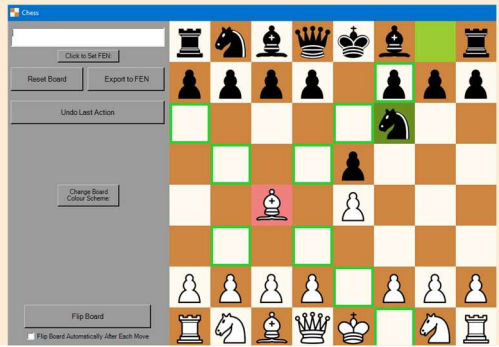
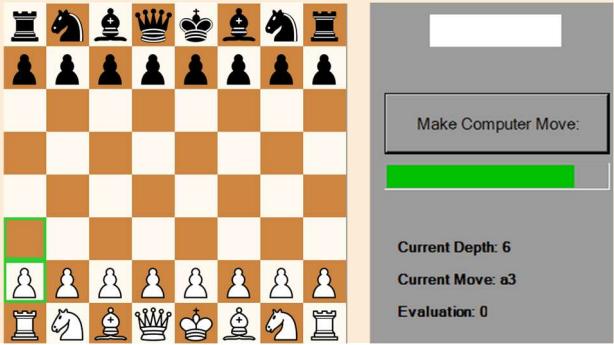
For some of the below tests, it is necessary to provide evidence in a video format rather than through screenshots. To access these tests, please view this private YouTube video: <https://youtu.be/rAxARmMeF7Q> (all included tests can be found as timestamps in the video's description). The beginning of this testing video contains a test of my system's robustness, involving a typical game with my AI. This is then proceeded by the test evidence for the required tests below.

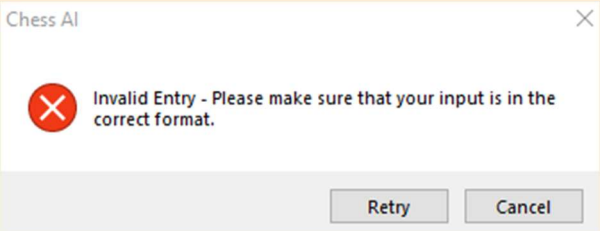
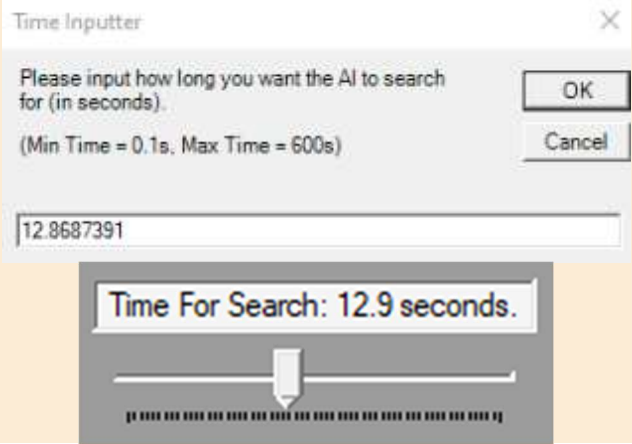
TESTING THE GUI & ITS FEATURES

No	Section	Test Description	Data Used	Expected Outcome
		Actual Outcome		Test Evidence
1	Test Details	Testing that a checkerboard pattern is made on the screen at the start of the program.	N/A	8x8 checkerboard pattern of alternating colours produced.
	Test Results	Same as expected.		
2	Test Details	Testing that all the chess pieces are successfully placed on the board for the starting position of chess.	N/A	32 PictureBoxes appropriately placed on their starting squares on the board – centred in their squares.
	Test Results	Same as expected.		
3	Test Details	Testing that pieces can be picked up, dragged, and dropped to any location on the screen, which then 'snap' into their appropriate box's centre.	Typical (e2->e3/e4)	White pawn being moved to the centre of the board (not into centre of square), then being snapped into place.
	Test Results	Same as expected.	Please refer to 5:03 of my testing video.	

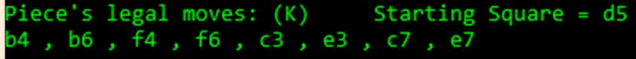
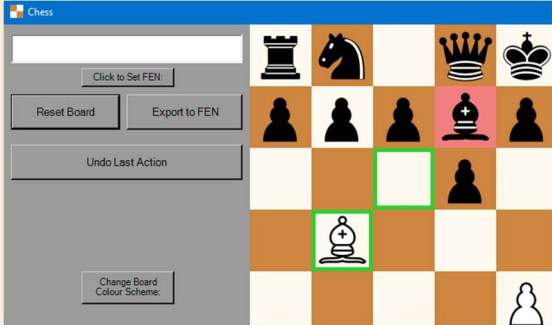
4	Test Details	Testing that only pseudo-legal moves specified by the PieceLegalMove subroutine are allowed to be made on the board.	Typical (moving e2 pawn).	When moving the e2 pawn, only the squares e3 and e4 are allowed to be made. If the user makes any other move, then the pawn is sent back to its starting square.
	Test Results	Same as expected (a multitude of tests for all the different pieces were also performed separately).	Please refer to 5:03 of my testing video.	
5	Test Details	Testing that the kings are allowed to castle kingside & queenside, <u>if that move is legal</u> .	Typical (Kg1, Kg8, Kc1, Kc8)	If the move is legal, then the castling move should perform – an act that involves moving the correct rook, updating castling privileges, and playing the castling sound.
	Test Results	Same as expected.	Please refer to 5:31 of my testing video.	
6	Test Details	Testing that the board, at any time, can be reset to the starting position.	N/A	When pressing the 'Reset Board' button, the position should revert back to the starting position.
	Test Results	Same as expected.	Please refer to 5:54 of my testing video.	
7	Test Details	Testing that the user is able to enter in a valid FEN, and have the move be represented on the board.	Typical - r1bqkb1r/pppp1ppp/2n2n2/4p1b1/4P3/3P1N2/PPP2PPP/RN1QKB1R b KQkq -	The FEN position is correctly translated onto the board, and all the necessary position details are modified.
	Test Results	Same as expected.		
8	Test Details	Testing that only valid FENs can be inputted into the system.	Erroneous – length checks, format checks, type checks, and more (according to the rules of chess). Please see my FEN validation process for more details.	If the FEN is invalid, then nothing should change on the board. The user should also be given an appropriate message explaining why the FEN is invalid.
	Test Results	Same as expected.		

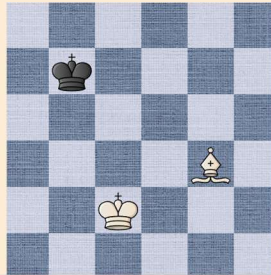
9	Test Details	Testing that the current board position can be converted to its equivalent FEN.	Typical (sample board positions).	When the user presses 'Export to FEN', the board position (and all its details) should be condensed down into its FEN, then displayed to the user.
	Test Results	Same as expected.		
10	Test Details	Testing that, when a player is put into check, or when the game has ended, the user is notified of this visually and audibly	Typical (black king put into check by bishop).	When the player is put into check, their king should take on a red 'glow', and the CheckBox should contain the word "Check". The checking sound should also play.
	Test Results	Same as expected.		
11	Test Details	Testing that, when a player is checkmated or the game results in a stalemate, the game should end, and no further moves can be made on the board.	Typical (black king put into checkmate).	White checkmates black, and therefore ends the game. The user should not be able to make any more moves on the board.
	Test Results	Same as expected.	Please refer to 6:09 of my testing video.	
12	Test Details	Testing that the board position can be flipped 180° in real time.	N/A	When the user presses the 'Flip Board' button, the entire chessboard should rotate from white's POV to black's POV (180°).
	Test Results	Same as expected.	Please refer to 6:29 of my testing video.	

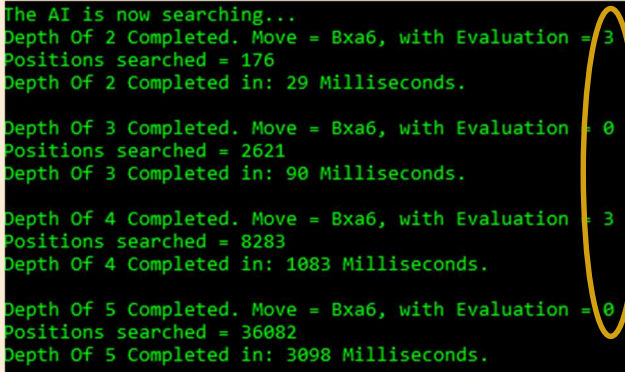
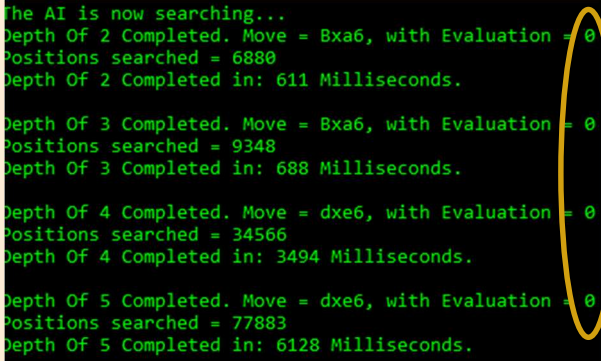
13	Test Details	Testing that the squares on the chessboard are coloured appropriately for specific actions.	Typical (Nf6, then moving white's bishop)	The previously made move should be highlighted on the chessboard (green squares). When the user clicks on a piece, that square should be coloured (pink) and that piece's legal moves should be outlined (lime).
	Test Results	Same as expected.		
14	Test Details	Testing that AI's chosen move is presented to the user via the GUI (whilst the AI is thinking).	Typical (starting position).	When the AI is searching, it should give the user details about its current search. This includes the AI's progress, its current move, its evaluation, and the move's depth. This move should also be highlighted on the GUI.
	Test Results	Same as expected.		
15	Test Details	Testing that, at the start of the program, the MainMenu form should appear, and allows the user to select their game mode, along with any necessary customisations.	N/A	The program should open with the MainMenu form, which should play an opening animation and present the user with three game modes. If the user selects the 1/2 Player Modes, then they should be taken to a separate form where they can customise their game. After this, the user will be redirected to the Chess form.
	Test Results	Same as expected.	Please refer to 6:55 of my testing video.	

16	Test Details	Testing that, depending on what game mode the user selected in the Main Menu, the Chess form should be displayed differently.	Typical (Gamemode = 1, 2, 3)	If the user selects Analysis (3) Mode, then all the GUI features should be available. Otherwise, they will be presented with a smaller form that only has the controls necessary to their game mode. For more details on the differences between the modes, please see this section .
	Test Results	Same as expected.	Please refer to 6:55 of my testing video.	
17	Test Details	Testing that, when the user inputs how long they want the AI to search for, they can only enter a valid decimal from 0.1s to 600s.	Boundary & Erroneous – length check “”, type check “hello”, range check “0”, “650”, format check “1..5”.	If the user enters an invalid entry, the system should flag this and ask the user to enter in a new value.
	Test Results	Same as expected.		
18	Test Details	Testing that, when the user inputs how long they want the AI to search for, their input will be rounded to one decimal place. Prevents the time not fitting correctly into the progress bar.	Boundary – 12.8687391	The system should convert this time to 12.9 seconds, and display this in the TimeBox.
	Test Results	Same as expected.		

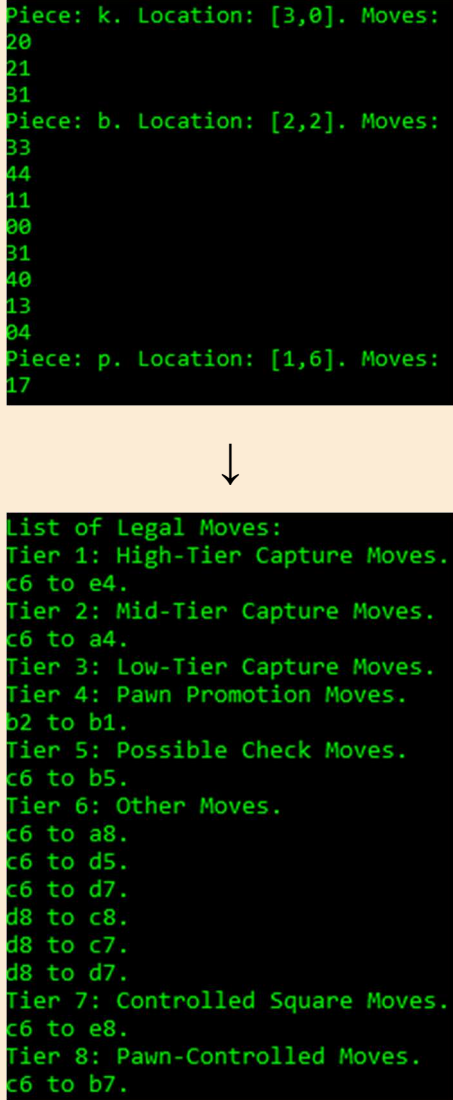
TESTING AI FEATURES

No	Section	Test Description	Data Used	Expected Outcome
		Actual Outcome		Test Evidence
1	Test Details	Testing that the computer can calculate all the pseudo-legal moves of any piece on the board, according to the standard rules of chess.	Typical (moving a knight from d5 on an empty board).	When needed, the program will generate a list of any piece's pseudo-legal moves. This includes the squares: b4, b6, c3, c7, e3, e7, f4, and f6.
	Test Results	Same as expected.		 <pre>Piece's legal moves: (K) Starting Square = d5 b4 , b6 , f4 , f6 , c3 , e3 , c7 , e7</pre>
2	Test Details	Testing that the computer restricts the movement of pinned pieces, according to the standard rules of chess.	Typical (moving a pinned bishop on d7).	In this case, the black bishop should only be able to move top-right and bottom-right. Any other movement would cause the king to be taken, so those moves are illegal.
	Test Results	Same as expected.		
3	Test Details	Testing that, when the user is in check, the computer will only accept the pseudo-legal moves that do not help escape check.	Typical - rnbqkbnr/ppp1p1pp /3p1p2/1B6/4P3/2N5/ PPPP1PPP/R1BQK1NR b KQkq -	When the computer generates black's legal moves, only those that resolve check should be included. These are: Nc6, Nd7, c6, Bd7, Qd7, Kf7.
	Test Results	Same as expected.		 <pre>All legal moves: KING IN CHECK - Moves filtered. Nd7 , Nc6 , Bd7 , Qd7 , Kf7 , c6</pre>

4	Test Details	Testing that FENs that involve invalid checks (black in check with white to move, or vice versa) or touching kings, will be blocked from playing with.	Erroneous - 8/8/2k5/ 3K4/8/8/8/8 w - - 0 I and 4K3/4q3/3k4 /8/8/8/8/8 b - - 0 I	When entering in this illegal FEN, the AI should flag this and stop the game from running. The user should not be able to move any pieces, and will have to enter in another FEN in order to continue playing.
	Test Results	Same as expected.	Please refer to 8:28 of my testing video.	
5	Test Details	Testing that the AI can correctly evaluate if a given position is an end state (checkmate, stalemate, etc).	Typical (black has checkmated white)	When running the CheckForEndState function in the AI, it should return the code "c" for checkmate.
	Test Results	Same as expected.	N/A	
6	Test Details	Testing that the system can detect draws by insufficient material.	Typical (king vs king + knight)	The computer should flag this position as a draw, and should terminate the game.
	Test Results	Same as expected.	 <pre> Checking for end states: Low material count detected: checking for insufficient material... Possible King vs King + (Bishop / Knight) detected. Searching... END STATE FOUND - Reason = Draw by Insufficient Material. </pre>	
7	Test Details	Testing that the AI uses multithreading to run multiple AIs simultaneously, whilst not hindering GUI performance.	N/A	All the AIs (of multiple depths) should run on the same position simultaneously and update the user with its progress. The user should also be able to move the window during this search.
	Test Results	Same as expected.	<pre> The AI is now searching... Depth Of 2 Completed. Move = a3, with Evaluation = 0 Positions searched = 63 Depth Of 2 Completed in: 28 Milliseconds. Depth Of 3 Completed. Move = a3, with Evaluation = 0 Positions searched = 525 Depth Of 3 Completed in: 41 Milliseconds. Depth Of 4 Completed. Move = a3, with Evaluation = 0 Positions searched = 1530 Depth Of 4 Completed in: 202 Milliseconds. Depth Of 5 Completed. Move = a3, with Evaluation = 0 Positions searched = 13078 Depth Of 5 Completed in: 504 Milliseconds. Depth Of 6 Completed. Move = a3, with Evaluation = 0 Positions searched = 44826 Depth Of 6 Completed in: 4208 Milliseconds. Depth Of 7 Completed. Move = a3, with Evaluation = 0 Positions searched = 363027 Depth Of 7 Completed in: 10313 Milliseconds. </pre>	

8	Test Details	Testing that the AI effectively uses the MiniMax algorithm to calculate optimal moves.	Typical - r1bqkbnr/pppp1ppp /8/4p3/2BnP3/5Q2/ PPPP1PPP/RNB1K1NR w KQkq -	In this position, the definitive best move for white is to play Qxf7, which results in white checkmating black. Therefore, the AI must play this move.
	Test Results	Same as expected.	Please refer to 9:07 of my testing video.	
9	Test Details	Testing that the AI can effectively use Quiescence to generate stable evaluations of positions.	Typical - r3k2r/p1ppqpb1 /bn2pnp1/3PN3/1p2P3 /2N2Q1p/PPPBPPPP /R3K2R w KQkq	With Quiescence enabled, the computer should be evaluating this complex position with much more accuracy – evaluating it as being close to 0.
	Test Results	Same as expected.	Without Quiescence:  With Quiescence: 	
10	Test Details	Testing that, if all the AIs have completed their searches before the time has expired, searches at higher depths are conducted.	Typical - 8/8/1k6/8/8 /3K3P/8/8 w -	In this example, all 5 AIs will search from a depth of 1-5. Once the depths of 2&3 have completed, a depth of 6&7 should be conducted using the same AIs. If these searches finish in time then the search stops early.

	Test Results	Same as expected.	<pre> The AI is now searching... Time for Search = 3 Seconds. Quiescence Mode: True Depth Of 1 Completed. Move = Kd4, with Evaluation = 1.41 Positions searched = 9 Depth Of 1 Completed in: 0 Milliseconds. Moving to a Depth of: 6 Depth Of 2 Completed. Move = Kd4, with Evaluation = 1.30 Positions searched = 42 Depth Of 2 Completed in: 4 Milliseconds. Moving to a Depth of: 7 Depth Of 3 Completed. Move = Kc4, with Evaluation = 1.30 Positions searched = 348 Depth Of 3 Completed in: 13 Milliseconds. Depth Of 4 Completed. Move = Kc4, with Evaluation = 1.19 Positions searched = 1219 Depth Of 4 Completed in: 48 Milliseconds. Depth Of 5 Completed. Move = Kc4, with Evaluation = 1.19 Positions searched = 7909 Depth Of 5 Completed in: 230 Milliseconds. Depth Of 6 Completed. Move = Kc4, with Evaluation = 1.19 Positions searched = 16738 Depth Of 6 Completed in: 452 Milliseconds. Depth Of 7 Completed. Move = Kc4, with Evaluation = 1.19 Positions searched = 77782 Depth Of 7 Completed in: 1696 Milliseconds. All AIs Finished Search Early - Aborting Search. Search Complete. Total Time: 1708.7068 Milliseconds. </pre>	
I1	Test Details	Testing that the user cannot interact with core GUI elements whilst the AI is searching (to prevent crashes)	Typical & Erroneous	When the AI is searching, the user should not be able to interact with the chessboard, the FEN setter, the undo button, the TimeForSearch changers, and the Reset button. All other controls must be available, however.
	Test Results	Same as expected.	Please refer to 9:23 of my testing video.	
I2	Test Details	Testing that the AI will not crash if it is unable to make a move on the board (if no legal move exists).	Boundary	Before running the AI, we test if there are any legal moves possible (using the CheckForEndStates function). If there are then we run the AI – if not then we stop the AI from running. The latter should occur here.
	Test Results	Same as expected.	N/A	
I3	Test Details	Testing that the AI will make forced moves instantly, without having to search.	Typical - r1bqkbnr/pppp1Bpp /8/4p3/3nP3/5Q2 /PPPP1PPP/RNBIK1NR b KQkq -	In this position, black has only one legal move. When the user clicks the “Make Computer Move” button, the AI should notice this forced move and play it without running MiniMax.

	Test Results	Same as expected.	 The AI is now searching... Time for Search = 10 Seconds. Quiescence Mode: True Only one move detected in position: (Ke7). Playing move.	
I4	Test Details	Testing that the AI can effectively order moves from 'best to worst' given certain criteria (which can be found here).	Typical - 3k4/8/PIb5 /8/N3R3/8/ Ip2K3/8 b -	The AI should separate black's moves into 8 tiers ranging from best to worst, in the order: Bxe4, Bxa4, b1=Q, Bb5, [Ba8, Bd5, Bd7, Kc8, Kc7, Kd7] (any order), Be8, Bb7. This reflects the move ordering criteria as specified by my move ordering process.
	Test Results	Same as expected.	 Piece: k. Location: [3,0]. Moves: 20 21 31 Piece: b. Location: [2,2]. Moves: 33 44 11 00 31 40 13 04 Piece: p. Location: [1,6]. Moves: 17 ↓ List of Legal Moves: Tier 1: High-Tier Capture Moves. c6 to e4. Tier 2: Mid-Tier Capture Moves. c6 to a4. Tier 3: Low-Tier Capture Moves. Tier 4: Pawn Promotion Moves. b2 to b1. Tier 5: Possible Check Moves. c6 to b5. Tier 6: Other Moves. c6 to a8. c6 to d5. c6 to d7. d8 to c8. d8 to c7. d8 to d7. Tier 7: Controlled Square Moves. c6 to e8. Tier 8: Pawn-Controlled Moves. c6 to b7.	

TRACKING AI PROGRESS

There are many methods of comparison to track how my AI has improved over time, such as search efficiency, ELO, online puzzle rating, etc. However, in this section I will be focussing on how long it takes each version of my AI to complete a search at a given depth, as I believe this method provides the most accurate and easily comparable scale of how my AI system has developed. To increase the accuracy of these tests, they will be performed on three sample chess positions: a typical opening position, a typical middlegame positions, and a typical endgame position, with white to move in all three cases.

To make the tests more apt for real-world comparisons, I will also calculate the centipawn loss for the AI's moves. This is calculated by finding the difference between Stockfish's best move in the position, and Stockfish's evaluation of the AI's chosen move – I have chosen the Stockfish AI as the method of comparison as it is the undisputed greatest chess-playing entity in the world right now.

Each AI will search on the position two times in a row, and its lowest time of the two will be recorded. Its time will then be plotted onto a base 2 logarithmic graph (chosen as it best illustrates the differences between the AIs). For more information on these tests, along with extra comparative data, please visit the below link to access an online text document:

<https://drive.google.com/file/d/16tNZ2fLQL-gRGBwyEPYDaiTpmDRvhH>

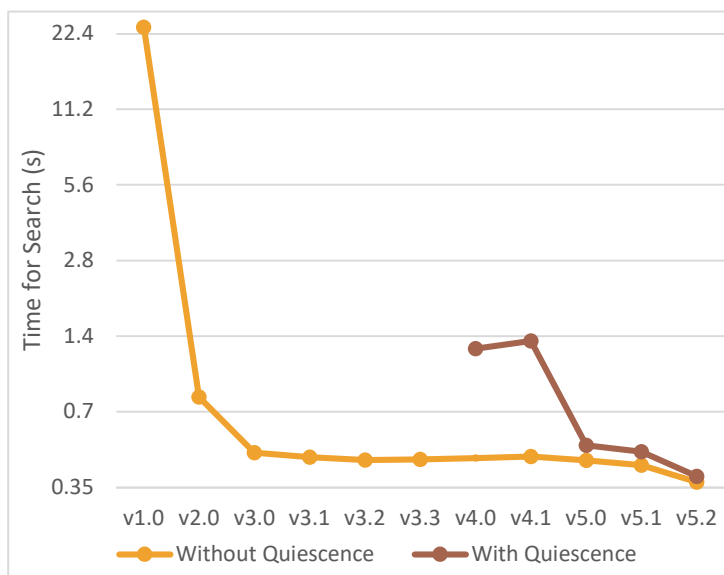
OPENING TEST

The position used for the opening test will be the chess starting position, denoted in FEN notation with (rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq -). This was chosen for the position's simplicity and greatest real-game comparison. Our AI will search on this position with a static depth of 5.



Sample Opening Position.

Best Move (Stockfish) = [e4, Nf3, d4]



Current results (without Quiescence): 21,415 positions searched in 0.367 seconds. Move = b3 (centipawn loss = 30).

Current results (with Quiescence): 13,084 positions searched in 0.387 seconds. Move = a3 (centipawn loss = 30).

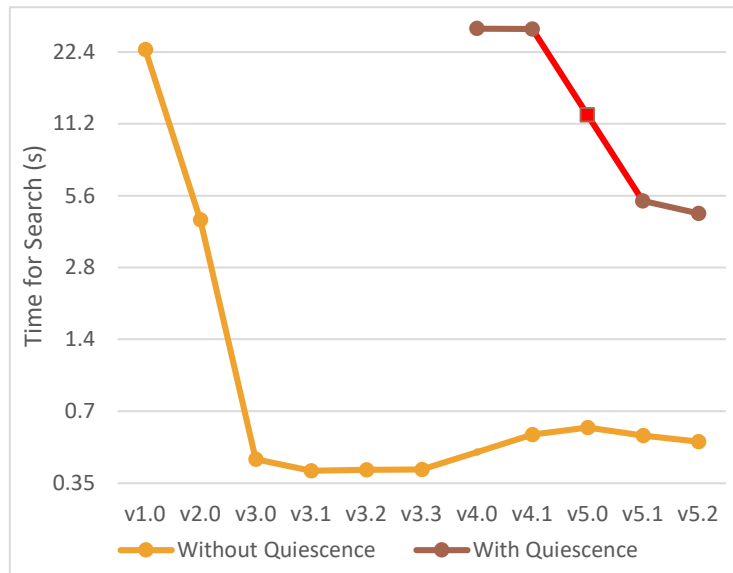
MIDDLEGAME TEST

The middlegame position is this complex position, denoted in FEN notation with (r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/2N2Q1p/PPPBPPPP/R3K2R w KQkq -). This position was chosen for its complexity, and for having many possible moves from a wide range of evaluations. Our AI will search on this position with a static depth of 4.



Sample Middlegame Position.

Best Move (Stockfish) = [Bxa6, dxe6]



Current results (without Quiescence): 6,692 positions searched in 0.522 seconds. Move = Bxa6 (centipawn loss = 0).

Current results (with Quiescence): 56,377 positions searched in 4.723 seconds. Move = dxe6 (centipawn loss = 10).

Please note the square red marker for v5.0's test score with Quiescence. Unfortunately, this test was not able to complete in an acceptable duration of time due to my old implementation of Quiescence containing a huge issue. This bug caused the Quiescence AI to search through many more positions than it needed to in a given search, thus making it incredibly slow in complex positions (such as the one above). This flaw was first introduced in v4.0 of my AI, was 'fixed' using an incorrect method in v5.0 (hence v5.0's great time in the opening & endgame tests, yet shocking performance in the above test), before being fully fixed in v5.1. You can read more about this issue, and how I fixed it, in the [evaluation section](#) of this report.

Another point of interest in this test is the subtle decrease in efficiency from v3.3 to v4.1 (without Quiescence). This results from my change in approach to multithreading in my program (which you can read about [here](#)) – essentially, the huge efficiency increase of my move ordering algorithm (first implemented into v4.0) negated that of multithreading in the vast majority of positions, and I therefore changed my methodology of multithreading in my code. However, in rare circumstances (such as the above position), and somewhat due to the position's complexity and its large amount of available capture moves, the move ordering algorithm is somewhat unable to accurately sort the players' capture moves from best to worst, as they are all placed in similar tiers. Therefore, the efficiency gain from move ordering is overshadowed by the algorithm's time intensity, and hence we see a dip in efficiency.

Thankfully, this subtle speed deficit is only noticeable in very few positions, and for the vast majority the move ordering algorithm has an incredible effect on performance – as you will soon see in the endgame test!

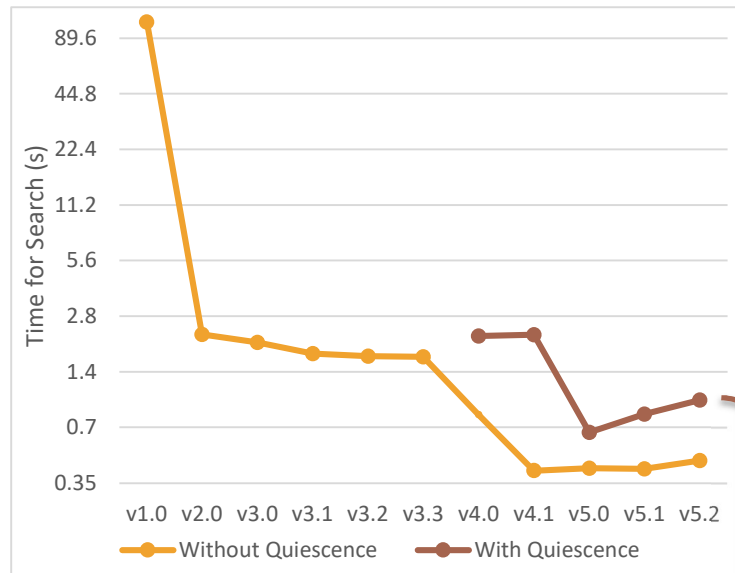
ENDGAME TEST

The final position is a medium-complex endgame, denoted in FEN notation with (2r3k1/8/6p1/ppP3Kp/4RP1P/6P1/P7/8 w -). This position was chosen due to the limited number of moves that are winning for white, as well as it striking a good balance of complexity. Our AI will search on this position with a static depth of 6.



Sample Endgame Position.

Best Move (Stockfish) = [Kxg6]



Current results (without Quiescence): 13,334 positions searched in 0.463 seconds. Move = Kxg6 (centipawn loss = 0).

Current results (with Quiescence): 24,614 positions searched in 0.984 seconds. Move = Kxg6 (centipawn loss = 0).

(We see a slight dip in efficiency in v5.2 due to the addition of my endgame heuristic.)

SECTION 5: EVALUATION

SUPERVISOR ASSESSMENT

To initiate my evaluation process, I met up with my project supervisor and showed him my program. I then conducted an interview with him, where he answered the following questions:

Transcript:

After playing around with my program, would you say it has met your expectations?

Yes - I didn't detect any bug or crashes when I used the program, the computer stopped me from making any illegal moves, and I really liked some of the included features. I'm very happy with the finished product.

Do you believe that my program has fit the purpose you originally laid out for it?

Definitely. I was able to analyse positions with the program, play against the AI or with a friend, and I had lots of fun using it.

What it is like to play using the One Player Mode?

I found it very good. I especially appreciated what the computer said about its move, like its highest depth, evaluation, and current move. I also liked that I could see the AI's move on the board while it was thinking, and that I could undo my move if I accidentally made a mistake.

What it is like to play using the Two Player Mode?

I found the Two Player Mode really easy and intuitive to use. One thing I liked was the simplicity of the layout (which I originally wanted), and the fact that only necessary features were included. I also liked the ability to view the last move on the board, and that the board would automatically flip after each turn.

What were your thoughts about the included Analysis Mode? Did you find it useful or not?

Personally, I found this mode the most useful out of them all – I used this feature to search deeply into positions, and I liked the layout of the mode. I found the included features to be very useful (such as being able to change how long the AI can search for, import / export FENs. etc) and I liked the level of customisability available.

In a nutshell, how would you rate my AI's performance?

The AI itself was much better than I expected it to be. It made its moves in a short amount of time, and can make very challenging moves too! I found it cool that there were multiple AIs running simultaneously so that each search would take the same amount of time, and the difficulty options were a great bonus. I also really liked that the AI knew beforehand when it could checkmate me, and so it didn't have to search for as long when a mating pattern was found. However, I found the AIs with Quiescence to be much worse than the ones without, even though they were supposed to be more difficult to beat. The Quiescence AIs often blundered pieces and constantly made illogical moves, and so it was very easy to beat.

How would you rate the level of interaction and user-friendliness between you and my program? (How easy was it to move pieces, how easy to use / beneficial were the GUI controls, how did the program interact with you, etc.)

The program was very easy to interact with – I liked the fact that the system told me all the legal moves I could make with a piece when I clicked on it. The Main Menu and pre-game customisations were also easy to use, and helped me set up a game to fit my specific needs. The fact that the program was simply laid out made the program very pleasant to use. There was some slight visual glitching when I moved pieces over each other, however it wasn't large enough to distract me from a game.

Did you benefit from the customisation available in my program?

Yes – I was able to change the AI difficulty to suit my playing ability (which I liked), and the ability to set custom FEN positions was really appreciated. I also found the ability to change the board colour scheme, along with little touches like having my chosen colour scheme persisting amongst sessions, to be very useful.

What would you say is/are the biggest negative(s) in my program currently?

Other than the illogical moves that the Quiescence AIs made and the slight graphical glitching, I couldn't spot any significant negatives in the program.

NOTES ON SUPERVISOR'S COMMENTS

Overall, I am very happy with my supervisor's comments – it appears that I have included all the features he deemed important, and have created a program that fits his specific purpose. It also seems that the addition of my customisation options, board flipper, and move undo-er have been a real success. More importantly, my supervisor made it clear that my program fits his original needs – he was pleasantly surprised by the AI's performance and its ability to multitask, and found each mode (from the simple and intuitive design of the Two-Player Mode to the feature-rich and customisable options available in the Analysis Mode) important and useful in their own ways. Therefore, I would call this project an overall success from this point of view.

I am very pleased that my supervisor didn't detect any bugs or crashes during his experience, however I would be somewhat surprised if he did experience any, due to how long I spent testing my FEN validation process and move generation functions. Regarding the 'slight visual tearing', this issue has unfortunately been present since the first version of my program, and results from a technical limitation of VB.net and how it handles transparent elements in PictureBoxes. To learn more about this limitation, please read my thoughts on this in my [Personal Evaluation](#).

FIXING QUIESCENCE

Perhaps the most important topic of discussion stems from my supervisor's comment about the performance of my AI, in which he stated that the AIs with Quiescence enabled were worse than the AIs with it disabled; after conducting my own tests, I can confirm that the Quiescence AIs are not performing as well as they should be. So, I had a deep dive into my system in strict environments, and after a while found a huge issue in my implementation...

```

The AI is now searching...
Depth Of 1 Completed. Move = Qe8, with Evaluation = -8
Positions searched = 75
Depth Of 1 Completed in: 12 Milliseconds.

Depth Of 2 Completed. Move = Bxe3, with Evaluation = 2
Positions searched = 621
Depth Of 2 Completed in: 109 Milliseconds.

Depth Of 3 Completed. Move = Bxe3, with Evaluation = -1
Positions searched = 1295
Depth Of 3 Completed in: 144 Milliseconds.

Depth Of 4 Completed. Move = Bxe3, with Evaluation = 2
Positions searched = 54728
Depth Of 4 Completed in: 7394 Milliseconds.
    
```



Above is a sample position that illustrates the problem quite well – as you can see, the AI's top move at a depth of 1 (Qe8) has been given an evaluation of -8! This indicates that there is something seriously wrong with Quiescence, so what's going on here?!

Well, let's go back to our initial description of Quiescence (which, if you need a quick refresh on, you can find [here](#)) – the AI's search is conducted as normal, however when leaf positions are reached (when the depth limit is met), instead of stopping we continue searching, but only include capture moves in the move generation. This continues until we reach a 'quiet' position, or one where there are no more capture moves available. So, with that in mind, let's have a look at what our buggy AI is thinking:

The AI searches at a depth of 1 – first considering the move Qe8. It then notices that the depth limit has been reached, and so it initiates Quiescence mode – only considering capture moves from this point onwards. This means that white, in response to the move Qe8, is forced to play one of two capture moves – Qxb7 or Qxe7 – both of which loses the queen on the next move. And hence we have found the problem: when Quiescence is activated, the AI is forced to make capture moves – even when the only capture moves available are bad for the player. To solve this, when Quiescence has been activated we need to first evaluate each position, and set that score as the move to beat. This prevents bad capture moves from influencing the search, as they won't be strong enough to beat the evaluation of the position. Here's what this addition looks like in code:

```

'This block of code is placed at the start of MiniMax, just after
the TFTable generator.
Dim BestMove As Decimal
If isWhite Then
    BestMove = Integer.MinValue '-infinity.
    If depth <= 0 And Not WInCheck.IsInCheck Then
        'Quiescence mode activated.
        'Evaluation of board is the current move to beat.
        BestMove = Evaluate(Board, MaterialCount)
        Alpha = Math.Max(Alpha, BestMove)
    End If
Else
    BestMove = Integer.MaxValue '+infinity.
    If depth <= 0 And Not BInCheck.IsInCheck Then
        'Quiescence mode activated.
        'Evaluation of board is the current move to beat.
        BestMove = Evaluate(Board, MaterialCount)
        Beta = Math.Min(Beta, BestMove)
    End If
End If
    
```

One small element to note in this code is that we don't calculate the BestMove lower bound if the player to move is in check. This is because, if the position we have reached is a checkmate, the checkmating score ($\pm\infty$) will be replaced by the evaluation score^[4] – thus producing invalid results.

Now, if we run our AI on the same position with that code in place, we get these results:

```
The AI is now searching...
Depth Of 1 Completed. Move = Qe8, with Evaluation = 0
Positions searched = 36
Depth Of 1 Completed in: 216 Milliseconds.

Depth Of 2 Completed. Move = Qe8, with Evaluation = 0
Positions searched = 319
Depth Of 2 Completed in: 274 Milliseconds.

Depth Of 3 Completed. Move = Qe8, with Evaluation = 0
Positions searched = 1445
Depth Of 3 Completed in: 453 Milliseconds.

Depth Of 4 Completed. Move = Qe8, with Evaluation = 0
Positions searched = 9785
Depth Of 4 Completed in: 1664 Milliseconds.
```



Much better! What you may have noticed, however, is that with our bug fixed, the AI is significantly faster for higher depths, and also searches through less positions! But this isn't just a one-off – below are the statistics for the starting position, before and after the bug was fixed...

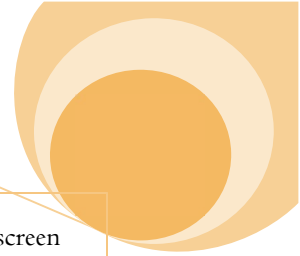
	Depth Searched	Time 1 (s)	Time 2 (s)	Time 3 (s)	Average (s)	% Faster
Before Fix	6	6.963	6.807	6.881	6.884	56.57%
After Fix	6	2.964	2.958	2.984	2.969	
Before Fix	7	38.364	37.325	37.538	37.742	69.14%
After Fix	7	11.643	11.670	11.632	11.648	

This proves how beneficial this new implementation has been to my program, and with it in place my AI plays much stronger.

OBJECTIVE ASSESSMENT

Essential objectives:

Objective No.	Objective	Was this Objective Achieved? (R/A/G)	Explanation
I	The project must be written in VB.net with the Windows Forms Application to create a graphical interface.	G	Multiple forms are used, along with many customisable options and buttons, in the aim to provide the user with a user-friendly yet feature-packed GUI.



2	The user should be presented with a graphical (and accurately scaled) interface of a chessboard, along with its pieces.	G	My chessboard is scaled to fit on the screen window, and it is symmetrical. The pieces fit onto their appropriate squares perfectly, are appropriately sized, and use the same images as other top chess websites, to help avoid confusion.
3	The user should be able to drag and drop the pieces on the board, and if that move is legal (see Objective (4) for more details), have it be represented on the board.	G	This drag-and-drop mechanic works as intended, allowing the user to make their move on the board. When their move is made, we check to see if it is legal, and if so submit it as a legal move. There is some slight jitteriness when moving pieces, but this does not lead to any bugs or crashes, and should not affect user experience.
4	The program should use the standard rules of chess to determine if a move the user has entered in is legal or not (and refuse it if it's not). This may require context about the board, such as restricting certain moves if a piece is blocked by another piece, or if a piece is pinned by an opponent's piece.	G	My PieceLegalMove subroutine works hand-in-hand with the DoesMoveResolveCheck subroutine to determine if a given move is legal. The PieceLegalMove subroutine generates all the pseudo-legal moves a piece can make (including pins), and if the user is in check, we pass all these moves onto the DoesMoveResolveCheck subroutine to eliminate those that do not help to escape a check. If a move successfully makes its way through all these steps, then we deem the move as legal.
5	The user should be able to Castle both king-side and queen-side whilst conforming to the rules of chess.	G	This feature works as intended, allowing only legal castling to be performed. This is handled by my PieceLegalMove subroutine.
6	The user should be able to enter in a chess FEN, and if it is legal (which will be determined by the program), it will be presented onto the chessboard.	G	This can be done either at the start of a user's game, or midway through (if they are using Analysis Mode). If it is legal, we convert the FEN to a board position, and display it on the board. (Please see here for my FEN validation process.)
7	The user should be able to export the current chess position on the board to a FEN string that can be used on other chess websites / engines.	G	This process is done automatically whenever the user (or the AI) makes a move, so that it can be presented to the user whenever they please. This FEN can be easily copied and used on every FEN-accepting website. To achieve this, we condense all the details of a chess position into a FEN string (using the ConvertToFEN algorithm), which is then presented to the user.
8	The user can promote a pawn to a queen if it reaches the end of the board.	G	This feature works as intended by replacing the pawn's board index and swapping its PictureBox.

9	The program should identify when a check has occurred, and will notify the user of this both visually and audibly. This should also be used to restrict the moves an opponent can make, as checks need to be resolved immediately.	G	Checks are calculated via the FixTFTTable algorithm by scanning the legal moves of a piece that could intersect the enemy king. If a check does occur, we use the CheckChecker subroutine, which notifies the user of this both visually and audibly.
10	When a user is put into check, the program should accurately determine whether this is a Checkmate or a regular check. If it is a Checkmate then the game should terminate.	G	This process is achieved via the CheckForEndState function in my AI class by scanning for all the legal moves a player can make on their turn. If there are no legal moves, and the user is in check, then the function returns this code – making the game end due to checkmate. There are also visual and audible indicators for this.
11	The user, at any time, should be able to let my chess AI make a move from the current position (for either player). This move should be represented on the chessboard.	G	In the Analysis Mode, this feature works completely, and the user can make the AI search at any time they please. However, in an effort to improve user-experience, I have purposely stopped the user from making the AI play for them during the One-Player Mode (as that would remove all sense of difficulty). My supervisor has approved this subtle change.
12	The AI should be given either a certain amount of time to search for, and at the conclusion of this time its move should be chosen and displayed.	G	In my program, the system allocates a given amount of time (which is fixed in the One-Player Mode, but variable in the Analysis Mode), and creates a number of instantiations of AIs – all searching to different depths. At the conclusion of this time, the search finished at the highest depth is chosen and displayed.
13	My AI should be modular and separate from my main chess program via classes and object-oriented programming.	G	My AI is separate and modular to my main program, and is controlled as I originally planned it to be. The input variables to my AI are the position's FEN, the required depth, and if Quiescence mode is enabled. The AI also contains some optional functions which interact with the main class (which control such things as: making a move on a temporary board, finding a piece's legal moves, etc), however interaction is as limited as possible, and I have kept the class modular.



Desired objectives:

Objective No.	Objective	Was this Objective Achieved? (R/A/G)	Explanation
1	When allowed to, the user should be able to take En Passant.	G	This feature works as intended – there are some positions where it is illegal to take En Passant (please see here to learn more); my system is able to detect these positions and only let the user take En Passant when legal.
2	The game should end if only kings are left on the board, or in other special circumstances (please see https://www.chessstrategyonline.com/content/tutorials/how-to-play-chess-draws for more detail).	A	All the drawing rules are working correctly when the user or the AI are making their moves. However, in an effort to improve the efficiency of my AI, the computer does not detect these positions <u>during the MiniMax process</u> (however, each of the AIs' final moves are scanned for these end positions). I have found that this compromise has the best balance between efficiency and user-experience, and as a result I am happy with my chosen implementation.
3	The user will be given the option at all times to undo their / their opponent's last move.	G	This feature works as intended by swapping the current FEN string with the FEN string of the previous position, then displaying the position again. Furthermore, I have also included some smart features in this process, such as undoing two sets of moves in the One-Player Mode (as the computer always makes its move automatically).
4	If the AI detects a forced checkmate at a shallow depth, it should terminate its searching and instantly play the move that results in that checkmate.	G	As we have multiple versions of the AI running simultaneously, I have designed an ABORT mechanism into the AI, so that it can be easily terminated when another AI has detected a mating pattern. This feature has improved my program's user-experience significantly.
5	When the AI is creating its move, the user should be updated in real-time on its progress and current evaluation.	G	There is lots of interaction between the computer and the user whilst the AIs are making their moves. This includes the current highest depth searched to, this depth's best move, the current evaluation for the position, and a graphical indication of the AI's current move (squares are highlighted on the board appropriately to signify the AI's move).
6	The user should be able to select multiple / variable difficulties of the AI.	G	As the AIs are solely restricted by time, we can easily adjust the difficulty of the AI by controlling how much time it has to search for. Furthermore, the easier difficulties of the AI in the One-Player Mode do not use the Quiescence algorithm – making it a more forgiving opponent for beginners.

7	By clicking on a piece, the user should be told visually the squares it can move to.	G	Through lots of testing to determine the best way of displaying a piece's legal moves graphically, I have found that surrounding the legal squares in a green border (when the user clicks on a piece) is the most aesthetically pleasing. We calculate the piece's legal moves, then represent these moves on the Boolean array 'LegalMoveSquares'.
---	--	---	--

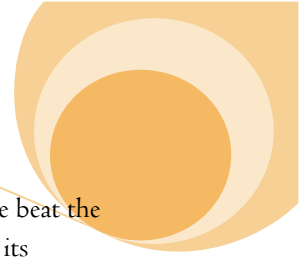
Preferred objectives:

Objective No.	Objective	Was this Objective Achieved? (R/A/G)	Explanation
1	The user should have their last move be illustrated on the chessboard graphically.	G	This is represented by two green squares on the GUI that represent the last move made – stored in the SquareHistory array.
2	The user should be able to move the program window around at all times, <u>including</u> when waiting for the AI to make its move.	G	As I am creating multiple instantiations of my AI that run on separate threads simultaneously, I am able to reserve the system's 'main' thread to controlling the GUI, along with its objects. This includes allowing the user to move around the program window, as well as allowing them to interact with <u>some</u> GUI objects.
3	The user can pick from a variety of board colour schemes that can be modified in real-time. This setting should be stored in a file, so that when the user opens the application, their preferred colour scheme is retrieved and applied seamlessly.	G	The user is able to choose between the current colour schemes: "Default", "Blue 1" (light), "Blue 2" (dark), "Gold", "Green", "Purple", and "Grey". This is selected via a drop-down menu, and once the user selects an option, the board colours are updated instantly. Their preferences are written to the file "ColourPreferences", which is read from whenever the user boots up the program. Because of this, a user's colour profile will remain even if they switch game modes.

CALCULATING MY AI'S STRENGTH

To calculate the relative strength of my AI, I will be using the ELO system – a number which represents a chess entity's general ability. To generate this value for my AI, I put the system against the chess.com adaptable engine (which can be accessed through this link: <https://www.chess.com/play/computer>), and played multiple games against it on increasing difficulties. For the specific details about each game, including all the game PGNs, please access the below link:

<https://drive.google.com/file/d/1IRLYgLFrb8FYXQmwHGtIOvC5LsQpaJaD>



In a nutshell, however, my AI was able to comfortably beat the 1500 ELO rated engine, and in one case beat the 1700 ELO engine in a truly fantastic game (in which my AI played a brilliant move that capitalised on its opponent's hard-to-see blunder). Therefore, using all this information, I estimated the ELO of my AI to be **1650**. For scale, the ELO of an average player on chess.com (the website where these games were played) is **790²¹**, and the ELO of a typical Grandmaster is ~2500-2600. This shows that, whilst my AI won't be beating Magnus Carlsen anytime soon, it is more than strong enough to beat amateurs, and even poses a challenge to advanced players. Therefore, I would say that my AI is a real success in this area, and fulfils its purpose as a chess-playing entity.

PERSONAL EVALUATION

Probably the most unexpected thing I found difficult during the completion of this project was knowing when to stop. As creating a chess AI is an incredibly competitive challenge amongst the artificial intelligence community, there are countless features and improvements that I would have loved to add to my program - including Zobrist Hashing, transposition tables, more evaluation heuristics, genetic functions / learning, the 50 Move Rule, binary structures, more variable AI difficulties (using multiple algorithms), the implementation of NegaMax, the addition of checks into the move ordering function, etc. Because of all this, I initially found it difficult to take time constraints into account – however, as I became more experienced with coding, I found it easier to plan out my program, and to only include feature that my supervisor and I deemed important.

I believe that the current version of my program contains two primary 'issues' / annoyances, and two main possible improvements. Beginning with the annoyances:

- 1) As my supervisor mentioned in our [most recent interview](#), when the user is dragging a piece, some graphical tearing appears when moving it over other pieces. This is unfortunately a technical downside of using PictureBoxes and transparent objects – all my pieces are transparent in the context of the checkerboard, but not in context of each other. To fix this issue would mean completely redesigning my GUI to use sprites, and modifying the way that the user interacts with my GUI; therefore, I didn't deem this issue worth the immense time required to fix it. Were I to begin this project from scratch, however, I would definitely switch to using sprites.
- 2) In a typical game of chess, the user can choose from whichever piece they want when they promote a pawn. However, my program only allows for queen promotions. This issue boils down to the huge speed deficit this would result in in my AI, along with (yet again) the limitations of the PictureBox system – to allow the user to (for example) promote to a knight, I would need to create 8 new knight PictureBoxes (one for each potentially promoting pawn), along with all the controls and algorithms that the pieces rely on. I would then need to complete this for both the rooks and bishops, along with creating the promoting GUI that the user would interact with. I would also need to redesign the move generation algorithms, as pawn promotion is the only move in chess that results in multiple 'possibilities' per move. All this would be for a mechanic that, in the years that I've played chess for, I have only used a handful of times. Therefore, it is extremely unlikely that my AI would need to promote to anything other than a queen.

Other than these annoyances, I believe the biggest possible improvement to my program would be my AI's evaluation function – more specifically, its performance during the opening phase of a chess game. Currently, the AI makes very strange and quite random opening moves – naïve that this will lead to trouble long-term. Furthermore, because there are no evaluation heuristics present in the opening & middlegame phases of the chess

game, there are many positions where the AI just moves its pieces back and forth – waiting for the human player to make their attack. There are multiple methods to improving this, however the most effective (yet complicated) method would be implementing a ‘Piece-Square Table’^[3]. This states that, for a given piece, there are ‘optimal’ and ‘less optimal’ squares for that piece to lie on, and therefore we evaluate positions higher where all a player’s pieces lie on these optimal squares. Not only should this encourage the AI to keep its king safe in the corner of the board, it would also improve its opening & middlegame performance significantly. Unfortunately, I couldn’t find an implementation of this algorithm that didn’t decrease the depth efficiency of my AI to an extreme degree, and therefore I concluded that, until I could make my AI significantly faster, implementing ‘Piece-Square Tables’ into my code wouldn’t be worthwhile.

Furthermore, a huge addition to my program would be implementing randomness into my AI’s moves. Currently, when my AI plays a game against itself, it makes the same set of moves every time, and typically ends up in a position where the two AIs endlessly shuffles its pieces – making no progress. Adding an element of randomness to my AI’s moves would most likely fix this issue, and would also make its games more interesting. However, implementing this feature would result in a complete restructuring of Alpha-Beta Pruning (as the score for each MiniMax branch is based on the score for Alpha / Beta, and therefore we cannot tell if there are multiple ‘best moves’ in a position).

Despite these points, however, I am incredibly happy and proud of the program I have produced. I have provided a user-friendly, crash-free, and feature-rich chess-playing environment that contains lots of details that improves user experience (such as being able to flip the board, import & export custom positions, displaying a piece’s legal moves by clicking on it, etc). Furthermore, I have created a competent AI system that is able to play chess to a respectable level, whilst taking time controls into account. This AI can comfortably search to a depth of 5/6 on even the most complex of positions, but can reach as high as 12 in endgames - making it a formidable opponent to play against. I am especially proud of the level of interaction between the user and the computer – the AI stops searching when it finds a checkmating pattern (to help save the user’s time), and constantly gives the user visual updates on its current move, along with its constantly updating evaluation of the position. I truly hope that these features come together to create an easy-to-use and ‘fun’ program that has its place in the world of chess-playing AIs.

REFERENCES

- [1] = https://www.chessprogramming.org/Quiescence_Search (“Stand pat is not allowed if we are in check ... because we are not sure that there is a move that can match alpha - in many positions a check can mean a serious threat that cannot be resolved.”).
- [2] = <https://www.chess.com/leaderboard/live/rapid> (“Avg Rating = 789.86”).
- [3] = https://www.chessprogramming.org/Piece-Square_Tables (“Piece-Square Tables: a simple way to assign values to specific pieces on specific squares. A table is created for each piece of each colour, and values [are] assigned to each square.”).